

Master Dissertation

**Social Network Analysis via a Sentiment Dictionary for identifying the most promising opportunities within a local and industry-specific context:
The case for small to medium-sized hotels**

Submitted June 2021, in partial fulfilment of the conditions of the award of the degree
MSc Information Systems

By xxx xxx

School of Computing and Engineering

University of West London

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature  _____

Date 22 / 05 / 2021

Copyright

The copyright of this dissertation belongs to the author under the terms of the UK Copyright acts as amended by The University of West London regulations.
Due acknowledgement must always be made of the use of any materials contained in, or derived from this thesis.

Abstract

Customer voices about items and services expressed on social media platforms are regarded as authentic thereby presenting chances for entrepreneurs to learn from the critical portion of these reviews in order to produce additional wealth for the clients as well as for themselves in the future. Therefore this study utilises an opportunity system on sentiment connoted hospitality articles so that particular hotel services exhibiting a relatively low amount of satisfaction plus are demonstrating an elevated degree of relevance for a broad scale of guests are identified for the purpose of deriving valuable service-specific recommendations. Furthermore especially the resource-restricted and data-privacy-concerned situation of small to medium-sized hotels is considered by applying offline the locally-confined, language-specialised as well as branch-limited Naive Bayes sentiment dictionaries in combination with Latent Dirichlet allocation topic models, which represent the two major components of the opportunity system. Moreover the opportunity approach has been entirely conducted twice per individual real-world Twitter dataset addressing one of the three topic categories (bar, entertainment, wedding) published within Las Vegas in English respectively thereby integrating two sentiment lexica separately, which were previously trained on Yelp data and selected due to their promising classification capabilities on the proxy test data stemming as well from Yelp ((20 000 review-strong sentiment dictionary: F-beta score = 0.765 - 0.805; precision = 0.769 - 0.801; recall = 0.746 - 0.820; accuracy = 0.787), (200 000 article-rich sentiment lexicon: F-beta score = 0.803 - 0.848 ; precision = 0.805 - 0.846; recall = 0.794 - 0.855; accuracy = 0.829))). In addition performed experiments reveal that the optimal topic models feature strengthened coherence as well as perplexity scores, if the intersection between one of the gathered three Twitter datasets and rather the sentiment lexicon consisting out of 20 000 recensions than the independently chosen one comprising 200 000 critiques is incorporated. In the end this research quantifies that for a minimum of 80% of hotel service topics, meaningful indications for a beneficial development may be inferred.

Acknowledgements

Herewith I express the deepest gratefulness towards my beloved family as well as friends for illuminating my heart especially on darker days. I would also like to thank my supervisor Mr. Dr. Jie and the entirety of my teachers, who I had the pleasure to meet during my stay at the University of West London, because they permanently pushed me to the maximum of my God-given capabilities and refined me to deliver a degree of excellence, which I previously would have never believed to accomplish. Lastly my everlasting appreciation is addressed to the noble country of the United Kingdom and its citizens due to them providing me the not taken-for-granted privilege to receive the globally recognised superb British education.

Table of contents

Copyright	II
Abstract	III
Acknowledgements	IV
Table of figures	VII
1.) Introduction	1
1.1.) Aim and research question	2
1.2.) Objectives	4
2.) Literature review	7
3.) Methodology	12
3.1.) Development and validation of a sentiment lexicon (Phase 1)	12
3.1.1.) The suitability of Yelp data for the training and proxy test phase	12
3.1.2.) Data Preparation for Yelp critiques	13
3.1.3.) Computing sentiment points via Naive Bayes probabilities	14
3.1.4.) Measuring the categorisation reliability of a sentiment lexicon	17
3.2.) The opportunity algorithm applied on domain-specific reviews (Phase 2)	19
3.2.1.) Data Collection of Tweets	19
3.2.2.) Data Preparation of Tweets	21
3.2.3.) LDA Topic Modelling	21
3.2.4.) Calculation of opportunity scores and their illustration	22
4.) Results & Discussion	24
4.1.) Naive Bayes models' performance on sentiment word classification	25
4.1.1.) Selecting specific sentiment dictionaries for further proceedings	30
4.2.) Characteristics of the examined topic models	33
4.2.1.) Comparison of the generated LDA topic models	33
4.2.2.) Inference of topic titles	35
4.3.) Opportunity score visualisations & their analysis	39
5.) Limitations	47

6.) Conclusion	48
7.) Future Work	50
8.) References	51
9.) Word count	59
10.) Appendix	60
10.1.) Dropbox link to this master dissertation's research documents	60
10.2.) Supplementary illustrations	60
10.3.) Code of the two selected opportunity systems	70

Table of figures

Figure 1	- The Opportunity Equation	7
Figure 2	- The Phase 1 Data Preparation Process	13
Figure 3	- The Naive Bayes Formula	14
Figure 4	- Phase 1 - Design for the generation of a viable domain-specific sentiment dictionary	15
Figure 5	- The Phase 2 Data Preparation Process	19
Figure 6	- The complete opportunity system (i.e. Phase 1 + Phase 2)	20
Figure 7	- The allocation of open businesses across the cities of the Yelp dataset	24
Figure 8	- The review sentiment distribution for hotels located in Las Vegas	24
Figure 9	- Categorisation quality on strongly positive sentiment	25
Figure 10	- Categorisation quality on slightly positive sentiment	26
Figure 11	- Categorisation quality on slightly negative sentiment	27
Figure 12	- Categorisation quality on strongly negative sentiment	28
Figure 13	- Categorisation quality on negative sentiment	29
Figure 14	- Categorisation quality on positive sentiment	30
Figure 15	- Classification performance of positive and negative sentiment (200 000 reviews - training set portion = 99%)	31
Figure 16	- Classification performance of positive and negative sentiment (20 000 reviews - training set portion = 10%)	31
Figure 17	- Classification performance of strongly as well as slightly positive and negative sentiment (20 000 reviews - training set portion = 10%)	60
Figure 18	- Classification performance of strongly as well as slightly positive and negative sentiment (200 000 reviews - training set portion = 99%)	60
Figure 19	- Confusion matrix of positivity and negativity (20 000 reviews - training set portion = 10%)	61
Figure 20	- Confusion matrix of slight as well as strong positivity and negativity (20 000 reviews - training set portion = 10%)	61
Figure 21	- Confusion matrix of positivity and negativity (200 000 reviews - training set portion = 99%)	61
Figure 22	- Confusion matrix of slight as well as strong positivity and negativity (200 000 reviews - training set portion = 99%)	61
Figure 23	- Quality assessment table of the explored topic models	34
Figure 24	- Coherence and perplexity scores over the topics of topic category bar (20 000 reviews - training set portion = 10%)	62

Figure 25	- Coherence and perplexity scores over the topics of topic category entertainment (20 000 reviews - training set portion = 10%)	62
Figure 26	- Coherence and perplexity scores over the topics of topic category wedding (20 000 reviews - training set portion = 10%)	62
Figure 27	- Coherence and perplexity scores over the topics of topic category bar (200 000 reviews - training set portion = 99%)	63
Figure 28	- Coherence and perplexity scores over the topics of topic category entertainment (200 000 reviews - training set portion = 99%)	63
Figure 29	- Coherence and perplexity scores over the topics of topic category wedding (200 000 reviews - training set portion = 99%)	63
Figure 30	- Identified bar topics (20 000 reviews - training set portion = 10%)	64
Figure 31	- Identified entertainment topics (20 000 reviews - training set portion = 10%)	65
Figure 32	- Identified wedding topics (20 000 reviews - training set portion = 10%)	66
Figure 33	- Identified bar topics (200 000 reviews - training set portion = 99%)	67
Figure 34	- Identified entertainment topics (200 000 reviews - training set portion = 99%)	68
Figure 35	- Identified wedding topics (200 000 reviews - training set portion = 99%)	69
Figure 36	- Dash Table sorted per default by opportunity score (200 000 reviews - training set portion = 99%)	39
Figure 37	- Dash Table alphabetically sorted according to the topic category (200 000 reviews - training set portion = 99%)	39
Figure 38	- Dash scatterplot with all examined topic categories (left side) and the Dash scatterplot excluding topics of the topic category wedding (right side) (200 000 reviews - training set portion = 99%)	40
Figure 39	- Summary of the monitored hotel service opportunities (20 000 reviews - training set portion = 10%)	41
Figure 40	- Summary of the monitored hotel service opportunities (200 000 reviews - training set portion = 99%)	44

1.) Introduction

Since the start of the 21st century, social media (SM) has continuously expanded its presence in the daily routine for a large-scale of the global population, who are interested to contribute and absorb ideas in the formats of videos, voice messages, photos or texts via these digital channels. In fact Koiranen et al. (2019) discovered that in the timespan from 2008 to 2016 the probability of possessing a SM account in Finland was elevated while being unaffected by diverse socio-demographic factors, whereas Ofcom (2017) reported that the social media penetration rate of the UK has surged among all age brackets between the years 2012 and 2016. Furthermore Koiranen et al. (2019) brought to light that the distribution of the social media adoption rate is skewed, therefore resulting in a relatively lower growth of the attendance levels for the elderly when compared to the younger generation. Moreover research by Ofcom (2017) indicates that each respective social grade group displays its individual propensity in joining certain SM platforms.

The opportunity arising is the profound analysis of these social media data, namely social media analytics (SMA). To illustrate according to Lee (2018) SMA is composed out of first observing the communication occurring in social media networks and second it is involving the extensive evaluation from these streams of information to extract client preferences concerning goods or services. For example SMA has been utilised by Kim and Kim (2019) to assess fans' satisfaction levels concerning a football match resulting in the fans' suggestion that the former coach should return in order to replace the current one. These kind of SMA conclusions based on customer feedback are tremendously appreciated by small to medium-sized enterprises (SMEs) in order to boost the features of their products or services ((Ahmad, Ahmad, and Bakar, 2018), (Liu et al., 2020)), if data privacy and cyber security requirements are fulfilled to an acceptable degree (Rajabion, 2018). Typically SMEs's relatively small range of workers is a criterion for classifying companies as SMEs, so that for instance in the US, they are defined as firms, which exhibit under 500 employees (Rajabion, 2018). Furthermore the operational constraints of SMEs are characterised by a narrow allocation flexibility in the area of budget (Ahmad, Ahmad, and Bakar, 2018) as well as human resources and to acknowledge their finite capacity of technological expertise (Liu et al., 2020) is crucial. Due to the significantly scarce amount of proficient SMA analysts in the US (Rajabion, 2018) and the firmly required research to correctly quantify customers' emotions for the tourism sector (Kirilenko et al., 2018) the relevance of realisable SMA procedures particularly for small to medium-sized hotels (SMH) may not be underestimated.

Supplementing the preceding essentiality Davras and Caber (2019) urge to recognise that the various factors of hotel services are impacting the total customer sentiment remarkably thereby giving reason to put emphasis on the service quality management. Similarly Nazari et al. (2020) were able to demonstrate that 52.4% of hotel performance was produced by client sentiment, while Geetha, Singha and Sinha (2017) delineate that the positivity or negativity of hotel customers' written critiques correspond to their generated feedback scores for the respective establishment. Moreover SM websites provide through their hotel reviews a point of reference for a considerable portion of tourists (Park and Kwon Lee, 2021), thus influencing the purchasing behaviour of potential hotel customers (An et al., 2020). In addition the latter scholars ascertain that the extent of service quality is positively correlated to the number of review photos and that if the photos are consistent with the review texts then the review helpfulness is amplified but An et al. (2020) also admitted the difficulty of evaluating visual content in comparison to textual data. Therefore the analysis of videos and photos seems to be unsuitable for SMEs given their strict resource limitations. However in a distinct SMA approach, a novel potential of SM data was harvested by Fuentes-Moraleda et al. (2020). These academics studied in an analytical fashion TripAdvisor critiques focusing on robot services for natural persons within a hotel context resulting in the obtainment of evidence for hotel robots being primarily beneficial for utilitarian purposes as perceived by the guests, which possibly leads to adjustments in the division of hotel work between machines and humans. Alternatively Davras and Caber (2019) identified a positive impact on customers' happiness in their case study focusing on Turkish hotels if the service executed by the hotel's staff is enhanced. Likewise Nazari et al. (2020) uncovered that in a SMSH environment, customer satisfaction and as a consequence the hotel's competitive market position may be strengthened, if the responsible executive accomplishes to improve service standards.

1.1.) Aim and research question

Twitter posts present a cornucopia of potential customer perspectives. Due to them exhibiting a highly significant ratio of negative sentiment ((Liu, Burns and Hou, 2017), (Blagus and Zitnik, 2018)), which is partly directed at services to a relevant proportion (Hu et al., 2017), they are likely to be beneficial for SMA since they partially indicate unfulfilled desires for novel or established customer services ((Lee, 2018), (Ulwick, 2005), (Liu, Burns and Hou, 2017)) as well as they might include latent characteristics of specific services concerning the hotel industry (Davras and Caber, 2019). In fact the service quality

is in particular crucial for the low-cost hotel sector as experiments by Geetha, Singha and Sinha (2017) portray. In addition the more diminished the degree of services is, the more extensive grows the number of client photos in conjunction with their corresponding negative customer reviews (An et al., 2020). Even in the instance of robot hotels, service still manifests a significant characteristic which is coming to light due to guests criticising the delivery duration of hotel room service robots (Fuentes-Moraleda et al., 2020). Furthermore the contributions by Liu, Burns and Hou (2017) highlight that unsatisfied clients tend to post thrice as much as satisfied ones on Twitter.

Maroufkhani et al. (2020) encountered that the utilisation of big data analytics (BDA), which feature SMA as a topical subsection, benefits especially SMEs in the terms of boosting profits and enhancing audience awareness, while the identical intellectuals assert on the other hand, that risk aversion, the absence of simplicity and the lack of knowledge are existing barriers for SMES' BDA adoption. Moreover while a demand for the aforementioned kinds of evaluation methods is noticeable (Ahmad, Ahmad, and Bakar, 2018), circa 50% of US SMEs fail to examine this precious SM feedback ((Statista Research Department, 2016a), (Deloitte, 2019)), therefore artificially limiting their SME's potential to attain an optimal market position ((Lee, 2018), (Maroufkhani et al., 2020)). However it is necessary to reflect upon SMEs' situation therewith taking into account their compact corset in terms of resources ((Ahmad, Ahmad, and Bakar, 2018), (Liu et al., 2020)), of technological capabilities (Liu et al., 2020) and of data protection principles (Rajabion, 2018) to settle for the most appropriate compromise. As a result the following research question was formulated:

May a SMA algorithm be realised in a real-world environment to provide beneficial guidance on how to refine individual hotel service aspects after their respective chances of success are determined according to the combination of their relevance and sentiment ratings, while SMSH's resource requirements, the approach's complexity of implementation and the technique's exposure to cyber attacks are reduced to a minimum?

Consequently the aim of this master dissertation arises from the stated research question above and is hereby declared as follows: This research article pursues to originate an inexpensive, effective technological framework with which SMSH are enabled first to gather Twitter reviews, second to scrutinise Twitter users's opinions concerning specific topics as well as its subtopics respectively and third to deduce goals for a successful future development of their services.

1.2.) Objectives

The aim is divided into seven components indicating that the objectives have to be executed in the subsequent order outlined below for achieving the ultimate goal, which is located in a context of hotel services:

Data Collection

The acquisition of customer reviews, which are pre-labelled with the corresponding client sentiment as well as featuring the property of incorporating industry-specific details expressed within a selected location, is demanded and then provided for the composition and evaluation stages of the nominated supervised classifier. Next the capturing of uncategorised SM posts in terms of their emotional tendency emerging from the identical region previously chosen is necessary to be performed concerning the similar branch of the economy, thus only the minimal required amount of data is utilised for further analytical processes.

Data Preparation

The training set, proxy test set and the SM critiques of unknown sentiment are converted into a standardised format which is more likely to allow comparability between these distinct data entities to a great extent and proportional sampling has to be executed in a random manner for attaining the maximum degree of representativity. A second aspect to focus on is the preparation of multiple training and proxy test sets, which in total exhibit the identical number of data points, while they differ in the ratio between the training and test set in order to interpret *ceteris paribus* the effects of this relationship in the succeeding phases.

Generation of validated domain-specific sentiment dictionaries

The selected classifiers have to be constructed by inputting training sets of multiple sizes and numerous training set to test set ratios. Furthermore the intersection of training and testing words is required to be determined resulting in the emission of identical words between both datasets. Next the elimination of words reflecting overall neutral sentiment in at least one of the two examined datasets needs to be conducted due to their uninformative nature from an emotional point of view. Lastly the sentiments of the remaining non-neutral words belonging to the training or the test set have to be compared via conventional machine learning evaluation metrics like precision, recall, accuracy and f-beta score.

Origination and selection of topic models

Models comprising communicated topics depending on the intersection dataset consisting out of the overlap between validated words of the particular sentiment dictionary and the gathered SM review vocabulary have to be developed. By searching for the correct configuration parameter, namely the number of topics, which have to be specified in advance, measures of coherence and perplexity are indispensable to be recorded for the identification of the most suitable topic model per respective sentiment dictionary.

Computation of topic relevance and topic sentiment

The calculation of a single key performance indicator (KPI) combining customers' emotions and clients' perceived essentiality for each certain topic is to be delivered as well as it is desirable for presentation purposes. Furthermore these KPIs are required to be ranked within their discrete topic categories thereby the necessity is formulated to create a topic category-specific baseline for the dimensions of importance and satisfaction as well as for the compounded criterion. Moreover the topics are to be assigned their individual and adequate group names, which may be derived from their ten most frequent words and the respective search terms.

Visualisation of the results appropriate for end user navigation

A proper graphical representation is to be produced. Therefore a scatterplot will have to be generated in order to overview all KPI dimensions per topic when hovering over the particular data point. In addition each topic is required to be colour coded according to their respective topic category, while particular topic categories may be hidden on demand. Furthermore the plot may be set at a default viewing angle whereas it still should be possible to rotate the chart in order to grasp distances between distinct topics visually. Into the bargain an interactive table is to be provided, which an end user may sort according to the table's categories or delete particular rows if they are not of current interest. Equally important is the necessity to display within the referred table the ten most negative words for potential viewers so that they may deduce beneficial advancement directions.

Comprehensive documentation and their accessibility

Last but not least the documentation of the proposed approach ought to be rigorously detailed via first permitting the reader of this master dissertation unrestricted access to a commented version of the written as well as of the executable code. Secondly any

artefacts collected during this research must be available in a digital form and saved in a cloud storage repository in case one may download copies of these artefacts and codes.

2.) Literature review

The opportunity equation invented by Ulwick (2005) embraces the concepts of topic importance and topic satisfaction by blending them into a single score (as depicted in Figure 1) which attains its maximum value once first the emotions addressed towards a theme are the most negative as well as second the subject's relevance is peaking in a concurrent fashion, whereas the score reaches its bottom if the opposite conditions are true. Furthermore the aforementioned algorithm presents an option to support firms by ordering their individual business chances according to the levels of customer priority.

$$\text{Opportunity Score} = \text{Importance} + \text{Max}(\text{Importance} - \text{Satisfaction}, 0)$$

Figure 1:
The Opportunity Equation

Equally of relevance is that although Hu et al. (2017) brought to light that a majority of positive emotions are addressed towards the manufacturing sector, the scholars also revealed that negative sentiment is primarily expressed towards serving industries, which generalises the findings by Farizah Ibrahim and Wang (2019), who specifically ascertain the tremendous level of negative feedback for retail customer and retail delivery services. On the other hand Jeong, Yoon and Lee (2019) overlooked the algorithm's adequacy for services when designing their own opportunity discovery approach deduced from the opportunity formula by Ulwick (2005) (illustrated in Figure 1) due to conducting their procedure on critiques limited to a product capable of performing various tasks. Therefore a clear gap could be identified by applying the opportunity procedure by Jeong, Yoon and Lee (2019) on services, which are consisting out of a range of diverse activities.

The chance of acquiring tremendous insights from scrutinising massive volumes of data (Bednár, 2017) being disseminated in SM networks everyday is opening up for SMEs. At the same time research conducted by Ahmad, Ahmad and Bakar (2018) noticed the demand by SMEs of being enabled to capture and evaluate the communication of customers for first developing their novel goods and services, for second innovating their existing products as well as services and for third enhancing their understanding of the current business environment in order to make more sophisticated choices. Although Ahmad, Ahmad and Bakar (2018) were also criticising the extremely limited studies concerning the use-cases of SM for SMEs the researchers Liu et al. (2020) identified a similar desire by SMEs as well to execute BDA for the purpose of gaining knowledge about customers' purchasing motivation. Despite Farizah Ibrahim and Wang (2019) solely

recognising the enormous potential of SMA applications within the domain of retail, which following the definition by the Office of Management and Budget (2017) excludes the hospitality branch of the economy, Liu, Burns and Hou (2017) encourage the implementation of SMA approaches for the evolution of customer services to foster quality enhancements in this area. Furthermore the need for a framework has been expressed by Davras and Caber (2019), which captures the effects of discrete hotel service attributes on the happiness of the clients. To illustrate according to Nazari et al. (2020) the SMSH, which master how to differentiate the various stages of customer happiness, are the prospective ones of winning ground in the hotel business. In addition Davras and Caber (2019) communicated, that in their examination of a five star hotel this key factor being positive customer sentiment is amplified for novel guests as well as for regulars if the service executed by the hotel employees is enhanced thereby the scientific professionals were also emphasising the essentiality for monitoring on the one hand the delivered hotel services while on the other hand paying close attention to customer aspirations. This prior finding is expanded upon by recommendations postulating a better situated market position of budget hotels in contrast to luxury hotels is attained, if the service standards are elevated (Geetha, Singha and Sinha, 2017). Consequently a second deficiency became more obvious, namely the desire especially by SMSH to determine significant trends in customer feedback concerning hotel services and to make business decisions based on a vastly enhanced foundation of information.

Even though the integration of BDA is yielding significant monetary as well as promotional advantages for SMEs (Maroufkhani et al., 2020), studies by Statista Research Department (2016a) and Deloitte (2019) have surprisingly revealed that only around half of SMEs are operating under the assistance of these approaches. In fact Lee (2018) asserts that companies as a whole boost their competitiveness due to an enhanced quality and quantity of information concerning their field of business, if SMA is specifically performed to monitor first their market players' actions and second clients' preferences of business rivals as well as of their own customer base. Of equal relevance are the results observed by Dong and Yang (2020), which indicate that SMEs are privileged in terms of market performance if they are utilising BDA across various SM platforms compared to firms of large scale. To illustrate further SMEs are also constraint in the adoption of BDA by factors like their insecurity (Maroufkhani et al., 2020) and their concerns about cyber security of their data (Rajabion, 2018). Moreover according to Liu et al. (2020) the suppressed development potential of items and services by SMEs is caused by a lack of data evaluation not only limited to the analysis of external data but also addressed to the

scrutinies of SMEs' corporate documentation. Although Liu et al. (2020) are advocating an approach, which includes the utilisation of cloud infrastructure, in order to resolve the previously addressed issue, SMEs are advised to be aware of the drawbacks which one is exposed to as a consequence of outsourcing the personal cloud architecture to third-party enterprises. To illustrate Amazon Web Services (AWS) has demonstrated during recent years that trust was abused due to the fact that Amazon presented its in-house music streaming service (Nicolaou, 2016) a couple of months after their customer Spotify decided to operate its business on hardware rather owned by Google than possessed by Amazon (Donavan, 2016), which has been similarly repeated by Amazon in the case of the platform Netflix for movie and tv-show streaming services ((Venugopal and Phartiyal, 2016), (Amazon Web Services, 2021)). In addition the instance of AWS abruptly and completely denying their client Parler to host their services (British Broadcasting Corporation, 2021) manifests the maximal potential damage to business success by relying on third party providers. Therefore one may consider the appropriateness of a cloud architecture being the property of the SME thereby implicitly minimising the exposure to unauthorised product or service replications and to unsolicited withdrawal of administrator rights at any given moment. As a result the absence of suitable SME solutions in order to perform BDA has been illuminated in this paragraph.

The ethnic culture is impacting client recensions and exhibiting at least the same amount of influence in the field of service reviews as is true for product critiques (Nakayama and Wan, 2019). Furthermore when Nakayama and Wan (2019) were studying the cultural effects on perceptions and preferences within restaurant critiques originated on the one hand by Western customers and on the other hand by Japanese diners, the scholars were able to measure that while the foods' constitution was the major topic of interest comprising an elevated degree of positivity as well as of complaints for both cultures, the remaining restaurant characteristics were prioritised in a dissimilar manner. Because of the strong correlation between cultural like-minded individuals to their physical locational proximity, it is more feasible in some situations to analyse defined territories like it happened in the example of IKEA (a global furniture company) for which Li and Fleyeh (2018) were assessing the level of happiness by locals for four cities as positive. Alternatively in the instance of the published descriptive visitor data concerning Las Vegas by Tourism Economics (2020, quoted in Las Vegas Convention and Visitors Authority, 2021, p. 1) demonstrating an enormous variety in the nationality of the hosted international tourists between the years 2010 and 2019, Statista Research Department (2016b) put this first impression into perspective by amending that the extent of domestic travellers in the

timespan from 2006 to 2015 accounted for at least 80% of the total volume, while the corresponding numbers for foreign visitors were ranging between 12 to 20 percent. Therefore it might be assumed that if such a prominent tourist hotspot like Las Vegas hosts predominantly domestic travellers, that these statistics embody a representative profile for a spectrum of additional travel destinations where one would expect a similar distribution. Moreover it seems rational from the perspective of SMSH located in Las Vegas to focus first on analysing the customer preferences of the majority travelling to Las Vegas, which makes a domain-specific SMA approach, confined to within the regional borders indispensable before proceeding to the second step of evaluating foreigners' travel desires. In particular the facts presented by Tourism Economics (2020, quoted in Las Vegas Convention and Visitors Authority, 2021, p. 1) suggest a small variation in non-North-American tourist demand within the most recently monitored eight years until 2019, so that hoteliers may not fear a drastic trend change which is additionally supported by the data collected by Statista Research Department (2016b), which depict moderate deviations.

Emotions directed at identical topics vary to a significant degree across the numerous sectors of an economy (Hu et al., 2017). In addition Tanantong, Sanglerdsinlapachai and Donkhampai (2020) assert that the performance of a general sentiment lexicon is inferior compared to a well-trained industry-specific sentiment dictionary in terms of accuracy and F1-score, while asserting that the most notable classification gains are realised in the rubrics of positive and negative emotions respectively. Of equal relevance is the written statement by Hu et al. (2017), who claim that SM posts are mirroring to an overwhelming level the honest feeling perceived by their corresponding authors, which is supplemented by declaring that especially clients share their feedback publicly via SM networks (Farizah Ibrahim and Wang, 2019). As a consequence Geetha, Singha and Sinha (2017) demand as a form of self-criticism concerning their own research that the particular machine learning algorithm Naive Bayes, which was executed by the scientists utilising a dictionary consisting out of non-neutral words as input, may be exposed to certain words occurring in reviews belonging to the hotel domain in order to further tweak the algorithm. Thereby Geetha, Singha and Sinha (2017) argued in a similar fashion to Kirilenko et al. (2018), who insisted more generally that machine learning techniques as well as methods relying on a dictionary may be optimised through training them on data stemming from the identical topical domain as the anticipated thematic background of the test dataset. Based on the culminated amount of arguments described above, the presence of research insufficiency for a hotel-contextual dictionary approach utilising Naive Bayes was determined.

In order to select a suitable SM network for proceeding with further analyses a comparative study concerning the platform properties by Blagus and Zitnik (2018) is inspected. These researchers proclaim that Twitter comprises the highest level of negative opinions in comparison to Facebook, Google+, Tumblr and YouTube. However Blagus and Zitnik (2018) added that they were able to ascribe just a minority of about 24% of the posts, which supports the findings by Farizah Ibrahim and Wang (2019), who were capable to only classify 17% into positive or negative sentiment categories. In sharp contrast supplemental research by Liu, Burns and Hou (2017) unveiled contradicting evidence, which specifies that circa two-thirds of Twitter texts could be related to a corresponding emotional value and nearly 50% of the 1.7 million Twitter posts were assigned the status of negativity, while only 16.9% of the data were connoted with positive sentiment. Although out of the 26.6% of Twitter users who could be categorised by gender, a majority of male users was determined (Blagus and Zitnik, 2018), a broader study by Ofcom (2017) painted a contrary picture, by affirming that 78% of the female and 73% of the male population within the UK are making use of SM. However Singh and Verma (2020) articulated that tweets' author metadata is negligible for SMA, whereas of crucial relevance is the analysis of their texts, while Molinillo et al. (2019) ascertain that the communication with the lowest rate of occurrence in SM networks are pro-active interactions like comments, whereas likes or shares are happening more often. In the end Blagus and Zitnik (2018) acknowledged that Twitter is the SM network among the previously mentioned five, which exhibits the most persuasive influences on its specific community.

3.) Methodology

The entire proposed procedure (as illustrated in Figure 6) is divided into two sections (Phase 1 and Phase 2) and is explained in the following.

3.1.) Development and validation of a sentiment lexicon (Phase 1)

3.1.1.) The suitability of Yelp data for the training and proxy test phase

At the start of data gathering two crucial elements, namely the data characteristics and the custom APIs (Application Programming Interfaces) provided by social networks must be considered in order to perform SMA in an effective style and to maximise the return of invested resources. Therefore first the Yelp dataset is determined as suitable for the production procedure of the sentiment lexicon (as pictured in Figure 4) due to the data properties of exhibiting guests' hotel reviews, which are originated and annotated with sentiment ratings by the customers themselves. In contrast tweets (Twitter posts) are lacking the latter feature, thereby requiring the scholar(s) to assign sentiment to texts either by hand, therewith implying a notable amount of working hours or a significant budget expense for the delegation of this task, which is still likely to convey biased judgements i.e. misclassifications whereas on the other hand one may implement a partially or fully automated approach. Furthermore the researchers Li, Li and Jin (2020) and Li and Fleyeh (2018) carried out a computer-driven method for estimating SM users' opinions depending on the emoticons found within their particular review texts, which is appropriate if no labelled sentiment data is available, however if customer emitted sentiment information is accessible, the proposed procedure by the intellectuals is at the maximum equivalent to clients' perspective although extremely probable to deliver a sub-optimal sentiment classification performance. In sharp contrast the second essential aspect outlined above presents the reason why the Yelp dataset is not selected for the execution of Phase 2 (as demonstrated in Figure 6). To illustrate the Yelp Fusion API releases three critiques upon request (Yelp, 2021), whereas Twitter API permits per request to extract 100 tweets in its Sandbox configuration and 500 tweets if one is willing to pay for the premium edition of its API (Twitter, 2021b). In addition an option within Twitter's premium product portfolio enables the ability to search its total repository of past records (Twitter, 2021a). This represents a precious feature for obtaining hotel critiques published before the commencement of the Covid-19 pandemic in the year 2020, which furthermore evoked unparalleled government regulations impacting the hotel industry significantly thereby rendering the reviews from the year 2020 until the containment of this virus as unrepresentative.

The specific question arises in which programming language the SMA implementation is to be coded. According to Gallagher and Trendafilov (2018) the examined programming languages R and Python are both serviceable in terms of scrutinising the extensive volumes of input data, whereas due to SMEs possessing relatively limited technological knowledge, the recommendations by Gallagher and Trendafilov (2018) are taken as guidance to decide that Python is more beneficial than R, because its stated attributes of reduced complexity and user friendliness. Therefore the succeeding Python libraries are required to be installed and imported in order to perform Phase 1: Pandas, matplotlib, seaborn, math, re, nltk and sklearn.

3.1.2.) Data Preparation for Yelp critiques

At the start of Phase 1 after all of the Yelp data has been downloaded, first a complete list of business IDs concerning the operating hotels located within the Las Vegas city borders is to be identified and permanently saved in order to filter out in a second step reviews, which are not being associated with these domain-specific IDs. Furthermore the remaining critiques of interest are converted into a csv file and securely stored on a hard drive. In the succeeding stage within Phase 1 the data preparation is conducted as portrayed in Figure 2. Therefore reviews containing Asian characters are sorted out and the sampling is conducted according to the definition by Etikan and Bala (2017) leading to the training and test set reflecting the overall sentiment distribution of the entire hotel review data for Las Vegas, which is designated by Barragán-Landy et al. (2020) as a representative

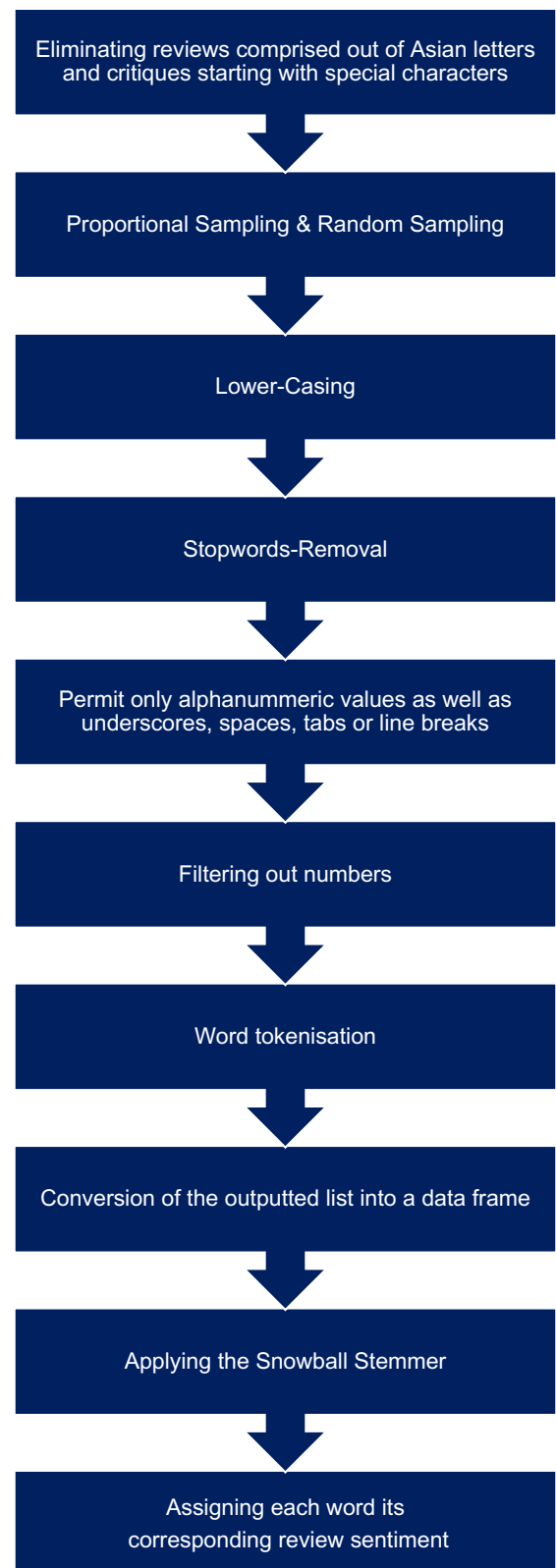


Figure 2:
The Phase 1
Data Preparation Process

sample. Moreover while proportional sampling has been performed in terms of sentiment ratings, random sampling is applied in addition so that the specific reviews determined to be in a particular sample category are selected randomly under the prerequisite of a matching congruent sentiment between the critique and the sentiment class. These actions have been repeatedly practised for training total sample size ratios of 10%, 25%, 50%, 75% as well as 90% for 20 000 reviews and ratios of 10%, 25%, 50%, 75% as well as 99% for 200 000 critiques. Sticking to the procedure outlined in Figure 2 the addressed tasks are executed in a sequential order. While the elimination of stop words, numbers and special characters was exercised due to observing identical preprocessing stages applied in the experiment by Rustam et al. (2019) on SM data, lower casing and stemming have been implemented due to their evidenced capability of positively effecting classifier accuracy (Resyanto, Sibaroni and Romadhony, 2019). The residual tasks of word tokenisation and the conversion of the outputted list by the prior stage are employed due to them accomplishing the auxiliary function of transforming input data into an adequate format for the Snowball stemmer. Last but not least the examined words are individually assigned the sentiment rating of their respective review for the training and the proxy test set. As a result the training and proxy test set have been set up to a sufficient level, which is illustrated in Figure 4 as the fourth point of Phase 1.

3.1.3.) Computing sentiment points via Naive Bayes probabilities

The following calculations of the frequency and the likelihood table are indispensable for the computation of the particular Naive Bayes sentiment score based on the specific conditional probability (i.e. posterior probability - $P(s|w)$) that indicates if an individual word is given, a certain expressed sentiment. In fact the aforementioned stages comprise the counting of words being assigned to sentiment contexts, therewith one is able to aggregate the counts in order to gain an enhanced view on the sentiment distribution of particular words within the training set and these sections of Phase 1 determine as well the respective word likelihood (i.e. predictor prior probability - $P(w)$), the specific probability of a sentiment class (i.e. class prior probability - $P(s)$), the individual probability that given a sentiment class a word is occurring (i.e. likelihood - $P(w|s)$) and last but not least the referred to posterior probability for which the Naive Bayes formula is depicted in Figure 3.

$$P(s|w) = \frac{P(w|s) \cdot P(s)}{P(w)}$$

Figure 3:

The Naive Bayes Formula

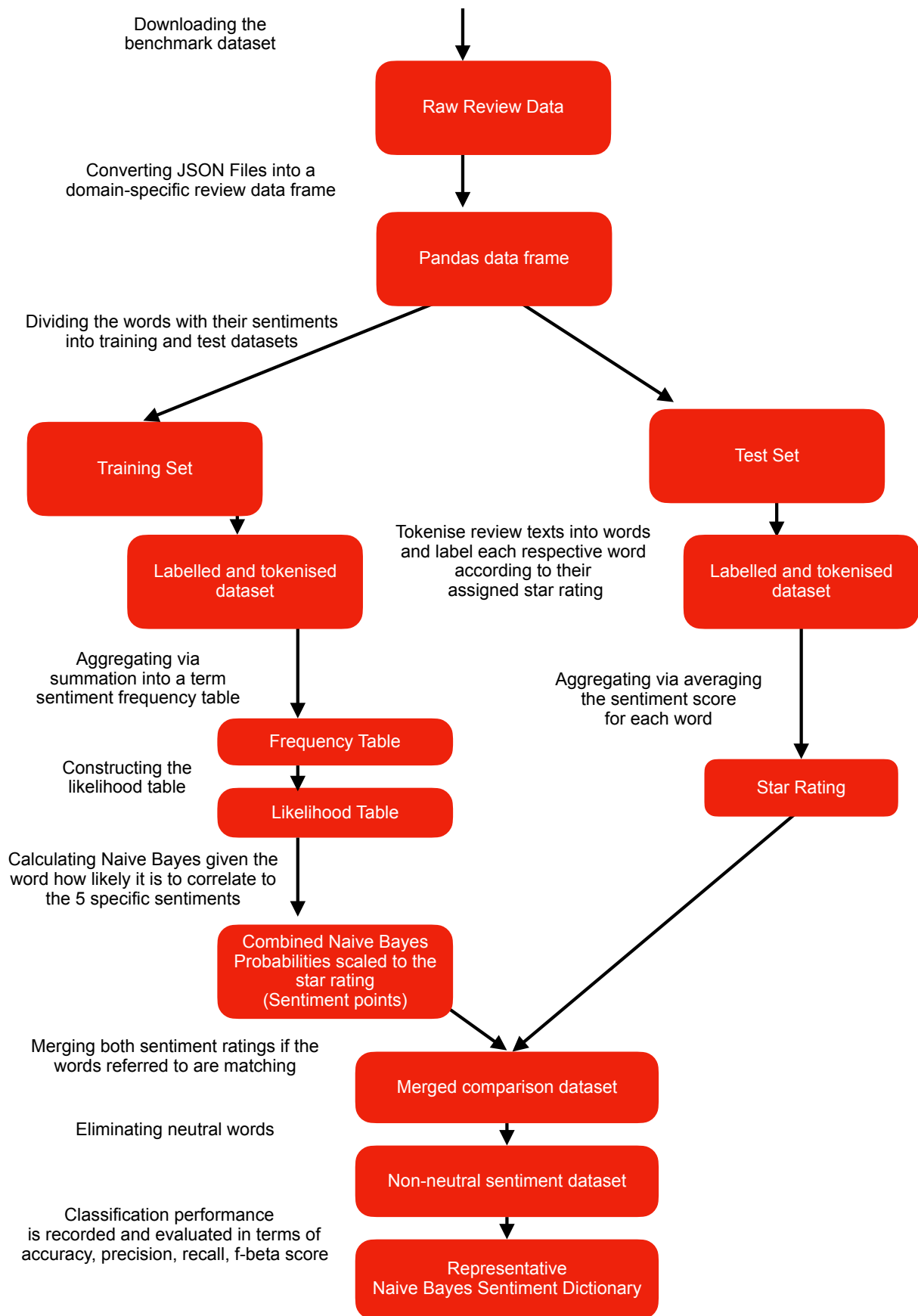


Figure 4:

Phase 1 - Design for the generation of a viable domain-specific sentiment dictionary

The rationale for the application of the Naive Bayes classifier is justified through the analysis of the academic literature. The methods logistic regression and Naive Bayes were outperforming the technique of decision trees (Elzayady, Badran and Salama, 2018), while it was highlighted in an alternative study by Kunal et al. (2018) that despite a necessary adaptation of the stop word vocabulary depending on the studied language, Naives Bayes is capable to operate irrespective to the examined review language as well as to the critique's thematic orientation while Naive Bayes demonstrates also according to the scientists a superior precision when set into comparison to the decision tree classification. Of equal if not greater relevance are the findings by Kirilenko et al. (2018), who discovered that if the training and testing data are collected from the same topical domain Naive Bayes like support vector machines (SVM) are especially suitable for categorisation endeavours. In addition Kritiyanti et al. (2018) brought to light in their research concerning politicians' popularity in front of an election that SVM is not a match for Naive Bayes when just a limited amount of data is available.

In a separate twitter-driven evaluation of emotions it was emphasised that Naive Bayes and SVM outclass the techniques of decision trees, logistic regression, K-nearest neighbours (KNN) as well as the lexicon approach SentiStrength (Krouska, Troussas and Virvou, 2017). Putting the recent result into perspective sentiment lexicons may still be viable if no integration of machine learning classifiers is affordable due to a strict budget or time scheduling constraints (Kirilenko et al., 2018). Despite being not contrasted against Naive Bayes, Mukhtar, Abid Khan and Chiragh (2018) revealed that their dictionary approach is on the hand surpassing KNN, SVM and decision trees in the attribution of emotions while on the other hand running under a fraction of the expenses required by the remaining algorithms respectively. Furthermore Naive Bayes was appreciated due to its ease of implementation as well as its effectiveness (Li, Li and Jin, 2020) and moreover a Naive Bayes sentiment dictionary attained top grades compared to its peer techniques (N-gram Naive Bayes, SVM and convolutional neural networks) particularly for the classification of complaints measured in the metric dimensions of accuracy, F1-score and precision, while ranking at second place in terms of recall according to Li, Li and Jin (2020). Although it was determined that deep learning techniques are more applicable for the job of categorising opinions than Naive Bayes and residual machine learning methods (Jain and Kaushal, 2018), Jain and Kaushal (2018) acknowledged the niche for machine learning algorithms in the referred field by stating that if there are just minor performance differences between the classes of approaches than a machine learning classifier might be preferred due to its relative cost advantage. Besides Gaussian Naive Bayes was discarded

as an option due to the poor accuracy results in comparison to distinct machine learning procedures as outlined in the experiment by Rustam et al. (2019). Also Kirilenko et al. (2018) reported that sentiment-dictionary-based and machine learning classification are of the same capability to human readers on data that excludes noise, however according to Kirilenko et al. (2018) when the dataset includes more disturbances the inorganic classifiers underperform human beings in terms of accuracy, although one must consider that the computational knowledge of the algorithms was strictly limited to the input data in contrast to the human judges, which may have used their expertise from past experiences. Proceeding in the process pictured in Figure 4, the sentiment points are created for each linked unique word by first multiplying the Naive Bayes probabilities for the five sentiment classes of one word respectively with the corresponding values of Yelp's emotion categorisation scheme ranging from one to five and second the five resulting scores are aggregated via summation. In contrast the star rating per word of the proxy test set is reached through averaging. Furthermore both scores of the training and testing set are rounded in order to fit in the categorical sentiment groups used by Yelp. Next the datasets are merged on the unique word representing the unique identifier for the purpose of evaluating the emotional classes for identical words. In the succeeding step the elimination of words which are assigned an overall neutral value in the training or proxy test set, is conducted due to these words not containing any beneficial emotional information, whereas words which possess extreme non-neutral properties may be weighted through their frequent occurrence in neutral review texts as slightly negative or moderately positive.

3.1.4.) Measuring the categorisation reliability of a sentiment lexicon

A representative sentiment dictionary is finally approved for further research if the classification performance either of the four remaining opinion classes (1, 2, 4 or 5) or the categorisation results of the classes positive (i.e. the sentiment categories: 4 + 5) and negative (i.e. the sentiment categories: 1 + 2) achieves an acceptable level of accuracy, recall, precision and F-beta score. Additionally all Naive Bayes sentiment lexicons will be permanently stored and published. To clarify the F-beta score ($\beta = 0.5$) has been chosen as more suitable to assess classifier performance than the F1 score ($\beta = 1.0$), because the F-beta score is adjustable in terms of prioritising precision or recall, whereas the F1-score provides an equal weight to both metrics. To illustrate further classifier precision is seen as more relevant in comparison to the recall dimension, which is explained by the following worst case scenarios. If a classifier exhibits excellent precision, but suboptimal recall it has the consequence in this research project that for instance actual negative words are

ignored, while if the algorithm indicates a negative sentiment of an expression it is to an enormous degree correct. In contrast for a model featuring low precision, but superb recall characteristics the severe ramification is for example that non-negative words are wrongly assigned into the group of negative sentiment, thereby a user will not be able to distinguish between the actual negative words and the falsely labelled ones.

3.2.) The opportunity algorithm applied on domain-specific reviews (Phase 2)

In order to run Phase 2 the succeeding libraries are mandatory to be installed and called: Pandas, tweepy, re, nltk, gensim, matplotlib, seaborn, dash, plotly, dash_core_components, dash_html_components and dash_table.

3.2.1.) Data Collection of Tweets

The first point on the agenda of Phase 2 is the tweet collection concerning the general topics bar, entertainment and wedding (as depicted in Figure 6). Therefore 500 english tweets were acquired for each of the aforementioned themes per quarter of the year 2019. Furthermore the first group of search terms included “bartender”, “bartending”, “cocktail” and “cocktails”, whereas the applied keywords for the second selection were “Event” as well as “Show”, while the last category utilised the expressions “marriage”, “wedding”, “married”, “bride” and “marry” in order to identify tweets from the certain topics bar, entertainment and wedding respectively. In addition the rationale behind exempting tweets, which incorporated certain topic-specific words like for instance “TV” for entertainment, topic-diluting words like “recruiting” or which were composed by SM influencers, was first to target

in the case of the theme entertainment, services being directly adjustable by the hotels and second to increase the chances of obtaining actual customer reviews instead of for example marketing material. Also after the IDs, texts and screen names have been gathered and linked correspondingly to each other, the screen names were inspected for clarifying that no tweets were authored by hotels themselves.

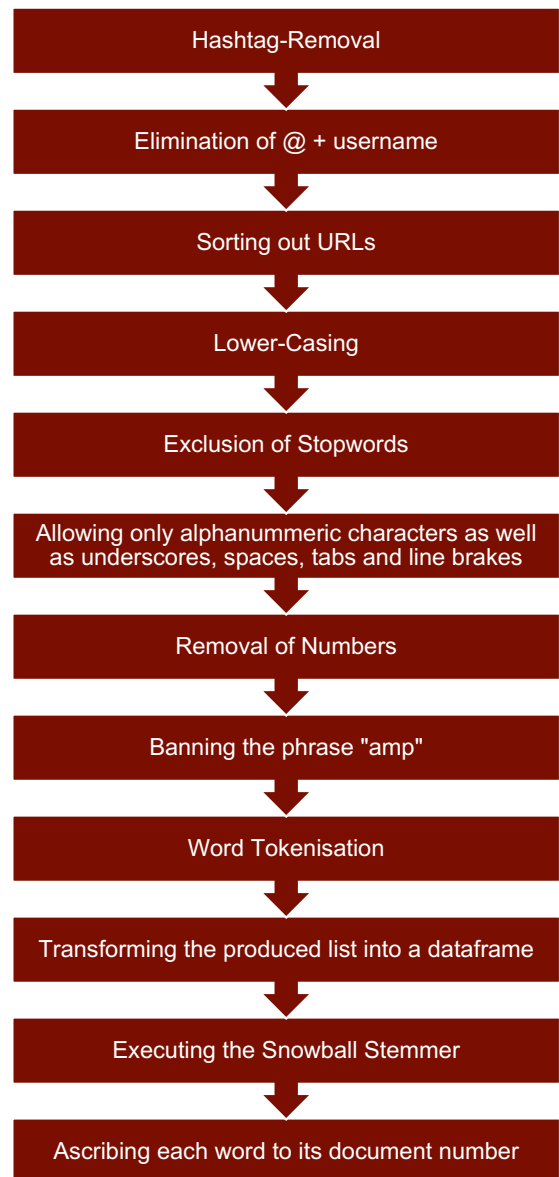


Figure 5:
The Phase 2
Data Preparation Process

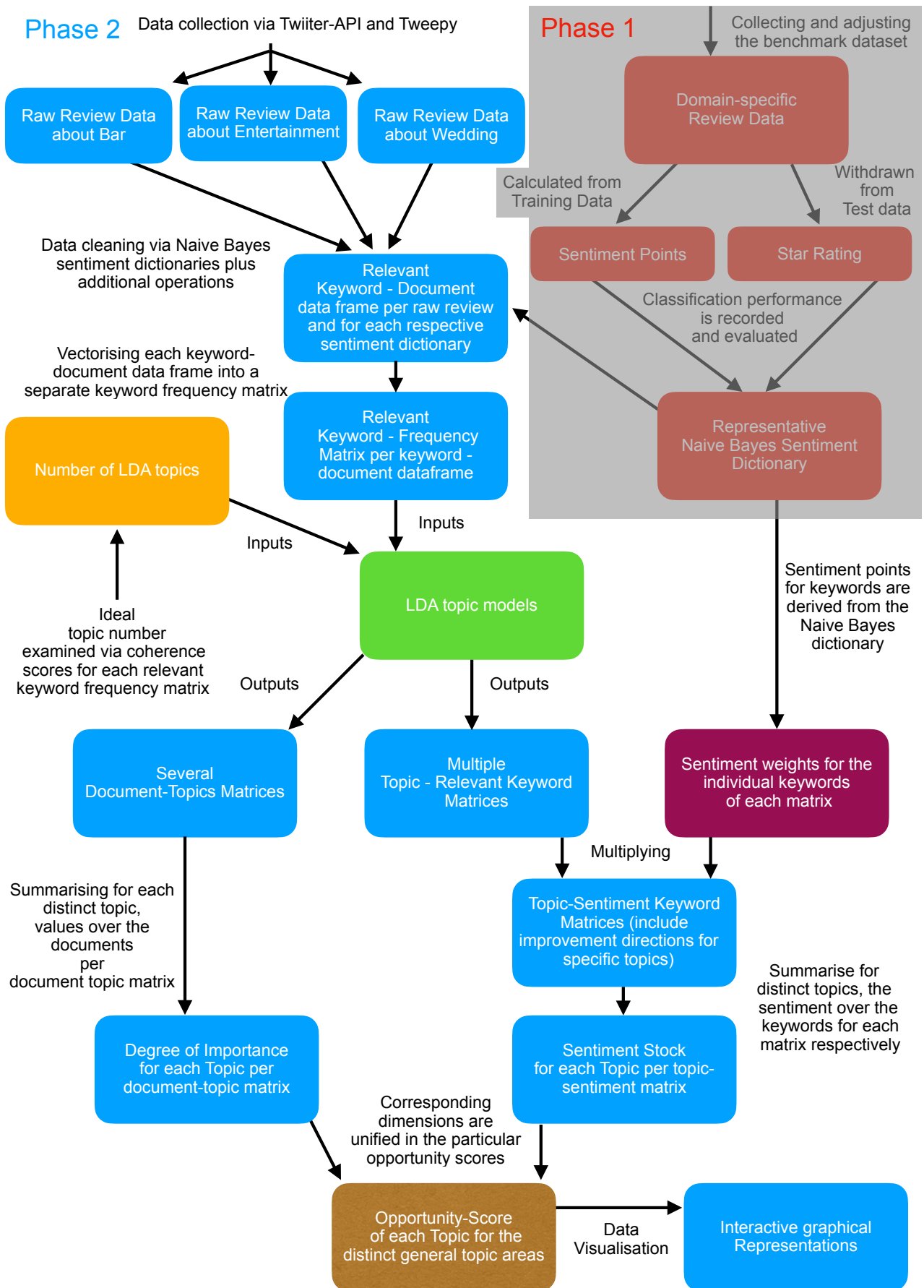


Figure 6:
The complete opportunity system (i.e. Phase 1 + Phase 2)

3.2.2.) Data Preparation of Tweets

Subsequently these twitter data has been cleaned and modified step by step as is demonstrated in Figure 5 for the refinement of the input data and in order to insure the comparability between the tweets' words and the vocabulary of the particular sentiment dictionary. To implement the comparability standards was crucial, because the pool of words which does not match to the one of the sentiment lexicon is discarded due to its unknown emotional values thereby emphasising once more the importance of the sentiment dictionary being representative. Through these actions the keyword document data frame was produced and furthermore will be vectorised in the next distinct step according to the process flow of Figure 6. Then a dictionary of unique words is assembled as well as the input text has to be presented in a list of lists format so that one may proceed with the determination of the ideal number of topics and with the selection of the most appropriate topic model.

3.2.3.) LDA Topic Modelling

The machine learning approach LDA (Latent Dirichlet allocation) topic modelling was adopted due to its capability to scrutinise texts of extensive quantities (Liu, Burns and Hou, 2017). Despite the naming process for the particular topic groups, which is performed by the researchers Jeong, Yoon and Lee (2019) as well, being subject to the human judgement of the specific evaluator, Farizah Ibrahim and Wang (2019) brought to light two novel themes, which were priorly considered to be latent by applying LDA. Of equal significance is that the academics Vayansky and Kumar (2020) are the originators of a roadmap, which is aimed at supporting the selection process of topic models. Therefore the decision is taken to execute LDA keyword aggregation based on the scholars' recommendation to choose this kind of topic modelling technique if the conditions of first text documents comprising less than a number of 50 words measured by their arithmetic mean, of second these documents exhibiting various content-wise associations and lastly of the mentioned records featuring available details about keywords are valid.

When attempting to deduce the ideal amount of LDA topics Chang et al. (2009) argue that one should rather concentrate more on realistic work environments than to direct his or her attention towards the more theoretical metric of perplexity. Furthermore Mifrah and Habib Benlahmar (2020) advocate for the execution of LDA combined with the topic coherence criteria especially in the human end-user context, while Röder, Both and Hinneburg (2015) contributed that the most similar topic coherence type to human evaluations is called the Cv measure. Moreover Mifrah and Habib Benlahmar (2020) explain that this particular

variant is depending on a sliding window, the separation of the most probable words and on an abstract validation mechanism, which itself is relying on cosine similarity and normalised pointwise mutual information (NPMI). In addition Röder, Both and Hinneburg (2015) suggest about their invented indicator that a more enhanced performance is obtained when examining the top ten words instead of the top five words and the researchers recommend as well to set the window size to at least 50, of which the latter requirement is fulfilled by the gensim Cv coherence models via setting per default this parameter to 110 (Rehurek, 2021). Although the code, which is spotting coherence scores for a range of LDA topic models is in essence written by Prabhakaran (2018) and therefore major credit should be assigned to him, small alterations have been implemented by the author of this master dissertation so that first the method generates the perplexity value corresponding to the respective model over the configured range of topics, secondly features the characteristic of being reproducible and thirdly the parameter of passes is updated to 20 in order to enhance topic model performance.

3.2.4.) Calculation of opportunity scores and their illustration

Once the LDA topic modelling is finished, one document topic matrix and one topic words matrix are emitted for each combination of the particular sentiment dictionary and the specific raw review dataset as indicated in Figure 6. Then the document-topic matrix is summarised via summing up the topic occurrence rates over all tweets, whereas in step one a multiplication between the topic-keyword matrix and the matrix containing the emotional dictionary values for each of the unique words from the twitter dataset is conducted. Furthermore in step two the sentiment towards various topics is examined by adding the individual keyword scores of a particular topic respectively. Next the subtopics are assigned with their names, which are deduced by associating the top ten most likely words with each other and in certain cases in which this method is not outputting sufficiently detailed categories the theme titles are inferred by semantically connecting the top indicative words of the top ten words with the specific keywords implemented for the purpose of searching Twitter. Subsequently the emotional connoted topic-keyword matrix and the document-topics matrix will be normalised leading to them ranging in a spectrum between zero and ten thereby indicating that if one feature attains the score of ten the topmost importance or satisfaction is attained respectively, whereas a score of zero stands for the lowest degree of topic importance or negativity towards a topic. Now the possibility to calculate the opportunity score is used in order to prioritise topics of elevated importance and inferior satisfaction magnitude, for which the topic-sentiment-keyword

matrix specifies what aspect of a service is the most complained one thereby advising that if this particular service element is improved a benefit for the customer's as well as for the hotelier's wealth will be the result. Ultimately non-service topics are omitted manually first by eliminating external and independent events, second by distinguishing between service and product concerned themes and third by differentiating between services themselves, thus the services are included which may be realistically offered when considering the existing capabilities and expertise of hotels thereby neglecting services like for instance offering legal advice due to the unfamiliar character of this domain to the hospitality industry.

The last element in the opportunity system represents the stage of data visualisation. It is essential due to the demand by SMSH to reduce complexity in order to extract in a quick and effective manner which services are regarded as the most promising opportunities and what are vital steps of advancement to exploit these opportunities. Therefore the Python Dash package is assessed as suitable for end-users, because of its ease of use (Burch et al., 2019), whereas the libraries matplotlib and seaborn will be utilised for the purpose of generating more scientific graphics addressed to the IT experts employed by the respective hotels in order to monitor and evaluate the technical key points of the opportunity system's performance.

4.) Results & Discussion

The first outcome of this research concerned to identify the general review sentiment distribution for all hotels within Las Vegas, who are listed as being still in business. Therefore first Figure 7 was generated in order to provide an overview of the top ten cities, which feature open businesses in the Yelp dataset. By further inspecting Figure 7 one is able to notice, that Las Vegas is the city with the largest extent of registered open businesses, whereas the 2nd most elevated place exhibits nearly 35% less of this characteristic. In addition when contrasted against the 4th highest ranked city for this criteria, Las Vegas demonstrates almost two times more open business records.

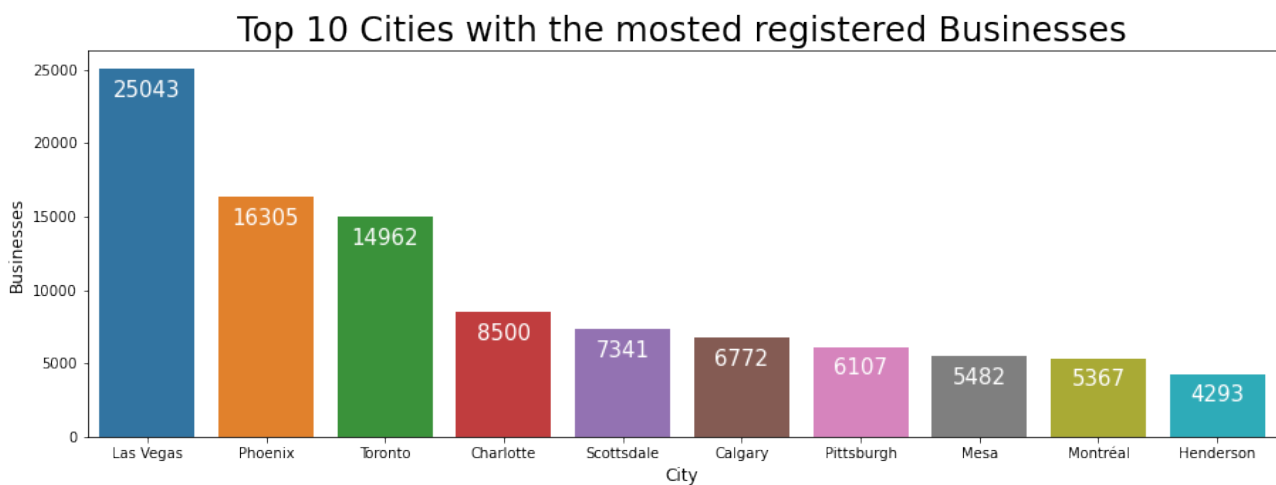


Figure 7:

The allocation of open businesses across the cities of the Yelp dataset

Secondly more sentiment information addressing specifically the hotel industry in Las Vegas was computed from critiques as shown in Figure 8. Also the reader may observe in this illustration that the total sentiment distribution is more shifted towards positive opinions than to negative emotions. In fact the slightly and strongly positive ratings outnumber their respective negative counterparts, while neutral sentiment remains relatively low.

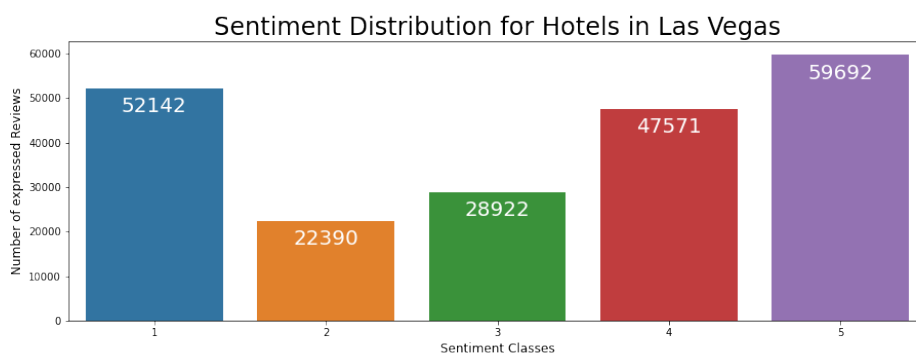


Figure 8:

The review sentiment distribution for hotels located in Las Vegas

4.1.) Naive Bayes models' performance on sentiment word classification

In the following the categorisation scores of the various Naive Bayes dictionaries trained on domain-specific reviews are outlined according to the examined levels of sentiment. In addition entire column headers will be composed out of review amount, scale of the training set divided by the total of training and test set, F-beta score, precision, recall, accuracy and last but not least the overlap between words occurring in the training as well as in the proxy test set thereby determining the sentiment dictionaries' vocabulary.

Classifier performance on strongly positive sentiment

The table concerning strongly positive sentiment (depicted in Figure 9) outlines that there is a positive correlation between the training sets' percentage of the overall dataset to the F-beta score regardless of the inputted review size except for the cases where the training set is assigned a proportion of 90% and 99% respectively. While a relative increase in the extent of the training data set also corresponds to a raise in the degree of precision in general, an outlier of this trend is materialising for the instance of 20 000 articles and the training set portion being set to 90%. In sharp contrast it seems to be true when excluding the data point produced for 20 000 reviews and a training data segment of 25% that the more the input data for the algorithm is expanded, the more the quality of the recall metric suffers. The accuracy appears to be uncorrelated to the relative magnitude of the training set, to the quantity of critiques and to the area of intersection, although the volatility of the recorded accuracy stays within moderate thresholds (+/- 10%). In addition the mutual vocabulary size progresses in an inverted u-shaped manner when at the same time the proportion of the training data is soaring, however for the review size of 20 000 this tendency is nearly symmetrical, whereas the path exercised by the 200 000 recensions is asymmetrical due to experimenting with a training set ratio of 99% instead of a 90% one.

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
20 000	10%	0.177	0.155	0.417	0.507	3 613
20 000	25%	0.272	0.249	0.431	0.529	5 080
20 000	50%	0.320	0.323	0.309	0.556	5 816
20 000	75%	0.378	0.433	0.251	0.545	5 177
20 000	90%	0.298	0.368	0.169	0.521	3 417
200 000	10%	0.229	0.203	0.468	0.541	11 686

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
200 000	25%	0.289	0.270	0.407	0.551	17 468
200 000	50%	0.342	0.342	0.341	0.544	20 418
200 000	75%	0.346	0.369	0.277	0.549	17 228
200 000	99%	0.302	0.570	0.105	0.541	3 580

Figure 9:
Categorisation quality on strongly positive sentiment

Classifier performance on slightly positive sentiment

The categorisation capability of slightly positive emotions by multiple selected sentiment dictionaries is pictured in Figure 10. Controversially this table is expressing that as the training set is playing a more extensive part in percentage terms, the less the F-beta score will be due to its continuously falling component, namely the precision, whereas the recall values are being boosted concurrently during this procedure. To clarify the datasets consisting out of 200 000 reviews are generally displaying more desirable F-beta scores than the ones entailing 20 000 reviews, which is not true for the individual comparison between the 20 000 review dataset and the 200 000 critiques entry in Figure 10, which are both utilising 50% of the entire data for training. Of equal importance are the identical accuracies and vocabulary sizes as presented in Figure 9 for which the same description previously articulated is applicable for Figure 10, Figure 11, Figure 12.

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
20 000	10%	0.697	0.746	0.552	0.507	3 613
20 000	25%	0.671	0.700	0.578	0.529	5 080
20 000	50%	0.665	0.665	0.665	0.556	5 816
20 000	75%	0.621	0.602	0.715	0.545	5 177
20 000	90%	0.578	0.547	0.743	0.521	3 417
200 000	10%	0.727	0.782	0.568	0.541	11 686
200 000	25%	0.698	0.722	0.617	0.551	17 468
200 000	50%	0.658	0.659	0.654	0.544	20 418
200 000	75%	0.650	0.639	0.702	0.549	172 28

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
200 000	99%	0.604	0.565	0.833	0.541	3580

Figure 10:
Categorisation quality on slightly positive sentiment

Classifier performance on slightly negative sentiment

When scrutinising the slightly negative data shown in Figure 11, the similarity to the F-beta score's course to the one portrayed in Figure 10 is undeniable, so that while the training portion rises the F-beta score declines, although the difference to Figure 10 is that the more preferable values are arising from the datasets featuring the total volume of 20 000 critiques. Likewise Figure 11 presents evidence that as an increased percentage of data is assigned to the training set, the precision characteristic is shrinking, whereas recall is strengthening. On the one hand recall is attaining higher evaluations for the 200 000 review datasets than for the 20 000 article ones except for the instances of the training set portion being 25 or 50 percent, whereas on the other hand the precision scores are greater for the 20 000 than for the 200 000 critiques dataset.

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
20 000	10%	0.650	0.716	0.475	0.507	3 613
20 000	25%	0.634	0.666	0.533	0.529	5 080
20 000	50%	0.619	0.625	0.596	0.556	5 816
20 000	75%	0.577	0.558	0.669	0.545	5 177
20 000	90%	0.551	0.523	0.701	0.521	3 417
200 000	10%	0.646	0.689	0.519	0.541	11 686
200 000	25%	0.608	0.629	0.536	0.551	17 468
200 000	50%	0.561	0.559	0.571	0.544	20 418
200 000	75%	0.550	0.531	0.641	0.549	17 228
200 000	99%	0.531	0.492	0.777	0.541	3580

Figure 11:
Categorisation quality on slightly negative sentiment

Classifier performance on strongly negative sentiment

The classification results concerning well-negatively connoted emotions are made transparent in Figure 12, which are furthermore similar to the ones produced in Figure 9. In fact as the training set is comprising a more relatively extensive amount of the total data, the F-beta score is enhanced, except under the circumstance when utilising 99% of the 200 000 reviews dataset. Although that the precision levels of the records, which incorporate 200 000 critiques, are more superior than the ones including 20 000 recensions, it is delineated in Figure 12 that when the ratio of the training set to the entire dataset is enlarged, the precision quality is augmented. On the contrary the recall decimal numbers are decreasing concurrently as the portion of the training data is growing irrespective of the analysed dataset. As a consequence Figure 12 continues the trend commenced by Figure 9 for extreme positive sentiment, while Figure 10 and Figure 11 indicate the opposite characteristics for small extents of emotions.

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
20 000	10%	0.235	0.209	0.475	0.507	3 613
20 000	25%	0.281	0.260	0.415	0.529	5 080
20 000	50%	0.367	0.358	0.410	0.556	5 816
20 000	75%	0.383	0.409	0.304	0.545	5 177
20 000	90%	0.397	0.514	0.207	0.521	3 417
200 000	10%	0.290	0.259	0.544	0.541	11 686
200 000	25%	0.368	0.348	0.482	0.551	17 468
200 000	50%	0.404	0.406	0.400	0.544	20 418
200 000	75%	0.418	0.451	0.323	0.549	17 228
200 000	99%	0.397	0.650	0.155	0.541	3580

Figure 12:

Categorisation quality on strongly negative sentiment

Classifier performance on negative sentiment

The statistical documentation features in Figure 13 as well as in Figure 14 are recorded succeeding the modifications to group the classes slightly negative and strongly negative into the negative sentiment category as well as to combine both variants of the positive sentiment into the single genre of positivity. The most superior dataset in terms of F-beta

score, precision, recall and accuracy is the one which encompasses 200 000 reviews and allocates 99% of the entire data to the training stage. Equally of relevance is that when focusing only on datasets exhibiting 20 000 reviews that the second best F-beta value as well as the precision score and the greatest accuracy is attained by a model which is utilising just 10% of the overall dataset as input data. Despite being outclassed in the areas of F-beta score and precision concerning classifiers, which are learning from 20 000 recensions, by the dataset which commits 90% of the records for training, these minor performance differences may be seen as trade-off for being able to conduct sentiment analysis by excluding 10% of the total dataset instead of omitting 90%. Although the vocabulary size is the identical one as in Figure 9, Figure 10, Figure 11, Figure 12 as well as Figure 14, the reader may notice a significant enhancement in general accuracy (as demonstrated in Figure 13 or Figure 14) in contrast to the previously examined sentiment classifications, which differentiated to a higher granularity the classes of positivity and negativity.

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
20 000	10%	0.765	0.769	0.746	0.787	3 613
20 000	25%	0.756	0.758	0.749	0.777	5 080
20 000	50%	0.744	0.742	0.753	0.774	5 816
20 000	75%	0.749	0.746	0.763	0.782	5 177
20 000	90%	0.766	0.777	0.728	0.774	3 417
200 000	10%	0.749	0.742	0.776	0.795	11 686
200 000	25%	0.723	0.722	0.728	0.771	17 468
200 000	50%	0.712	0.711	0.717	0.761	20 418
200 000	75%	0.730	0.730	0.733	0.773	17 228
200 000	99%	0.803	0.805	0.794	0.829	3580

Figure 13:

Categorisation quality on negative sentiment

Classifier performance on positive sentiment

The last categorisation performance table which was taken into account before selecting the most promising sentiment dictionaries is illustrated in Figure 14. Therefore by scrutinising the qualities in distinguishing between positive emotions and negative voices,

the resulting most appropriate classifier in terms of F-beta score, precision, recall as well as accuracy was identified to be the one, which were provided 99% of the 200 000 critiques in order to train. Equally when concentrating on classifiers which received 20 000 reviews, the one being trained on 75% of the entire dataset delivers the most preferable F-beta and precision scores, whereas similar to the instance of Figure 13, the second most suitable approach in the fields of F-beta score as well as precision is the one trained on 10% of the review data, whereas this latter technique features the most attractive properties in terms of recall and accuracy for evaluation tasks being given 20 000 sentiment connoted articles. Lastly Figure 14 and Figure 13 are not presenting any further implications or trends like they are observed from Figure 9 to Figure 12.

Number of reviews	Training set portion	F-beta score	Precision	Recall	Accuracy	Vocabulary intersection size
20 000	10%	0.805	0.801	0.820	0.787	3 613
20 000	25%	0.795	0.793	0.801	0.777	5 080
20 000	50%	0.799	0.801	0.791	0.774	5 816
20 000	75%	0.808	0.811	0.796	0.782	5 177
20 000	90%	0.780	0.772	0.815	0.774	3 417
200 000	10%	0.830	0.835	0.808	0.795	11 686
200 000	25%	0.805	0.806	0.801	0.771	17 468
200 000	50%	0.796	0.797	0.792	0.761	20 418
200 000	75%	0.805	0.805	0.803	0.773	17 228
200 000	99%	0.848	0.846	0.855	0.829	3580

Figure 14:

Categorisation quality on positive sentiment

4.1.1.) Selecting specific sentiment dictionaries for further proceedings

After all the notion is developed that generally the introduced sentiment classifiers operate to a more enhanced degree rather if the issue to be solved concerns the categorisation between positive and negative sentiment than in the case of differentiating between strong positive, weak positive, modest negative and intense negative feedback. Furthermore the F-beta top score attained for any of these both positive sentiment classes was 0.727 and the corresponding maximum value for the two specific negative emotional groups materialised as 0.650. Especially the most superior classifier in this study comprising 200

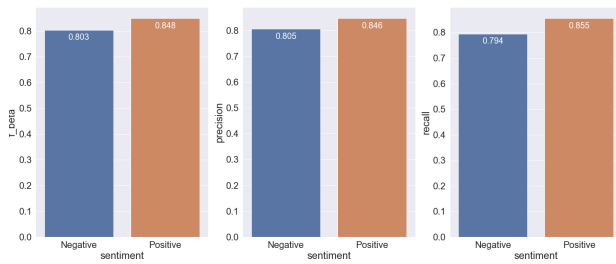


Figure 15:

Classification performance of
positive and negative sentiment
(200 000 reviews -
training set portion = 99%)

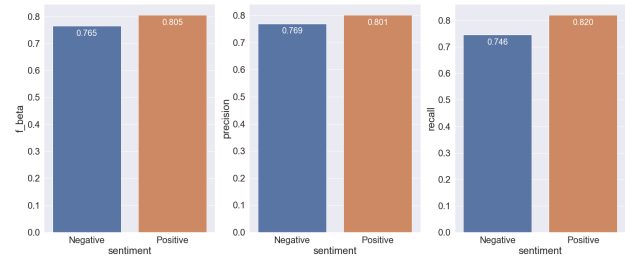


Figure 16:

Classification performance of
positive and negative sentiment
(20 000 reviews -
training set portion = 10%)

000 reviews and 99% training data accomplished a F-beta outcome of 0.848 on positively and 0.803 on negatively directed opinions as depicted in Figure 15. Thus this sentiment lexicon in its alternative form to the one achieving suboptimal results (illustrated in Figure 18 attached in the appendix) having classified only between positive and negative words is tremendously qualified for the succeeding research stages of Phase 2 as it represents the best in class approach and embodies the technique, which was most exposed to training data in terms of its absolute and relative amount. In addition confusion matrices of the two classification tasks are painted in Figure 21 as well as Figure 22 (both displayed in the appendix) for the aforementioned approach therefore demonstrating the respective outcomes in absolute numbers, which might be beneficial in order to comprehend the divergences between them.

When determining a reliable classifier executing under scarcer conditions than the previously mentioned one, the decision is not clear at first glance due to the most adequate modus operandi for correctly identifying positive sentiments encompasses 20 000 reviews and allocates 75% of its data resources to its learning stage, whereas in slight contrast the distinct algorithm, which is being trained on 90% of its 20 000 reviews, surpasses the remaining models in terms of F-beta score if the total dataset is limited to 20 000 recensions and the categorisation of negativity is tested. However a third dictionary involving emotional content is evaluated as appealing for classifying opinions when given 20 000 articles. Despite not attaining first place on the latest two referred to undertakings, the procedure incorporating 10% of the total dataset for the purpose of training delivers substantial results (as displayed in Figure 16) thereby coming in second place for positive as well as negative classification performance, respectively. Namely this individual sentiment lexicon saves enormous portions of data, which are ranging between 65% to

80% of all collected data in comparison to the prior techniques leading to a significant competitive advantage in two areas of data analysis due to the fact that while the bar for the amount of training data is lowered, the absolute portion of the real-world test dataset is at the same time enlarged. The second rationale for including this particular sentiment dictionary is revealed due to its relevance in practice. While in reality even the 200 000 review dataset might be of assistive character in order to scrutinise 2000 critiques (as depicted in Figure 18), a more crucial emphasis is put on the outcomes of an emotional lexicon, which is trained on a 2000 recension dataset and may classify 18 000 real-world customer feedbacks. Additionally despite being inferior in comparison the performance of the 20 000 review classifier, which is learning from a 10% training dataset is illustrated in Figure 17 (located in the appendix), thereby confirming the general observations for the categorisation problem regarding the four addressed sentiments so that when the Naive Bayes sentiment lexica are applied, they are more suitable for determining slightly positive or moderately negative emotions while lacking in the categorisation of extremely optimistic or immensely pessimistic opinions. Likewise the confusion matrices of the second selected sentiment dictionary may be viewed in detail for categorising the two (as indicated in Figure 19 within the appendix) or four sentiment groups (as revealed via Figure 20 within the appendix) respectively, therewith disclosing the exact numbers on which the classification performance metrics are based on. Last but not least the entire code for computing the two most promising approaches will be outlined in the appendix in text form plus the code of every examined technique will be available to download as an executable script as well as a Jupyter Notebook via the provided Dropbox link placed in the appendix section of this report.

4.2.) Characteristics of the examined topic models

4.2.1.) Comparison of the generated LDA topic models

Figure 23 comprises key facts supporting the evaluation of the particularly created six optimal LDA topic models. Therefore each of these generated topic models is listed according to the specific lexicon of emotions and the topical type of the individually cleaned tweet dataset. In addition although the ideal amount of topics is identical for the topic categories of entertainment and wedding respectively, the reader may notice that a divergence occurs in this aspect within the broader topic area of bar. Due to the fact that the most suitable number for LDA topic models is determined based on the corresponding global peak of its coherence score, the particular values (0.670, 0.669) represented in the previously mentioned table are both more favourable than if the topic model including the 20 000 review dataset is provided with the number 17 thereby attaining a score of about 0.650, whereas the topic model incorporating 200 000 critiques is accomplishing a rating of circa 0.640 if the topic number 22 is inserted. Next one may discover that the extend of service topics, which is partially influenced by human subjectivity, differs in the topic category cases of entertainment and wedding to a tolerable degree (~7% - ~18%), whereas when looked at this criterion for the theme spectrum concerning bar a significantly more elevated deviation of 40% is noticeable, which is more than twice to more than five times as much as the models earlier referred to.

In sharp contrast are the discrepancies in terms of coherence and perplexity scores. Surprisingly the performance gaps in perplexity and coherence between the two distinct models for each topic category are narrow for the general topic field of bar (coherence divergence: ~0.2% - perplexity divergence: ~3.2%), widen in the domain of entertainment (coherence divergence: ~2.1% - perplexity divergence: ~5.3%) and finish the most outspread (coherence divergence: ~7.9% - perplexity divergence: ~7.5%) in the thematic section of wedding. Of equal relevance is that when comparing the emitted outcomes by both sentiment lexica according to the prior communicated metrics, it becomes obvious that models integrating the 20 000 review-strong emotional dictionary, outperform the remaining approaches in their particular thematic spheres. In fact if the LDA models incorporating the 20 000 article dataset are contrasted against each other over different topic categorises the most superior result (0.682) is attained in the métier of wedding followed by the outcome of the thematic class entertainment (0.674) while the performance for the bar topic group is ranked at third place (0.670) in terms of coherence scores. On the other hand the same type of LDA model achieves the most preferable perplexity rating (-7.37) for the entertainment category, while it exhibits the second most desirable

perplexity value (-7.03) in the topic area of bar whereas the technique is rewarded with the relatively lowest perplexity grade (-7.02) in the topical field of wedding. Alternatively when a comparison is conducted between the topic models, which are partially based on the sentiment dictionary consisting out of 200 000 reviews, the most appropriate application areas in terms of coherence are first the topic domain bar (0.669), second the topic category entertainment (0.660) and third the thematic sector of wedding (0.628). Likewise when focusing on these specific groups of LDA approaches from the perspective of perplexity, they are prioritised in their suitability in the succeeding fashion: The best in class method is executed on entertainment tweets (-6.98), while a result of a mediocre acceptance level (-6.80) is produced for content comprising the broader spanning theme of bar whereas the topic category wedding is the least advantageous option (-6.49) of all scrutinised variants. Furthermore in order to demonstrate these fluctuating courses of coherence and perplexity for each topic model, which are themselves all partially featuring exactly one of the two selected sentiment dictionaries, the visualisations spanning from Figure 24 to Figure 29 are revealed in the appendix thereby providing additional specifications of these metrics in the range between two and fifty topics for each respective topic category.

Sentiment Dictionary	Topic Category	Optimal Number of Topics	Service Topics	Coherence Score	Perplexity Value
20 000 reviews - training set portion = 10%	Bar	22	10	0.670	-7.03
200 000 reviews - training set portion = 99%	Bar	17	6	0.669	-6.80
20 000 reviews - training set portion = 10%	Entertainment	24	14	0.674	-7.37
200 000 reviews - training set portion = 99%	Entertainment	24	15	0.660	-6.98
20 000 reviews - training set portion = 10%	Wedding	23	11	0.682	-7.02
200 000 reviews - training set portion = 99%	Wedding	23	9	0.628	-6.49

Figure 23:

Quality assessment table of the explored topic models

4.2.2.) Inference of topic titles

Interpreting the individual service topics on the foundation of the LDA outputs i.e. the most probable words for each unified dataset being composed out of the cleaned tweets concerning the specific topic category and the respective sentiment lexicon, which has been announced to be on the declared short list, entails vagueness to a certain extent due to ambivalences of topological grey areas and the researcher's inability of complete objectivity. This having been stated, the following section will answer why service topic names have been assigned in that style.

Commencing first with the approach featuring the element of a sentiment dictionary, trained on 10% of 20 000 hotel reviews for the thematic field of bar, the topmost entry has been labelled as "strip club" due to the prominent words of club and strip, which is rated as service topic (as portrayed in Figure 30 within the appendix) due to the activity of striping. Lounge nudity is perceived as a possible service venue for hotels within the domain of bar, which is evidenced by the expressions "lounge" and "underwear" while the relationship between monetary costs, emphasised through words like "dollar" or "pay" and cooking performance being highlighted via "yum" or the stemmed expression "delici" is additionally regarded as a service. Likewise the theme "magic in the library" consisting out of the exemplary vocabulary of "magic" as well as "librari" is evaluated as a complementary factor for attracting customers to a bar and is therefore treated as an existing service dimension, which is also applicable for the traditional waitressing service indicated in this certain analysis by the words "salad", "told" and "pizza" in conjunction to the bar theme. Besides the emitted topic of grilling identified by the most likely term "grill" was recognised to present a service aspect like it is true for the singing quality at the dining table, which comprised the frequently occurring vocables "pm", "dine" and "sing", as well as it is ascertained that the touristic facet of brokering golf and brewery tours inferred via the words "golf", "tour", "breweri" belongs to the service class. The last two deemed services are the roasting of pineapples (words of determination: "live", "roast", "pineappl") and the verbal communication by the staff (words of determination: "talk", "shop", "team").

On the other hand when examining the topic model based on the intersectional data between the previously integrated sentiment lexicon and the filtered entertainment tweets, the on-stage acting performance is assessed by the keywords "cast" and "live", while alternatively a singing and style contest could be discovered through the use of the terms "fashion", "battl" as well as "sing". Equally important is that the health of the viewers was distinguished by the expressions "organ", "hold" and "everything", whereas the vocables "movie", "flat" as well as "visit" were defining the sub-theme of visiting a movie apartment.

Interestingly the topic “car magic” was uncovered by its terms “magic” and “car” (as may be extracted from Figure 31 located in the appendix), which represents identical words in comparison to the earlier stated content about magic in a library. In addition this approach was able to group the vocables of “sale” and “sell” within an entertainment context, so that a sales event was assigned as a subtopic title. Likewise the subjects emergency call, dancing show and talking talent were discerned by the respective words of first “call”, “shot” and “inform”, second the expression “danc” mentioned within a show environment and third due to the formulations of “talk” and “talent”. Another topic concerned wrestling, which was verified by the words “tour” as well as “wrestling”, while the theme charity was confirmed by the output “chariti”. In the end services for the entertainment topic categories were revealed like event cancellation by the term “cancel” within the topical area of entertainment, excuse messages (words of determination: “excus” and “messag”) and the anticipation of a club scene (words of determination: “club” and “scene”).

The last LDA model being operated on the joined dataset composed out of the polished tweet dataset concerning the topic category wedding and the chosen sentiment lexicon exhibiting 20 000 reviews, hinted that fraud (words of determination: “fraud”, “victim”, “claim”), wedding entertainment (words of determination: “entertain” and “classic”) as well as wedding photography (words of determination: “photograph” and “style”) were just a few instances of the discussed service topics in this thematic area. Moreover the unexpected topic of “dressing up like a bride for prom” was assumed by the expressions “prom” and “wifey” placed within a wedding context, while an Elvis performance in a chapel became evident by the descriptive words of “elvi” and “chapel”. Likewise the services of preserving customers health (words of determination: “health” and “custom”), renewing the marriage (words of determination: “license”, “renew”, “princess”, “bridgetob”) and organising a wedding (words of determination: “wine” and “compani”) were obtained from the analysis of the key terms. Finally the remaining three service topics addressed the exclusivity of the ceremony (words of determination: “ceremoni” and “privat”), the capabilities of the wedding photographer (words of determination: “color” and “shot”) as well as astonishingly Valentine’s day photography (words of determination: “valentin” and “photograph”), which are all depicted in Figure 32 in the appendix section.

Instead of the utilised topic models incorporating as a major component the selected sentiment dictionary featuring 20 000 reviews, the labels of the LDA service topics, which are based on the separately cleaned twitter datasets focused on exactly one of the three aforementioned topic categories and which are also relying on the corresponding second nominated sentiment lexicon (comprising 200 000 written feedbacks), are explained in

more detail in the following. Therefore by starting with the topic category bar (illustrated in Figure 33 inside of the appendix chapter), the first service topic is named art of cooking due its constituting words “perform”, “kitchen” and “art”, while the second theme is labelled dancing due to the key term “danc”, whereas the third subject is covering the nightlife, which is based on the vocables “spirit” as well as “nightlif”. Moreover the theme title of oyster bar was assigned, due to the expression “oyster” occurring as one of the predominant words of the topic area bar (as pictured in Figure 33 within the appendix). Ultimately the residual two themes are comprising karaoke (words of determination: “karaok” and “nightclub”) and lounge nudity (words of determination: “loug” and “underwear”), of which the latter one is eponymous to the labelled topic generated via including the other sentiment lexicon and the identical review dataset as topic modelling inputs like it is outlined above.

Analysing under the assistance of the identical sentiment dictionary frequently mentioned in the preceding paragraph the basket of words outputted for each topic within the category entertainment, the first service theme is designated as encompassing singing performance (words of determination: “song” and “singer”), whereas the second one is named as acting performance (words of determination: “movi”, “act”, “actor”), while the third encompasses corporate celebration (words of determination: “celebr” and “corpor”) as depicted in Figure 34 within the appendix. Of equal importance are the succeeding topics: Advertisement for buying theatre tickets (words of determination: “friend”, “theatr”, “sale” and “tomorrow”), cinematic effects (words of determination: “movie” and “earth”), producing a score (words of determination: “happen” and “score”), plot comedy (words of determination: “comedi”, “raider”), live performance (words of determination: “live” and “perform”), security staff dealing with inappropriate behaviour (words of determination: “secur”, “inform” and “pee”), comedy performance supported by lasers (words of determination: “comedi”, “perform” and “laser”), seminar for singing (words of determination: “sing”, “seminar”, “slam” and “showroom”), amusement tour (words of determination: “tour”, “clown”, “museum” and “firework”), advertising tickets with friends for a charity event (words of determination: “sale”, “friend” and “chariti”), a detective play (words of determination: “justic”, “cheat”, “evid” and “truth”) and feedback on a tour payment process (words of determination: “credit”, “paid” and “tour”).

The service topics produced by the LDA approach comprising overlapping words between the dataset of wedding critiques and the certain emotional lexicon trained on 99% of 200 000 hotel recensions, are labeled for instance first as the renewal of the marriage vow, which is proven by the expressions like “vow”, “renew” and “marriag” and confirmed by the

generated results of the residual sentiment dictionary, second as wedding expenditures, which are signalling the terms “marriag” as well as “money”, or third as waiting for a marriage license indicated by the formulations “wait”, “marriag” and “license”. In addition the theme ceremony with Elvis photography may be extracted from the words “photographi”, “elvi” and “ceremoni”, while a separate subject is identified as properties of a singer due to the expressions “song”, “brian”, “phantom” and “sexual”. Besides the topics dancing in a chapel (words of determination: “chapel” and “danc”), wedding celebration style (words of determination: “celebr”, “wild”, “cute”, “romant”), becoming engaged (words of determination: “engag”, “marriag”, “love”, “bond” and “paper”) and the importance of hair for the photography (words of determination: “hair” and “photographi”) are brought to light and are presented in Figure 35 within the appendix.

Last but not least it is of crucial interest to publish the three criteria why topics are purposely excluded thereby classified as a non-service area. The first rule involves the rejection of external subjects as is executed in the instances of “drought”, “sports team in the finals” and “sexual violence within a marriage”. The second developed guideline states that themes which concern primarily products like for example “afternoon dessert”, “taste of wine” or “comedy rock album” are disregarded. The third principle omits topics, which have no reasonable chance to be exploited by hotels like it is true for the cases “dancing in the kitchen with friends”, “asking in an unordinary fashion for a nightclub dance” or “legal disputes”.

4.3.) Opportunity score visualisations & their analysis

Two outputs (depending exclusively on one of the distinct two sentiment dictionaries) are fabricated in order to depict the topics' opportunity scores, the top ten negative words for each theme, the corresponding topic names and their particular topical class membership as well as the individual components of the opportunity algorithm per thematic element. Namely first a table, which is ranking distinct topics according to their linked opportunity scores, is originated thereby also incorporating the topic titles, the certain topic category as

Service Opportunity Table												
Topic Name	Topic Category	Opportunity Score	1st neg. word	2nd neg. word	3rd neg. word	4th neg. word	5th neg. word	6th neg. word	7th neg. word	8th neg. word	9th neg. word	10th neg. word
A detective play	Entertainment	10.96	speak	record	truth	sorri	determin	cheat	evid	someone	suppos	report
Ceremony with Elvis photography	Wedding	10	suck	cri	edit	bye	injury	speak	racist	embarrass	disgust	threaten
Oyster bar	Bar	10	pm	sheet	suppos	industri	truck	bill	offic	alert	shift	thier
Wedding expenditures	Wedding	5.33	money	agre	send	said	front	final	cheat	forc	instead	noth
Nightlife	Bar	4.4	said	spirit	pm	respons	hire	bitch	car	threat	threaten	wouldnt
Comedy performance supported by laser lights	Entertainment	4.18	shit	honor	later	lori	hide	somewher	then	stuck	caus	somebodi
Corporate celebration	Entertainment	4.07	wors	tomorrow	corpor	gm	write	this	never	left	racist	discuss
Advertising tickets with friends for a charity event	Entertainment	4.07	wait	final	number	sale	pray	emot	homeless	rob	nightmar	yesterday
Advertismet for buying theater tickets	Entertainment	3.78	pm	tomorrow	sale	truck	final	sell	yesterday	femal	interview	noth
Dancing in a chapel	Wedding	3.39	illeg	intent	somebodi	instead	shed	final	trash	hasnt	hmmm	cheat
Live performance	Entertainment	2.97	weed	voic	car	blame	nobodi	forc	underwear	click	health	woman
Cinematic effects	Entertainment	2.57	said	k	what	lie	hell	hold	record	pay	relay	third
Lounge nudity	Bar	2.39	ask	underwear	front	told	said	act	paid	chief	paper	final
Waiting for a marriage license	Wedding	2.21	wait	licens	bureau	joke	steer	subject	third	screw	tomorrow	shame
Dancing	Bar	1.99	tell	rip	rep	shock	key	lawyer	bag	horibl	noth	no
Wedding celebration style	Wedding	1.77	later	hell	never	law	lori	sad	bill	rob	told	sorri

Figure 36:

Dash Table sorted per default by the opportunity score
(200 000 reviews - training set portion = 99%)

Service Opportunity Table												
Topic Name	Topic Category	Opportunity Score	1st neg. word	2nd neg. word	3rd neg. word	4th neg. word	5th neg. word	6th neg. word	7th neg. word	8th neg. word	9th neg. word	10th neg. word
Oyster bar	Bar	10	pm	sheet	suppos	industri	truck	bill	offic	alert	shift	thier
Nightlife	Bar	4.4	said	spirit	pm	respons	hire	bitch	car	threat	threaten	wouldnt
Lounge nudity	Bar	2.39	ask	underwear	front	told	said	act	paid	chief	paper	final
Dancing	Bar	1.99	tell	rip	rep	shock	key	lawyer	bag	horibl	noth	no
Karaoke	Bar	1.77	wait	tomorrow	manag	shit	said	lawyer	caus	told	pos	lock
Art of cooking	Bar	1.63	card	pm	honor	tomorrow	credit	write	compani	somebodi	hide	zero
A detective play	Entertainment	10.96	speak	record	truth	sorri	determin	cheat	evid	someone	suppos	report
Comedy performance supported by laser lights	Entertainment	4.18	shit	honor	later	lori	hide	somewher	then	stuck	caus	somebodi
Corporate celebration	Entertainment	4.07	wors	tomorrow	corpor	gm	write	this	never	left	racist	discuss
Advertising tickets with friends for a charity event	Entertainment	4.07	wait	final	number	sale	pray	emot	homeless	rob	nightmar	yesterday
Advertismet for buying theater tickets	Entertainment	3.78	pm	tomorrow	sale	truck	final	sell	yesterday	femal	interview	noth
Live performance	Entertainment	2.97	weed	voic	car	blame	nobodi	forc	underwear	click	health	woman
Cinematic effects	Entertainment	2.57	said	k	what	lie	hell	hold	record	pay	relay	third
Security staff dealing with inappropriate behaviour	Entertainment	1.7	call	tomorrow	yesterday	secur	inform	fox	pee	cc	discuss	advertis
Feedback on a tour payment process	Entertainment	1.61	wtf	credit	paid	record	edit	manag	respons	later	bill	someone
Acting performance	Entertainment	1.48	never	ask	act	hell	credit	replac	hair	situat	awar	titl

Figure 37:

Dash Table alphabetically sorted according to the topic category
(200 000 reviews - training set portion = 99%)

well as the ten most negative words per theme (as outlined in Figure 36). Furthermore the option has been implemented to sort according to features like topic category (shown in Figure 37) or topic name due to the service opportunity table displaying per default a prioritisation from the theme comprising the greatest opportunity score to the one, which exhibits the lowest business chance. Moreover users may interactively delete rows of the dash table in order to gain a more customised overview if required. Secondly a scatterplot is drawn within the spectrum of the three dimensions of opportunity score, importance and satisfaction, so that an end-user may interact with the individual data points by rotating the plot or by hovering over a specific topic leading to the description of the topic name, opportunity score, importance rating and the satisfaction value, which is true for all thematic data points. In addition one may hide or unhide some topics belonging to the identical topic category, if demanded like it is outlined in Figure 38. Therefore by establishing both graphical representations the reader is advised to notice that topical elements belonging to the topic category of bar, entertainment or wedding are shown in its standard configuration within the identical plot and table, thus differentiating in the part of visualisation only according to the selected sentiment lexica.

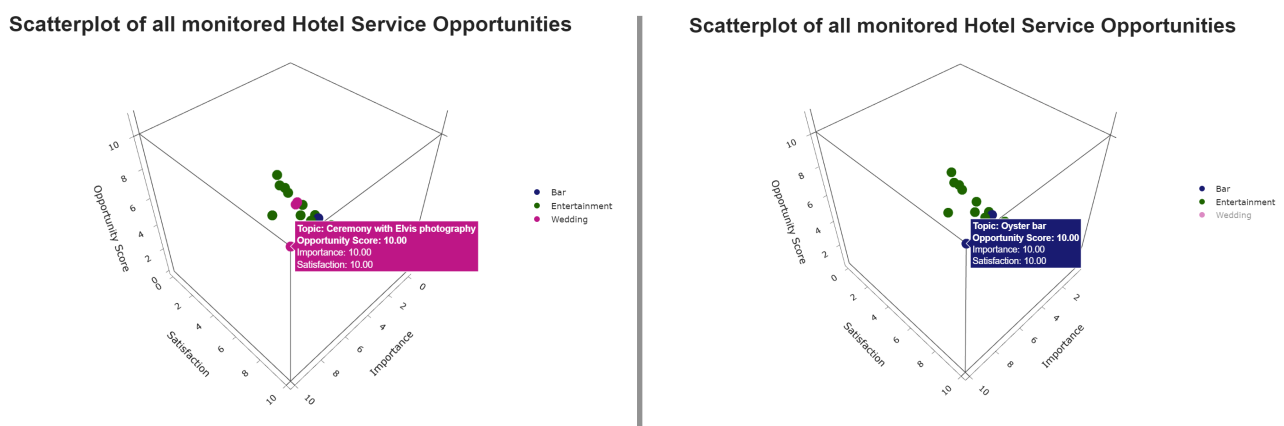


Figure 38:

Dash scatterplot with all examined topic categories (left side) and
the Dash scatterplot excluding topics of the topic category wedding (right side)
(200 000 reviews - training set portion = 99%)

When examining the dash table developed by integrating the selected sentiment dictionary consisting out of 20 000 reviews (portrayed in Figure 39), the author of this paper has to admit that no substantial ways of refinement could be deduced for six out of the 35 service topics (i.e. Health of customers, to waitress, magic with a car, roasting pineapples live, event cancellation and magic in the library) due to the fact that the ten words connoted

Dash Table

Topic Name	Topic Category	Opportunity Score	1st neg. word	2nd neg. word	3rd neg. word	4th neg. word	5th neg. word	6th neg. word	7th neg. word	8th neg. word	9th neg. word	10th neg. word
Health of the viewers	Entertainment	13.69	shown	fuck	post	repres	poor	hold	state	siam	beep	soccer
Lounge nudify	Bar	10.0	leav	underwear	post	clog	messag	anam	dirli	report	ashtray	embarrass
Wedding photography	Wedding	9.26	dont	alreadi	discov	news	what	crimin	money	dump	disabl	talk
Fraud	Wedding	9.16	fraud	victim	claim	month	obtain	report	final	file	nobodi	subject
Health of the customers	Wedding	6.74	sorri	health	custom	honor	marcus	hour	receiv	broke	practic	understand
Talking talent	Entertainment	6.73	talk	never	yesterday	Imao	listen	honor	aw	no	scratch	fool
Grilling by a restaurant	Bar	4.64	said	applebe	compani	fuck	sec	leav	promis	ridicul	gas	anoth
Elvis performance in a chapel	Wedding	4.1	fail	bag	said	paper	attorney	welp	hour	district	drug	somebodi
Valentine's day photography	Wedding	4.02	talk	fuck	rental	later	guarante	whoever	screw	amend	told	speak
To waitress	Bar	3.99	fuck	tom	told	tomorrow	fire	then	manag	loyalti	legit	shut
Magic with a car	Entertainment	3.38	dont	tue	car	affect	alien	attitud	homeless	speak	chariti	quot
Renewing the marriage	Wedding	2.96	licens	Imao	bureau	suck	hour	pay	church	Imma	mistak	certif
Exclusivity of the ceremony	Wedding	2.88	no	instead	leav	pls	crime	tomorrow	min	report	off	clue
Excuse messages	Entertainment	2.72	shit	anoth	excus	messag	happen	prom	femal	repli	pos	fox
Organising a wedding	Wedding	2.39	shit	wouldnt	said	compani	sent	car	adapt	record	prison	refer
Dressing up like a bride for prom	Wedding	2.22	understand	prom	desk	commit	fb	poor	race	corpor	custom	remov
Visiting a movie apartment	Entertainment	2.09	said	listen	hell	pald	pls	snth	attorney	ask	pray	cancel
Charity	Entertainment	2.05	honor	bill	guard	offic	rob	chariti	instead	titl	listen	ta
Unprofessional wedding photographer	Wedding	1.96	happen	cheat	Imma	wrong	shed	fuck	final	leak	obvious	eachoth
Price performance ratio of eating	Bar	1.94	hour	Imao	dollar	pay	min	anoth	later	off	written	straw
Emergency call	Entertainment	1.8	hour	call	inform	someone	phone	wouldnt	polic	cash	f	quot
Live cast of actors	Entertainment	1.59	horror	record	voic	month	happen	corpor	click	forc	investig	poison
Sales event	Entertainment	1.57	happen	sale	tomorrow	sell	practic	steal	yesterday	pay	lexus	replac
Wedding entertainment	Wedding	1.23	agre	jake	bouquet	charg	formal	loyal	alreadi	chariti	dirli	mess
Retail seller's verbal communication	Bar	1.21	talk	ta	month	happen	said	never	post	fuckin	chief	leav
Roasting pineapples live	Bar	1.13	sh	corpor	tell	car	avoid	blood	health	slip	cheat	offic
Awaiting a club scene	Entertainment	1.09	tomorrow	news	reach	discov	anoth	joke	titl	file	someone	crime
Sing & fashion style competition	Entertainment	1.09	battl	where	nikki	tie	min	apolog	print	written	understand	attack
Dancing show	Entertainment	0.99	alreadi	told	log	fox	church	polic	worst	chief	realiz	respond
Strip club	Bar	0.98	listen	honor	custom	replac	concern	batteri	newbi	record	weed	attempt
Singing at the dinner table	Bar	0.95	pm	nobodi	write	intent	print	wast	requir	situat	cc	tomorrow
Event cancellation	Entertainment	0.89	cancel	sorri	evid	off	cheat	no	ask	report	wrong	dont
Magic in the library	Bar	0.29	recip	tomorrow	car	record	beep	pos	realiz	click	key	attack
Wrestling	Entertainment	0.26	leav	account	compani	harm	submit	strike	dollar	georg	document	carpet
Golf & brewery tour	Bar	0.19	money	call	breweri	speak	bare	emerg	ppl	told	trap	paper

Figure 39:

Summary of the monitored hotel service opportunities
(20 000 reviews - training set portion = 10%)

with negativity did not provide a totally clear indication. On the contrary the most overall business potential (opportunity score = 13.69) has been observed for the theme “Health of the viewers” thereby suggesting the reduction of acoustic noise and the omission of the soccer components for the area of entertainment. Furthermore by analysing “Lounge nudity” it became more obvious that dirt, especially in the form of underwear, ashtrays and clogs is not appreciated, while for the topical element “Wedding photography” the communication of the pricing is emphasised. Next the recommendations basing on the subject “Fraud” located within the topic category wedding demonstrate first the shortening of the final report’s duration until it is obtained and secondly emphasise the identification of the responsible person, who has caused this situation. Of equal significance is that when customers were assessing the topic “Talking talent”, they rated the level of intellect as inferior thereby one may enhance these capabilities by inspecting the semantic and syntactic skills of the speaker and then offer assistance for the exact weaknesses detected. Alternatively clients were as well complaining about the restaurant Applebee’s grilling with gas, so that one may infer that consumers prefer food from a coal or an electrical grill, which might be valuable for hotels as well. Moreover hotels operating in Las Vegas may especially profit from exercising drug tests on their staff since it presented an issue in the case of the theme “Elvis performance in a chapel”.

The unexpected topic “Valentine’s day photography” in the domain of wedding demonstrated the relevance of transparent terms and conditions, because the demanding side expressed negativity if this matter is not sufficiently articulated. In addition married couples, who aimed to renew their marriage, underlined the cruciality of correctly including the entirety of the wedding information on their certificate, whereas one may reason for the wedding thematic element “Exclusivity of the ceremony” that the appropriate degree of security has to be established or maintained in order to minimise the dangerous exposure to criminal activities. Into the bargain while the one wedding topic “Organising a wedding” stresses the significance of a well-functioning car service, the other theme, namely “Dressing up like a bride for prom” hints towards a more racially sensitive customer interaction in this field. However the insight, which may be withdrawn from the entertainment subject “Excuse messages” highlights the essentiality of keeping the amount of these notifications relatively low, although a more profound analysis for this theme is likely to uncover more details so that the key reason(s) of this phenomenon may be identified and addressed in full. A distinct aspect is brought to attention by the entertainment theme of “Visiting a movie apartment” therewith accentuating the implications of cancellation fees.

Continuing by further examining the dash table, which relies on the chosen sentiment lexicon composed out of 20 000 recensions for the thematic subject “Charity”, evidence was discovered that spotlighted the necessity for guarding charity events from being robbed. Next the potential source for the theme “Unprofessional wedding photographer” is assumed to be the lack of discretion, while the negativity expressed towards the matter “Price performance ratio of eating” seems to be caused by paying for straws. Furthermore within the domain of entertainment in order to advance in the individual topics “Emergency call”, “Live cast of actors” and “Sales Event” it was particularly diagnosed that the response time by the police is required to be minimised, that the vocal performance by actors is needed to be refined as well as the forced investigation is advised to be discarded, and that lastly it was detected that the price of Lexus (i.e. a car produced by a specific company) replacements should be decreased. Of equal importance is a decently clean bouquet within the realm of “Wedding entertainment”, whereas “Retail seller’s verbal communication” may be boosted by excluding to promise non-existing service features, while the thematic element “Awaiting a club scene” is able to be enhanced via a more precise formulation of the event’s title. In addition it is indicated on the one hand that a “Sing & fashion style competition” is able to be tweaked by filtering out aggressive applicants. On the other hand the findings reveal that the topic “Dancing show” is upgraded if the police officer, which is assumably a dancing role, is working on his or her skills. However within the bar theme category the service topic “Strip club” may offer an augmented customer experience by preventing customer attempts of either recording or smuggling in marihuana. Likewise the theme “Singing at the dinner table” located in the identical category as the previously mentioned subject may be positively influenced by a clear communication due to some consumers perceiving this action as unnecessary, which triggers disappointment if these guests are not or insufficiently informed apriori. Last but not least the second lowest opportunity score was assigned to the matter of “Wrestling” thereby rendering the optimisation direction of alleviating the harm as not relevant for the masses while the topic “Golf & brewery tour” is presenting the business chance (opportunity score = 0.19) with the tiniest expected benefit, although this issue might be solved through strengthening the liaison of communicative character between clients and the brewery.

The second dash table generated includes the component of the selected and 200 000 review-strong sentiment dictionary as it is pictured in Figure 40. Furthermore the most promising topic “A detective play” (opportunity score = 10.96) reveals through its top ten negative expressions that the audience is demanding a more thought-out and logical

Dash Table

Topic Name	Topic Category	Opportunity Score	1st neg. word	2nd neg. word	3rd neg. word	4th neg. word	5th neg. word	6th neg. word	7th neg. word	8th neg. word	9th neg. word	10th neg. word
A detective play	Entertainment	10.96	speak	record	truth	sorri	determin	cheat	evid	someone	suppos	report
Ceremony with Elvis photography	Wedding	10.0	suck	cri	edit	bye	injury	speak	racist	embarrass	disgust	threaten
Oyster bar	Bar	10.0	pm	sheet	suppos	industri	truck	bill	offic	alert	shift	thier
Wedding expenditures	Wedding	5.33	money	agre	send	said	front	final	cheat	forc	instead	noth
Nightlife	Bar	4.4	said	spirit	pm	respons	hire	bitch	car	threat	threaten	wouldnt
Comedy performance supported by laser lights	Entertainment	4.18	shit	honor	later	lori	hide	somewher	then	stuck	caus	somebodi
Corporate celebration	Entertainment	4.07	wors	tomorrow	corpor	gm	write	this	never	left	racist	discuss
Advertising tickets with friends for a charity event	Entertainment	4.07	wait	final	number	sale	play	emot	homeless	rob	nightmar	yesterday
Advertisement for buying theater tickets	Entertainment	3.78	pm	tomorrow	sale	truck	final	sell	yesterday	femal	interview	noth
Dancing in a chapel	Wedding	3.39	illeg	intent	somebodi	instead	shed	final	trash	hasnt	hmm	cheat
Live performance	Entertainment	2.97	weed	voic	car	blame	nobodi	forc	underwear	click	health	woman
Cinematic effects	Entertainment	2.57	said	k	what	lie	hell	hold	record	pay	relay	third
Lounge nudity	Bar	2.39	ask	underwear	front	told	said	act	paid	chief	paper	final
Waiting for a marriage license	Wedding	2.21	wait	licens	bureau	joke	steer	subject	third	screw	tomorrow	shame
Dancing	Bar	1.99	tell	rip	rep	shock	key	lawyer	bag	horribl	noth	no
Wedding celebration style	Wedding	1.77	later	hell	never	law	lori	sad	bill	rob	told	sorri
Karaoke	Bar	1.77	wait	tomorrow	manag	shit	said	lawyer	caus	told	pos	lock
Security staff dealing with inappropriate behaviour	Entertainment	1.7	call	tomorrow	yesterday	secur	inform	fox	pee	cc	discuss	advertis
Art of cooking	Bar	1.63	card	pm	honor	tomorrow	credit	write	compani	somebodi	hide	zero
Feedback on a tour payment process	Entertainment	1.61	wtf	credit	paid	record	edit	manag	respons	later	bill	someone
Acting performance	Entertainment	1.48	never	ask	act	hell	credit	replac	hair	situat	avar	ttl
Producing a score	Entertainment	1.24	this	happen	no	instead	excus	evid	crimin	durango	proceed	discov
Beautiful hair when being photographed	Wedding	1.2	hair	tell	call	final	ask	nobodi	woke	dead	assum	hold
Plot comedy	Entertainment	1.15	poor	phone	state	smh	report	attorney	carpet	disrespect	happen	ttl
Seminar for singing	Entertainment	0.88	fuck	post	announc	clip	seminar	slam	guard	someone	pathet	citizen
Renewing the marriage vow	Wedding	0.86	tomorrow	told	no	tell	ask	clue	corpor	whoever	later	wors
Amusement tour	Entertainment	0.76	clown	money	pm	lori	soccer	imposs	tell	final	ttl	explain
Becoming engaged	Wedding	0.76	paper	desk	ignor	sad	audac	later	switch	favor	stupid	realiz
Singing performance	Entertainment	0.38	noth	someone	joke	wont	pay	voic	whoever	shouldnt	reaction	law
Features of a singer	Wedding	0.0	crime	sexual	ask	lawyer	attitud	baltimor	anymor	cheat	paper	detain

Figure 40:

Summary of the monitored hotel service opportunities

(200 000 reviews - training set portion = 99%)

argumentation structure. Next by analysing the subject “Ceremony with Elvis photography” (opportunity score = 10.00) a powerful call for stopping racist remarks and for ending physical or psychological violence was concluded to be beneficial for business operations. Likewise for the matter “Oyster bar” being of equal opportunity when compared to the previous referred theme, it is advised that an enhanced coordination of presumably staff between their shifts is needed in order to bill customers to a sufficient level. Moreover for the topic “Wedding expenditures” an absence of goodwill and reimbursement options was inferred, while for the different subject “Nightlife”, which is situated within the topic area of bar, the lacking response to a drunken person aiming to drive was accentuated. In contrast the themes “Comedy performance supported by laser lights”, “Corporate celebration” as well as “Advertising tickets with friends for a charity event”, which are all lying within the topical entertainment class, respectively addressed the stuck laser lights caused by human interaction, the racist discussion seemingly conducted at a GM (i.e. General Motors) event and the waiting time. In addition the theme “Dancing in the chapel” uncovered the demand for ordinary dancing hours inside a chapel, whereas the point of abandoning chav attributes is stressed for the subject “Live performance”, while the critical keywords of “Lounge Nudity” are focused on performing a payment, which may be interpreted as a fine transaction for guests’ misbehaviour.

In the distinct case of “Waiting for a marriage license” the issue’s epicentre was the process waiting time, thus recommending that hotels may profit via a service which may mitigate the temporal expenses of wedding pairs. Besides the findings outputted by the topics “Wedding celebration style” and “Karaoke” demonstrate that for the former aspect guards might be adequate in order to counteract stealing endeavours, while for the latter service a well-posted information desk might prove to be suitable in order to avoid situations where clients are locked out due to changes in the opening hours. Alternative thematic variants, namely the “Security staff dealing with inappropriate behaviour” and the “Art of cooking”, exhibited the determining negative vocables, which lead to the reasoning that in order to improve the first mentioned topic an action is required to be exercised in an immediate fashion, while for enhancing the second theme the fact of non-transparent costs is indispensable to be tackled. A related difficulty arouse for the topic “Feedback on a tour payment process”, which in the end of the day is underlining the essentiality of transferring the purchasing price to the rightfully authorised contact person(s) as well as it foregrounds the significance of a quickly reactive trouble-shooting by the management. Also it was discovered for the topic “Beautiful hair when being photographed” that instructions towards the customers should be communicated, whereas in thematic contrast the weakness of

“Plot comedy” was its unclear connection between title and stage programme. Different perspectives concerning the entertainment topic category were evidenced for “Seminar for singing”, “Amusement tour” and “Singing performance” thereby the advancement directions were identified and are as well described in the following. The first insight comprises that the shortening of the seminar is not advised to be repeated as well as the seminar may be grouped into classes containing members of a similar vocal skill level, whereas the second notion to be deduced advocates for a precise definition of the amusement tour title as well as for the relinquishment of future tour elements like clowns or football (i.e. soccer), while third the suboptimal singing performance may be rectified although it is not specified in which style. Finally the topics exhibiting the least opportunity scores from the wedding domain are “Becoming engaged” and “Features of a singer” of which the former one implies a beneficial development if the condition holds true that none of the engagement documents are ignored, whereas the latter theme designates an advantageous future progression under the prerequisite that the singer does not possess a sexual or criminal attitude. Last but not least the reader is prompted to take into account that from the assessment of the 30 topics conducted by evaluating the dash table illustrated in Figure 40, the succeeding six subjects did not feature clearly valuable information for inferring helpful recommendations on which further adjustments can be realised: “Advertisement for buying theater tickets”, “Renewing the marriage vow”, “Dancing”, “Cinematic effects”, “Acting performance” and “Producing a score”.

5.) Limitations

While the opportunity system developed in this master dissertation on the basis of the research framework by Jeong, Yoon and Lee (2019) provides an easy to understand and implement SM service sentiment evaluation mechanism especially for SMSH, admittedly the drawback still exists that SMSH are still required to employ an IT-expert, who customises and operates the semi-automated technique, which lowers the barriers for SMSH's SMA adoption, however it does not fully eliminate them. Furthermore the procedure is coded as a non-scalable model due to the absence of cluster computing and cluster data storage capabilities.

The first vulnerability of the proposed sentiment lexica is the fundamental independence assumption of Naive Bayes leading to the perception that words are always generated irrespective of their context, which is evidently not correct. Consequently the second weakness comprises that the sentiment lexica may not comprehend negations like in "do not favour" or they are blind to amplifications like "extremely admire". Third the sentiment dictionaries are not adjusting to changes in language over time in the area of novel word expressions as well as in the semantical meaning of their respective vocabulary. An extra assumption implies the selection of the β for the generation of the F-beta score. The sentiment dictionaries were evaluated from a risk-adverse perspective ($\beta = 0.5$), whereas risk-neutral ($\beta = 1.0$) or risk-tolerant ($\beta > 1.0$) standpoints may be justified when including the individual preferences of certain SMSH owners. Besides the applicant should be aware of the fact that 17.14% to 20% of all hotel relevant service topics featured immaterial recommendations for advancing in their specific thematic field.

The presented opportunity system suffers as well from the lack of differentiation between on the one hand domestic as well as international consumers and on the other hand between first-time guests and patrons. Moreover due to Blagus and Zitnik (2018) recognising that tweets where the gender could be identified (26.6%), are emitted to an enormous extent by men, the explanatory power of these findings may be limited to males only, if this impression is also confirmed in general by additional academic projects. Likewise the assumption that Yelp data demonstrates similar if not identical characteristics to the one published on Twitter when filtered by location and business branch has to be taken into account by the reader, while it lastly can not be ruled out that a portion of the recognised client posts may be authored by fraudulent Twitter influencers.

6.) Conclusion

This academic article contributes to the modern scientific discussion concerning SMA for SMSH by training Naive Bayes classifiers specifically on communicated words within the hotel industry as demanded by Geetha, Singha and Sinha (2017). Therefore these classifiers are attaining sufficient categorisation levels in the sphere of precision, recall, accuracy and F-beta score for the two selected sentiment dictionaries, when tested on the task to differentiate between positivity and negativity of expressions. Despite the entire monitored four sentiment classification lexica are underperforming in the aforementioned fields when contrasted against the both previously referred to ones, these emotional Naive Bayes dictionaries were achieving tremendously enhanced results for the determination of moderate sentiment when compared to the terms conveying firmly positive or negative feelings. Therefore the impact of an increase in the relative amount of the training data portion generally lead to a declining precision, to a decreased F-beta score, but an elevated recall value in the case of slightly connoted sentiment, whereas the opposite characteristics are normally true if the training size is raised in percentage terms for the correct identification of vocables containing strong emotional magnitudes.

This research additionally examined that topic models integrating the emotional lexicon which has been scarcely supplied with 10% of 20 000 hotel reviews, are superior to LDA approaches, which are utilising the sentiment dictionary comprising 99% of 200 000 hospitality critiques, when measured in coherence and perplexity for all three analysed topic categories, whereas the latter sentiment lexicon distinguishes emotional word tendency in a more desirable fashion. Furthermore while approaches including both emotional lexica are differing in their optimal number of topics in one out of three topic categories, the ranking of the volatility between both models' perplexity and topic coherence proved to be identical across distinct thematic categories. Equally important is that the integrated semi-automatic technique of LDA allows experts of the hotel domain to utilise their off-record knowledge obtained through their real-world experiences to interpret topic titles and to infer beneficial service development practices, which is believed to be more accurate than if conducted by an external researcher due to the applicant of the SMSH representing a domain-specific judge for the dimensions of location and business area himself.

Answering the research question outlined in the beginning of this master dissertation, although the ease of execution could not be completely established as described in the limitation section, the key benefit of the presented opportunity system is that from at least 80% of the hotel service topics assistive recommendations may be reasonably concluded

thereby indicating at the minimum a starting point for further service advancements if not in some cases even specifying detailed suggestions about how to optimise particular aspects of the respective service. Moreover the approach is designed to be operated on a single computer, which recognises the scarce resources of a SME while only an extremely limited possibility exists that criminal hackers may interfere in the process due to the actions of downloading the Yelp dataset, of registering as a Twitter developer, of searching and downloading tweets as well as of obtaining the necessary Python libraries from the internet. Besides the seeking of tweets published within a year for each topic category complies to the findings communicated by Choi et al. (2020), who were discovering in their study that 93.3% of their classified opportunities were lasting for a one day period, however the same academics expressed that the remaining themes were of great considerable interest. Finally the substitution of the external sentiment classifier applied by Jeong, Yoon and Lee (2019) for a domain-specific sentiment dictionary, which represents the final output of Phase 1, enables the researchers and technical experts to check and tweak the classifier's quality.

7.) Future Work

Further studies may develop the opportunity system into an end-user friendly application thereby introducing a simple graphical user interface and as a consequence making the required IT-specialist obsolete. Likewise additional research may tackle the constraints of the current approach by contributing a scalable solution either for SMEs, which grow to mid-sized and eventually to large enterprises or for the purpose of meeting extreme analytical demands expressed by certain SMEs. Different academic paths may lead to first exploring the informative benefit of customers' voice messages on SM platforms like Clubhouse for the domain of SMEs, whereas second the opportunity algorithm is still missing a suitability examination for multifunctional product-service systems as it is also attested by Jeong, Yoon and Lee (2019). *Ceteris paribus* variations to the characteristics of the evaluated review data in terms of its origination location, its ratio between domestic and international travellers over time or alterations in the business domain may deliver remarkable insights. Moreover first the performance of the opportunity system having integrated an unsupervised sentiment classification technique, second the results by the opportunity procedure utilising SM critiques from a broad range of SM networks as well as third a case study scrutinising the applicants' satisfaction of the opportunity approach when β is set corresponding to his or her risk profile are all promising to be of tremendous academic interest. Last but not least an up-to-date representative investigation concerning active users' demographic attributes (like for instance gender) of the most popular SM networks as well as a mechanism distinguishing between actual customer recensions and advertisements is probable to immensely assist future scientific activity in the area of SMA, while an experiment measuring the classification performance of domain-specific Naive Bayes sentiment dictionaries, in which the four referred to sentiments are grouped on the one hand into a class of extreme emotions and on the other hand into the category of moderate sentiment, might lead to fascinating inferences so that texts may be particularly filtered by their granular sentiment amplitudes.

8.) References

Ahmad, S., Ahmad, N. and Bakar, A. (2018) 'Reflections of entrepreneurs of small and medium-sized enterprises concerning the adoption of social media and its impact on performance outcomes: Evidence from the UAE', *Telematics and Informatics*, 35 (1), pp. 6-17. doi: <https://doi.org/10.1016/j.tele.2017.09.006>.

Amazon Web Services (2021) *Netflix on AWS*. Available at: <https://aws.amazon.com/solutions/case-studies/netflix/> (Accessed: 15th of May in 2021).

An, Q. et al. (2020) 'Role of user-generated photos in online hotel reviews: An analytical approach', *Journal of Hospitality and Tourism Management*, 45, pp. 633-640. doi: <https://doi.org/10.1016/j.jhtm.2020.11.002>.

Barragán-Landy, M. et al. (2020) 'A Proposed Representative Sampling Methodology', *European Conference on Research Methodology for Business and Management Studies*. Aveiro (Portugal), 18th to 19th of June in 2020. Reading (UK): Academic Conferences International Limited, pp. 8-17

Bednár, P. (2017) 'Conceptualization of Social Big Data: Keynote', *12th International Workshop on Semantic and Social Media Adaption and Personalization (SMAP)*. Bratislava (Slovakia), 9th to 10th of July in 2017. New Jersey: Institute of Electrical and Electronics Engineers, pp. 21-22.

Blagus, N. and Zitnik, S. (2018) 'Social media comparison and analysis: The best data source for research?', *12th International Conference on Research Challenges in Information Science (RCIS)*, Nantes (France), 29th to 31st of May in 2018. New Jersey: Institute of Electrical and Electronics Engineers, pp. 1-10.

British Broadcasting Corporation (2021) *Parler: Amazon to remove site from web hosting service*. Available at: <https://www.bbc.com/news/technology-55608081> (Accessed: 15th of May in 2021).

Burch et al. (2019) 'Finding the outliers in scan path data', *ETRA '19: Proceedings of the 11th ACM Symposium on Eye Tracking Research & Applications*. Denver (USA), 25th to

28th of June in 2019. New York (USA): Association for Computing Machinery, pp. 1-5. doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1145/3317958.3318225>

Chang, J. et al. (2009) 'Reading Tea Leaves: How Humans interpret Topic Models', *22nd International Conference on Neural Information Processing Systems*. Vancouver (Canada), 7th to 9th of December in 2009. Red Hook (USA): Curran Associates Inc., pp. 288-296.

Choi, J. et al. (2020) 'Identification of time-evolving product opportunities via social media mining', *Technological Forecasting and Social Change*, 156, pp. 1-12. doi: <https://doi.org/10.1016/j.techfore.2020.120045>.

Davras, Ö. and Caber, M. (2019) 'Analysis of hotel services by their symmetric and asymmetric effects on overall customer satisfaction: A comparison of market segments', *International Journal of Hospitality Management*, 81, pp. 83-93. doi: <https://doi.org/10.1016/j.ijhm.2019.03.003>.

Deloitte (2019) *The performance of Small and Medium Sized Businesses in a digital world: A report for the Connected Commerce Council 2019*. Available at: <https://www2.deloitte.com/content/dam/Deloitte/es/Documents/Consultoria/The-performance-of-SMBs-in-digital-world.pdf> (Accessed: 24th of January in 2021).

Donavan, J. (2016) 'Spotify Announces Google Cloud Platform Partnership'. Available at: <https://techcrunch.com/2016/02/23/spotify-announces-google-cloud-platform-partnership/> (Accessed: 15th of May in 2021).

Dong, J. and Yang, C. (2020) 'Business value of big data analytics: A systems-theoretic approach and empirical test', *Information & Management*, 57 (1), pp. 1-9, doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1016/j.im.2018.11.001>.

Elzayady, H., Badran, K. and Salama, G. (2018) 'Sentiment Analysis on Twitter Data using Apache Spark Framework', *13th International Conference on Computer Engineering and Systems (ICCES)*. Cairo, Egypt, 18th to 19th of December in 2018. New Jersey: Institute of Electrical and Electronics Engineers, pp. 171-176.

Etikan, I. and Bala, K. (2017) 'Sampling and Sampling Methods', *Biometrics & Biostatistics International Journal*, 5 (6), pp. 215-217. doi: [10.15406/bbij.2017.05.00149](https://doi.org/10.15406/bbij.2017.05.00149).

Farizah Ibrahim, N. and Wang, X. (2019) 'A text analytics approach for online retailing service improvement: Evidence from Twitter', *Decision Support Systems*, 121, pp. 37-50. doi: <https://doi.org/10.1016/j.dss.2019.03.002>.

Fuentes-Moraleda, L. et al. (2020) 'Interaction between hotel service robots and humans: A hotel-specific Service Robot Acceptance Model (sRAM)', *Tourism Management Perspectives*, 36, pp. 1-14. doi: <https://doi.org/10.1016/j.tmp.2020.100751>.

Gallagher, M. and Trendafilov, R. (2018) 'R vs: Python: Ease of use and numerical accuracy', *Journal of Business and Accounting*, 11 (1), pp. 117-126.

Geetha, M., Singha, P. and Sinha S. (2017) 'Relationship between customer sentiment and online customer ratings for hotels - An empirical analysis', *Tourism Management*, 61, pp. 43-54. doi: <https://doi.org/10.1016/j.tourman.2016.12.022>.

Hu, G. et al. (2017) 'Analyzing users' sentiment towards popular consumer industries and brands on Twitter', *IEEE International Conference on Data Mining Workshops*. New Orleans, LA, USA, 18th to 21st of November in 2017. New Jersey: Institute of Electrical and Electronics Engineers, pp. 381-388.

Jain, K. and Kaushal S. (2018) 'A comparative study of Machine Learning and Deep Learning Techniques for Sentiment Analysis', *7th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*. Noida (India), 29th to 31st of August in 2018. New Jersey: Institute of Electrical and Electronics Engineers, pp. 483-487.

Jeong, B., Yoon, J. and Lee, J. (2019) 'Social media mining for product planning: A product opportunity mining approach based on topic modeling and sentiment analysis', *International Journal of Information Management*, 48, pp. 280-290. doi: <https://doi.org/10.1016/j.ijinfomgt.2017.09.009>.

Kim, Y. and Kim, M. (2019) 'A Wisdom of Crowds': Social Media Mining for Soccer Match Analysis'. *IEEE Access*, 7, pp. 52634-52639. doi: <https://doi.org/10.1109/ACCESS.2019.2912009>.

Kirilenko et al. (2018) 'Automated Sentiment Analysis in Tourism: Comparison of Approaches', *Journal of Travel Research*, 57 (8), pp. 1012-1025. doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1177%2F0047287517729757>.

Koiranen, I. et al. (2019) *Changing patterns of social media use? A population-level study of Finland*. Available at: <https://link.springer.com/article/10.1007/s10209-019-00654-1> (Accessed: 29th of October in 2020).

Kritiyanti, D. et al. (2018) 'Comparison of SVM & Naïve Bayes Algorithm for Sentiment Analysis Toward West Java Governor Candidate Period 2018-2023 Based on Public Opinion on Twitter', *6th International Conference on Cyber and IT Service Management (CITSM)*. Parapat (Indonesia), 7th to 9th of August in 2018. New Jersey: Institute of Electrical and Electronics Engineers, pp. 1-6.

Krouska, A., Troussas, C. and Virvou, M. (2017) 'Comparative Evaluation of Algorithms for Sentiment Analysis over Social Networking Services', *Journal of Universal Computer Science*, 23 (8), pp. 755-768.

Kunal, S. et al. (2018) 'Textual Dissection Of Live Twitter Reviews Using Naive Bayes', *Procedia Computer Science*, 132, pp. 307-313. doi: <https://doi.org/10.1016/j.procs.2018.05.182>.

Las Vegas Convention and Visitors Authority (2021) *Las Vegas International Visitation Estimates*. Available at: https://assets.simpleviewcms.com/simpleview/image/upload/v1/clients/lasvegas/2019_Intl_Visitation_Country_and_World_Region_023a2502-e1a3-40c8-9af6-e69fd4bdc48a.pdf (Accessed: 15th of May in 2021).

Lee, I. (2018) 'Social media analytics for enterprises: Typology, methods, and processes', *Business Horizons*, 61 (2), pp. 199-210. doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1016/j.bushor.2017.11.002>.

Li, Y. and Fleyeh H. (2018) 'Twitter Sentiment Analysis of New IKEA Stores Using Machine Learning', *International Conference on Computer and Applications (ICCA)*. Beirut, Lebanon, 25th to 26th of August in 2018. New Jersey: Institute of Electrical and Electronics Engineers, pp. 4-11.

Li, Z., Li, R. and Jin, G. (2020) 'Sentiment Analysis of Danmaku Videos Based on Naïve Bayes and Sentiment Dictionary', *IEEE Access*, 8, pp. 75073-75084. doi: <https://doi.org/10.1109/ACCESS.2020.2986582>.

Liu, X., Burns, A. and Hou, Y. (2017) 'An Investigation of Brand-Related User-Generated Content on Twitter', *Journal of Advertising*, 46 (2), pp. 236-247. doi: 10.1080/00913367.2017.1297273.

Liu, Y. et al. (2020) 'Cloud-based big data analytics for customer insight-driven design innovation in SMEs', *International Journal of Information Management*, 51, pp. 1-12, doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1016/j.ijinfomgt.2019.11.002>.

Maroufkhani, P. et al. (2020) 'Big data analytics adoption: determinants and performances among small to medium-sized enterprises', *International Journal of Information Management*, 54, pp.1-15. doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1016/j.ijinfomgt.2020.102190>.

Mifrah, S. and Habib Benlahmar, E. (2020) 'Topic Modeling Coherence: A Comparative Study between LDA and NMF Models using COVID' 19 Corpus', *International Journal of Advanced Trends in Computer Science and Engineering*, 9, pp. 5756-5761. doi: <https://doi.org/10.30534/ijatcse/2020/231942020>.

Molinillo, S. et al. (2019) 'Smart city communication via social media: Analysing residents' and visitors' engagement', *Cities*, 94, pp. 247-255. doi: <https://doi.org/10.1016/j.cities.2019.06.003>.

Mukhtar, N., Abid Khan, M. and Chiragh, N. (2018) 'Lexicon-based approach outperforms Supervised Machine Learning approach for Urdu Sentiment Analysis in multiple domains', *Telematics and Informatics*, 35 (8), pp. 2173-2183. doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1016/j.tele.2018.08.003>.

Nakayama, M. and Wan, Y. (2019) 'The cultural impact on social commerce: A sentiment analysis on Yelp ethnic restaurant reviews', *Information & Management*, 56 (2), pp. 271-279. doi: <https://doi.org/10.1016/j.im.2018.09.004>.

Nazari, N. et al. (2020) 'The effect of customer satisfaction on the performance of the small and medium-sized hotels', *Tourism and Hospitality Management*, 26 (1), pp. 69-96. doi: <https://doi.org/10.20867/thm.26.1.5>.

Nicolaou, A. (2016) *Amazon launches music streaming service*. Available at: <https://www.ft.com/content/5d0f5152-9086-11e6-a72e-b428cb934b78> (Accessed: 15th of May in 2021).

Ofcom (2017) *Adults' media use and attitudes Report 2017*. Available at: https://www.ofcom.org.uk/_data/assets/pdf_file/0020/102755/adults-media-use-attitudes-2017.pdf (Accessed: 29th of October in 2020).

Office of Management and Budget (2017) *North American Industry Classification System*. Available at: https://www.census.gov/eos/www/naics/2017NAICS/2017_NAICS_Manual.pdf (Accessed: 26th of January in 2021)

Park, J. and Kwon Lee, B. (2021) 'An opinion-driven decision-support framework for benchmarking hotel service', *Omega*, 103, pp. 1-20. doi: <https://doi.org/10.1016/j.omega.2021.102415>.

Prabhakaran, S. (2018) *Topic Modeling with Gensim [Python]*. Available at: <https://www.machinelearningplus.com/nlp/topic-modeling-gensim-python/> (Accessed: 15th of May in 2021).

Rajabion, L. (2018) 'Application and Adoption of Big Data Technologies in SMEs', *International Conference on Computational Science and Computational Intelligence (CSCI)*. Las Vegas (USA), 12th to 14th of December in 2018. New Jersey: Institute of Electrical and Electronics Engineers, pp. 1133-1135.

Rehurek, R. (2021) '*model.coherencemodel - Topic coherence pipeline*'. Available at: <https://radimrehurek.com/gensim/models/coherencemodel.html> (Accessed: 15th of May in 2021).

Resyanto, F., Sibaroni, Y. and Romadhony, A. (2019), 'Choosing The Most Optimum Text Preprocessing Method for Sentiment Analysis: Case: iPhone Tweets', *Fourth International Conference on Informatics and Computing (ICIC)*, Semarang (Indonesia), 16th to 17th of October in 2019. New Jersey: Institute of Electrical and Electronics Engineers, pp. 1-5.

Rustam, F. et al. (2019) 'Tweets Classification on the Base of Sentiments for US Airline Companies', *Entropy*, 21 (11), pp. 1078-1099. doi: <https://doi.org/10.3390/e21111078>.

Röder, M., Both, A. and Hinneburg, A. (2015) 'Exploring the Space of Topic Coherence Measures', *WSDM '15: Proceedings of the Eight ACM International Conference on Web Search and Data Mining*. Shanghai (China), 31st of January to 6th of February in 2015. New York City (USA): Association for Computing Machinery, pp. 399-408.

Singh, R. and Verma, H. (2020) 'Effective Parallel Processing Social Media Analytics Framework', *Journal of King Saud University - Computer and Information Sciences*, Not yet assigned to volumes/issues, pp. 1-11. doi: <https://doi.org/10.1016/j.jksuci.2020.04.019>.

Statista Research Department (2016a) *Digital presence of U.S. SMEs 2016*. Available at: <https://www.statista.com/statistics/680426/sme-owners-digital-presence-usa/> (Accessed: 27th of January in 2021)

Statista Research Department (2016b) *Share of domestic and international visitors to Las Vegas from 2006 to 2015*. Available at: <https://www.statista.com/statistics/480962/las-vegas-international-and-domestic-visitors/> (Accessed: 15th of May in 2021).

Tanantong, T., Sanglerdsinlapachai, N. and Donkhampai, U. (2020) 'Sentiment Classification on Thai Social Media Using a Domain-Specific Trained Lexicon', *17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, Phuket (Thailand), 24th to 27th of June in 2020. New Jersey: Institute of Electrical and Electronics Engineers, pp. 580-583.

Twitter (2021a) *Twitter's search endpoints*. Available at: <https://developer.twitter.com/en/docs/twitter-api/search-overview> (Accessed: 16th of January in 2021).

Twitter (2021b) *Premium search APIs*. Available at: <https://developer.twitter.com/en/docs/twitter-api/premium/search-api/api-reference/premium-search#Pagination> (Accessed: 29th of January in 2021).

Ulwick, A. (2005) *'What Customers Want: Using Outcome-Driven Innovation to Create Breakthrough Products and Services'*. New York City (USA): McGraw-Hill Education Ltd.

Vayansky, I. and Kumar, S. (2020) 'A review of topic modeling methods', *Informations Systems*, 94, pp. 1-15. doi: <https://doi-org.ezproxy.uwl.ac.uk/10.1016/j.is.2020.101582>.

Venugopal, A. and Phartiyal, S. (2016) *Amazon launches Prime Video globally, leaves out China*. Available at: <https://www.reuters.com/article/us-amazon-com-prime-video-idUSKBN1431EW> (Accessed: 15th of May in 2021).

Yelp (2021) */business/{id}/reviews*. Available at: https://www.yelp.com/developers/documentation/v3/business_reviews (Accessed: 16th of January in 2021).

9.) Word count

This master dissertation comprises 15 743 words.

10.) Appendix

10.1.) Dropbox link to this master dissertation's research documents

<https://www.dropbox.com/sh/5m26nkixkeep73/AADurvHps4vBGIMITrigl8zua?dl=0>

10.2.) Supplementary illustrations

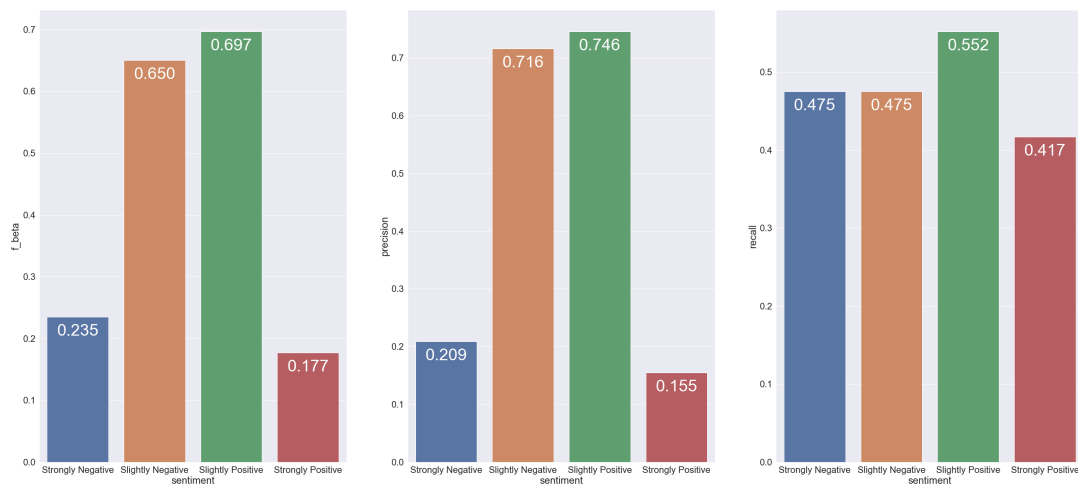


Figure 17:

Classification performance of strongly as well as slightly positive and negative sentiment (20 000 reviews - training set portion = 10%)

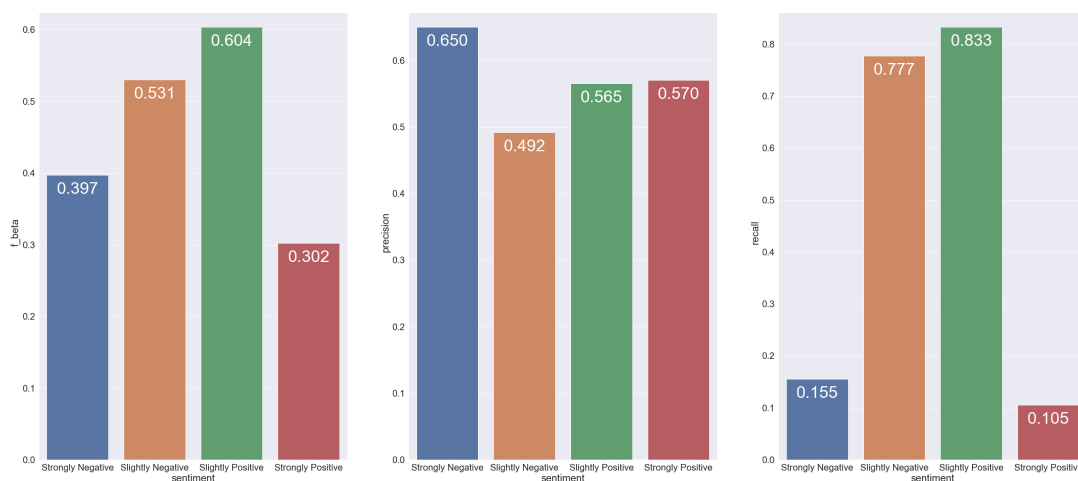


Figure 18:

Classification performance of strongly as well as slightly positive and negative sentiment (200 000 reviews - training set portion = 99%)

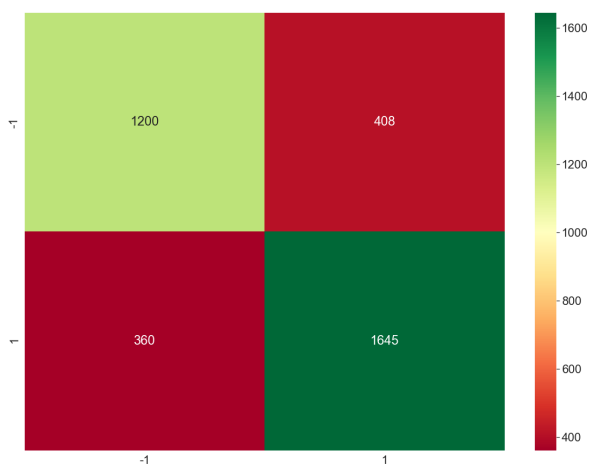


Figure 19:
Confusion matrix
of positivity and negativity
(20 000 reviews -
training set portion = 10%)

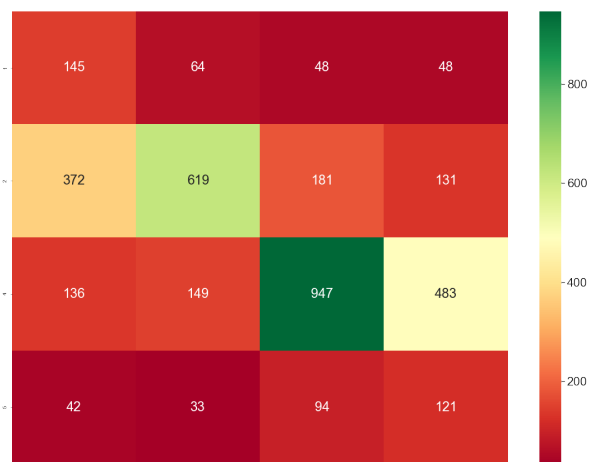


Figure 20:
Confusion matrix of slight as well as
strong positivity and negativity
(20 000 reviews -
training set portion = 10%)

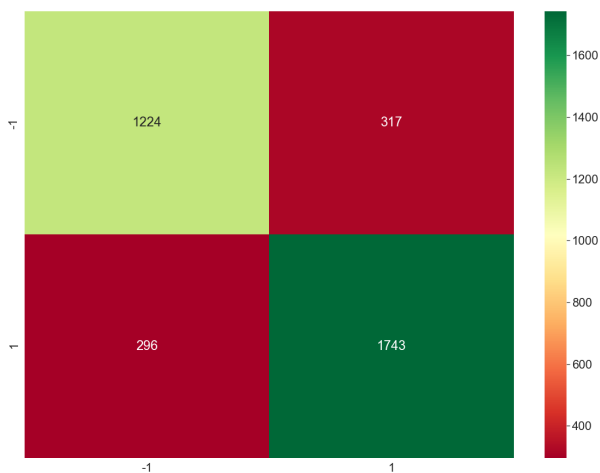


Figure 21:
Confusion matrix
of positivity and negativity
(200 000 reviews -
training set portion = 99%)

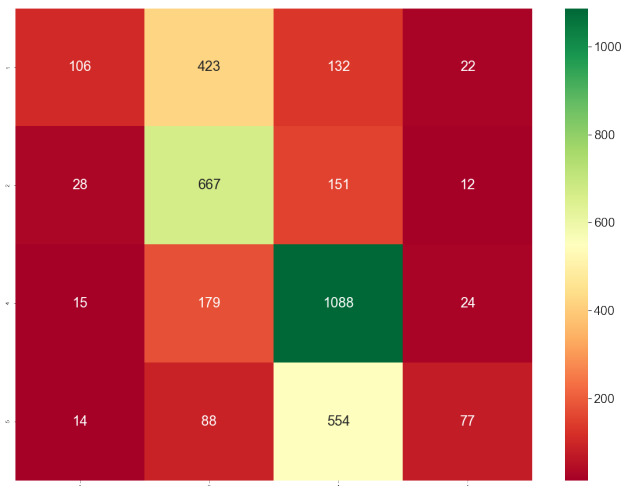


Figure 22:
Confusion matrix of slight as well as
strong positivity and negativity
(200 000 reviews -
training set portion = 99%)

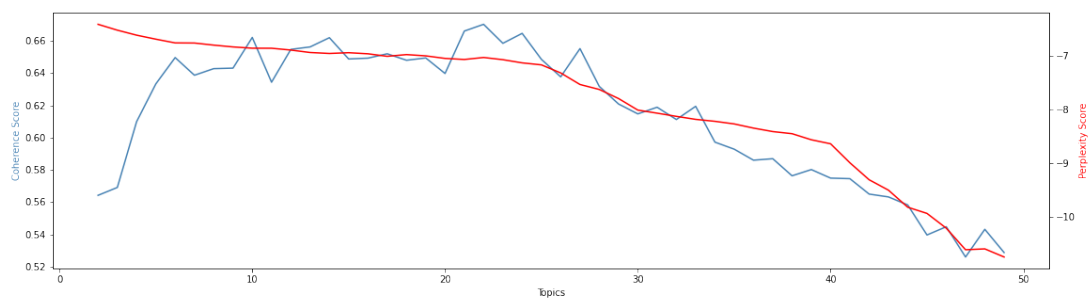


Figure 24:
Coherence and perplexity scores
over the topics of the topic category bar
(20 000 reviews - training set portion = 10%)

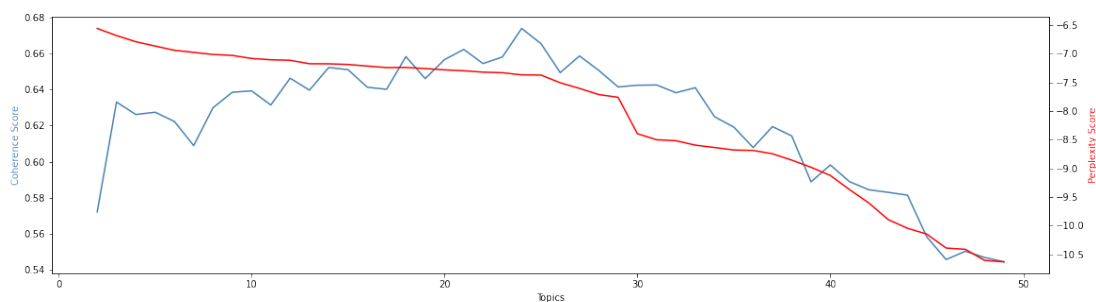


Figure 25:
Coherence and perplexity scores
over the topics of the topic category entertainment
(20 000 reviews - training set portion = 10%)

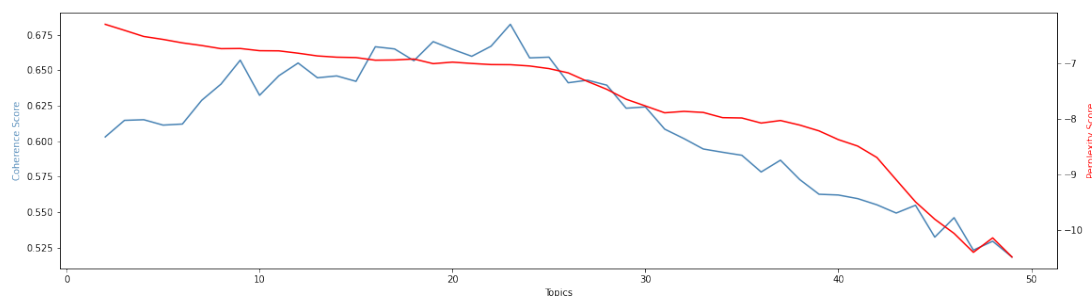


Figure 26:
Coherence and perplexity scores
over the topics of the topic category wedding
(20 000 reviews - training set portion = 10%)

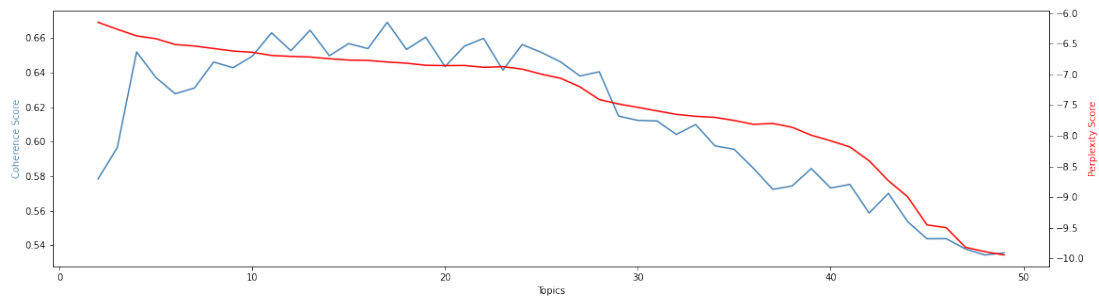


Figure 27:
Coherence and perplexity scores
over the topics of the topic category bar
(200 000 reviews - training set portion = 99%)

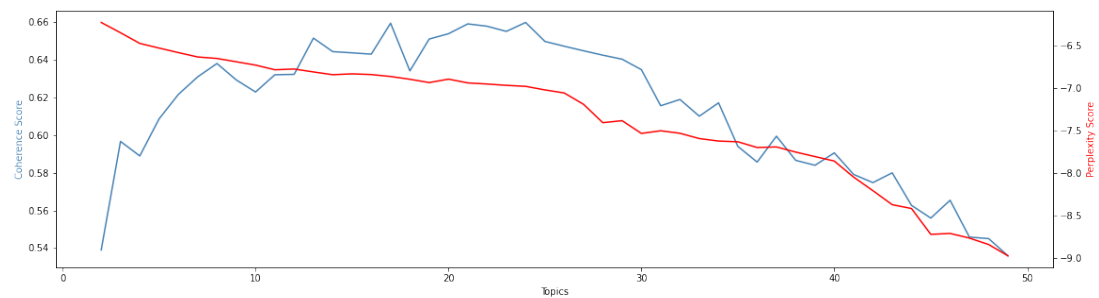


Figure 28:
Coherence and perplexity scores
over the topics of the topic category entertainment
(200 000 reviews - training set portion = 99%)

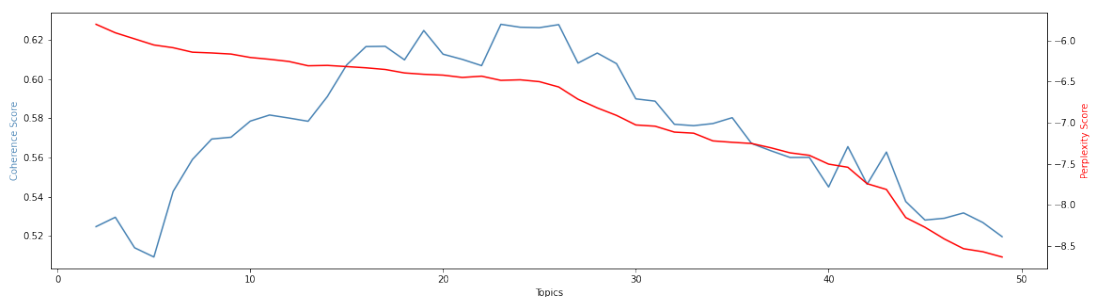


Figure 29:
Coherence and perplexity scores
over the topics of the topic category wedding
(200 000 reviews - training set portion = 99%)

Topic	Topic Category	Topic Name	Service Topic?	Top Ten Words
0	Bar	Strip club	Yes	0.097**club" + 0.054**amaz" + 0.063**strip" + 0.029**pink" + 0.025**seafood" + 0.025**recommend" + 0.025**listen" + 0.024**honor" + 0.024**lit" + 0.020**crave"
1	Bar	Eating shrimps under the moonlight	No	0.089**state" + 0.049**invit" + 0.033**moon" + 0.033**dogg" + 0.027**shrimp" + 0.022**therapi" + 0.022**underground" + 0.017**mob" + 0.017**self" + 0.017**art"
2	Bar	Lounge nudity	Yes	0.346**lounge" + 0.310**phenix" + 0.024**amaz" + 0.013**always" + 0.013**leav" + 0.013**underwear" + 0.011**post" + 0.009**ss" + 0.008**hug" + 0.007**lit"
3	Bar	Dancing in the kitchen with friends	No	0.190**great" + 0.120**favorit" + 0.065**danc" + 0.042**kitchen" + 0.035**candi" + 0.030**enjoy" + 0.026**anoth" + 0.023**friend" + 0.022**iyan" + 0.022**aloha"
4	Bar	Price performance ratio of eating	Yes	0.146**hour" + 0.061**cool" + 0.043**imao" + 0.041**thank" + 0.038**great" + 0.032**dollar" + 0.028**yum" + 0.017**variety" + 0.027**summer" + 0.024**pay"
5	Bar	Magic in the library	Yes	0.132**live" + 0.064**recip" + 0.033**tomorrow" + 0.022**team" + 0.017**librar" + 0.017**car" + 0.017**magic" + 0.016**record" + 0.011**beep"
6	Bar	Drinking shots with friends	No	0.195**friend" + 0.034**shot" + 0.032**aug" + 0.027**david" + 0.022**bless" + 0.021**high" + 0.016**toller" + 0.016**jade" + 0.016**somebodi" + 0.016**prime"
7	Bar	To waitress	Yes	0.243**love" + 0.126**fuck" + 0.034**k" + 0.027**wore" + 0.027**scene" + 0.024**tom" + 0.020**bow" + 0.020**salad" + 0.020**told" + 0.018**pizza"
8	Bar	Fashion vendors	No	0.134**beauti" + 0.047**super" + 0.045**someon" + 0.019**fashio" + 0.019**vendor" + 0.019**winner" + 0.015**hour" + 0.015**season" + 0.015**media" + 0.015**yesterday"
9	Bar	Grilling by a restaurant	Yes	0.343**grill" + 0.086**said" + 0.039**viva" + 0.032**cafe" + 0.025**applebe" + 0.022**self" + 0.014**pub" + 0.014**compan" + 0.012**fuck"
10	Bar	Singing at the dinner table	Yes	0.302**pm" + 0.021**dine" + 0.021**aria" + 0.021**hella" + 0.021**nobodi" + 0.021**sing" + 0.016**puff" + 0.016**affil" + 0.015**write" + 0.011**intent"
11	Bar	Golf & brewery tour	Yes	0.074**money" + 0.064**call" + 0.058**bacon" + 0.042**golf" + 0.037**puff" + 0.021**nobodi" + 0.021**tour" + 0.021**breweri" + 0.021**speak" + 0.021**bomb" + 0.016**amaz"
12	Bar	Roasting pineapples live	Yes	0.114**always" + 0.042**cit" + 0.037**roast" + 0.033**haha" + 0.028**pineappl" + 0.028**classic" + 0.023**jewel" + 0.021**live" + 0.019**st" + 0.019**pan"
13	Bar	Retail seller's verbal communication	Yes	0.145**talk" + 0.044**season" + 0.041**ta" + 0.040**month" + 0.031**happen" + 0.030**shop" + 0.027**said" + 0.027**never" + 0.018**bubbl" + 0.017**team"
14	Bar	Afternoon dessert	No	0.125**ask" + 0.107**post" + 0.040**grand" + 0.037**pm" + 0.035**spirit" + 0.030**signatur" + 0.030**hell" + 0.025**biscuit" + 0.025**what" + 0.015**enjoy"
15	Bar	Bar ambience	No	0.109**cheer" + 0.059**secret" + 0.039**tel" + 0.037**chandeli" + 0.032**yummi" + 0.030**food" + 0.014**pm" + 0.014**screen" + 0.014**vs" + 0.014**summer"
16	Bar	Singing with the family	No	0.234**lol" + 0.157**fun" + 0.031**sing" + 0.022**cosmopolitan" + 0.018**strat" + 0.014**call" + 0.013**expert" + 0.013**fan" + 0.013**area" + 0.009**classic"
17	Bar	Bistro garden	No	0.081**alreadi" + 0.081**never" + 0.065**final" + 0.053**hold" + 0.022**perfect" + 0.022**die" + 0.021**dream" + 0.016**bistro" + 0.016**credit" + 0.012**garden"
18	Bar	Chocolate	No	0.122**best" + 0.045**told" + 0.039**cheer" + 0.032**everyth" + 0.021**chocol" + 0.020**fun" + 0.018**delici" + 0.018**featur" + 0.018**no" + 0.014**awesom"
19	Bar	Design of a chandelier	No	0.124**dont" + 0.072**art" + 0.027**design" + 0.024**definit" + 0.021**shock" + 0.021**tel" + 0.016**call" + 0.016**chandeli" + 0.016**eag" + 0.016**space"
20	Bar	Legal phone examination	No	0.060**law" + 0.058**anoth" + 0.056**phone" + 0.055**exam" + 0.055**trump" + 0.023**defin" + 0.023**shot" + 0.020**accept" + 0.015**tan" + 0.015**appreci"
21	Bar	Traditional eating style	No	0.158**shit" + 0.078**wine" + 0.043**high" + 0.029**sheet" + 0.029**rip" + 0.024**vibe" + 0.024**blast" + 0.015**tradit" + 0.015**eaten" + 0.015**style"

Figure 30:

Identified bar topics

(20 000 reviews - training set portion = 10%)

Topic	Topic Category	Topic Name	Service Topic?	Top Ten Words
0	Entertainment	Misleading show title	No	0.292**pm" + 0.029**lie" + 0.026**titi" + 0.018**manag" + 0.013**doctor" + 0.013**suppos" + 0.013**invit" + 0.010**thank" + 0.009**rsp"
1	Entertainment	Live cast of actors	Yes	0.258**live" + 0.025**opportun" + 0.022**cast" + 0.022**horror" + 0.019**citi" + 0.015**record" + 0.013**preview" + 0.013**runway" + 0.013**voic" + 0.009**trump"
2	Entertainment	Place of the theater	No	0.128**anoth" + 0.110**annual" + 0.059**great" + 0.038**theater" + 0.034**wonder" + 0.026**news" + 0.021**hook" + 0.021**ugh" + 0.017**iol" + 0.013**stadium"
3	Entertainment	Youtube series actor in the library	No	0.045**visit" + 0.036**moneys" + 0.035**ask" + 0.026**friend" + 0.026**youtub" + 0.026**seri" + 0.021**actor" + 0.021**proceed" + 0.016**flight" + 0.016**libran"
4	Entertainment	Sing & fashion style competition	Yes	0.091**featur" + 0.069**friend" + 0.064**fashion" + 0.048**trump" + 0.025**area" + 0.024**battl" + 0.020**sing" + 0.020**style" + 0.015**asada" + 0.015**flower"
5	Entertainment	Health of the viewers	Yes	0.221**great" + 0.156**shown" + 0.069**luck" + 0.049**post" + 0.027**repres" + 0.014**poor" + 0.014**admiss" + 0.014**organ" + 0.014**hold" + 0.012**everyth"
6	Entertainment	Visiting a movie apartment	Yes	0.101**said" + 0.064**movi" + 0.056**listen" + 0.048**hell" + 0.024**paid" + 0.020**plis" + 0.020**smh" + 0.020**trump" + 0.012**flat" + 0.012**visit"
7	Entertainment	Vibrance in the vision of colours	No	0.176**dont" + 0.037**record" + 0.022**state" + 0.017**liit" + 0.017**recommend" + 0.017**color" + 0.017**quick" + 0.017**vision" + 0.013**tomorrow" + 0.013**cirqui"
8	Entertainment	Magic with a car	Yes	0.148**favorit" + 0.116**alway" + 0.048**great" + 0.040**magic" + 0.025**dont" + 0.022**tue" + 0.021**afford" + 0.021**friend" + 0.021**genius" + 0.017**car"
9	Entertainment	Plot scenery	No	0.103**marketplac" + 0.074**nice" + 0.057**what" + 0.035**pay" + 0.029**agre" + 0.023**k" + 0.023**brian" + 0.017**museum" + 0.017**speak" + 0.017**design"
10	Entertainment	Sales event	Yes	0.070**happen" + 0.055**incred" + 0.047**sale" + 0.039**inspl" + 0.037**tomorrow" + 0.028**dream" + 0.028**sell" + 0.024**great" + 0.024**cultur" + 0.019**perfect"
11	Entertainment	Art of a woman	No	0.093**art" + 0.051**mrs" + 0.028**media" + 0.028**anoth" + 0.023**wrong" + 0.023**wtf" + 0.023**then" + 0.021**suck" + 0.019**desert" + 0.017**report"
12	Entertainment	Emergency call	Yes	0.097**hour" + 0.091**call" + 0.045**definit" + 0.039**inform" + 0.039**someon" + 0.034**shot" + 0.029**phone" + 0.026**everyth" + 0.015**fam" + 0.015**wonder"
13	Entertainment	Dancing show	Yes	0.083**alread" + 0.081**danc" + 0.073**welcom" + 0.048**pacific" + 0.031**told" + 0.026**haha" + 0.021**privat" + 0.021**log" + 0.016**fox" + 0.016**church"
14	Entertainment	Drought	No	0.251**best" + 0.044**la" + 0.031**fire" + 0.027**month" + 0.022**wors" + 0.018**number" + 0.014**bless" + 0.014**gm" + 0.011**fresh" + 0.010**bowl"
15	Entertainment	Talking talent	Yes	0.203**talk" + 0.101**never" + 0.081**talent" + 0.030**great" + 0.029**yesterday" + 0.027**imao" + 0.019**bout" + 0.016**listen" + 0.012**honor" + 0.012**aw"
16	Entertainment	Smoking marijuana	No	0.424**love" + 0.064**iol" + 0.041**K" + 0.019**rsp" + 0.016**desert" + 0.014**shoutout" + 0.014**candi" + 0.014**sweet" + 0.014**vape" + 0.011**weed"
17	Entertainment	Wrestling	Yes	0.069**super" + 0.045**tour" + 0.030**wrestl" + 0.030**leav" + 0.025**memori" + 0.025**greatest" + 0.025**account" + 0.015**govern" + 0.015**bello" + 0.014**fun"
18	Entertainment	Charity	Yes	0.285**amaz" + 0.098**beauti" + 0.056**honor" + 0.014**invit" + 0.010**bill" + 0.010**guard" + 0.010**definit" + 0.010**offic" + 0.010**rob" + 0.010**chanti"
19	Entertainment	Event cancellation	Yes	0.049**cancel" + 0.038**sorr" + 0.037**justic" + 0.037**evid" + 0.029**beyond" + 0.029**off" + 0.029**cheat" + 0.028**no" + 0.025**trump" + 0.017**high"
20	Entertainment	Sports team in the finals	No	0.141**season" + 0.136**team" + 0.070**final" + 0.031**citi" + 0.030**tell" + 0.023**chariti" + 0.021**iol" + 0.019**too" + 0.015**secur" + 0.012**hoard"
21	Entertainment	Excuse messages	Yes	0.212**shit" + 0.133**thank" + 0.025**strat" + 0.021**view" + 0.017**easi" + 0.014**anoth" + 0.013**excus" + 0.012**hilar" + 0.010**fun" + 0.009**messag"
22	Entertainment	Awaiting a club scene	Yes	0.149**awesom" + 0.098**tomorrow" + 0.036**club" + 0.033**studio" + 0.030**scene" + 0.024**oct" + 0.019**great" + 0.015**addict" + 0.015**crowd" + 0.015**entertain"
23	Entertainment	Summer fun with friends	No	0.208**fun" + 0.069**great" + 0.048**thank" + 0.042**summer" + 0.039**enjoy" + 0.032**die" + 0.025**dope" + 0.025**cheer" + 0.025**friend" + 0.024**cool"

Figure 31:

Identified entertainment topics

(20 000 reviews - training set portion = 10%)

Topic	Topic Category	Topic Name	Service Topic?	Top Ten Words
0	Wedding	Chapel	No	0.344**"chapel" + 0.112**"amaz" + 0.063**"post" + 0.038**"flower" + 0.029**"david" + 0.015**"haha" + 0.015**"daniel" + 0.015**"violat" + 0.011**"mon" + 0.011**"blatant"
1	Wedding	Fashion style of wedding clothes	No	0.106**"tell" + 0.069**"anoth" + 0.065**"perfect" + 0.049**"tomorrow" + 0.033**"art" + 0.033**"job" + 0.027**"nah" + 0.023**"best" + 0.022**"fashion" + 0.022**"pizza"
2	Wedding	Decoration	No	0.522**"love" + 0.034**"live" + 0.018**"state" + 0.012**"too" + 0.012**"everyth" + 0.012**"forc" + 0.009**"scene" + 0.009**"batti" + 0.009**"sugar" + 0.009**"rabbit"
3	Wedding	Fraud	Yes	0.284**"beauti" + 0.135**"fraud" + 0.099**"victim" + 0.046**"claim" + 0.036**"month" + 0.036**"obtain" + 0.033**"report" + 0.030**"final" + 0.018**"file" + 0.015**"nobodi"
4	Wedding	Desert atmosphere	No	0.085**"favorit" + 0.070**"said" + 0.041**"ia" + 0.040**"everyth" + 0.031**"desert" + 0.030**"love" + 0.024**"phone" + 0.019**"enjoy" + 0.018**"sock" + 0.018**"secur"
5	Wedding	News about an eloped aunt	No	0.310**"lol" + 0.087**"elop" + 0.058**"definit" + 0.025**"invi" + 0.017**"listen" + 0.013**"media" + 0.013**"aunt" + 0.013**"intent" + 0.013**"cool" + 0.009**"love"
6	Wedding	Traditional gun of the grandparents	No	0.059**"yesterday" + 0.049**"gun" + 0.031**"favourit" + 0.031**"tradit" + 0.025**"grand" + 0.019**"moder" + 0.019**"stun" + 0.018**"post" + 0.018**"violenc"
7	Wedding	Wedding entertainment	Yes	0.056**"agre" + 0.050**"lash" + 0.031**"should" + 0.030**"jake" + 0.025**"sometim" + 0.024**"strip" + 0.019**"favorit" + 0.019**"bouquet" + 0.019**"entertain" + 0.019**"classic"
8	Wedding	Joking about marriage	No	0.195**"fuck" + 0.170**"never" + 0.060**"thank" + 0.044**"die" + 0.026**"rich" + 0.025**"bless" + 0.022**"nice" + 0.019**"joke" + 0.018**"send" + 0.016**"said"
9	Wedding	Personality of the wedding partner	No	0.184**"month" + 0.182**"someone" + 0.038**"legal" + 0.030**"later" + 0.024**"portrait" + 0.024**"happier" + 0.019**"visit" + 0.014**"rose" + 0.014**"shame" + 0.013**"broke"
10	Wedding	Wedding photography	Yes	0.291**"dont" + 0.084**"alreadi" + 0.038**"gorgeous" + 0.028**"discov" + 0.025**"news" + 0.024**"what" + 0.018**"photograph" + 0.014**"hilar" + 0.014**"crimin" + 0.014**"style"
11	Wedding	Dressing up like a bride for prom	Yes	0.184**"fun" + 0.146**"k" + 0.075**"brian" + 0.022**"blast" + 0.022**"understand" + 0.022**"obsess" + 0.016**"prom" + 0.016**"kim" + 0.016**"wifey" + 0.011**"desk"
12	Wedding	Elvis performance in a chapel	Yes	0.167**"chapel" + 0.137**"graceland" + 0.127**"elvi" + 0.068**"crystal" + 0.020**"fail" + 0.020**"bag" + 0.018**"said" + 0.015**"paper" + 0.015**"attorney" + 0.010**"justic"
13	Wedding	Exclusivity of the ceremony	Yes	0.056**"ceremoni" + 0.045**"no" + 0.036**"instead" + 0.035**"privat" + 0.033**"leav" + 0.031**"cool" + 0.031**"plis" + 0.030**"danc" + 0.026**"summer" + 0.021**"crime"
14	Wedding	Health of the customers	Yes	0.346**"friend" + 0.235**"best" + 0.019**"team" + 0.012**"everyth" + 0.011**"sorri" + 0.011**"health" + 0.011**"cusion" + 0.011**"honor" + 0.010**"live"
15	Wedding	Shop for wedding utensils	No	0.080**"awesom" + 0.070**"trump" + 0.065**"wonder" + 0.036**"afford" + 0.030**"shop" + 0.022**"featur" + 0.022**"ugli" + 0.022**"recommend" + 0.021**"practic" + 0.015**"law"
16	Wedding	Renewing the marriage	Yes	0.090**"licens" + 0.084**"imao" + 0.051**"renew" + 0.040**"bureau" + 0.034**"suck" + 0.033**"hour" + 0.028**"princess" + 0.023**"bridelob" + 0.017**"jason" + 0.017**"pay"
17	Wedding	Engagement memories	No	0.150**"engag" + 0.055**"movi" + 0.042**"call" + 0.037**"memori" + 0.031**"cook" + 0.031**"hold" + 0.025**"promis" + 0.023**"number" + 0.018**"poor" + 0.018**"energi"
18	Wedding	Organising a wedding	Yes	0.161**"shit" + 0.061**"citi" + 0.054**"live" + 0.039**"wouldnt" + 0.034**"cute" + 0.022**"wore" + 0.017**"said" + 0.011**"wine" + 0.011**"compani" + 0.011**"hook"
19	Wedding	Reflections on wedding dream(s)	No	0.176**"great" + 0.160**"ask" + 0.055**"dream" + 0.034**"told" + 0.031**"high" + 0.028**"lie" + 0.016**"spous" + 0.015**"memori" + 0.014**"final" + 0.012**"wtf"
20	Wedding	Unprofessional wedding photographer	Yes	0.119**"happen" + 0.041**"season" + 0.038**"cheat" + 0.032**"imma" + 0.032**"color" + 0.027**"quick" + 0.016**"wrong" + 0.016**"shed" + 0.016**"shot" + 0.012**"luck"
21	Wedding	Dealing with attention during a wedding	No	0.156**"alway" + 0.069**"museum" + 0.051**"money" + 0.043**"honor" + 0.039**"neon" + 0.030**"propos" + 0.022**"suppos" + 0.017**"avoid" + 0.017**"situat" + 0.017**"mob"
22	Wedding	Valentine's day photography	Yes	0.082**"talk" + 0.042**"valentin" + 0.034**"mrs" + 0.021**"amaz" + 0.020**"fuck" + 0.018**"photograph" + 0.017**"rental" + 0.013**"later" + 0.013**"club" + 0.013**"garante"

Figure 32:

Identified wedding topics

(20 000 reviews - training set portion = 10%)

Topic	Topic Category	Topic Name	Service Topic?	Top Ten Words
0	Bar	Art of cooking	Yes	0.101**perform" + 0.091**vega" + 0.065**art" + 0.050**kitchen" + 0.036**card" + 0.027**tour" + 0.020**david" + 0.020**dream" + 0.019**honor"
1	Bar	Evening preferences for groceries	No	0.177**lol" + 0.053**state" + 0.027**shrimp" + 0.027**noth" + 0.024**pm" + 0.023**moon" + 0.019**appreci" + 0.016**secret" + 0.016**mike" + 0.016**whiskey"
2	Bar	Dancing	Yes	0.130**great" + 0.049**danc" + 0.044**tell" + 0.033**strip" + 0.029**puff" + 0.021**enjoy" + 0.018**rip" + 0.018**beauti" + 0.014**haha" + 0.014**live"
3	Bar	Nightlife	Yes	0.252**grill" + 0.187**love" + 0.047**high" + 0.043**said" + 0.021**signatur" + 0.019**spirit" + 0.016**nightlif" + 0.015**selfi" + 0.014**pm" + 0.011**lil"
4	Bar	Oyster bar	Yes	0.472**vega" + 0.067**pm" + 0.045**oyster" + 0.022**live" + 0.021**best" + 0.017**craft" + 0.013**bourbon" + 0.013**pizza" + 0.012**menu" + 0.011**sheet"
5	Bar	Legal disputes	No	0.060**call" + 0.055**bitch" + 0.045**law" + 0.041**phone" + 0.037**told" + 0.026**min" + 0.023**no" + 0.019**defin" + 0.019**d" + 0.015**legal"
6	Bar	Lounge nudity	Yes	0.101**always" + 0.084**lounge" + 0.076**amaz" + 0.054**ask" + 0.031**bacon" + 0.030**vega" + 0.019**underwear" + 0.019**front" + 0.016**told" + 0.016**awesom"
7	Bar	Drink & drive	No	0.180**fuck" + 0.077**best" + 0.043**celebr" + 0.022**bless" + 0.018**car" + 0.013**offic" + 0.013**protein" + 0.013**compani" + 0.013**sunset" + 0.013**hangov"
8	Bar	Singing & snacks	No	0.045**sing" + 0.044**someon" + 0.041**patio" + 0.040**hold" + 0.035**foodi" + 0.031**vega" + 0.028**happen" + 0.023**delici" + 0.020**heart" + 0.019**yummi"
9	Bar	Fire screen	No	0.138**vega" + 0.091**post" + 0.062**beauti" + 0.036**season" + 0.027**thank" + 0.019**live" + 0.018**super" + 0.015**welcom" + 0.015**fire" + 0.012**screen"
10	Bar	Taste of wine	No	0.070**cool" + 0.046**juic" + 0.028**secret" + 0.020**great" + 0.019**thank" + 0.019**she" + 0.019**everyth" + 0.019**master" + 0.019**vintag" + 0.016**delici"
11	Bar	Traditional wine	No	0.038**yesterday" + 0.025**wine" + 0.025**tradit" + 0.025**xoxo" + 0.025**number" + 0.019**accept" + 0.019**mapl" + 0.019**joe" + 0.013**easi" + 0.013**afford"
12	Bar	Karaoke	Yes	0.098**karaok" + 0.089**wait" + 0.051**rock" + 0.041**tomorrow" + 0.034**nightclub" + 0.031**manag" + 0.027**shit" + 0.027**said" + 0.026**chocol" + 0.025**huge"
13	Bar	Bar atmosphere	No	0.141**fun" + 0.091**favorit" + 0.087**shit" + 0.020**cri" + 0.020**movi" + 0.020**vibe" + 0.020**live" + 0.017**sweet" + 0.014**lol"
14	Bar	Affordability of wine	No	0.076**never" + 0.065**wine" + 0.048**money" + 0.045**k" + 0.028**nice" + 0.020**brian" + 0.020**realiz" + 0.017**ask" + 0.016**bubbl" + 0.016**what"
15	Bar	Brew shop	No	0.429**lounge" + 0.040**left" + 0.029**hell" + 0.026**candi" + 0.026**lil" + 0.016**brew" + 0.016**sad" + 0.013**shop" + 0.013**enjoy" + 0.007**hoppin"
16	Bar	Interior design of a pub	No	0.161**friend" + 0.032**this" + 0.032**pub" + 0.032**great" + 0.029**everyth" + 0.028**chandell" + 0.024**dollar" + 0.022**best" + 0.022**fashion" + 0.021**pay"

Figure 33:
Identified bar topics
(200 000 reviews - training set portion = 99%)

Topic	Topic Category	Topic Name	Service Topic?	Top Ten Words
0	Entertainment	Singing performance	Yes	0.066""song" + 0.052""wonder" + 0.045""noti" + 0.037""someone" + 0.032""haha" + 0.031""wont" + 0.025""privat" + 0.025""pay" + 0.019""singer"
1	Entertainment	Acting performance	Yes	0.123""never" + 0.080""movi" + 0.076""ask" + 0.063""incred" + 0.037""opportun" + 0.032""act" + 0.027""brian" + 0.021""actor" + 0.011""hant" + 0.011""hell"
2	Entertainment	Corporate celebration	Yes	0.419""great" + 0.044""celebr" + 0.043""fun" + 0.020""than" + 0.017""alway" + 0.014""wors" + 0.014""afford" + 0.012""tomorrow" + 0.010""corpor" + 0.010""gm"
3	Entertainment	Costume design	No	0.487""love" + 0.035""fashion" + 0.025""everyth" + 0.019""huge" + 0.018""thank" + 0.013""pose" + 0.013""dead" + 0.010""friend" + 0.009""mall" + 0.008""cit"
4	Entertainment	Studio emotions	No	0.043""cool" + 0.042""tell" + 0.041""pm" + 0.037""epic" + 0.036""dic" + 0.031""emot" + 0.031""memori" + 0.026""studio" + 0.025""sad"
5	Entertainment	Advertisement for buying theater tickets	Yes	0.176""pm" + 0.154""vega" + 0.058""friend" + 0.035""theater" + 0.035""tomorrow" + 0.021""citi" + 0.021""sale" + 0.018""lheat" + 0.015""celebr" + 0.015""valley"
6	Entertainment	Cinematic effects	Yes	0.128""said" + 0.112""K" + 0.059""what" + 0.043""le" + 0.037""hell" + 0.032""earth" + 0.025""quick" + 0.022""hold" + 0.022""movi" + 0.016""rich"
7	Entertainment	Decor design	No	0.169""lol" + 0.074""art" + 0.071""nice" + 0.047""heart" + 0.019""suck" + 0.018""nfl" + 0.014""flower" + 0.014""left" + 0.014""comfort" + 0.014""sell"
8	Entertainment	Producing a score	Yes	0.115""this" + 0.095""happen" + 0.044""no" + 0.034""east" + 0.034""instead" + 0.023""jo" + 0.023""mike" + 0.023""score" + 0.017""excus" + 0.017""fresh"
9	Entertainment	Plot comedy	Yes	0.104""beauti" + 0.057""raider" + 0.027""nfl" + 0.026""comed" + 0.026""poor" + 0.026""phone" + 0.021""state" + 0.021""smh" + 0.016""report" + 0.016""legend"
10	Entertainment	Entertainment craft	No	0.327""best" + 0.029""left" + 0.024""thank" + 0.023""craft" + 0.023""offic" + 0.017""bing" + 0.017""caus" + 0.016""truli" + 0.012""wors" + 0.012""disabl"
11	Entertainment	Thoughts about (a) plot(s)	No	0.273""venu" + 0.037""fire" + 0.027""page" + 0.016""featur" + 0.016""compani" + 0.016""stole" + 0.016""thoma" + 0.011""chemic" + 0.011""contract" + 0.011""answe"
12	Entertainment	The fox as figure in history and culture	No	0.283""amaz" + 0.045""honor" + 0.041""histori" + 0.030""cancel" + 0.028""agre" + 0.024""thank" + 0.023""fox" + 0.018""cultur" + 0.018""greatest" + 0.017""vip"
13	Entertainment	Asking in an unordinary fashion for a nightclub dance	No	0.056""off" + 0.048""color" + 0.035""definit" + 0.031""tribut" + 0.028""never" + 0.025""jon" + 0.024""fashion" + 0.019""ask" + 0.018""nightclub" + 0.013""danc"
14	Entertainment	Live performance	Yes	0.210""live" + 0.158""vega" + 0.077""perform" + 0.017""weed" + 0.014""candi" + 0.012""sweet" + 0.011""celebr" + 0.011""voic" + 0.011""car"
15	Entertainment	Security staff dealing with inappropriate behaviour	Yes	0.103""call" + 0.063""tomorrow" + 0.047""high" + 0.046""yesterday" + 0.031""secur" + 0.031""dope" + 0.031""inform" + 0.016""stone" + 0.011""fox" + 0.011""pee"
16	Entertainment	Comedy performance supported by laser lights	Yes	0.179""shit" + 0.159""fun" + 0.049""alway" + 0.041""welcom" + 0.015""comed" + 0.014""perform" + 0.014""bellagio" + 0.014""laser" + 0.014""pie" + 0.011""honor"
17	Entertainment	Comedy rock album	No	0.112""thank" + 0.077""featur" + 0.071""comed" + 0.062""rock" + 0.026""album" + 0.021""send" + 0.020""blit" + 0.019""hilar" + 0.017""entertain" + 0.016""dollar"
18	Entertainment	Seminar for singing	Yes	0.108""luck" + 0.077""post" + 0.053""magic" + 0.047""announce" + 0.031""sing" + 0.021""clip" + 0.017""vega" + 0.016""seminar" + 0.016""showroom" + 0.016""slam"
19	Entertainment	An interview about the dream of dancing	No	0.237""season" + 0.062""danc" + 0.051""dream" + 0.033""told" + 0.023""interview" + 0.022""till" + 0.017""pleasur" + 0.017""mistak" + 0.017""wayn" + 0.017""reach"
20	Entertainment	Amusement tour	Yes	0.106""vega" + 0.061""enjoy" + 0.061""inspir" + 0.052""invit" + 0.041""tour" + 0.036""down" + 0.031""museum" + 0.026""money" + 0.026""firework" + 0.021""neon"
21	Entertainment	Advertising tickets with friends for a charity event	Yes	0.211""wait" + 0.128""favorit" + 0.081""final" + 0.021""number" + 0.017""admiss" + 0.017""space" + 0.017""health" + 0.017""sale" + 0.014""friend" + 0.013""charit"
22	Entertainment	A detective play	Yes	0.571""vega" + 0.019""speak" + 0.018""record" + 0.016""justic" + 0.015""truth" + 0.015""sori" + 0.015""determin" + 0.015""cheat" + 0.013""evid" + 0.007""someone"
23	Entertainment	Feedback on a tour payment process	Yes	0.147""awesom" + 0.058""super" + 0.032""too" + 0.027""favorit" + 0.023""tli" + 0.023""shout" + 0.021""wif" + 0.018""credit" + 0.018""paid"

Figure 34:

Identified entertainment topics

(200 000 reviews - training set portion = 99%)

Topic	Topic Category	Topic Name	Service Topic?	Top Ten Words
0	Wedding	Renewing the marriage vow	Yes	0.091**"vow" + 0.086**"thank" + 0.071**"renew" + 0.051**"tomorrow" + 0.045**"old" + 0.025**"vega" + 0.020**"garden" + 0.020**"hilar" + 0.019**"marriage"
1	Wedding	Disputes about real estate	No	0.274**"fuck" + 0.163**"amaz" + 0.045**"noth" + 0.028**"jana" + 0.023**"super" + 0.017**"blitch" + 0.014**"said" + 0.014**"bag" + 0.014**"steve"
2	Wedding	Broken marriage	No	0.127**"k" + 0.096**"live" + 0.064**"love" + 0.050**"no" + 0.025**"marriage" + 0.024**"broke" + 0.019**"beast" + 0.019**"sock" + 0.019**"privat"
3	Wedding	Misbehaving actor at a wedding	No	0.142**"marriage" + 0.039**"legal" + 0.039**"state" + 0.025**"job" + 0.023**"actor" + 0.019**"yell" + 0.013**"protect" + 0.013**"bye" + 0.013**"secur"
4	Wedding	Beauty of a chapel	No	0.229**"beaut" + 0.099**"chapel" + 0.090**"graceland" + 0.071**"post" + 0.040**"love" + 0.029**"season" + 0.019**"vega" + 0.016**"elvi" + 0.016**"eric" + 0.013**"hold"
5	Wedding	Wedding expenditures	Yes	0.455**"marriage" + 0.048**"money" + 0.029**"agre" + 0.019**"send" + 0.017**"said" + 0.013**"best" + 0.013**"front" + 0.012**"wine" + 0.011**"live"
6	Wedding	Waiting for a marriage license	Yes	0.182**"wait" + 0.092**"marriage" + 0.063**"licens" + 0.037**"cool" + 0.035**"neon" + 0.033**"museum" + 0.032**"bureau" + 0.027**"elop" + 0.026**"poke" + 0.018**"steer"
7	Wedding	Ceremony with Elvis photography	Yes	0.500**"vega" + 0.211**"love" + 0.057**"fun" + 0.023**"photographi" + 0.022**"elvi" + 0.010**"ceremoni" + 0.008**"enjoy" + 0.007**"d" + 0.006**"memori" + 0.006**"bear"
8	Wedding	Eloped woman	No	0.097**"blitch" + 0.083**"elop" + 0.042**"invit" + 0.034**"too" + 0.030**"afford" + 0.030**"avoid" + 0.024**"princess" + 0.024**"health" + 0.024**"marriage" + 0.018**"suck"
9	Wedding	Marrying at the museum	No	0.106**"happen" + 0.072**"love" + 0.047**"awesom" + 0.037**"rich" + 0.036**"museum" + 0.026**"marriage" + 0.019**"bec" + 0.019**"recommend" + 0.014**"maric" + 0.014**"entertain"
10	Wedding	Sexual violence within a marriage	No	0.102**"ask" + 0.101**"marriage" + 0.095**"assault" + 0.063**"emot" + 0.059**"sexual" + 0.051**"sexual" + 0.046**"claim" + 0.043**"fals" + 0.038**"report" + 0.027**"movi"
11	Wedding	Features of a singer	Yes	0.110**"brian" + 0.071**"bless" + 0.065**"song" + 0.039**"favourit" + 0.039**"crime" + 0.032**"sexual" + 0.024**"phantom" + 0.024**"secret" + 0.023**"adventur" + 0.016**"vegg"
12	Wedding	Wedding ring	No	0.328**"lol" + 0.168**"ring" + 0.027**"huge" + 0.023**"shop" + 0.016**"ceremoni" + 0.014**"later" + 0.014**"mansion" + 0.014**"appar" + 0.014**"ill" + 0.009**"decor"
13	Wedding	Appearance of women at the marriage	No	0.200**"woman" + 0.070**"marriage" + 0.069**"ci" + 0.032**"venu" + 0.021**"everyth" + 0.021**"loral" + 0.016**"vega" + 0.015**"hair" + 0.015**"pure" + 0.015**"wit"
14	Wedding	Wedding expectations	No	0.137**"shit" + 0.050**"marriage" + 0.043**"said" + 0.039**"perfect" + 0.039**"wouldnt" + 0.039**"gorgeous" + 0.034**"david" + 0.022**"live" + 0.018**"love" + 0.017**"hope"
15	Wedding	Omitting the goal of marrying	No	0.179**"never" + 0.084**"marriage" + 0.067**"love" + 0.047**"sad" + 0.046**"caus" + 0.038**"elvi" + 0.037**"left" + 0.025**"nice" + 0.025**"fail"
16	Wedding	Dancing in a chapel	Yes	0.364**"chapel" + 0.221**"vega" + 0.034**"lower" + 0.022**"danc" + 0.011**"illeg" + 0.011**"intent" + 0.011**"somebod" + 0.011**"instead" + 0.008**"shed" + 0.008**"final"
17	Wedding	Traditional values in a marriage	No	0.083**"honor" + 0.070**"someone" + 0.061**"everyth" + 0.050**"suppos" + 0.029**"crimin" + 0.029**"tradit" + 0.029**"wont" + 0.027**"marriage" + 0.022**"ray"
18	Wedding	Happiness among the best friends due to the wedding	No	0.286**"friend" + 0.199**"best" + 0.143**"great" + 0.041**"dream" + 0.013**"what" + 0.011**"venu" + 0.010**"tour" + 0.010**"phone" + 0.010**"realiz" + 0.010**"glad"
19	Wedding	Wedding celebration style	Yes	0.230**"celebr" + 0.109**"vega" + 0.078**"favorit" + 0.029**"quick" + 0.018**"later" + 0.018**"wild" + 0.018**"romant" + 0.018**"hell" + 0.015**"friend"
20	Wedding	Becoming engaged	Yes	0.133**"engag" + 0.070**"marriage" + 0.037**"love" + 0.032**"art" + 0.025**"color" + 0.022**"cook" + 0.016**"inspir" + 0.016**"bitt" + 0.016**"bond" + 0.016**"paper"
21	Wedding	Mother's feelings about a marriage	No	0.165**"alway" + 0.122**"someone" + 0.039**"chil" + 0.036**"yesterday" + 0.031**"mother" + 0.030**"marriage" + 0.026**"heart" + 0.022**"happier" + 0.018**"said" + 0.018**"dick"
22	Wedding	Beautiful hair when being photographed	Yes	0.153**"hair" + 0.094**"photographi" + 0.086**"tear" + 0.051**"call" + 0.036**"final" + 0.030**"beaut" + 0.027**"ask" + 0.021**"nobod" + 0.016**"hook" + 0.016**"woke"

Figure 35:

Identified wedding topics

(200 000 reviews - training set portion = 99%)

10.3.) Code of the two selected opportunity systems

..... Start - 1 Business data to Business ID

In[1]:

#Importing the libraries

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

In[2]:

#Reading & Converting the JSON file into a pandas dataframe (r=Raw String)

business_df=pd.read_json(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\1 Yelp Data\business.json", lines=True)

In[3]:

#Select businesses which are still open for the most up to date sentiment dictionary

business_df = business_df[business_df["is_open"]==1]

In[4]:

#Inspecting the columns of the dataset

business_df.head()

In[5]:

#Counting the open businesses per city

city_business_number=business_df["city"].value_counts()

In[6]:

#Selecting the Top 10 cities with open businesses

city_business_number = city_business_number[:10,]

In[7]:

#Plotting the Top 10 Cities with open businesses

plt.figure(figsize=(15, 5))

sbar=sns.barplot(x=city_business_number.index,y=city_business_number.values)

```

#Annotating the plot
for p in sbar.patches:
    sbar.annotate(format(int(p.get_height()), "1.0f"),
                  (p.get_x() + p.get_width() / 2., p.get_height()),
                  ha = "center", va = "center", fontsize=15,
                  xytext = (0, -15), color="white",
                  textcoords = "offset points")

plt.title("Top 10 Cities with the most registered Businesses", fontsize=24)
plt.ylabel("Businesses", fontsize=12)
plt.xlabel("City", fontsize=12)

#Displaying the plot
plt.show()

#Saving the plot as png
fig = sbar.get_figure()
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\2 Preparing the Yelp Proxy data for experiments\Top 10 Cities with the
most registered Businesses.png")

# In[8]:
#Remove the irrelevant columns (axis=1 =>columns)
irrelevant_c=["name","address","state","review_count","postal_code","stars","latitude","long
itude","is_open","attributes","hours"]
business_df=business_df.drop(irrelevant_c,axis=1)

# In[9]:
#Checking if the irrelevant column removal was conducted appropriately
business_df.head()

# In[10]:
#Selecting only open businesses, which are located in Las Vegas
business_df = business_df[business_df["city"]=="Las Vegas"]

# In[11]:

```

#Checking if the city column is homogenous

```
business_df.head()
```

In[12]:

#Selecting the hotel industry as business category while keeping business ID and review_count

```
business_df["Hotel?"]=business_df["categories"].str.contains("Hotel")
```

```
domain_business_df = business_df[business_df["Hotel?"] == True]
```

In[13]:

#Check how many open businesses fulfil the location and business industry requirements

```
print(domain_business_df)
```

In[14]:

#Drop for the second time new non-essential columns for further proceeding

```
irrelevant_cc=["categories","city","Hotel?"]
```

```
domain_business_id_df=domain_business_df.drop(irrelevant_cc,axis=1)
```

In[15]:

#Control if the 1030 businesses ids are occurring which are output by earlier statement

```
"print(domain_business_df)"
```

```
print(domain_business_id_df)
```

In[16]:

#Save the business IDs as a csv file

```
domain_business_id_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\2 Preparing the Yelp Proxy data for experiments\Las Vegas Hotels Business IDs.csv",index=False)
```

..... End - 1 Business data to Business ID

..... Start - 2 Link the review and sentiment data domain specifically

In[1]:

#Importing the libraries

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

In[2]:

#Importing the extensive review dataset in chunks and with predefined datatypes

```
review_df=pd.read_json(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\1 Yelp  
Data\review.json",lines=True,dtype={"review_id":str,"user_id":str,"business_id":str,"stars":in  
t,"date":str,"text":str,"useful":int,"funny":int,"cool":int}, chunksize=600000)
```

In[3]:

#Loading in the domain specific business ids

```
business_df=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\2 Preparing the Yelp Proxy data for experiments\Las Vegas Hotels  
Business IDs.csv")
```

```
print(business_df)
```

In[4]:

#For each chunk of the massive review dataset, select just the review data of the domain-specific IDs

```
chunk_reviews = []
```

```
for chunk_review in review_df:
```

```
    chunk_review = chunk_review.drop(["review_id","user_id","date","useful","funny","cool"],  
axis=1)
```

```
    wanted_info = pd.merge(business_df, chunk_review, on="business_id", how="inner")
```

#Correct BusinessId match (Inner Join)

```
    chunk_reviews.append(wanted_info)
```

#Connecting the domain-specific review chunks by the xaxis into one dataframe

```
domain_yelp_df = pd.concat(chunk_reviews, ignore_index=True, join="outer", axis=0)
```

```

# In[5]:
#Inspecting the newly created domain specific review dataset
print(domain_yelp_df)

# In[6]:
#Select only the review text and the rating (i.e. stars) of the domain specific review dataset
essential_domain_yelp_df=domain_yelp_df[["text","stars"]]

# In[7]:
#Check if the prior operation has been applied successfully
print(essential_domain_yelp_df)

# In[8]:
#Count how many occurrence of each star category are in the domain specific review
dataset
star_count=pd.value_counts(essential_domain_yelp_df["stars"])

# In[9]:
#Double Checking the star scale is from 1 to 5
star_count=star_count[:5]

# In[10]:
#Plotting the Sentiment Distribution (i.e. the number of star occurrences for each star
class) for Hotels in Las Vegas
plt.figure(figsize=(15, 5))
sbar=sns.barplot(x=star_count.index, y=star_count.values)
#Annotating the plot
for p in sbar.patches:
    sbar.annotate(format(int(p.get_height()), "1.0f"),
                  (p.get_x() + p.get_width() / 2., p.get_height()),
                  ha = "center", va = "center", fontsize=20,
                  xytext = (0, -18), color="white",
                  textcoords = "offset points")
plt.title("Sentiment Distribution for Hotels in Las Vegas", fontsize=24)

```

```
plt.ylabel("Number of expressed Reviews", fontsize=12)
plt.xlabel("Sentiment Classes", fontsize=12)
```

```
#Displaying the plot in Python
plt.show()
```

```
#Saving the plot
fig = sbar.get_figure()
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\2 Preparing the Yelp Proxy data for experiments\Sentiment Distribution
for Hotels in Las Vegas.png")
```

```
# In[11]:
#Exporting the pandas dataframe into a csv in order to store the file on disk
essential_domain_yelp_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\2 Preparing the Yelp Proxy data for experiments\Las
Vegas Domain-specific review and sentiment data.csv",encoding="utf-8", index=False)
```

..... End - 2 Link the review and sentiment data domain specifically

..... Start - 3.1. Getting the Tweets about Bars over 4 quarters

```
# In[1]:
#Importing the libraries
import tweepy
import pandas as pd
```

```
# In[2]:
#Twitter Authentication
```

```
consumer_key ="ysyr81OjCUwsuWuNETOeO1DcF"
consumer_secret ="jlcvj3x0LGfn5G7dQQ3oVdVqRbq26Mfg3HtU1yYwCA9S2mSmlp"
access_token ="1023802587093655552-G0zhM3iEaFzjbE5fy7Gdjj352G7bml"
access_token_secret ="ReMCB7Sw8JpbwovUYXOuC1TxjNH2VHzw2qJLQ6gAg6KUB"
```

```
auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_token_secret)
```

```
api = tweepy.API(auth)
```

```
# In[3]:
```

```
#First Search Request for Tweet IDs of English Tweets concerning the topic Bar in Las Vegas for the 1st Quarter of 2019
```

```
IDs_1=[]
```

```
results_1=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas" lang:en ("bartender" OR "bar" OR "bartending" OR "cocktail" OR "cocktails") -from:Patientzero722 -from:iluvnappytime -customer -you -your -KevLove -restaurant -airport -table -guests -guest -providing -give -package -plan -call -present -offer -provide -announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift -voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs -technology))""", fromDate="201901010000", toDate="201903312359",maxResults="500")
for tweet in results_1:
```

```
    IDs_1.append(tweet.id)
```

```
# In[4]:
```

```
#Showing the number of IDs of Tweets which are found by the First Search Request
print(len(IDs_1))
```

```
# In[5]:
```

```
#Chunking the IDs of the First Search Request to lists of 100 IDs
```

```
n=100
```

```
first_500_IDs = [IDs_1[i:i + n] for i in range(0, len(IDs_1), n)]
```

```
list1= first_500_IDs[0]
```

```
list2= first_500_IDs[1]
```

```
list3= first_500_IDs[2]
```

```
list4= first_500_IDs[3]
```

```
list5= first_500_IDs[4]
```

```
# In[6]:
```

#Second Search Request for Tweet IDs of English Tweets concerning the topic Bar in Las Vegas for the 2nd Quarter of 2019

```
IDs_2=[]
```

```
results_2=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("bartender" OR "bar" OR "bartending" OR "cocktail" OR "cocktails")
-from:Patientzero722 -from:iluvnappytime -customer -you -your -KevLove -restaurant
-airport -table -guests -guest -providing -give -package -plan -call -present -offer -provide
-announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift
-voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs
-technology))""", fromDate="201904010000", toDate="201906302359",maxResults="500")
```

```
for tweet in results_2:
```

```
    IDs_2.append(tweet.id)
```

```
# In[7]:
```

```
#Showing the number of IDs of Tweets which are found by the Second Search Request
print(len(IDs_2))
```

```
# In[8]:
```

```
#Chunking the IDs of the Second Search Request to lists of 100 IDs
```

```
n=100
```

```
second_500_IDs = [IDs_2[i:i + n] for i in range(0, len(IDs_2), n)]
```

```
list6= second_500_IDs[0]
```

```
list7= second_500_IDs[1]
```

```
list8= second_500_IDs[2]
```

```
list9= second_500_IDs[3]
```

```
list10= second_500_IDs[4]
```

```
# In[9]:
```

```
#Third Search Request for Tweet IDs of English Tweets concerning the topic Bar in Las Vegas for the 3rd Quarter of 2019
```

```
IDs_3=[]
```

```
results_3=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("bartender" OR "bar" OR "bartending" OR "cocktail" OR "cocktails")
-from:Patientzero722 -from:iluvnappytime -customer -you -your -KevLove -restaurant
-airport -table -guests -guest -providing -give -package -plan -call -present -offer -provide
```

```
-announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift  
-voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs  
-technology))""", fromDate="201907010000", toDate="201909302359",maxResults="500")  
for tweet in results_3:
```

```
    IDs_3.append(tweet.id)
```

```
# In[10]:
```

```
#Showing the number of IDs of Tweets which are found by the Third Search Request  
print(len(IDs_3))
```

```
# In[11]:
```

```
#Chunking the IDs of the Third Search Request to lists of 100 IDs
```

```
n=100
```

```
third_500_IDs = [IDs_3[i:i + n] for i in range(0, len(IDs_3), n)]
```

```
list11=third_500_IDs[0]
```

```
list12=third_500_IDs[1]
```

```
list13=third_500_IDs[2]
```

```
list14=third_500_IDs[3]
```

```
list15=third_500_IDs[4]
```

```
# In[12]:
```

```
#Fourth Search Request for Tweet IDs of English Tweets concerning the topic Bar in Las  
Vegas for the 4th Quarter of 2019
```

```
IDs_4=[]
```

```
results_4=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"  
lang:en (("bartender" OR "bar" OR "bartending" OR "cocktail" OR "cocktails")  
-from:Patientzero722 -from:iluvnappytime -customer -you -your -KevLove -restaurant  
-airport -table -guests -guest -providing -give -package -plan -call -present -offer -provide  
-announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift  
-voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs  
-technology))""", fromDate="201910010000", toDate="201912312359",maxResults="500")  
for tweet in results_4:
```

```
    IDs_4.append(tweet.id)
```

```
# In[13]:
```

```
#Showing the number of IDs of Tweets which are found by the Fourth Search Request
print(len(IDs_4))
```

```
# In[14]:
```

```
#Chunking the IDs of the Fourth Search Request to lists of 100 IDs
```

```
n=100
```

```
fourth_500_IDs = [IDs_4[i:i + n] for i in range(0, len(IDs_4), n)]
```

```
list16 =fourth_500_IDs[0]
```

```
list17 =fourth_500_IDs[1]
```

```
list18 =fourth_500_IDs[2]
```

```
list19 =fourth_500_IDs[3]
```

```
list20 =fourth_500_IDs[4]
```

```
# In[ ]:
```

```
##*Start: Requesting for each list of 100 Tweet IDs the Tweet Text and the Author  
Screenname, then combining these features with their corresponding ID into a Dataframe
```

```
# In[15]:
```

```
text_of_tweets_1=[]
```

```
author_screenname_1=[]
```

```
tweets=api.statuses_lookup(list1,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_1.append(tweet.full_text)
```

```
    author_screenname_1.append(tweet.author.screen_name)
```

```
query_df_1 = pd.DataFrame(list(zip(list1, text_of_tweets_1,  
author_screenname_1)),columns =["ID", "Text", "Screenname"])
```

```
# In[16]:
```

```
text_of_tweets_2=[]
```

```
author_screenname_2=[]
```

```
tweets=api.statuses_lookup(list2,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_2.append(tweet.full_text)
```

```
    author_screenname_2.append(tweet.author.screen_name)
```

```
query_df_2 = pd.DataFrame(list(zip(list2, text_of_tweets_2,
author_screenname_2)),columns=["ID", "Text", "Screenname"])
```

```
# In[17]:
```

```
text_of_tweets_3=[]
```

```
author_screenname_3=[]
```

```
tweets=api.statuses_lookup(list3,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_3.append(tweet.full_text)
```

```
    author_screenname_3.append(tweet.author.screen_name)
```

```
query_df_3 = pd.DataFrame(list(zip(list3, text_of_tweets_3,
author_screenname_3)),columns=["ID", "Text", "Screenname"])
```

```
# In[18]:
```

```
text_of_tweets_4=[]
```

```
author_screenname_4=[]
```

```
tweets=api.statuses_lookup(list4,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_4.append(tweet.full_text)
```

```
    author_screenname_4.append(tweet.author.screen_name)
```

```
query_df_4 = pd.DataFrame(list(zip(list4, text_of_tweets_4,
author_screenname_4)),columns=["ID", "Text", "Screenname"])
```

```
# In[19]:
```

```
text_of_tweets_5=[]
```

```
author_screenname_5=[]
```

```
tweets=api.statuses_lookup(list5,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_5.append(tweet.full_text)
```

```
    author_screenname_5.append(tweet.author.screen_name)
```

```
query_df_5 = pd.DataFrame(list(zip(list5, text_of_tweets_5,
author_screenname_5)),columns=["ID", "Text", "Screenname"])
```

```
# In[20]:
```

```
text_of_tweets_6=[]
```



```

author_screename_6=[]
tweets=api.statuses_lookup(list6,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_6.append(tweet.full_text)
    author_screename_6.append(tweet.author.screen_name)
query_df_6 = pd.DataFrame(list(zip(list6, text_of_tweets_6,
author_screename_6)),columns=["ID", "Text", "Screenname"])

# In[21]:
text_of_tweets_7=[]
author_screename_7=[]
tweets=api.statuses_lookup(list7,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_7.append(tweet.full_text)
    author_screename_7.append(tweet.author.screen_name)
query_df_7 = pd.DataFrame(list(zip(list7, text_of_tweets_7,
author_screename_7)),columns=["ID", "Text", "Screenname"])

# In[22]:
text_of_tweets_8=[]
author_screename_8=[]
tweets=api.statuses_lookup(list8,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_8.append(tweet.full_text)
    author_screename_8.append(tweet.author.screen_name)
query_df_8 = pd.DataFrame(list(zip(list8, text_of_tweets_8,
author_screename_8)),columns=["ID", "Text", "Screenname"])

# In[23]:
text_of_tweets_9=[]
author_screename_9=[]
tweets=api.statuses_lookup(list9,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_9.append(tweet.full_text)
    author_screename_9.append(tweet.author.screen_name)

```

```
query_df_9 = pd.DataFrame(list(zip(list9, text_of_tweets_9,
author_screenname_9)),columns=["ID", "Text", "Screenname"])
```

```
# In[24]:
```

```
text_of_tweets_10=[]
```

```
author_screenname_10=[]
```

```
tweets=api.statuses_lookup(list10,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_10.append(tweet.full_text)
```

```
    author_screenname_10.append(tweet.author.screen_name)
```

```
query_df_10 = pd.DataFrame(list(zip(list10, text_of_tweets_10,
author_screenname_10)),columns=["ID", "Text", "Screenname"])
```

```
# In[25]:
```

```
text_of_tweets_11=[]
```

```
author_screenname_11=[]
```

```
tweets=api.statuses_lookup(list11,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_11.append(tweet.full_text)
```

```
    author_screenname_11.append(tweet.author.screen_name)
```

```
query_df_11 = pd.DataFrame(list(zip(list11, text_of_tweets_11,
author_screenname_11)),columns=["ID", "Text", "Screenname"])
```

```
# In[26]:
```

```
text_of_tweets_12=[]
```

```
author_screenname_12=[]
```

```
tweets=api.statuses_lookup(list12,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_12.append(tweet.full_text)
```

```
    author_screenname_12.append(tweet.author.screen_name)
```

```
query_df_12 = pd.DataFrame(list(zip(list12, text_of_tweets_12,
author_screenname_12)),columns=["ID", "Text", "Screenname"])
```

```
# In[27]:
```

```
text_of_tweets_13=[]
```

```

author_screenname_13=[]
tweets=api.statuses_lookup(list13,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_13.append(tweet.full_text)
    author_screenname_13.append(tweet.author.screen_name)
query_df_13 = pd.DataFrame(list(zip(list13, text_of_tweets_13,
author_screenname_13)),columns=["ID", "Text", "Screenname"])

# In[28]:
text_of_tweets_14=[]
author_screenname_14=[]
tweets=api.statuses_lookup(list14,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_14.append(tweet.full_text)
    author_screenname_14.append(tweet.author.screen_name)
query_df_14 = pd.DataFrame(list(zip(list14, text_of_tweets_14,
author_screenname_14)),columns=["ID", "Text", "Screenname"])

# In[29]:
text_of_tweets_15=[]
author_screenname_15=[]
tweets=api.statuses_lookup(list15,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_15.append(tweet.full_text)
    author_screenname_15.append(tweet.author.screen_name)
query_df_15 = pd.DataFrame(list(zip(list15, text_of_tweets_15,
author_screenname_15)),columns=["ID", "Text", "Screenname"])

# In[30]:
text_of_tweets_16=[]
author_screenname_16=[]
tweets=api.statuses_lookup(list16,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_16.append(tweet.full_text)
    author_screenname_16.append(tweet.author.screen_name)

```

```
query_df_16 = pd.DataFrame(list(zip(list16, text_of_tweets_16,
author_screenname_16)),columns=["ID", "Text", "Screenname"])
```

```
# In[31]:
```

```
text_of_tweets_17=[]
```

```
author_screenname_17=[]
```

```
tweets=api.statuses_lookup(list17,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_17.append(tweet.full_text)
```

```
    author_screenname_17.append(tweet.author.screen_name)
```

```
query_df_17 = pd.DataFrame(list(zip(list17, text_of_tweets_17,
author_screenname_17)),columns=["ID", "Text", "Screenname"])
```

```
# In[32]:
```

```
text_of_tweets_18=[]
```

```
author_screenname_18=[]
```

```
tweets=api.statuses_lookup(list18,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_18.append(tweet.full_text)
```

```
    author_screenname_18.append(tweet.author.screen_name)
```

```
query_df_18 = pd.DataFrame(list(zip(list18, text_of_tweets_18,
author_screenname_18)),columns=["ID", "Text", "Screenname"])
```

```
# In[33]:
```

```
text_of_tweets_19=[]
```

```
author_screenname_19=[]
```

```
tweets=api.statuses_lookup(list19,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_19.append(tweet.full_text)
```

```
    author_screenname_19.append(tweet.author.screen_name)
```

```
query_df_19 = pd.DataFrame(list(zip(list19, text_of_tweets_19,
author_screenname_19)),columns=["ID", "Text", "Screenname"])
```

```
# In[34]:
```

```
text_of_tweets_20=[]
```

```

author_screename_20=[]
tweets=api.statuses_lookup(list20,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_20.append(tweet.full_text)
    author_screename_20.append(tweet.author.screen_name)
query_df_20 = pd.DataFrame(list(zip(list20, text_of_tweets_20,
author_screename_20)),columns=["ID", "Text", "Screenname"])

# In[ ]:
#*End: Requesting for each list of 100 Tweet IDs the Tweet Text and the Author
Screenname, then combining these features with their corresponding ID into a Dataframe

# In[35]:
#Creating dataframe, which represents an aggregated version (by attaching new queries
as new rows) of the 20 queries
final_tweet_df=pd.concat([query_df_1, query_df_2, query_df_3, query_df_4, query_df_5,
query_df_6, query_df_7, query_df_8, query_df_9, query_df_10, query_df_11,
query_df_12, query_df_13, query_df_14, query_df_15, query_df_16, query_df_17,
query_df_18, query_df_19, query_df_20],axis=0, ignore_index=True)

# In[36]:
#Controlling that no Tweet was emitted by a user with Hotel in its name in order to filter out
advertisements
authors=final_tweet_df["Screenname"].str.contains("Hotel").sum()
if authors>0:
    print ("There are {a} tweets by Hotel(s)".format(a=authors))
else:
    print("No Hotel Posts or Retweets")

# In[37]:
#Export the combined dataset of queries
final_tweet_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\3 Obtaining the Twitter Data\Twitter Data\Search by topic
bar.csv",index=False)

```

..... End - 3.1. Getting the Tweets about Bars over 4 quarters

..... Start - 3.2. Getting the Tweets about Events over 4 quarters

In[1]:

#Importing the libraries

import tweepy

import pandas as pd

In[2]:

#Twitter Authentication

consumer_key ="ysyr81OjCUwsuWuNETOeO1DcF"

consumer_secret ="jlcvj3x0LGfn5G7dQQ3oVdVqRbq26Mfg3HtU1yYwCA9S2mSmIp"

access_token ="1023802587093655552-G0zhM3iEaFzjbE5fy7Gdjj352G7bml"

access_token_secret ="ReMCB7Sw8JpbwovUYXOuC1TxjNH2VHzw2qJLQ6gAg6KUB"

auth = tweepy.OAuthHandler(consumer_key, consumer_secret)

auth.set_access_token(access_token, access_token_secret)

api = tweepy.API(auth)

In[3]:

#First Search Request for Tweet IDs of English Tweets concerning the topic Stationary Entertainment in Las Vegas for the 1st Quarter of 2019

IDs_1=[]

results_1=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas" lang:en ("Event" OR "Show") -TV -customer -you -your -KevLove -restaurant -airport -table -guests -guest -providing -give -package -plan -call -present -offer -provide -announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift -voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs -technology))""", fromDate="201901010000", toDate="201903312359",maxResults="500")

for tweet in results_1:

```
IDs_1.append(tweet.id)
```

```
# In[4]:
```

```
#Showing the number of IDs of Tweets which are found by the First Search Request  
print(len(IDs_1))
```

```
# In[5]:
```

```
#Chunking the IDs of the First Search Request to lists of 100 IDs
```

```
n=100
```

```
first_500_IDs = [IDs_1[i:i + n] for i in range(0, len(IDs_1), n)]
```

```
list1= first_500_IDs[0]
```

```
list2= first_500_IDs[1]
```

```
list3= first_500_IDs[2]
```

```
list4= first_500_IDs[3]
```

```
list5= first_500_IDs[4]
```

```
# In[6]:
```

```
#Second Search Request for Tweet IDs of English Tweets concerning the topic Stationary  
Entertainment in Las Vegas for the 2nd Quarter of 2019
```

```
IDs_2=[]
```

```
results_2=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"  
lang:en (("Event" OR "Show") -TV -customer -you -your -KevLove -restaurant -airport  
-table -guests -guest -providing -give -package -plan -call -present -offer -provide  
-announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift  
-voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs  
-technology))""", fromDate="201904010000", toDate="201906302359",maxResults="500")
```

```
for tweet in results_2:
```

```
    IDs_2.append(tweet.id)
```

```
# In[7]:
```

```
#Showing the number of IDs of Tweets which are found by the Second Search Request  
print(len(IDs_2))
```

```
# In[8]:
```

```
#Chunking the IDs of the Second Search Request to lists of 100 IDs
```

```

n=100
second_500_IDs = [IDs_2[i:i + n] for i in range(0, len(IDs_2), n)]
list6= second_500_IDs[0]
list7= second_500_IDs[1]
list8= second_500_IDs[2]
list9= second_500_IDs[3]
list10= second_500_IDs[4]

# In[9]:
#Third Search Request for Tweet IDs of English Tweets concerning the topic Stationary
Entertainment in Las Vegas for the 3rd Quarter of 2019
IDs_3=[]
results_3=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("Event" OR "Show") -TV -customer -you -your -KevLove -restaurant -airport
-table -guests -guest -providing -give -package -plan -call -present -offer -provide
-announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift
-voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs
-technology))""", fromDate="201907010000", toDate="201909302359",maxResults="500")
for tweet in results_3:
    IDs_3.append(tweet.id)

# In[10]:
#Showing the number of IDs of Tweets which are found by the Third Search Request
print(len(IDs_3))

# In[11]:
#Chunking the IDs of the Third Search Request to lists of 100 IDs
n=100
third_500_IDs = [IDs_3[i:i + n] for i in range(0, len(IDs_3), n)]
list11=third_500_IDs[0]
list12=third_500_IDs[1]
list13=third_500_IDs[2]
list14=third_500_IDs[3]
list15=third_500_IDs[4]

```


In[12]:

#Fourth Search Request for Tweet IDs of English Tweets concerning the topic Stationary Entertainment in Las Vegas for the 4th Quarter of 2019

IDs_4=[]

```
results_4=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("Event" OR "Show") -TV -customer -you -your -KevLove -restaurant -airport
-table -guests -guest -providing -give -package -plan -call -present -offer -provide
-announce -booking -blog -sector -available -rent -product -book -hiring -recruiting -gift
-voucher -contact -join -award -purchase -us -opened -tools -podcast -guide -coupon -KPIs
-technology))""", fromDate="201910010000", toDate="201912312359",maxResults="500")
```

for tweet in results_4:

IDs_4.append(tweet.id)

In[13]:

#Showing the number of IDs of Tweets which are found by the Fourth Search Request
print(len(IDs_4))

In[14]:

#Chunking the IDs of the Fourth Search Request to lists of 100 IDs

n=100

fourth_500_IDs = [IDs_4[i:i + n] for i in range(0, len(IDs_4), n)]

list16 =fourth_500_IDs[0]

list17 =fourth_500_IDs[1]

list18 =fourth_500_IDs[2]

list19 =fourth_500_IDs[3]

list20 =fourth_500_IDs[4]

In[]:

##*Start: Requesting for each list of 100 Tweet IDs the Tweet Text and the Author
Screenname, then combining these features with their corresponding ID into a Dataframe

In[15]:

text_of_tweets_1=[]

author_screenname_1=[]

```

tweets=api.statuses_lookup(list1,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_1.append(tweet.full_text)
    author_screenname_1.append(tweet.author.screen_name)
query_df_1 = pd.DataFrame(list(zip(list1, text_of_tweets_1,
author_screenname_1)),columns=["ID", "Text", "Screenname"])

```

In[16]:

```

text_of_tweets_2=[]
author_screenname_2=[]
tweets=api.statuses_lookup(list2,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_2.append(tweet.full_text)
    author_screenname_2.append(tweet.author.screen_name)
query_df_2 = pd.DataFrame(list(zip(list2, text_of_tweets_2,
author_screenname_2)),columns=["ID", "Text", "Screenname"])

```

In[17]:

```

text_of_tweets_3=[]
author_screenname_3=[]
tweets=api.statuses_lookup(list3,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_3.append(tweet.full_text)
    author_screenname_3.append(tweet.author.screen_name)
query_df_3 = pd.DataFrame(list(zip(list3, text_of_tweets_3,
author_screenname_3)),columns=["ID", "Text", "Screenname"])

```

In[18]:

```

text_of_tweets_4=[]
author_screenname_4=[]
tweets=api.statuses_lookup(list4,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_4.append(tweet.full_text)
    author_screenname_4.append(tweet.author.screen_name)

```

```
query_df_4 = pd.DataFrame(list(zip(list4, text_of_tweets_4,
author_screenname_4)),columns=["ID", "Text", "Screenname"])
```

```
# In[19]:
```

```
text_of_tweets_5=[]
```

```
author_screenname_5=[]
```

```
tweets=api.statuses_lookup(list5,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_5.append(tweet.full_text)
```

```
    author_screenname_5.append(tweet.author.screen_name)
```

```
query_df_5 = pd.DataFrame(list(zip(list5, text_of_tweets_5,
author_screenname_5)),columns=["ID", "Text", "Screenname"])
```

```
# In[20]:
```

```
text_of_tweets_6=[]
```

```
author_screenname_6=[]
```

```
tweets=api.statuses_lookup(list6,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_6.append(tweet.full_text)
```

```
    author_screenname_6.append(tweet.author.screen_name)
```

```
query_df_6 = pd.DataFrame(list(zip(list6, text_of_tweets_6,
author_screenname_6)),columns=["ID", "Text", "Screenname"])
```

```
# In[21]:
```

```
text_of_tweets_7=[]
```

```
author_screenname_7=[]
```

```
tweets=api.statuses_lookup(list7,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_7.append(tweet.full_text)
```

```
    author_screenname_7.append(tweet.author.screen_name)
```

```
query_df_7 = pd.DataFrame(list(zip(list7, text_of_tweets_7,
author_screenname_7)),columns=["ID", "Text", "Screenname"])
```

```
# In[22]:
```

```
text_of_tweets_8=[]
```

```

author_screenname_8=[]
tweets=api.statuses_lookup(list8,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_8.append(tweet.full_text)
    author_screenname_8.append(tweet.author.screen_name)
query_df_8 = pd.DataFrame(list(zip(list8, text_of_tweets_8,
author_screenname_8)),columns=["ID", "Text", "Screenname"])

# In[23]:
text_of_tweets_9=[]
author_screenname_9=[]
tweets=api.statuses_lookup(list9,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_9.append(tweet.full_text)
    author_screenname_9.append(tweet.author.screen_name)
query_df_9 = pd.DataFrame(list(zip(list9, text_of_tweets_9,
author_screenname_9)),columns=["ID", "Text", "Screenname"])

# In[24]:
text_of_tweets_10=[]
author_screenname_10=[]
tweets=api.statuses_lookup(list10,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_10.append(tweet.full_text)
    author_screenname_10.append(tweet.author.screen_name)
query_df_10 = pd.DataFrame(list(zip(list10, text_of_tweets_10,
author_screenname_10)),columns=["ID", "Text", "Screenname"])

# In[25]:
text_of_tweets_11=[]
author_screenname_11=[]
tweets=api.statuses_lookup(list11,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_11.append(tweet.full_text)
    author_screenname_11.append(tweet.author.screen_name)

```

```
query_df_11 = pd.DataFrame(list(zip(list11, text_of_tweets_11,
author_screenname_11)),columns=["ID", "Text", "Screenname"])
```

```
# In[26]:
```

```
text_of_tweets_12=[]
```

```
author_screenname_12=[]
```

```
tweets=api.statuses_lookup(list12,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_12.append(tweet.full_text)
```

```
    author_screenname_12.append(tweet.author.screen_name)
```

```
query_df_12 = pd.DataFrame(list(zip(list12, text_of_tweets_12,
author_screenname_12)),columns=["ID", "Text", "Screenname"])
```

```
# In[27]:
```

```
text_of_tweets_13=[]
```

```
author_screenname_13=[]
```

```
tweets=api.statuses_lookup(list13,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_13.append(tweet.full_text)
```

```
    author_screenname_13.append(tweet.author.screen_name)
```

```
query_df_13 = pd.DataFrame(list(zip(list13, text_of_tweets_13,
author_screenname_13)),columns=["ID", "Text", "Screenname"])
```

```
# In[28]:
```

```
text_of_tweets_14=[]
```

```
author_screenname_14=[]
```

```
tweets=api.statuses_lookup(list14,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_14.append(tweet.full_text)
```

```
    author_screenname_14.append(tweet.author.screen_name)
```

```
query_df_14 = pd.DataFrame(list(zip(list14, text_of_tweets_14,
author_screenname_14)),columns=["ID", "Text", "Screenname"])
```

```
# In[29]:
```

```
text_of_tweets_15=[]
```

```

author_screenname_15=[]
tweets=api.statuses_lookup(list15,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_15.append(tweet.full_text)
    author_screenname_15.append(tweet.author.screen_name)
query_df_15 = pd.DataFrame(list(zip(list15, text_of_tweets_15,
author_screenname_15)),columns=["ID", "Text", "Screenname"])

# In[30]:
text_of_tweets_16=[]
author_screenname_16=[]
tweets=api.statuses_lookup(list16,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_16.append(tweet.full_text)
    author_screenname_16.append(tweet.author.screen_name)
query_df_16 = pd.DataFrame(list(zip(list16, text_of_tweets_16,
author_screenname_16)),columns=["ID", "Text", "Screenname"])

# In[31]:
text_of_tweets_17=[]
author_screenname_17=[]
tweets=api.statuses_lookup(list17,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_17.append(tweet.full_text)
    author_screenname_17.append(tweet.author.screen_name)
query_df_17 = pd.DataFrame(list(zip(list17, text_of_tweets_17,
author_screenname_17)),columns=["ID", "Text", "Screenname"])

# In[32]:
text_of_tweets_18=[]
author_screenname_18=[]
tweets=api.statuses_lookup(list18,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_18.append(tweet.full_text)
    author_screenname_18.append(tweet.author.screen_name)

```

```
query_df_18 = pd.DataFrame(list(zip(list18, text_of_tweets_18,
author_screenname_18)),columns=["ID", "Text", "Screenname"])
```

```
# In[33]:
```

```
text_of_tweets_19=[]
```

```
author_screenname_19=[]
```

```
tweets=api.statuses_lookup(list19,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_19.append(tweet.full_text)
```

```
    author_screenname_19.append(tweet.author.screen_name)
```

```
query_df_19 = pd.DataFrame(list(zip(list19, text_of_tweets_19,
author_screenname_19)),columns=["ID", "Text", "Screenname"])
```

```
# In[34]:
```

```
text_of_tweets_20=[]
```

```
author_screenname_20=[]
```

```
tweets=api.statuses_lookup(list20,tweet_mode="extended")
```

```
for tweet in tweets:
```

```
    text_of_tweets_20.append(tweet.full_text)
```

```
    author_screenname_20.append(tweet.author.screen_name)
```

```
query_df_20 = pd.DataFrame(list(zip(list20, text_of_tweets_20,
author_screenname_20)),columns=["ID", "Text", "Screenname"])
```

```
# In[ ]:
```

```
##*End: Requesting for each list of 100 Tweet IDs the Tweet Text and the Author  
Screenname, then combining these features with their corresponding ID into a Dataframe
```

```
# In[35]:
```

```
#Creating dataframe, which represents an aggregated version (by attaching new queries  
as new rows) of the 20 queries
```

```
final_tweet_df=pd.concat([query_df_1, query_df_2, query_df_3, query_df_4, query_df_5,  
query_df_6, query_df_7, query_df_8, query_df_9, query_df_10, query_df_11,  
query_df_12, query_df_13, query_df_14, query_df_15, query_df_16, query_df_17,  
query_df_18, query_df_19, query_df_20],axis=0, ignore_index=True)
```

```
# In[36]:
#Controlling that no Tweet was emitted by a user with Hotel in its name in order to filter out
advertisements
authors=final_tweet_df["Screenname"].str.contains("Hotel").sum()
if authors>0:
    print ("There are {a} tweets by Hotel(s)".format(a=authors))
else:
    print("No Hotel Posts or Retweets")
```

```
# In[37]:
#Export the combined dataset of queries
final_tweet_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\3 Obtaining the Twitter Data\Twitter Data\Search by topic
event.csv",index=False)
```

..... End - 3.2. Getting the Tweets about Events over 4 quarters

..... Start - 3.3. Getting the Tweets about Weddings over 4 quarters

```
# In[1]:
#Importing the libraries
import tweepy
import pandas as pd
```

```
# In[2]:
#Twitter Authentication

consumer_key ="ysyr81OjCUwsuWuNETOeO1DcF"
consumer_secret ="jlcvj3x0LGfn5G7dQQ3oVdVqRbq26Mfg3HtU1yYwCA9S2mSmlp"
access_token ="1023802587093655552-G0zhM3iEaFzjbE5fy7Gdjj352G7bml"
access_token_secret ="ReMCB7Sw8JpbwovUYXOuC1TxjNH2VHzw2qJLQ6gAg6KUB"

auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_token_secret)
```



```

api = tweepy.API(auth)

# In[3]:
#First Search Request for Tweet IDs of English Tweets concerning the topic Wedding in
Las Vegas for the 1st Quarter of 2019
IDs_1=[]
results_1=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("marriage" OR "wedding" OR "married" OR "bride" OR "marry") -customer -you
-your -KevLove -restaurant -airport -table -guests -guest -providing -give -package -plan
-call -present -offer -provide -announce -booking -blog -sector -available -rent -product
-book -hiring -recruiting -gift -voucher -contact -join -award -purchase -us -opened -tools
-podcast -guide -coupon -KPIs -technology))""", fromDate="201901010000",
toDate="201903312359",maxResults="500")
for tweet in results_1:
    IDs_1.append(tweet.id)

# In[4]:
#Showing the number of IDs of Tweets which are found by the First Search Request
print(len(IDs_1))

# In[5]:
#Chunking the IDs of the First Search Request to lists of 100 IDs
n=100
first_500_IDs = [IDs_1[i:i + n] for i in range(0, len(IDs_1), n)]
list1= first_500_IDs[0]
list2= first_500_IDs[1]
list3= first_500_IDs[2]
list4= first_500_IDs[3]
list5= first_500_IDs[4]

# In[6]:
#Second Search Request for Tweet IDs of English Tweets concerning the topic Wedding in
Las Vegas for the 2nd Quarter of 2019
IDs_2=[]

```

```
results_2=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("marriage" OR "wedding" OR "married" OR "bride" OR "marry") -customer -you
-your -KevLove -restaurant -airport -table -guests -guest -providing -give -package -plan
-call -present -offer -provide -announce -booking -blog -sector -available -rent -product
-book -hiring -recruiting -gift -voucher -contact -join -award -purchase -us -opened -tools
-podcast -guide -coupon -KPIs -technology))""", fromDate="201904010000",
toDate="201906302359",maxResults="500")
```

```
for tweet in results_2:
```

```
    IDs_2.append(tweet.id)
```

```
# In[7]:
```

```
#Showing the number of IDs of Tweets which are found by the Second Search Request
```

```
print(len(IDs_2))
```

```
# In[8]:
```

```
#Chunking the IDs of the Second Search Request to lists of 100 IDs
```

```
n=100
```

```
second_500_IDs = [IDs_2[i:i + n] for i in range(0, len(IDs_2), n)]
```

```
list6= second_500_IDs[0]
```

```
list7= second_500_IDs[1]
```

```
list8= second_500_IDs[2]
```

```
list9= second_500_IDs[3]
```

```
list10= second_500_IDs[4]
```

```
# In[9]:
```

```
#Third Search Request for Tweet IDs of English Tweets concerning the topic Wedding in
Las Vegas for the 3rd Quarter of 2019
```

```
IDs_3=[]
```

```
results_3=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("marriage" OR "wedding" OR "married" OR "bride" OR "marry") -customer -you
-your -KevLove -restaurant -airport -table -guests -guest -providing -give -package -plan
-call -present -offer -provide -announce -booking -blog -sector -available -rent -product
-book -hiring -recruiting -gift -voucher -contact -join -award -purchase -us -opened -tools
-podcast -guide -coupon -KPIs -technology))""", fromDate="201907010000",
toDate="201909302359",maxResults="500")
```

```

for tweet in results_3:
    IDs_3.append(tweet.id)

# In[10]:
#Showing the number of IDs of Tweets which are found by the Third Search Request
print(len(IDs_3))

# In[11]:
#Chunking the IDs of the Third Search Request to lists of 100 IDs
n=100
third_500_IDs = [IDs_3[i:i + n] for i in range(0, len(IDs_3), n)]
list11=third_500_IDs[0]
list12=third_500_IDs[1]
list13=third_500_IDs[2]
list14=third_500_IDs[3]
list15=third_500_IDs[4]

# In[12]:
#Fourth Search Request for Tweet IDs of English Tweets concerning the topic Wedding in
Las Vegas for the 4th Quarter of 2019
IDs_4=[]
results_4=api.search_full_archive(environment_name="dis", query="""(place:"Las Vegas"
lang:en (("marriage" OR "wedding" OR "married" OR "bride" OR "marry") -customer -you
-your -KevLove -restaurant -airport -table -guests -guest -providing -give -package -plan
-call -present -offer -provide -announce -booking -blog -sector -available -rent -product
-book -hiring -recruiting -gift -voucher -contact -join -award -purchase -us -opened -tools
-podcast -guide -coupon -KPIs -technology))""", fromDate="201910010000",
toDate="201912312359",maxResults="500")
for tweet in results_4:
    IDs_4.append(tweet.id)

# In[13]:
#Showing the number of IDs of Tweets which are found by the Fourth Search Request
print(len(IDs_4))

```

```

# In[14]:
#Chunking the IDs of the Fourth Search Request to lists of 100 IDs
n=100
fourth_500_IDs = [IDs_4[i:i + n] for i in range(0, len(IDs_4), n)]

list16 =fourth_500_IDs[0]
list17 =fourth_500_IDs[1]
list18 =fourth_500_IDs[2]
list19 =fourth_500_IDs[3]
list20 =fourth_500_IDs[4]

# In[ ]:
**Start: Requesting for each list of 100 Tweet IDs the Tweet Text and the Author
Screenname, then combining these features with their corresponding ID into a Dataframe

# In[15]:
text_of_tweets_1=[]
author_screenname_1=[]
tweets=api.statuses_lookup(list1,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_1.append(tweet.full_text)
    author_screenname_1.append(tweet.author.screen_name)
query_df_1 = pd.DataFrame(list(zip(list1, text_of_tweets_1,
author_screenname_1)),columns=["ID", "Text", "Screenname"])

# In[16]:
text_of_tweets_2=[]
author_screenname_2=[]
tweets=api.statuses_lookup(list2,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_2.append(tweet.full_text)
    author_screenname_2.append(tweet.author.screen_name)
query_df_2 = pd.DataFrame(list(zip(list2, text_of_tweets_2,
author_screenname_2)),columns=["ID", "Text", "Screenname"])

```

```

# In[17]:
text_of_tweets_3=[]
author_screenname_3=[]
tweets=api.statuses_lookup(list3,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_3.append(tweet.full_text)
    author_screenname_3.append(tweet.author.screen_name)
query_df_3 = pd.DataFrame(list(zip(list3, text_of_tweets_3,
author_screenname_3)),columns=["ID", "Text", "Screenname"])

# In[18]:
text_of_tweets_4=[]
author_screenname_4=[]
tweets=api.statuses_lookup(list4,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_4.append(tweet.full_text)
    author_screenname_4.append(tweet.author.screen_name)
query_df_4 = pd.DataFrame(list(zip(list4, text_of_tweets_4,
author_screenname_4)),columns=["ID", "Text", "Screenname"])

# In[19]:
text_of_tweets_5=[]
author_screenname_5=[]
tweets=api.statuses_lookup(list5,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_5.append(tweet.full_text)
    author_screenname_5.append(tweet.author.screen_name)
query_df_5 = pd.DataFrame(list(zip(list5, text_of_tweets_5,
author_screenname_5)),columns=["ID", "Text", "Screenname"])

# In[20]:
text_of_tweets_6=[]
author_screenname_6=[]
tweets=api.statuses_lookup(list6,tweet_mode="extended")
for tweet in tweets:

```

```

text_of_tweets_6.append(tweet.full_text)
author_screenname_6.append(tweet.author.screen_name)
query_df_6 = pd.DataFrame(list(zip(list6, text_of_tweets_6,
author_screenname_6)),columns=["ID", "Text", "Screenname"])

```

In[21]:

```

text_of_tweets_7=[]
author_screenname_7=[]
tweets=api.statuses_lookup(list7,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_7.append(tweet.full_text)
    author_screenname_7.append(tweet.author.screen_name)
query_df_7 = pd.DataFrame(list(zip(list7, text_of_tweets_7,
author_screenname_7)),columns=["ID", "Text", "Screenname"])

```

In[22]:

```

text_of_tweets_8=[]
author_screenname_8=[]
tweets=api.statuses_lookup(list8,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_8.append(tweet.full_text)
    author_screenname_8.append(tweet.author.screen_name)
query_df_8 = pd.DataFrame(list(zip(list8, text_of_tweets_8,
author_screenname_8)),columns=["ID", "Text", "Screenname"])

```

In[23]:

```

text_of_tweets_9=[]
author_screenname_9=[]
tweets=api.statuses_lookup(list9,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_9.append(tweet.full_text)
    author_screenname_9.append(tweet.author.screen_name)
query_df_9 = pd.DataFrame(list(zip(list9, text_of_tweets_9,
author_screenname_9)),columns=["ID", "Text", "Screenname"])

```

```

# In[24]:
text_of_tweets_10=[]
author_screenname_10=[]
tweets=api.statuses_lookup(list10,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_10.append(tweet.full_text)
    author_screenname_10.append(tweet.author.screen_name)
query_df_10 = pd.DataFrame(list(zip(list10, text_of_tweets_10,
author_screenname_10)),columns=["ID", "Text", "Screenname"])

# In[25]:
text_of_tweets_11=[]
author_screenname_11=[]
tweets=api.statuses_lookup(list11,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_11.append(tweet.full_text)
    author_screenname_11.append(tweet.author.screen_name)
query_df_11 = pd.DataFrame(list(zip(list11, text_of_tweets_11,
author_screenname_11)),columns=["ID", "Text", "Screenname"])

# In[26]:
text_of_tweets_12=[]
author_screenname_12=[]
tweets=api.statuses_lookup(list12,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_12.append(tweet.full_text)
    author_screenname_12.append(tweet.author.screen_name)
query_df_12 = pd.DataFrame(list(zip(list12, text_of_tweets_12,
author_screenname_12)),columns=["ID", "Text", "Screenname"])

# In[27]:
text_of_tweets_13=[]
author_screenname_13=[]
tweets=api.statuses_lookup(list13,tweet_mode="extended")
for tweet in tweets:

```

```

text_of_tweets_13.append(tweet.full_text)
author_screenname_13.append(tweet.author.screen_name)
query_df_13 = pd.DataFrame(list(zip(list13, text_of_tweets_13,
author_screenname_13)),columns=["ID", "Text", "Screenname"])

```

In[28]:

```

text_of_tweets_14=[]
author_screenname_14=[]
tweets=api.statuses_lookup(list14,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_14.append(tweet.full_text)
    author_screenname_14.append(tweet.author.screen_name)
query_df_14 = pd.DataFrame(list(zip(list14, text_of_tweets_14,
author_screenname_14)),columns=["ID", "Text", "Screenname"])

```

In[29]:

```

text_of_tweets_15=[]
author_screenname_15=[]
tweets=api.statuses_lookup(list15,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_15.append(tweet.full_text)
    author_screenname_15.append(tweet.author.screen_name)
query_df_15 = pd.DataFrame(list(zip(list15, text_of_tweets_15,
author_screenname_15)),columns=["ID", "Text", "Screenname"])

```

In[30]:

```

text_of_tweets_16=[]
author_screenname_16=[]
tweets=api.statuses_lookup(list16,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_16.append(tweet.full_text)
    author_screenname_16.append(tweet.author.screen_name)
query_df_16 = pd.DataFrame(list(zip(list16, text_of_tweets_16,
author_screenname_16)),columns=["ID", "Text", "Screenname"])

```



```

# In[31]:
text_of_tweets_17=[]
author_screenname_17=[]
tweets=api.statuses_lookup(list17,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_17.append(tweet.full_text)
    author_screenname_17.append(tweet.author.screen_name)
query_df_17 = pd.DataFrame(list(zip(list17, text_of_tweets_17,
author_screenname_17)),columns=["ID", "Text", "Screenname"])

# In[32]:
text_of_tweets_18=[]
author_screenname_18=[]
tweets=api.statuses_lookup(list18,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_18.append(tweet.full_text)
    author_screenname_18.append(tweet.author.screen_name)
query_df_18 = pd.DataFrame(list(zip(list18, text_of_tweets_18,
author_screenname_18)),columns=["ID", "Text", "Screenname"])

# In[33]:
text_of_tweets_19=[]
author_screenname_19=[]
tweets=api.statuses_lookup(list19,tweet_mode="extended")
for tweet in tweets:
    text_of_tweets_19.append(tweet.full_text)
    author_screenname_19.append(tweet.author.screen_name)
query_df_19 = pd.DataFrame(list(zip(list19, text_of_tweets_19,
author_screenname_19)),columns=["ID", "Text", "Screenname"])

# In[34]:
text_of_tweets_20=[]
author_screenname_20=[]
tweets=api.statuses_lookup(list20,tweet_mode="extended")
for tweet in tweets:

```

```

text_of_tweets_20.append(tweet.full_text)
author_screename_20.append(tweet.author.screen_name)
query_df_20 = pd.DataFrame(list(zip(list20, text_of_tweets_20,
author_screename_20)),columns=["ID", "Text", "Screenname"])

```

In[]:

#*End: Requesting for each list of 100 Tweet IDs the Tweet Text and the Author Screenname, then combining these features with their corresponding ID into a Dataframe

In[35]:

#Creating dataframe, which represents an aggregated version (by attaching new queries as new rows) of the 20 queries

```

final_tweet_df=pd.concat([query_df_1, query_df_2, query_df_3, query_df_4, query_df_5,
query_df_6, query_df_7, query_df_8, query_df_9, query_df_10, query_df_11,
query_df_12, query_df_13, query_df_14, query_df_15, query_df_16, query_df_17,
query_df_18, query_df_19, query_df_20],axis=0, ignore_index=True)

```

In[36]:

#Controlling that no Tweet was emitted by a user with Hotel in its name in order to filter out advertisements

```
authors=final_tweet_df["Screenname"].str.contains("Hotel").sum()
```

if authors>0:

```
    print ("There are {a} tweets by Hotel(s)".format(a=authors))
```

else:

```
    print("No Hotel Posts or Retweets")
```

In[37]:

#Export the combined dataset of queries

```

final_tweet_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\3 Obtaining the Twitter Data\Twitter Data\Search by topic
wedding.csv",index=False)

```

..... End - 3.3. Getting the Tweets about Weddings over 4 quarters

Start - 4.1. Data Cleaning and Splitting in Training and Test Set

.....
(20 000 reviews - Training set portion = 10%)
.....

In[1]:

#Importing the Pandas library

```
import pandas as pd
```

In[2]:

#Importing the domain specific review dataset

```
raw_data=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\2 Preparing the Yelp Proxy data for experiments\Las Vegas  
Domain-specific review and sentiment data.csv")
```

In[3]:

#Getting an overview of the dataset

```
print(raw_data)
```

In[4]:

#Sorting the text alphabetically in order to check for inappropriate domain specific reviews

```
raw_data=raw_data.sort_values("text")
```

```
raw_data = raw_data.reset_index(drop=True)
```

In[5]:

#Printing to get a feeling of how the highest and lowest sorted reviews look like

```
print(raw_data)
```

In[6]:

#Identification of Asian letters requires a more detailed selection of the data, therefore the viewing window was expanded

```
pd.set_option("display.max_rows", None)
```

In[7]:

#Listing the reviews, which rank lowest when sorted alphabetically

```
print(raw_data[200000:210718])
```

```

# In[8]:
#Checking also for unusual reviews in the beginning
print(raw_data[0:10001])

# In[9]:
#Through manually reading a Limit was set in order to eliminate the Asian characters
raw_data=raw_data[0:210624]

# In[ ]:
#*Start Proportional Sampling

# In[10]:
#Allocating all reviews into their sentiment class dataframes
one_star_rd=raw_data[raw_data["stars"] == 1]
two_star_rd=raw_data[raw_data["stars"] == 2]
three_star_rd=raw_data[raw_data["stars"] == 3]
four_star_rd=raw_data[raw_data["stars"] == 4]
five_star_rd=raw_data[raw_data["stars"] == 5]

# In[11]:
#Setting the total number of reviews, which will be split in a training and testing set
taken_amount=20000

# In[12]:
#Importing the math library
import math

# In[13]:
#Determining the discrete amount of reviews from each sentiment class
one=math.floor(taken_amount*25/100)
two=math.floor(taken_amount*11/100)
three=math.floor(taken_amount*14/100)
four=math.floor(taken_amount*22/100)
five=math.floor(taken_amount*28/100)
print(one)

```

```

print(two)
print(three)
print(four)
print(five)

# In[14]:
#Selecting a random proportional sample from each of the sentiment class dataframes

sampled_one_star_rd=one_star_rd.sample(n=one)
sampled_two_star_rd=two_star_rd.sample(n=two)
sampled_three_star_rd=three_star_rd.sample(n=three)
sampled_four_star_rd=four_star_rd.sample(n=four)
sampled_five_star_rd=five_star_rd.sample(n=five)

#Dropping the index of the prior dataframe
sampled_one_star_rd = sampled_one_star_rd.reset_index(drop=True)
sampled_two_star_rd = sampled_two_star_rd.reset_index(drop=True)
sampled_three_star_rd = sampled_three_star_rd.reset_index(drop=True)
sampled_four_star_rd = sampled_four_star_rd.reset_index(drop=True)
sampled_five_star_rd = sampled_five_star_rd.reset_index(drop=True)

# In[15]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[16]:
#Checking the dimensions of the proportionally sampled dataframes
print(sampled_one_star_rd)
print(sampled_two_star_rd)
print(sampled_three_star_rd)
print(sampled_four_star_rd)
print(sampled_five_star_rd)

# In[ ]:
#*End Proportional Sampling

```

```

# In[17]:
#Setting the portion of the training set, thereby also determining the portion of the test set
p=0.1

# In[18]:
#Computing the portions of training and testing data in absolute numbers
tr_one=math.floor(p*one)
te_one=math.floor((1-p)*one)
print(te_one/tr_one)

tr_two=math.floor(p*two)
te_two=math.floor((1-p)*two)
print(te_two/tr_two)

tr_three=math.floor(p*three)
te_three=math.floor((1-p)*three)
print(te_three/tr_three)

tr_four=math.floor(p*four)
te_four=math.floor((1-p)*four)
print(te_four/tr_four)

tr_five=math.floor(p*five)
te_five=math.floor((1-p)*five)
print(te_five/tr_five)

check=tr_one+te_one+tr_two+te_two+tr_three+te_three+tr_four+te_four+tr_five+te_five

if taken_amount==check:
    print("Everything could be sorted into the corresponding groups")
else:
    diff=taken_amount-check
    print("{} omitted reviews".format(diff))

```

```
# In[19]:
```

```
#Selecting the training dataset by choosing the absolute number of training data for each sentiment class (counting from the top of the dataframes)
```

```
training_1_star=sampled_one_star_rd.head(tr_one)
```

```
training_2_star=sampled_two_star_rd.head(tr_two)
```

```
training_3_star=sampled_three_star_rd.head(tr_three)
```

```
training_4_star=sampled_four_star_rd.head(tr_four)
```

```
training_5_star=sampled_five_star_rd.head(tr_five)
```

```
# In[20]:
```

```
#Selecting the test dataset by choosing the absolute number of test data for each sentiment class (counting from the bottom of the dataframes)
```

```
test_1_star=sampled_one_star_rd.tail(te_one)
```

```
test_2_star=sampled_two_star_rd.tail(te_two)
```

```
test_3_star=sampled_three_star_rd.tail(te_three)
```

```
test_4_star=sampled_four_star_rd.tail(te_four)
```

```
test_5_star=sampled_five_star_rd.tail(te_five)
```

```
# In[21]:
```

```
#Combining the training datasets from all sentiment classes into one single dataframe
```

```
pro_train_sample=pd.concat([training_1_star, training_2_star, training_3_star, training_4_star, training_5_star],axis=0,ignore_index=True)
```

```
# In[22]:
```

```
#Combining the test datasets from all sentiment classes into one single dataframe
```

```
pro_test_sample=pd.concat([test_1_star, test_2_star, test_3_star, test_4_star, test_5_star],axis=0,ignore_index=True)
```

```
# In[23]:
```

```
#Checking the proportions & stars(in the beginning and in the end) of the training dataset
```

```
print(pro_train_sample)
```

```
# In[24]:
```

```
#Checking the proportions & stars(in the beginning and in the end) of the test dataset
```

```
print(pro_test_sample)
```

```

# In[25]:
#Importing the Data Cleaning libraries
import re
from nltk.tokenize import word_tokenize

# In[26]:
#Necessary step to do for removing stop_words (downloading the stop-word list) and to
apply word tokenization (downloading punkt)
import nltk
nltk.download("stopwords")
nltk.download("punkt")

# In[27]:
#Importing, Initialising and Displaying the Stop-Word List
from nltk.corpus import stopwords
stop_words = stopwords.words("english")
print(stop_words)

# In[28]:
#Pandas function applied for the lower casing of the words (Here on the training partition)
pro_train_sample["text"]=pro_train_sample["text"].str.lower()

# In[29]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

# In[30]:
#Removing Stop-words = splitting each word except stop-words and creating a list
pro_train_sample["text"]=pro_train_sample["text"].apply(lambda x: [item for item in
str.split(x) if item not in stop_words])

# In[31]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

```


In[32]:

#Join all list items together by a space for each row of the particular text column

```
pro_train_sample["text"] = [" ".join(map(str, l)) for l in pro_train_sample["text"]]
```

In[33]:

#Viewing if the Output changed accordingly to the last code entered

```
print(pro_train_sample)
```

In[34]:

#Replace text which is not alphanumeric or underscore or space or tabs or line brakes with "" (i.e. Nothing)

```
abnormal_things = re.compile(r"^[^\w\s]+")
```

```
pro_train_sample["text"] = [abnormal_things.sub("", x) for x in  
pro_train_sample["text"].tolist()]
```

In[35]:

#Viewing if the Output changed accordingly to the last code entered

```
print(pro_train_sample)
```

In[36]:

#Removing Numbers i.e. replace Numbers with "" (\d+ means [0-9])

```
pro_train_sample["text"] = pro_train_sample["text"].str.replace("\d+", "")
```

In[37]:

#Viewing if the Output changed accordingly to the last code entered

```
print(pro_train_sample)
```

In[38]:

#Applying Word tokenization

```
train_list=[pro_train_sample["text"].apply(word_tokenize),pro_train_sample["stars"]]
```

In[39]:

#Converting the list into a dataframe

```
train_cleaner_df=pd.DataFrame(train_list)
```

```

# In[40]:
#Viewing if the Output changed accordingly to the last code entered
train_cleaner_df.head()

# In[41]:
#Transposing the dataframe for better visibility
train_cleaner_df=train_cleaner_df.T

# In[42]:
#Viewing if the Output changed accordingly to the last code entered
train_cleaner_df.head()

# In[43]:
#Import the Snowball Stemmer and save the Snowball Stemmer for English in a Variable
from nltk.stem.snowball import SnowballStemmer

sbs = SnowballStemmer("english")

# In[44]:
#Create a new column (stemmed_words [of the dataframe train_cleaner_df]) in which
every word (i) is stemmed in list x and saved
train_cleaner_df["stemmed_words"] = train_cleaner_df["text"].apply(lambda x: [sbs.stem(i)
for i in x])

# In[45]:
#Viewing if the Output changed accordingly to the last code entered
train_cleaner_df.head()

# In[46]:
#Removing the unstemmed text column
train_cleaner_df = train_cleaner_df.drop(columns=["text"])

# In[47]:
#Viewing if the Output changed accordingly to the last code entered

```

```
train_cleaner_df.head()
```

```
# In[48]:
```

```
#Reordering the data columns for better readability
```

```
train_reordered_df = train_cleaner_df[["stemmed_words", "stars"]]
```

```
# In[49]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(train_reordered_df)
```

```
# In[50]:
```

```
#List items (i.e. words) are all separately assigned to the list value (i.e. star value)
```

```
train_exp=train_reordered_df.explode("stemmed_words")
```

```
# In[51]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(train_exp)
```

```
# In[53]:
```

```
#See if the last tweet is also perfectly assigned
```

```
print(train_exp.loc[[1999]])
```

```
# In[54]:
```

```
#Checking for Alphabetical values again
```

```
train_exp["Alphabetical"]=train_exp["stemmed_words"].str.isalpha()
```

```
# In[55]:
```

```
#Count the number of Alphabetical values
```

```
train_exp["Alphabetical"].values.sum()
```

```
# In[56]:
```

```
#Keeping only the Alphabetical Values
```

```
train_set = train_exp[train_exp["Alphabetical"] == True]
```

```
# In[57]:
```

```

#Viewing if the Output changed accordingly to the last code entered
print(train_set)
train_set["Alphabetical"].values.sum()

# In[58]:
#Remove the column Alphabetical
train_set = train_set.drop(columns=["Alphabetical"])

# In[59]:
#Viewing if the Output changed accordingly to the last code entered
print(train_set)

# In[60]:
#Exporting the dataframe as a csv
train_set.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset
=10%\Data\train\Clean words + sentiments (unaggregated).csv", encoding="utf-8")

# In[ ]:
#The data cleaning processes from the training were applied to the test data as well
#So please see the comments and explanations for each step in the training section if
there are any questions

# In[ ]:
#*Start Data Cleaning for the test dataset

# In[61]:
pro_test_sample["text"]=pro_test_sample["text"].str.lower()

# In[62]:
print(pro_test_sample)

# In[63]:
pro_test_sample["text"]=pro_test_sample["text"].apply(lambda x: [item for item in str.split(x)
if item not in stop_words])

```

```

# In[64]:
print(pro_test_sample)

# In[65]:
pro_test_sample["text"] = [" ".join(map(str, l)) for l in pro_test_sample["text"]]

# In[66]:
print(pro_test_sample)

# In[67]:
abnormalities = re.compile(r"^\w\s+")
pro_test_sample["text"] = [abnormalities.sub("", x) for x in pro_test_sample["text"].tolist()]

# In[68]:
print(pro_test_sample)

# In[69]:
pro_test_sample["text"] = pro_test_sample["text"].str.replace("\d+", "")

# In[70]:
print(pro_test_sample)

# In[71]:
test_list=[pro_test_sample["text"].apply(word_tokenize),pro_test_sample["stars"]]

# In[72]:
test_cleaner_df=pd.DataFrame(test_list)

# In[73]:
test_cleaner_df.head()

# In[74]:
test_cleaner_df=test_cleaner_df.T

```

```

# In[75]:
test_cleaner_df.head()

# In[76]:
test_cleaner_df["stemmed_words"] = test_cleaner_df["text"].apply(lambda x: [sbs.stem(i)
for i in x])

# In[77]:
test_cleaner_df.head()

# In[78]:
test_cleaner_df = test_cleaner_df.drop(columns=["text"])

# In[79]:
test_cleaner_df.head()

# In[80]:
test_reordered_df = test_cleaner_df[["stemmed_words", "stars"]]

# In[81]:
print(test_reordered_df)

# In[82]:
test_exp=test_reordered_df.explode("stemmed_words")

# In[83]:
print(test_exp)

# In[84]:
print(test_exp.loc[[17999]])

# In[85]:
test_exp["Alphabetical"]=test_exp["stemmed_words"].str.isalpha()

# In[86]:

```

```
test_exp["Alphabetical"].values.sum()
```

```
# In[87]:
```

```
test_set = test_exp[test_exp["Alphabetical"] == True]
```

```
# In[88]:
```

```
print(test_set)
```

```
test_set["Alphabetical"].values.sum()
```

```
# In[89]:
```

```
test_set = test_set.drop(columns=["Alphabetical"])
```

```
# In[90]:
```

```
print(test_set)
```

```
# In[91]:
```

```
test_set.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset  
=10%\Data\test\Clean test words + sentiments (unaggregated).csv",encoding="utf-8")
```

```
# In[ ]:
```

```
#*End Data Cleaning for the test dataset
```

End - 4.1. Data Cleaning and Splitting in Training and Test Set

..... (20 000 reviews - Training set portion = 10%)

Start - 4.2. Data Cleaning and Splitting in Training and Test Set

..... (200 000 reviews - Training set portion = 99%)

```
# In[1]:
```

```
#Importing the Pandas library
```

```
import pandas as pd
```

```
# In[2]:
```

```
#Importing the domain specific review dataset
```

```
raw_data=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\2 Preparing the Yelp Proxy data for experiments\Las Vegas  
Domain-specific review and sentiment data.csv")
```

```
# In[3]:
```

```
#Getting an overview of the dataset  
print(raw_data)
```

```
# In[4]:
```

```
#Sorting the text alphabetically in order to check for inappropriate domain specific reviews  
raw_data=raw_data.sort_values("text")  
raw_data = raw_data.reset_index(drop=True)
```

```
# In[5]:
```

```
#Printing to get a feeling of how the highest and lowest sorted reviews look like  
print(raw_data)
```

```
# In[6]:
```

```
#Identification of Asian letters requires a more detailed selection of the data, therefore the  
viewing window was expanded  
pd.set_option("display.max_rows", None)
```

```
# In[7]:
```

```
#Listing the reviews, which rank lowest when sorted alphabetically  
print(raw_data[200000:210718])
```

```
# In[8]:
```

```
#Checking also for unusual reviews in the beginning  
print(raw_data[0:10001])
```

```
# In[9]:
```

```
#Through manually reading a Limit was set in order to eliminate the Asian characters  
raw_data=raw_data[0:210624]
```

```
# In[ ]:
```



```
#*Start Proportional Sampling
```

```
# In[10]:
```

```
#Allocating all reviews into their sentiment class dataframes
```

```
one_star_rd=raw_data[raw_data["stars"] == 1]
```

```
two_star_rd=raw_data[raw_data["stars"] == 2]
```

```
three_star_rd=raw_data[raw_data["stars"] == 3]
```

```
four_star_rd=raw_data[raw_data["stars"] == 4]
```

```
five_star_rd=raw_data[raw_data["stars"] == 5]
```

```
# In[11]:
```

```
#Setting the total number of reviews, which will be split in a training and testing set
```

```
taken_amount=200000
```

```
# In[12]:
```

```
#Importing the math library
```

```
import math
```

```
# In[13]:
```

```
#Determining the discrete amount of reviews from each sentiment class
```

```
one=math.floor(taken_amount*25/100)
```

```
two=math.floor(taken_amount*11/100)
```

```
three=math.floor(taken_amount*14/100)
```

```
four=math.floor(taken_amount*22/100)
```

```
five=math.floor(taken_amount*28/100)
```

```
print(one)
```

```
print(two)
```

```
print(three)
```

```
print(four)
```

```
print(five)
```

```
# In[14]:
```

```
#Selecting a random proportional sample from each of the sentiment class dataframes
```

```
sampled_one_star_rd=one_star_rd.sample(n=one)
```

```
sampled_two_star_rd=two_star_rd.sample(n=two)
sampled_three_star_rd=three_star_rd.sample(n=three)
sampled_four_star_rd=four_star_rd.sample(n=four)
sampled_five_star_rd=five_star_rd.sample(n=five)
```

```
#Dropping the index of the prior dataframe
```

```
sampled_one_star_rd = sampled_one_star_rd.reset_index(drop=True)
sampled_two_star_rd = sampled_two_star_rd.reset_index(drop=True)
sampled_three_star_rd = sampled_three_star_rd.reset_index(drop=True)
sampled_four_star_rd = sampled_four_star_rd.reset_index(drop=True)
sampled_five_star_rd = sampled_five_star_rd.reset_index(drop=True)
```

```
# In[15]:
```

```
#Resetting the max view option to the default value
```

```
pd.reset_option("display.max_rows")
```

```
# In[16]:
```

```
#Checking the dimensions of the proportionally sampled dataframes
```

```
print(sampled_one_star_rd)
print(sampled_two_star_rd)
print(sampled_three_star_rd)
print(sampled_four_star_rd)
print(sampled_five_star_rd)
```

```
# In[ ]:
```

```
##*End Proportional Sampling
```

```
# In[17]:
```

```
#Setting the portion of the training set, thereby also determining the portion of the test set
```

```
p=0.99
```

```
# In[18]:
```

```
#Computing the portions of training and testing data in absolute numbers
```

```
tr_one=math.floor(p*one)
```

```
te_one=math.floor((1-p)*one)
```

```

print(te_one/tr_one)

tr_two=math.floor(p*two)
te_two=math.floor((1-p)*two)
print(te_two/tr_two)

tr_three=math.floor(p*three)
te_three=math.floor((1-p)*three)
print(te_three/tr_three)

tr_four=math.floor(p*four)
te_four=math.floor((1-p)*four)
print(te_four/tr_four)

tr_five=math.floor(p*five)
te_five=math.floor((1-p)*five)
print(te_five/tr_five)

check=tr_one+te_one+tr_two+te_two+tr_three+te_three+tr_four+te_four+tr_five+te_five

if taken_amount==check:
    print("Everything could be sorted into the corresponding groups")
else:
    diff=taken_amount-check
    print("{} omitted reviews".format(diff))

# In[19]:
#Selecting the training dataset by choosing the absolute number of training data for each
sentiment class (counting from the top of the dataframes)
training_1_star=sampled_one_star_rd.head(tr_one)
training_2_star=sampled_two_star_rd.head(tr_two)
training_3_star=sampled_three_star_rd.head(tr_three)
training_4_star=sampled_four_star_rd.head(tr_four)
training_5_star=sampled_five_star_rd.head(tr_five)

```

In[20]:

#Selecting the test dataset by choosing the absolute number of test data for each sentiment class (counting from the bottom of the dataframes)

```
test_1_star=sampled_one_star_rd.tail(te_one)
```

```
test_2_star=sampled_two_star_rd.tail(te_two)
```

```
test_3_star=sampled_three_star_rd.tail(te_three)
```

```
test_4_star=sampled_four_star_rd.tail(te_four)
```

```
test_5_star=sampled_five_star_rd.tail(te_five)
```

In[21]:

#Combining the training datasets from all sentiment classes into one single dataframe

```
pro_train_sample=pd.concat([training_1_star, training_2_star, training_3_star,  
training_4_star, training_5_star],axis=0,ignore_index=True)
```

In[22]:

#Combining the test datasets from all sentiment classes into one single dataframe

```
pro_test_sample=pd.concat([test_1_star, test_2_star, test_3_star, test_4_star,  
test_5_star],axis=0,ignore_index=True)
```

In[23]:

#Checking the proportions & stars(in the beginning and in the end) of the training dataset

```
print(pro_train_sample)
```

In[24]:

#Checking the proportions & stars(in the beginning and in the end) of the test dataset

```
print(pro_test_sample)
```

In[25]:

#Importing Data Cleaning libraries

```
import re
```

```
from nltk.tokenize import word_tokenize
```

In[26]:

#Necessary step to do for removing stop_words (downloading the stop-word list) and to apply word tokenization (downloading punkt)

```

import nltk
nltk.download("stopwords")
nltk.download("punkt")

# In[27]:
#Importing, Initialising and Displaying the Stop-Word List
from nltk.corpus import stopwords
stop_words = stopwords.words("english")
print(stop_words)

# In[28]:
#Pandas function applied for the lower casing of the words (Here on the training partition)
pro_train_sample["text"]=pro_train_sample["text"].str.lower()

# In[29]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

# In[30]:
#Removing Stop-words = splitting each word except stop-words and creating a list
pro_train_sample["text"]=pro_train_sample["text"].apply(lambda x: [item for item in
str.split(x) if item not in stop_words])

# In[31]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

# In[32]:
#Join all list items together by a space for each row of the particular text column
pro_train_sample["text"] = [" ".join(map(str, l)) for l in pro_train_sample["text"]]

# In[33]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

```

```

# In[34]:
#Replace text which is not alphanumeric or underscore or space or tabs or line brakes with
"" (i.e. Nothing)

abnormal_things = re.compile(r"^\w\s+")
pro_train_sample["text"] = [abnormal_things.sub("", x) for x in
pro_train_sample["text"].tolist()]

# In[35]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

# In[36]:
#Removing Numbers i.e. replace Numbers with "" (\d+ means [0-9])
pro_train_sample["text"] = pro_train_sample["text"].str.replace("\d+", "")

# In[37]:
#Viewing if the Output changed accordingly to the last code entered
print(pro_train_sample)

# In[38]:
#Applying Word tokenization
train_list=[pro_train_sample["text"].apply(word_tokenize),pro_train_sample["stars"]]

# In[39]:
#Converting the list into a dataframe
train_cleaner_df=pd.DataFrame(train_list)

# In[40]:
#Viewing if the Output changed accordingly to the last code entered
train_cleaner_df.head()

# In[41]:
#Transposing the dataframe for better visibility
train_cleaner_df=train_cleaner_df.T

```

In[42]:

#Viewing if the Output changed accordingly to the last code entered

```
train_cleaner_df.head()
```

In[43]:

#Import the Snowball Stemmer and save the Snowball Stemmer for English in a Variable
from nltk.stem.snowball import SnowballStemmer

```
sbs = SnowballStemmer("english")
```

In[44]:

#Create a new column (stemmed_words [of the dataframe train_cleaner_df]) in which
every word (i) is stemmed in list x and saved

```
train_cleaner_df["stemmed_words"] = train_cleaner_df["text"].apply(lambda x: [sbs.stem(i)  
for i in x])
```

In[45]:

#Viewing if the Output changed accordingly to the last code entered

```
train_cleaner_df.head()
```

In[46]:

#Removing the unstemmed text column

```
train_cleaner_df = train_cleaner_df.drop(columns=["text"])
```

In[47]:

#Viewing if the Output changed accordingly to the last code entered

```
train_cleaner_df.head()
```

In[48]:

#Reordering the data columns for better readability

```
train_reordered_df = train_cleaner_df[["stemmed_words", "stars"]]
```

In[49]:

#Viewing if the Output changed accordingly to the last code entered

```
print(train_reordered_df)
```

```
# In[50]:
```

```
#List items (i.e. words) are all separately assigned to the list value (i.e. star value)
```

```
train_exp=train_reordered_df.explode("stemmed_words")
```

```
# In[51]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(train_exp)
```

```
# In[52]:
```

```
#See if the last tweet is also perfectly assigned
```

```
print(train_exp.loc[[197999]])
```

```
# In[53]:
```

```
#Checking for Alphabetical values again
```

```
train_exp["Alphabetical"]=train_exp["stemmed_words"].str.isalpha()
```

```
# In[54]:
```

```
#Count the number of Alphabetical values (Not indicated here)
```

```
train_exp["Alphabetical"].values.sum()
```

```
# In[55]:
```

```
#Keeping only the Alphabetical Values
```

```
train_set = train_exp[train_exp["Alphabetical"] == True]
```

```
# In[56]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(train_set)
```

```
train_set["Alphabetical"].values.sum()
```

```
# In[57]:
```

```
#Remove the column Alphabetical
```

```
train_set = train_set.drop(columns=["Alphabetical"])
```



```

# In[58]:
#Viewing if the Output changed accordingly to the last code entered
print(train_set)

# In[59]:
#Exporting the dataframe as a csv
train_set.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Data\train\Clean words + sentiments (unaggregated).csv",
encoding="utf-8")

# In[ ]:
#The data cleaning processes from the training were applied to the test data as well
#So please see the comments and explanations for each step in the training section if
there are any questions

# In[ ]:
#*Start Data Cleaning for the test dataset

# In[60]:
pro_test_sample["text"]=pro_test_sample["text"].str.lower()

# In[61]:
print(pro_test_sample)

# In[62]:
pro_test_sample["text"]=pro_test_sample["text"].apply(lambda x: [item for item in str.split(x)
if item not in stop_words])

# In[63]:
print(pro_test_sample)

# In[64]:
pro_test_sample["text"] = [" ".join(map(str, l)) for l in pro_test_sample["text"]]

```

```

# In[65]:
print(pro_test_sample)

# In[66]:
abnormalities = re.compile(r"^\w\s+")
pro_test_sample["text"] = [abnormalities.sub("", x) for x in pro_test_sample["text"].tolist()]

# In[67]:
print(pro_test_sample)

# In[68]:
pro_test_sample["text"] = pro_test_sample["text"].str.replace("\d+", "")

# In[69]:
print(pro_test_sample)

# In[70]:
test_list=[pro_test_sample["text"].apply(word_tokenize),pro_test_sample["stars"]]

# In[71]:
test_cleaner_df=pd.DataFrame(test_list)

# In[72]:
test_cleaner_df.head()

# In[73]:
test_cleaner_df=test_cleaner_df.T

# In[74]:
test_cleaner_df.head()

# In[75]:
test_cleaner_df["stemmed_words"] = test_cleaner_df["text"].apply(lambda x: [sbs.stem(i)
for i in x])

```

```
# In[76]:
test_cleaner_df.head()

# In[77]:
test_cleaner_df = test_cleaner_df.drop(columns=["text"])

# In[78]:
test_cleaner_df.head()

# In[79]:
test_reordered_df = test_cleaner_df[["stemmed_words", "stars"]]

# In[80]:
print(test_reordered_df)

# In[81]:
test_exp=test_reordered_df.explode("stemmed_words")

# In[82]:
print(test_exp)

# In[83]:
print(test_exp.loc[[1999]])

# In[84]:
test_exp["Alphabetical"]=test_exp["stemmed_words"].str.isalpha()

# In[85]:
test_exp["Alphabetical"].values.sum()

# In[86]:
test_set = test_exp[test_exp["Alphabetical"] == True]

# In[87]:
print(test_set)
```

```
test_set["Alphabetical"].values.sum()
```

```
# In[88]:
```

```
test_set = test_set.drop(columns=["Alphabetical"])
```

```
# In[89]:
```

```
print(test_set)
```

```
# In[90]:
```

```
test_set.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews  
Trainset = 99%\Data\test\Clean test words + sentiments  
(unaggregated).csv",encoding="utf-8")
```

```
# In[ ]:
```

```
##*End Data Cleaning for the test dataset
```

End - 4.2. Data Cleaning and Splitting in Training and Test Set

..... (200 000 reviews - Training set portion = 99%)

Start - 5.1. Way to likelihood table of the training set

..... (20 000 reviews - Training set portion = 10%)

```
# In[1]:
```

```
#Importing the Pandas Library
```

```
import pandas as pd
```

```
# In[2]:
```

```
#Importing the training set
```

```
train=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset  
=10%\Data\train\Clean words + sentiments (unaggregated).csv")
```

```
# In[3]:
```

```
#Inspecting the dataframe structure of the training set  
train.head()
```

```
# In[4]:  
#Removing the column Unnamed: 0  
train_set = train.drop(columns=["Unnamed: 0"])
```

```
# In[5]:  
#Viewing if the Output changed accordingly to the last code entered  
print(train_set)
```

```
# In[6]:  
#Sorting the training set alphabetically (according to the stemmed words)  
train_set=train_set.sort_values("stemmed_words")  
train_set = train_set.reset_index(drop=True)
```

```
# In[7]:  
#Viewing if the Output changed accordingly to the last code entered  
print(train_set)
```

```
# In[8]:  
#German words were identified at the end of the dataframe so a more detailed view inside  
the dataset was necessary  
pd.set_option("display.max_rows", None)
```

```
# In[9]:  
#Displaying the end of the data (i.e. words which rank lowest, when sorted alphabetically)  
print(train_set[150000:153407])
```

```
# In[10]:  
#The beginning of the dataframe was also inspected to check if German words were  
occurring  
print(train_set[0:1000])
```

```
# In[11]:
```

```
#Resetting the max view option to the default value
```

```
pd.reset_option("display.max_rows")
```

```
# In[12]:
```

```
#Dataset was limited by setting zero as the last word in the alphabetical sorted dataframe
```

```
train_set=train_set[0:153363]
```

```
print(train_set[153000:153363])
```

```
# In[13]:
```

```
#New sentiment columns are initialised with 0
```

```
train_set["1_star"]=0
```

```
train_set["2_star"]=0
```

```
train_set["3_star"]=0
```

```
train_set["4_star"]=0
```

```
train_set["5_star"]=0
```

```
# In[14]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(train_set)
```

```
# In[15]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
1
```

```
df_1_star = train_set.loc[train_set["stars"] == 1].copy()
```

```
print(df_1_star)
```

```
# In[16]:
```

```
#Each word (with the stars rating =1) was assigned a 1 in the 1_star column
```

```
df_1_star.loc[df_1_star["stars"] == 1, "1_star"] = 1
```

```
print(df_1_star)
```

```
# In[17]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
2
```

```
df_2_star = train_set.loc[train_set["stars"] == 2].copy()
```

```
print(df_2_star)
```

```
# In[18]:
```

```
#Each word (with the stars rating =2) was assigned a 1 in the 2_star column
```

```
df_2_star.loc[df_2_star["stars"] == 2, "2_star"] = 1
```

```
print(df_2_star)
```

```
# In[19]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
3
```

```
df_3_star = train_set.loc[train_set["stars"] == 3].copy()
```

```
print(df_3_star)
```

```
# In[20]:
```

```
#Each word (with the stars rating =3) was assigned a 1 in the 3_star column
```

```
df_3_star.loc[df_3_star["stars"] == 3, "3_star"] = 1
```

```
print(df_3_star)
```

```
# In[21]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
4
```

```
df_4_star = train_set.loc[train_set["stars"] == 4].copy()
```

```
print(df_4_star)
```

```
# In[22]:
```

```
#Each word (with the stars rating =4) was assigned a 1 in the 4_star column
```

```
df_4_star.loc[df_4_star["stars"] == 4, "4_star"] = 1
```

```
print(df_4_star)
```

```
# In[23]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
5
```

```
df_5_star = train_set.loc[train_set["stars"] == 5].copy()
```

```
print(df_5_star)
```

```

# In[24]:
#Each word (with the stars rating =5) was assigned a 1 in the 5_star column
df_5_star.loc[df_5_star["stars"] == 5, "5_star"] = 1
print(df_5_star)

# In[25]:
#All the copies of the training dataset were combined into a single dataset
freq_matrix_train=pd.concat([df_1_star, df_2_star, df_3_star, df_4_star,
df_5_star],axis=0,ignore_index=True)
print(freq_matrix_train)

# In[26]:
#The column stars was removed due to redundancy
freq_matrix_train = freq_matrix_train.drop(columns=["stars"])
print(freq_matrix_train)

# In[27]:
#The datatypes for each column were set
freq_matrix_train["1_star"]=freq_matrix_train ["1_star"].astype(int)
freq_matrix_train["2_star"]=freq_matrix_train ["2_star"].astype(int)
freq_matrix_train["3_star"]=freq_matrix_train ["3_star"].astype(int)
freq_matrix_train["4_star"]=freq_matrix_train ["4_star"].astype(int)
freq_matrix_train["5_star"]=freq_matrix_train ["5_star"].astype(int)
freq_matrix_train["stemmed_words"]=freq_matrix_train ["stemmed_words"].astype(str)

# In[28]:
#The frequency dataframe was exported as csv
freq_matrix_train.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews
Trainset = 10 %\Tables\Frequency\ Not aggregated frequency
table.csv",encoding="utf-8",index=False)

# In[29]:
#One aggregated frequency matrix is generated to see the sentiment distribution of each
word

```



```
aggregat_freq_matrix=freq_matrix_train.groupby(["stemmed_words"],as_index=False).sum()
print(aggregat_freq_matrix)
```

```
# In[30]:
```

```
#A total of word occurrences was added per word in the column All
```

```
aggregat_freq_matrix["All"]=aggregat_freq_matrix["1_star"]+aggregat_freq_matrix["2_star"]
+aggregat_freq_matrix["3_star"]+aggregat_freq_matrix["4_star"]
+aggregat_freq_matrix["5_star"]
print(aggregat_freq_matrix)
```

```
# In[31]:
```

```
#A validation value is computed
```

```
aggregat_freq_matrix["All"].sum()
```

```
# In[32]:
```

```
#The total of word occurrences for each sentiment class + for all sentiment classes is computed
```

```
a_1=aggregat_freq_matrix["1_star"].sum()
a_2=aggregat_freq_matrix["2_star"].sum()
a_3=aggregat_freq_matrix["3_star"].sum()
a_4=aggregat_freq_matrix["4_star"].sum()
a_5=aggregat_freq_matrix["5_star"].sum()
a_6=a_1+a_2+a_3+a_4+a_5
```

```
# In[33]:
```

```
#The total of word occurrences for each sentiment class + for all sentiment classes is converted into a dataframe
```

```
df_all_rows = pd.DataFrame({
    "stemmed_words":["NaN"],
    "1_star":[a_1],
    "2_star":[a_2],
    "3_star":[a_3],
    "4_star":[a_4],
    "5_star":[a_5],
```

```
"All":[a_6]],index=["All"])
```

```
# In[34]:
```

```
#The just established dataframe is going to be appended to the aggregated frequency matrix
```

```
aggregat_freq_matrix=aggregat_freq_matrix.append(df_all_rows)
```

```
# In[35]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(aggregat_freq_matrix)
```

```
# In[36]:
```

```
#Exporting the aggregated frequency matrix as csv
```

```
aggregat_freq_matrix.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset =10%\Tables\Frequency\ aggregated frequency table with totals.csv",encoding="utf-8")
```

```
# In[37]:
```

```
#The word likelihood is calculated and saved in a newly generated column
```

```
aggregat_freq_matrix["Word Likelihood"]=(aggregat_freq_matrix["All"]/  
aggregat_freq_matrix["All"].iloc[-1])
```

```
print(aggregat_freq_matrix)
```

```
# In[38]:
```

```
#Sentiment Class Probabilities are calculated and added as the last row
```

```
aggregat_freq_matrix.loc["class_prob"] = [None,aggregat_freq_matrix["1_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["2_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["3_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["4_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["5_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["All"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],None]  
print(aggregat_freq_matrix)
```

```
# In[39]:
#Formatting word likelihood as float in order to try to prevent a 1e-10 format from
displaying in the dataframe
aggregat_freq_matrix["Word Likelihood"]=aggregat_freq_matrix["Word
Likelihood"].astype(float)
print(aggregat_freq_matrix)
```

```
# In[40]:
#Viewing if the Output changed accordingly to the last code entered
aggregat_freq_matrix.head()
```

```
# In[41]:
#Exporting the Likelihood Table as csv
aggregat_freq_matrix.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University
of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews
Trainset =10%\Tables\Likelihood table - Training set.csv",encoding="utf-8")
```

End - 5.1. Way to likelihood table of the training set

(20 000 reviews - Training set portion = 10%)

Start - 5.2. Way to likelihood table of the training set

(200 000 reviews - Training set portion = 99%)

```
# In[1]:
#Importing the Pandas Library
import pandas as pd
```

```
# In[2]:
#Importing the training set
train=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Data\train\Clean words + sentiments (unaggregated).csv")
```

```
# In[3]:
```

```
#Inspecting the dataframe structure of the training set  
train.head()
```

```
# In[4]:  
#Removing the column Unnamed: 0  
train_set = train.drop(columns=["Unnamed: 0"])
```

```
# In[5]:  
#Viewing if the Output changed accordingly to the last code entered  
print(train_set)
```

```
# In[6]:  
#Sorting the training set alphabetically (according to the stemmed words)  
train_set=train_set.sort_values("stemmed_words")  
train_set = train_set.reset_index(drop=True)
```

```
# In[7]:  
#Viewing if the Output changed accordingly to the last code entered  
print(train_set)
```

```
# In[8]:  
#NaN words were identified at the end of the dataframe so a more detailed view inside the  
dataset was necessary  
pd.set_option("display.max_rows", None)
```

```
# In[9]:  
#Displaying the end of the data (i.e. words which rank lowest, when sorted alphabetically)  
print(train_set[15190000:15203695])
```

```
# In[10]:  
#The beginning of the dataframe was also inspected to check if NaN words were occurring  
print(train_set[0:10000])
```

```
# In[11]:  
#Resetting the max view option to the default value
```

```
pd.reset_option("display.max_rows")
```

```
# In[12]:
```

```
#Dataset was limited by setting zero as the last word in the alphabetical sorted dataframe
```

```
train_set=train_set[0:15199715]
```

```
print(train_set[15190000:15199715])
```

```
# In[13]:
```

```
#New sentiment columns are initialised with 0
```

```
train_set["1_star"]=0
```

```
train_set["2_star"]=0
```

```
train_set["3_star"]=0
```

```
train_set["4_star"]=0
```

```
train_set["5_star"]=0
```

```
# In[14]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(train_set)
```

```
# In[15]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
1
```

```
df_1_star = train_set.loc[train_set["stars"] == 1].copy()
```

```
print(df_1_star)
```

```
# In[16]:
```

```
#Each word (with the stars rating =1) was assigned a 1 in the 1_star column
```

```
df_1_star.loc[df_1_star["stars"] == 1, "1_star"] = 1
```

```
print(df_1_star)
```

```
# In[17]:
```

```
#A copy of the training dataframe was made just including words with a sentiment rating of  
2
```

```
df_2_star = train_set.loc[train_set["stars"] == 2].copy()
```

```
print(df_2_star)
```

In[18]:

#Each word (with the stars rating =2) was assigned a 1 in the 2_star column

```
df_2_star.loc[df_2_star["stars"] == 2, "2_star"] = 1
```

```
print(df_2_star)
```

In[19]:

#A copy of the training dataframe was made just including words with a sentiment rating of 3

```
df_3_star = train_set.loc[train_set["stars"] == 3].copy()
```

```
print(df_3_star)
```

In[20]:

#Each word (with the stars rating =3) was assigned a 1 in the 3_star column

```
df_3_star.loc[df_3_star["stars"] == 3, "3_star"] = 1
```

```
print(df_3_star)
```

In[21]:

#A copy of the training dataframe was made just including words with a sentiment rating of 4

```
df_4_star = train_set.loc[train_set["stars"] == 4].copy()
```

```
print(df_4_star)
```

In[22]:

#Each word (with the stars rating =4) was assigned a 1 in the 4_star column

```
df_4_star.loc[df_4_star["stars"] == 4, "4_star"] = 1
```

```
print(df_4_star)
```

In[23]:

#A copy of the training dataframe was made just including words with a sentiment rating of 5

```
df_5_star = train_set.loc[train_set["stars"] == 5].copy()
```

```
print(df_5_star)
```

In[24]:

#Each word (with the stars rating =5) was assigned a 1 in the 5_star column

```
df_5_star.loc[df_5_star["stars"] == 5, "5_star"] = 1
```

```
print(df_5_star)
```

In[25]:

#All the copies of the training dataset were combined into a single dataset

```
freq_matrix_train=pd.concat([df_1_star, df_2_star, df_3_star, df_4_star,  
df_5_star],axis=0,ignore_index=True)
```

```
print(freq_matrix_train)
```

In[26]:

#The column stars was removed due to redundancy

```
freq_matrix_train = freq_matrix_train.drop(columns=["stars"])
```

```
print(freq_matrix_train)
```

In[27]:

#The datatypes for each column were set

```
freq_matrix_train["1_star"]=freq_matrix_train ["1_star"].astype(int)
```

```
freq_matrix_train["2_star"]=freq_matrix_train ["2_star"].astype(int)
```

```
freq_matrix_train["3_star"]=freq_matrix_train ["3_star"].astype(int)
```

```
freq_matrix_train["4_star"]=freq_matrix_train ["4_star"].astype(int)
```

```
freq_matrix_train["5_star"]=freq_matrix_train ["5_star"].astype(int)
```

```
freq_matrix_train["stemmed_words"]=freq_matrix_train ["stemmed_words"].astype(str)
```

In[28]:

#The frequency dataframe was exported as csv

```
freq_matrix_train.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews  
Train set = 99 %\Tables\Frequency\ Not aggregated frequency  
table.csv",encoding="utf-8",index=False)
```

In[29]:

#One aggregated frequency matrix is generated to see the sentiment distribution of each word

```
aggregat_freq_matrix=freq_matrix_train.groupby(["stemmed_words"],as_index=False).sum()
print(aggregat_freq_matrix)
```

```
# In[30]:
```

```
#A total of word occurrences was added per word in the column All
```

```
aggregat_freq_matrix["All"]=aggregat_freq_matrix["1_star"]+aggregat_freq_matrix["2_star"]
+aggregat_freq_matrix["3_star"]+aggregat_freq_matrix["4_star"]
+aggregat_freq_matrix["5_star"]
print(aggregat_freq_matrix)
```

```
# In[31]:
```

```
#A validation value is computed
```

```
aggregat_freq_matrix["All"].sum()
```

```
# In[32]:
```

```
#The total of word occurrences for each sentiment class + for all sentiment classes is computed
```

```
a_1=aggregat_freq_matrix["1_star"].sum()
a_2=aggregat_freq_matrix["2_star"].sum()
a_3=aggregat_freq_matrix["3_star"].sum()
a_4=aggregat_freq_matrix["4_star"].sum()
a_5=aggregat_freq_matrix["5_star"].sum()
a_6=a_1+a_2+a_3+a_4+a_5
```

```
# In[33]:
```

```
#The total of word occurrences for each sentiment class + for all sentiment classes is converted into a dataframe
```

```
df_all_rows = pd.DataFrame({
    "stemmed_words":["NaN"],
    "1_star":[a_1],
    "2_star":[a_2],
    "3_star":[a_3],
    "4_star":[a_4],
    "5_star":[a_5],
```



```
"All":{a_6}},index=["All"])
```

```
# In[34]:
```

```
#The just established dataframe is going to be appended to the aggregated frequency matrix
```

```
aggregat_freq_matrix=aggregat_freq_matrix.append(df_all_rows)
```

```
# In[35]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(aggregat_freq_matrix)
```

```
# In[36]:
```

```
#Exporting the aggregated frequency matrix as csv
```

```
aggregat_freq_matrix.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews Trainset =99%\Tables\Frequency\ aggregated frequency table with totals.csv",encoding="utf-8")
```

```
# In[37]:
```

```
#The word likelihood is calculated and saved in a newly generated column
```

```
aggregat_freq_matrix["Word Likelihood"]=(aggregat_freq_matrix["All"]/  
aggregat_freq_matrix["All"].iloc[-1])
```

```
print(aggregat_freq_matrix)
```

```
# In[38]:
```

```
#Sentiment Class Probabilities are calculated and added as the last row
```

```
aggregat_freq_matrix.loc["class_prob"] = [None,aggregat_freq_matrix["1_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["2_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["3_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["4_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["5_star"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],aggregat_freq_matrix["All"].iloc[-1]/  
aggregat_freq_matrix["All"].iloc[-1],None]  
print(aggregat_freq_matrix)
```

```
# In[39]:
#Formatting word likelihood as float in order to try to prevent a 1e-10 format from
displaying in the dataframe
aggregat_freq_matrix["Word Likelihood"]=aggregat_freq_matrix["Word
Likelihood"].astype(float)
print(aggregat_freq_matrix)
```

```
# In[40]:
#Viewing if the Output changed accordingly to the last code entered
aggregat_freq_matrix.head()
```

```
# In[41]:
#Exporting the Likelihood Table as csv
aggregat_freq_matrix.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University
of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Tables\Likelihood table - Training set.csv",encoding="utf-8")
```

End - 5.2. Way to likelihood table of the training set

(200 000 reviews - Training set portion = 99%)

Start - 6.1. Way to Naive Bayes Sentiment Scores

(20 000 reviews - Training set portion = 10%)

```
# In[1]:
#Importing the Pandas library
import pandas as pd
```

```
# In[2]:
#Importing the Likelihood Table
like_train=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews
Trainset =10%\Tables\Likelihood table - Training set.csv")
```

```
# In[3]:
#Inspecting the Likelihood Table
```

```
print (like_train)
```

```
# In[4]:
```

```
#Renaming column Unnamed: 0 as Index
```

```
like_train.rename(columns={"Unnamed: 0":"Index"}, inplace=True)
```

```
# In[5]:
```

```
#Setting column Index as index
```

```
like_train.set_index("Index", inplace=True)
```

```
# In[6]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print (like_train)
```

```
# In[7]:
```

```
#Calculating the probability that given a sentiment class an individual word is occurring (for all words and sentiment classes respectively)
```

```
like_train["NB_red_1_star"]=like_train["1_star"]/like_train.at["All","1_star"]
```

```
like_train["NB_red_2_star"]=like_train["2_star"]/like_train.at["All","2_star"]
```

```
like_train["NB_red_3_star"]=like_train["3_star"]/like_train.at["All","3_star"]
```

```
like_train["NB_red_4_star"]=like_train["4_star"]/like_train.at["All","4_star"]
```

```
like_train["NB_red_5_star"]=like_train["5_star"]/like_train.at["All","5_star"]
```

```
# In[8]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print (like_train)
```

```
# In[9]:
```

```
#Converting the class probabilities of the sentiments to columns
```

```
like_train["blue_class_prob_1_star"] = like_train.at["class_prob","1_star"]
```

```
like_train["blue_class_prob_2_star"] = like_train.at["class_prob","2_star"]
```

```
like_train["blue_class_prob_3_star"] = like_train.at["class_prob","3_star"]
```

```
like_train["blue_class_prob_4_star"] = like_train.at["class_prob","4_star"]
```

```
like_train["blue_class_prob_5_star"] = like_train.at["class_prob","5_star"]
```

```

# In[10]:
#Viewing if the Output changed accordingly to the last code entered
print(like_train)

# In[11]:
#Calculating a sentiment class probability given each word respectively (i.e. Naive Bayes
probability) for all of the sentiment classes respectively
like_train["yellow_NB_prob_1_star"]=like_train["NB_red_1_star"]*like_train["blue_class_pr
ob_1_star"]/like_train["Word Likelihood"]
like_train["yellow_NB_prob_2_star"]=like_train["NB_red_2_star"]*like_train["blue_class_pr
ob_2_star"]/like_train["Word Likelihood"]
like_train["yellow_NB_prob_3_star"]=like_train["NB_red_3_star"]*like_train["blue_class_pr
ob_3_star"]/like_train["Word Likelihood"]
like_train["yellow_NB_prob_4_star"]=like_train["NB_red_4_star"]*like_train["blue_class_pr
ob_4_star"]/like_train["Word Likelihood"]
like_train["yellow_NB_prob_5_star"]=like_train["NB_red_5_star"]*like_train["blue_class_pr
ob_5_star"]/like_train["Word Likelihood"]

# In[12]:
#Viewing if the Output changed accordingly to the last code entered
print(like_train)

# In[13]:
#Creating the sentiment points based on the Naive Bayes probabilities and the star
classification scheme by Yelp (1-5) for each word
like_train["Sentiment points"]=5*like_train["yellow_NB_prob_5_star"]
+4*like_train["yellow_NB_prob_4_star"]+3*like_train["yellow_NB_prob_3_star"]
+2*like_train["yellow_NB_prob_2_star"]+1*like_train["yellow_NB_prob_1_star"]

# In[14]:
#Viewing if the Output changed accordingly to the last code entered
like_train.head()

# In[15]:
#Exporting the most expansive Table after having performed all calculations

```

```
like_train.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset  
=10%\Tables\Maximal Table when conducting Naive Bayes Sentiment points calc.csv")
```

```
# In[16]:
```

```
#Deleting unnecessary columns
```

```
word_to_sentiment_points =  
like_train.drop(columns=["1_star", "2_star", "3_star", "4_star", "5_star",  
                        "All", "Word Likelihood", "NB_red_1_star", "NB_red_2_star",  
                        "NB_red_3_star", "NB_red_4_star", "NB_red_5_star",  
                        "blue_class_prob_1_star", "blue_class_prob_2_star",  
  
                        "blue_class_prob_3_star", "blue_class_prob_4_star", "blue_class_prob_5_star",  
  
                        "yellow_NB_prob_1_star", "yellow_NB_prob_2_star", "yellow_NB_prob_3_star",  
                        "yellow_NB_prob_4_star", "yellow_NB_prob_5_star"])
```

```
# In[17]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(word_to_sentiment_points)
```

```
# In[18]:
```

```
#Deleting the last two rows
```

```
word_to_sentiment_points=word_to_sentiment_points[0:9667]
```

```
print(word_to_sentiment_points)
```

```
# In[19]:
```

```
#Rounding the Sentiment points to a categorial value (1, 2, 3, 4 or 5)
```

```
word_to_sentiment_points["Sentiment points"]=word_to_sentiment_points["Sentiment  
points"].round(decimals=0)
```

```
print(word_to_sentiment_points)
```

```
# In[20]:
```

```
#Converting the Sentiment Points to a more visual attractive format
```

```
word_to_sentiment_points["Sentiment points"]=word_to_sentiment_points["Sentiment
points"].astype(int)
print(word_to_sentiment_points)
```

```
# In[21]:
#Changing the view window to maximum
pd.set_option("display.max_rows", None)
```

```
# In[22]:
#Taking a quick look at the vocabulary
print(word_to_sentiment_points)
```

```
# In[23]:
#Exporting the untested dictionary as word and sentiment points table
word_to_sentiment_points.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20
000 reviews Trainset =10%\Sentiment points\Word - Sentiment Points.csv",index=False)
```

```
..... End - 6.1. Way to Naive Bayes Sentiment Scores .....
(20 000 reviews - Training set portion = 10%)
```

```
..... Start - 6.2. Way to Naive Bayes Sentiment Scores .....
(200 000 reviews - Training set portion = 99%)
```

```
# In[1]:
#Importing the Pandas library
import pandas as pd
```

```
# In[2]:
#Importing the Likelihood Table
like_train=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Tables\Likelihood table - Training set.csv",low_memory=False)
```

```
# In[3]:
```

```
#Inspecting the Likelihood Table
```

```
print (like_train)
```

```
# In[4]:
```

```
#Renaming column Unnamed: 0 as Index
```

```
like_train.rename(columns={"Unnamed: 0":"Index"}, inplace=True)
```

```
# In[5]:
```

```
#Setting column Index as index
```

```
like_train.set_index("Index", inplace=True)
```

```
# In[6]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print (like_train)
```

```
# In[7]:
```

```
#Calculating the probability that given a sentiment class an individual word is occurring (for all words and sentiment classes respectively)
```

```
like_train["NB_red_1_star"]=like_train["1_star"]/like_train.at["All","1_star"]
```

```
like_train["NB_red_2_star"]=like_train["2_star"]/like_train.at["All","2_star"]
```

```
like_train["NB_red_3_star"]=like_train["3_star"]/like_train.at["All","3_star"]
```

```
like_train["NB_red_4_star"]=like_train["4_star"]/like_train.at["All","4_star"]
```

```
like_train["NB_red_5_star"]=like_train["5_star"]/like_train.at["All","5_star"]
```

```
# In[8]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print (like_train)
```

```
# In[9]:
```

```
#Converting the class probabilities of the sentiments to columns
```

```
like_train["blue_class_prob_1_star"] = like_train.at["class_prob","1_star"]
```

```
like_train["blue_class_prob_2_star"] = like_train.at["class_prob","2_star"]
```

```
like_train["blue_class_prob_3_star"] = like_train.at["class_prob","3_star"]
```

```
like_train["blue_class_prob_4_star"] = like_train.at["class_prob","4_star"]
```

```
like_train["blue_class_prob_5_star"] = like_train.at["class_prob","5_star"]
```

```
# In[10]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(like_train)
```

```
# In[11]:
```

```
#Calculating a sentiment class probability given each word respectively (i.e. Naive Bayes probability) for all of the sentiment classes respectively
```

```
like_train["yellow_NB_prob_1_star"]=like_train["NB_red_1_star"]*like_train["blue_class_prob_1_star"]/like_train["Word Likelihood"]
```

```
like_train["yellow_NB_prob_2_star"]=like_train["NB_red_2_star"]*like_train["blue_class_prob_2_star"]/like_train["Word Likelihood"]
```

```
like_train["yellow_NB_prob_3_star"]=like_train["NB_red_3_star"]*like_train["blue_class_prob_3_star"]/like_train["Word Likelihood"]
```

```
like_train["yellow_NB_prob_4_star"]=like_train["NB_red_4_star"]*like_train["blue_class_prob_4_star"]/like_train["Word Likelihood"]
```

```
like_train["yellow_NB_prob_5_star"]=like_train["NB_red_5_star"]*like_train["blue_class_prob_5_star"]/like_train["Word Likelihood"]
```

```
# In[12]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(like_train)
```

```
# In[13]:
```

```
#Creating the sentiment points based on the Naive Bayes probabilities and the star classification scheme by Yelp (1-5) for each word
```

```
like_train["Sentiment points"]=5*like_train["yellow_NB_prob_5_star"]  
+4*like_train["yellow_NB_prob_4_star"]+3*like_train["yellow_NB_prob_3_star"]  
+2*like_train["yellow_NB_prob_2_star"]+1*like_train["yellow_NB_prob_1_star"]
```

```
# In[14]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
like_train.head()
```

```
# In[15]:
```



```
#Exporting the most expansive Table after having performed all calculations
like_train.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Tables\Maximal Table when conducting Naive Bayes Sentiment points
calc.csv")
```

```
# In[16]:
```

```
#Deleting unnecessary columns
```

```
word_to_sentiment_points =
like_train.drop(columns=["1_star", "2_star", "3_star", "4_star", "5_star",
                        "All", "Word Likelihood", "NB_red_1_star", "NB_red_2_star",
                        "NB_red_3_star", "NB_red_4_star", "NB_red_5_star",
                        "blue_class_prob_1_star", "blue_class_prob_2_star",
                        "blue_class_prob_3_star", "blue_class_prob_4_star", "blue_class_prob_5_star",
                        "yellow_NB_prob_1_star", "yellow_NB_prob_2_star", "yellow_NB_prob_3_star",
                        "yellow_NB_prob_4_star", "yellow_NB_prob_5_star"])
```

```
# In[17]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(word_to_sentiment_points)
```

```
# In[18]:
```

```
#Deleting the last two rows
```

```
word_to_sentiment_points=word_to_sentiment_points[0:138593]
print(word_to_sentiment_points)
```

```
# In[19]:
```

```
#Rounding the Sentiment points to a categorial value (1, 2, 3, 4 or 5)
```

```
word_to_sentiment_points["Sentiment points"]=word_to_sentiment_points["Sentiment
points"].round(decimals=0)
print(word_to_sentiment_points)
```

```
# In[20]:
```

```
#Converting the Sentiment Points to a more visual attractive format
word_to_sentiment_points["Sentiment points"]=word_to_sentiment_points["Sentiment
points"].astype(int)
print(word_to_sentiment_points)
```

```
# In[21]:
#Taking a quick look at the first and last words in the vocabulary
print(word_to_sentiment_points)
```

```
# In[22]:
#Exporting the untested dictionary as word and sentiment points table
word_to_sentiment_points.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200
000 reviews Trainset =99%\Sentiment points\Word - Sentiment Points.csv",index=False)
```

End - 6.2. Way to Naive Bayes Sentiment Scores

```
..... (200 000 reviews - Training set portion = 99%) .....
```

Start - 7.1. Validation + Metrics

```
..... (20 000 reviews - Training set portion = 10%) .....
```

```
# In[1]:
#Importing the Pandas Library
import pandas as pd
```

```
# In[2]:
#Importing the words and the stars from the testset
test_stars=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews
Trainset =10%\Data\test\Clean test words + sentiments (unaggregated).csv")
```

```
# In[3]:
#Viewing if the Output changed accordingly to the last code entered
print(test_stars)
```

```

# In[4]:
#Removing the Unnamed: 0 column, sorting alphabetically according to the stemmed
words and then labelling the index according to the new ranking
test_stars = test_stars.drop(columns=["Unnamed: 0"])
test_stars=test_stars.sort_values("stemmed_words")
test_stars =test_stars.reset_index(drop=True)
print(test_stars)

# In[5]:
#Taking a more detailed look to inspect where the NaN words begin
pd.set_option("display.max_rows", None)

# In[6]:
#Displaying the end of the test dataframe
print(test_stars[1370000:1370854])

# In[7]:
#Displaying the beginning of the test dataframe
print(test_stars[0:1000])

# In[8]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[9]:
#Selecting all the single words in the dataframe until "zero"
test_stars=test_stars[0:1370460]
print(test_stars)

# In[10]:
#Averaging the star value of the same words across different recensions
aggregat_test_stars=test_stars.groupby(["stemmed_words"],as_index=False).mean()
print(aggregat_test_stars)

# In[11]:

```

```
#Rounding the stars value to discrete and categorical values (1, 2, 3, 4 or 5)
aggregat_test_stars["stars"]=aggregat_test_stars["stars"].round(decimals=0)
print(aggregat_test_stars)
```

```
# In[12]:
```

```
#Formatting the stars values for an easier visibility
aggregat_test_stars["stars"]=aggregat_test_stars["stars"].astype(int)
print(aggregat_test_stars)
```

```
# In[13]:
```

```
#Importing the untested dictionary as a Pandas dataframe
sen_points=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews
Trainset =10%\Sentiment points\Word - Sentiment Points.csv")
```

```
# In[14]:
```

```
#Inspecting the untested dictionary
sen_points.head()
```

```
# In[15]:
```

```
#Create one dataframe from the test dataframe and the untested dictionary on the same
words in order to compare stars and sentiment points
comparison_df=pd.merge(sen_points, aggregat_test_stars, on="stemmed_words",
how="inner")
```

```
# In[16]:
```

```
#Inspecting the newly created dataset
print(comparison_df)
```

```
# In[17]:
```

```
#Importing statistical Evaluation and plotting libraries
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import fbeta_score
import seaborn as sns
```

```

import matplotlib.pyplot as plt

# In[18]:
#For maximising the viewing window
pd.set_option("display.max_rows", None)

# In[19]:
#Making a copy of the newly created comparison dataframe
non_neutral_df=comparison_df.copy()
print(non_neutral_df)

# In[20]:
#Sorting out real neutral sentiment and predicted neutral sentiment
non_neutral_df = non_neutral_df[non_neutral_df["Sentiment points"] != 3]
non_neutral_df = non_neutral_df[non_neutral_df["stars"] != 3]
print(non_neutral_df)

# In[21]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[22]:
#Creating a confusion matrix on the basis of the sentiment points and stars column
non_neutral_sentiment_matrix=confusion_matrix(non_neutral_df["stars"],
non_neutral_df["Sentiment points"],labels=[1,2,4,5])

# In[23]:
#Setting x and y labels, the plot size, the font size before plotting according to the Red,
Yellow and Green scale, while the plot is exported as png in the end
x_axis_trial=[1,2,4,5]
y_axis_trial=[1,2,4,5]
plt.subplots(figsize=(20,15))
sns.set(font_scale=2)
sns_heat=sns.heatmap(non_neutral_sentiment_matrix, annot=True, fmt="d",
xticklabels=x_axis_trial, yticklabels=y_axis_trial , cmap="RdYlGn")

```

```
plt.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset  
=10%\Valuation Metrics\4 Sentiments for Vegas.png")
```

```
# In[24]:
```

```
#Providing the Sentiment class names, then performing an classification report with is  
saved as a Python dictionary
```

```
class_titles = ["Strongly Negative", "Slightly Negative", "Slightly Positive" ,"Strongly  
Positive"]
```

```
report=(classification_report(non_neutral_df["stars"], non_neutral_df["Sentiment points"],  
target_names=class_titles, labels=[1,2,4,5], output_dict=True))  
print(report)
```

```
# In[25]:
```

```
#Converting the dictionary into a dataframe, which is then transposed, while the F-beta  
score is calculated and saved
```

```
class_report=pd.DataFrame(report).T
```

```
f_beta=fbeta_score(non_neutral_df["stars"], non_neutral_df["Sentiment  
points"],average=None, beta=0.5)
```

```
print(class_report)
```

```
print(f_beta)
```

```
# In[26]:
```

```
#The accuracy is converted into a column, the F-beta score is assigned for each sentiment  
class, the sentiment is transformed from an index to a column and the columns are  
reordered
```

```
a=class_report.iloc[4]["precision"]
```

```
class_report["accuracy"]=a
```

```
class_report=class_report[0:4]
```

```
class_report["f_beta"]=f_beta
```

```
class_report["sentiment"] = class_report.index
```

```
class_report.reset_index(inplace = True)
```

```
class_report= class_report.drop(columns=["index"])
```

```
class_report = class_report[["sentiment", "accuracy", "f1 -
score", "f_beta", "precision", "recall"]]
print(class_report)
```

In[27]:

#Creating an individual plot for F-Beta, Precision and Recall listing each sentiment class separately with its corresponding values

```
sns.set(font_scale=2)
```

```
fig, axs = plt.subplots(1,3,figsize=(45,20))
```

```
for ax,col in zip(axs,["f_beta", "precision", "recall"]):
```

```
    sbar=sns.barplot(x="sentiment", y=col, data=class_report, ax=ax)
```

```
    for p in sbar.patches:
```

```
        sbar.annotate(format(p.get_height(), "1.3f"),
```

```
        (p.get_x() + p.get_width() / 2., p.get_height()),
```

```
        ha = "center", va = "center", fontsize=40,
```

```
        xytext = (0, -35), color="white",
```

```
        textcoords = "offset points")
```

In[28]:

#Saving the bar charts as png file

```
fig = sbar.get_figure()
```

```
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset
=10%\Valuation Metrics\Classifier performance for 4 sentiments.png")
```

In[29]:

#Labelling Strong and Slightly Positive sentiment as Positive sentiment as well as Strong and Slightly Negative sentiment as Negative sentiment

```
non_neutral_df["Sentiment points"].replace({1:-1 , 2: -1, 4:1, 5:1}, inplace=True)
```

```
non_neutral_df["stars"].replace({1:-1 , 2: -1, 4:1, 5:1}, inplace=True)
```

In[30]:

#Maximising the view window

```
pd.set_option("display.max_rows", None)
```

```

# In[31]:
#Viewing if the Output changed accordingly to the relabelling command
print(non_neutral_df)

# In[32]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[33]:
#Creating a new confusion matrix based on the two categories Positive and Negative
pos_and_neg_sentiment_matrix=confusion_matrix(non_neutral_df["stars"],
non_neutral_df["Sentiment points"],labels=[-1, 1])

# In[34]:
#Plotting the confusion matrix and saving the figure as png
x_axis_trial_pn=[-1,1]
y_axis_trial_pn=[-1,1]
plt.subplots(figsize=(20,15))
sns.set(font_scale=2)
sns_heat=sns.heatmap(pos_and_neg_sentiment_matrix, annot=True, fmt="d",
xticklabels=x_axis_trial_pn, yticklabels=y_axis_trial_pn , cmap="RdYlGn")
plt.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset
=10%\Valuation Metrics\2 Sentiments for Vegas.png")

# In[35]:
#Creating a new classification report based on the Negative and Positive classes for words
between the predicted and the actual class
class_titles = ["Negative", "Positive"]
report=(classification_report(non_neutral_df["stars"], non_neutral_df["Sentiment points"],
target_names=class_titles, labels=[-1,1], output_dict=True))
print(report)

# In[36]:

```



```
#Generating a dataframe from the classification dictionary, then transposing it and last but
not least calculating the F-beta score
class_report=pd.DataFrame(report).T
f_beta_2_sen=fbeta_score(non_neutral_df["stars"], non_neutral_df["Sentiment
points"],average=None, beta=0.5)
print(class_report)
print(f_beta_2_sen)
```

```
# In[37]:
```

```
#The accuracy is converted into a column, the F-beta score is assigned for each sentiment
class, the sentiment is transformed from an index to a column and the columns are
reordered
```

```
a=class_report.iloc[2]["precision"]
class_report["accuracy"]=a
```

```
class_report=class_report[0:2]
class_report["f_beta"]=f_beta_2_sen
```

```
class_report["sentiment"] = class_report.index
class_report.reset_index(inplace = True)
class_report= class_report.drop(columns=["index"])
```

```
class_report = class_report[["sentiment","accuracy", "f1 -
score","f_beta","precision","recall"]]
print(class_report)
```

```
# In[38]:
```

```
#Plotting the F-beta, Precision and Recall scores for each sentiment class
```

```
sns.set(font_scale=2)
fig, axs = plt.subplots(1,3,figsize=(25,10))
for ax,col in zip(axs,["f_beta", "precision", "recall"]):
    sbar=sns.barplot(x="sentiment", y=col, data=class_report, ax=ax)
    for p in sbar.patches:
        sbar.annotate(format(p.get_height(), "1.3f"),
            (p.get_x() + p.get_width() / 2., p.get_height()),
```

```
ha = "center", va = "center", fontsize=20,  
xytext = (0, -18), color="white",  
textcoords = "offset points")
```

```
# In[39]:
```

```
#Saving the figure as png file
```

```
fig = sbar.get_figure()
```

```
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset  
=10%\Valuation Metrics\Classifier performance for 2 sentiments.png")
```

```
# In[40]:
```

```
#Saving the dictionary part, which was tested, as the new dictionary
```

```
non_neutral_df = non_neutral_df.drop(columns=["stars"])
```

```
non_neutral_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews  
Trainset =10%\Ultimate dict.csv",index=False)
```

End - 7.1. Validation + Metrics

```
..... (20 000 reviews - Training set portion = 10%) .....
```

Start - 7.2. Validation + Metrics

```
..... (200 000 reviews - Training set portion = 99%) .....
```

```
# In[1]:
```

```
#Importing the Pandas Library
```

```
import pandas as pd
```

```
# In[2]:
```

```
#Importing the words and the stars from the testset
```

```
test_stars=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews  
Trainset =99%\Data\test\Clean test words + sentiments (unaggregated).csv")
```

```
# In[3]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(test_stars)
```

```
# In[4]:
```

```
#Removing the Unnamed: 0 column, sorting alphabetically according to the stemmed  
words and then labelling the index according to the new ranking  
test_stars = test_stars.drop(columns=["Unnamed: 0"])  
test_stars=test_stars.sort_values("stemmed_words")  
test_stars =test_stars.reset_index(drop=True)  
print(test_stars)
```

```
# In[5]:
```

```
#Taking a more detailed look to inspect where the German words begin  
pd.set_option("display.max_rows", None)
```

```
# In[6]:
```

```
#Displaying the end of the test dataframe  
print(test_stars[150000:151172])
```

```
# In[7]:
```

```
#Displaying the beginning of the test dataframe  
print(test_stars[0:1000])
```

```
# In[8]:
```

```
#Resetting the max view option to the default value  
pd.reset_option("display.max_rows")
```

```
# In[9]:
```

```
#Selecting all the single words in the dataframe until "zero"  
test_stars=test_stars[0:151137]  
print(test_stars)
```

```
# In[10]:
```

```
#Averaging the star value of the same words across different recensions  
aggregat_test_stars=test_stars.groupby(["stemmed_words"],as_index=False).mean()
```

```
print(aggreat_test_stars)
```

```
# In[11]:
```

```
#Rounding the stars value to discrete and categorical values (1, 2, 3, 4 or 5)
```

```
aggreat_test_stars["stars"]=aggreat_test_stars["stars"].round(decimals=0)
```

```
print(aggreat_test_stars)
```

```
# In[12]:
```

```
#Formatting the stars values for an easier visibility
```

```
aggreat_test_stars["stars"]=aggreat_test_stars["stars"].astype(int)
```

```
print(aggreat_test_stars)
```

```
# In[13]:
```

```
#Importing the untested dictionary as a Pandas dataframe
```

```
sen_points=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews  
Trainset =99%\Sentiment points\Word - Sentiment Points.csv")
```

```
# In[14]:
```

```
#Inspecting the untested dictionary
```

```
sen_points.head()
```

```
# In[15]:
```

```
#Create one dataframe from the test dataframe and the untested dictionary on the same  
words in order to compare stars and sentiment points
```

```
comparison_df=pd.merge(sen_points, aggreagat_test_stars, on="stemmed_words",  
how="inner")
```

```
# In[16]:
```

```
#Inspecting the newly created dataset
```

```
print(comparison_df)
```

```
# In[17]:
```

```
#Importing statistical Evaluation and plotting libraries
```

```
from sklearn.metrics import confusion_matrix
```

```

from sklearn.metrics import classification_report
from sklearn.metrics import fbeta_score
import seaborn as sns
import matplotlib.pyplot as plt

# In[18]:
#Making a copy of the newly created comparison dataframe
non_neutral_df=comparison_df.copy()
print(non_neutral_df)

# In[19]:
#Sorting out real neutral sentiment and predicted neutral sentiment
non_neutral_df = non_neutral_df[non_neutral_df["Sentiment points"] != 3]
non_neutral_df = non_neutral_df[non_neutral_df["stars"] != 3]
print(non_neutral_df)

# In[20]:
#Creating a confusion matrix on the basis of the sentiment points and stars column
non_neutral_sentiment_matrix=confusion_matrix(non_neutral_df["stars"],
non_neutral_df["Sentiment points"],labels=[1,2,4,5])

# In[21]:
#Setting x and y labels, the plot size, the font size before plotting according to the Red,
Yellow and Green scale, while the plot is exported as png in the end
x_axis_trial=[1,2,4,5]
y_axis_trial=[1,2,4,5]
plt.subplots(figsize=(20,15))
sns.set(font_scale=2)
sns_heat=sns.heatmap(non_neutral_sentiment_matrix, annot=True, fmt="d",
xticklabels=x_axis_trial, yticklabels=y_axis_trial , cmap="RdYlGn")
plt.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Valuation Metrics\4 Sentiments for Vegas.png")

# In[22]:

```

#Providing the Sentiment class names, then performing an classification report with is saved as a Python dictionary

```
class_titles = ["Strongly Negative", "Slightly Negative", "Slightly Positive", "Strongly Positive"]
```

```
report=(classification_report(non_neutral_df["stars"], non_neutral_df["Sentiment points"],  
target_names=class_titles, labels=[1,2,4,5], output_dict=True))  
print(report)
```

In[23]:

#Converting the dictionary into a dataframe, which is then transposed, while the F-beta score is calculated and saved

```
class_report=pd.DataFrame(report).T
```

```
f_beta=fbeta_score(non_neutral_df["stars"], non_neutral_df["Sentiment  
points"],average=None, beta=0.5)
```

```
print(class_report)
```

```
print(f_beta)
```

In[24]:

#The accuracy is converted into a column, the F-beta score is assigned for each sentiment class, the sentiment is transformed from an index to a column and the columns are reordered

```
a=class_report.iloc[4]["precision"]
```

```
class_report["accuracy"]=a
```

```
class_report=class_report[0:4]
```

```
class_report["f_beta"]=f_beta
```

```
class_report["sentiment"] = class_report.index
```

```
class_report.reset_index(inplace = True)
```

```
class_report= class_report.drop(columns=["index"])
```

```
class_report = class_report[["sentiment","accuracy", "f1 -  
score","f_beta","precision","recall"]]
```

```
print(class_report)
```

```

# In[25]:
#Creating an individual plot for F-Beta, Precision and Recall listing each sentiment class
separately with its corresponding values
sns.set(font_scale=2)
fig, axs = plt.subplots(1,3,figsize=(45,20))
for ax,col in zip(axs,["f_beta", "precision", "recall"]):
    sbar=sns.barplot(x="sentiment", y=col, data=class_report, ax=ax)
    for p in sbar.patches:
        sbar.annotate(format(p.get_height(), "1.3f"),
            (p.get_x() + p.get_width() / 2., p.get_height()),
            ha = "center", va = "center", fontsize=40,
            xytext = (0, -35), color="white",
            textcoords = "offset points")

# In[26]:
#Saving the bar charts as png file
fig = sbar.get_figure()
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Valuation Metrics\Classifier performance for 4 sentiments.png")

# In[27]:
#Labelling Strong and Slightly Positive sentiment as Positive sentiment as well as Strong
and Slightly Negative sentiment as Negative sentiment
non_neutral_df["Sentiment points"].replace({1:-1 , 2: -1, 4:1, 5:1}, inplace=True)
non_neutral_df["stars"].replace({1:-1 , 2: -1, 4:1, 5:1}, inplace=True)

# In[28]:
#Viewing if the Output changed accordingly to the relabelling command
print(non_neutral_df)

# In[29]:
#Creating a new confusion matrix based on the two categories Positive and Negative
pos_and_neg_sentiment_matrix=confusion_matrix(non_neutral_df["stars"],
non_neutral_df["Sentiment points"],labels=[-1, 1])

```

```
# In[30]:
#Plotting the confusion matrix and saving the figure as png
x_axis_trial_pn=[-1,1]
y_axis_trial_pn=[-1,1]
plt.subplots(figsize=(20,15))
sns.set(font_scale=2)
sns_heat=sns.heatmap(pos_and_neg_sentiment_matrix, annot=True, fmt="d",
xticklabels=x_axis_trial_pn, yticklabels=y_axis_trial_pn , cmap="RdYlGn")
plt.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Valuation Metrics\2 Sentiments for Vegas.png")
```

```
# In[31]:
#Creating a new classification report based on the Negative and Positive classes for words
between the predicted and the actual class
class_titles = ["Negative", "Positive"]
report=(classification_report(non_neutral_df["stars"], non_neutral_df["Sentiment points"],
target_names=class_titles, labels=[-1,1], output_dict=True))
print(report)
```

```
# In[32]:
#Generating a dataframe from the classification dictionary, then transposing it and last but
not least calculating the F-beta score
class_report=pd.DataFrame(report).T
f_beta_2_sen=fbeta_score(non_neutral_df["stars"], non_neutral_df["Sentiment
points"],average=None, beta=0.5)
print(class_report)
print(f_beta_2_sen)
```

```
# In[33]:
#The accuracy is converted into a column, the F-beta score is assigned for each sentiment
class, the sentiment is transformed from an index to a column and the columns are
reordered
a=class_report.iloc[2]["precision"]
```



```
class_report["accuracy"]=a
```

```
class_report=class_report[0:2]
```

```
class_report["f_beta"]=f_beta_2_sen
```

```
class_report["sentiment"] = class_report.index
```

```
class_report.reset_index(inplace = True)
```

```
class_report= class_report.drop(columns=["index"])
```

```
class_report = class_report[["sentiment","accuracy", "f1 -  
score","f_beta","precision","recall"]]  
print(class_report)
```

```
# In[34]:
```

```
#Plotting the F-beta, Precision and Recall scores for each sentiment class
```

```
sns.set(font_scale=2)
```

```
fig, axs = plt.subplots(1,3,figsize=(25,10))
```

```
for ax,col in zip(axs,["f_beta", "precision", "recall"]):
```

```
    sbar=sns.barplot(x="sentiment", y=col, data=class_report, ax=ax)
```

```
    for p in sbar.patches:
```

```
        sbar.annotate(format(p.get_height(), "1.3f"),
```

```
        (p.get_x() + p.get_width() / 2., p.get_height()),
```

```
        ha = "center", va = "center", fontsize=20,
```

```
        xytext = (0, -18), color="white",
```

```
        textcoords = "offset points")
```

```
# In[35]:
```

```
#Saving the figure as png file
```

```
fig = sbar.get_figure()
```

```
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews  
Trainset =99%\Valuation Metrics\Classifier performance for 2 sentiments.png")
```

```
# In[36]:
```

```
#Saving the dictionary part, which was tested, as the new dictionary
```

```

non_neutral_df = non_neutral_df.drop(columns=["stars"])
non_neutral_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews
Trainset =99%\Ultimate dict.csv",index=False)

```

End - 7.2. Validation + Metrics

```

..... (200 000 reviews - Training set portion = 99%) .....

```

Start - 8.1. Bar Data Cleaning Twitter data, Matching,

```

..... keyword matrix creation and Topic modelling .....
(Both sentiment dictionaries)

```

In[1]:

```

#Importing the Pandas library
import pandas as pd

```

In[2]:

```

#Importing the Twitter Data concerning the topic bar
twitter_data=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\3 Obtaining the Twitter Data\Twitter Data\Search by topic bar.csv")

```

In[3]:

```

#Inspecting the twitter dataframe
twitter_data.head()

```

In[4]:

```

#Deleting the Screenname
twitter_data = twitter_data.drop(columns=["Screenname"])
print(twitter_data)

```

In[5]:

```

#Maximising the view window to get an impression about the original tweets
pd.set_option("display.max_rows", None)

```

In[6]:

```

#Displaying the original tweets
print(twitter_data)

```

In[7]:

#Importing the Regular Expression library

import re

In[8]:

#Removing hashtag symbols from the text data (i.e. replacing it by Nothing [""])

hashtag= re.compile(r"#")

twitter_data["Text"]= twitter_data.Text.apply(lambda x: hashtag.sub("", x))

In[9]:

#Viewing if the Output changed accordingly to the last code entered

print(twitter_data)

In[10]:

#Removing @ + username from the text data (i.e. replacing it by Nothing [""])

mention= re.compile(r"s*@[\s]*")

twitter_data["Text"]= twitter_data.Text.apply(lambda x: mention.sub("", x))

In[11]:

#Viewing if the Output changed accordingly to the last code entered

print(twitter_data)

In[12]:

#Removing Urls from the text data (i.e. replacing it by Nothing [""])

url= re.compile(r"s*http[\s]*")

twitter_data["Text"]= twitter_data.Text.apply(lambda x: url.sub("", x))

In[13]:

#Viewing if the Output changed accordingly to the last code entered

print(twitter_data)

In[14]:

#Lower casing the text data

twitter_data["Text"]=twitter_data["Text"].str.lower()

In[15]:

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

In[16]:

```
#Importing the nltk (natural language toolkit) library  
#Necessary step for removing stop_words (downloading the stop-word list) and applying  
word tokenization (downloading the punkt package)  
import nltk  
nltk.download("stopwords")  
nltk.download("punkt")
```

In[17]:

```
#Importing the English stop-words  
from nltk.corpus import stopwords  
stop_words = stopwords.words("english")  
print(stop_words)
```

In[18]:

```
#Removing Stop-words = splitting each word except stop-words into a list for each tweet  
respectively by maintaining the dataframe data type  
twitter_data["Text"]=twitter_data["Text"].apply(lambda x: [item for item in str.split(x) if item  
not in stop_words])
```

In[19]:

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

In[20]:

```
#Join all list items together by a space for each row of the particular text column  
twitter_data["Text"] = [" ".join(map(str, l)) for l in twitter_data["Text"]]
```

In[21]:

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(twitter_data)
```

```
# In[22]:
```

```
#Replace text which is not alphanumeric or underscore or space or tabs or line brakes with  
"" (i.e. Nothing)
```

```
abnormal_things = re.compile(r"^\w\s+")
```

```
twitter_data["Text"] = [abnormal_things.sub("", x) for x in twitter_data["Text"].tolist()]
```

```
# In[23]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(twitter_data)
```

```
# In[24]:
```

```
#Removing Numbers i.e. replace Numbers with "" (\d+ means [0-9])
```

```
twitter_data["Text"] = twitter_data["Text"].str.replace("\d+", "")
```

```
# In[25]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(twitter_data)
```

```
# In[26]:
```

```
#Removing amp i.e. replace amp with ""
```

```
amp= re.compile(r"\s*amp[\s]*")
```

```
twitter_data["Text"] = twitter_data.Text.apply(lambda x: amp.sub("", x))
```

```
# In[27]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(twitter_data)
```

```
# In[28]:
```

```
#Importing the word tokenize function from nltk
```

```
from nltk.tokenize import word_tokenize
```

```
# In[29]:
```

```
#Applying Word tokenization
```

```
twitter_list=[twitter_data["Text"].apply(word_tokenize),twitter_data["ID"]]
```

```
# In[30]:
```

```
#Converting the list into a dataframe
```

```
twitter_df=pd.DataFrame(twitter_list)
```

```
# In[31]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
twitter_df.head()
```

```
# In[32]:
```

```
#Transposing the dataframe for better visibility
```

```
twitter_df=twitter_df.T
```

```
# In[33]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
twitter_df.head()
```

```
# In[34]:
```

```
#Importing the Snowball Stemmer and save the Snowball Stemmer for English in a Variable
```

```
from nltk.stem.snowball import SnowballStemmer
```

```
sbs = SnowballStemmer("english")
```

```
# In[35]:
```

```
#Create a new column (stemmed_words [of the dataframe train_cleaner_df]) in which every word (i) is stemmed in list x and saved
```

```
twitter_df["Text"] = twitter_df["Text"].apply(lambda x: [sbs.stem(i) for i in x])
```

```
# In[36]:
```

```
#Resetting the max view option to the default value
```

```
pd.reset_option("display.max_rows")
```

```
# In[37]:
```

```
#Inspecting the stemmed dataframe
```

```
print(twitter_df)
```

```
# In[38]:
```

```
#Exporting the data cleaned file as csv
```

```
twitter_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\5 Matching Twitter and dict\Cleaned Twitter data\Bar\Tweet lists.csv")
```

```
# In[39]:
```

```
#####Start Section (20 000 reviews Train = 10%)#####
```

```
# In[40]:
```

```
#Creating a column with the original index
```

```
twitter_df["Real_index"] = twitter_df.index
```

```
# In[41]:
```

```
#Renaming the Text column to stemmed_words
```

```
twitter_df.rename(columns={"Text":"stemmed_words"}, inplace=True)
```

```
print(twitter_df)
```

```
# In[42]:
```

```
#Removing the Tweet ID
```

```
twitter_df = twitter_df.drop(columns=["ID"])
```

```
print(twitter_df)
```

```
# In[43]:
```

```
#Having each word separately with its corresponding Real_index number
```

```
twitter_df_exp=twitter_df.explode("stemmed_words")
```

```
print(twitter_df_exp)
```

```
# In[44]:
```

```
#Importing the tested dictionary, then dropping the Sentiment points column of the tested  
dictionary
```

```
dict_20_000_10=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University  
of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews  
Trainset =10%\Ultimate dict.csv")  
dict_20_000_10 = dict_20_000_10.drop(columns=["Sentiment points"])  
print(dict_20_000_10)
```

```
# In[45]:
```

```
#Words which appear in the cleaned twitter data and the tested dictionary as well is stored  
in a cleaned dataframe
```

```
cleaned_df=pd.merge(twitter_df_exp, dict_20_000_10, on="stemmed_words",  
how="inner")  
print(cleaned_df)
```

```
# In[46]:
```

```
#Sorting according to the original Tweet index  
cleaned_df.sort_values("Real_index")
```

```
# In[47]:
```

```
#If the Real_index number is the same, allocate the stemmed words into their own list,  
then converting the Output into a Pandas dataframe again
```

```
relevant_clean_matrix=cleaned_df.groupby("Real_index")["stemmed_words"].apply(list)  
relevant_clean_df=pd.DataFrame(relevant_clean_matrix)  
print(relevant_clean_matrix)  
print(relevant_clean_df)
```

```
# In[48]:
```

```
#First creating a column with the words original tweet number, then a new index is  
initialised to indicate how many Tweet texts have been remaining
```

```
relevant_clean_df["Real Doc Numbers"] = relevant_clean_df.index  
relevant_clean_df = relevant_clean_df.reset_index(drop=True)  
print(relevant_clean_df)
```

```
# In[49]:
```

```
#Exporting the dataframe of stemmed_words and real document numbers as csv
```



```
relevant_clean_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of  
West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)  
\Bar\matched words to docs overview.csv")
```

```
# In[50]:
```

```
#Creating a separate list of lists and saving it under the variable text_list  
text_list=relevant_clean_df["stemmed_words"].tolist()
```

```
# In[51]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(text_list[1])
```

```
# In[52]:
```

```
#Importing the LDA Topic Model libraries  
import gensim  
from gensim import corpora  
from gensim.models import CoherenceModel
```

```
# In[53]:
```

```
#Creating a dictionary made out of unique words  
dictionary=corpora.Dictionary(relevant_clean_df["stemmed_words"])  
print(dictionary)
```

```
# In[54]:
```

```
#Creating the document term matrix where one document is one document (i.e. Tweet)  
from the relevant and cleaned dataframe  
doc_term_matrix=[dictionary.doc2bow(doc) for doc in  
relevant_clean_df["stemmed_words"] ]  
print(len(doc_term_matrix))
```

```
# In[55]:
```

```
#Code by Selva Prabhakaran from MachineLearningPlus although the passes parameter  
was added and adjusted to improve the topic model output and the random_state  
parameter was added to make the calculation reproducible  
def compute_coherence_values(dictionary,corpus,texts,limit, start=2, step=3):
```

```

perplexity_scores=[]
coherence_values=[]
model_list=[]
for num_topics in range (start, limit, step):
    model=gensim.models.ldamodel.LdaModel(corpus=corpus, num_topics=num_topics,
id2word=dictionary, random_state=100, passes=20)
    model_list.append(model)
    coherencemodel= CoherenceModel(model=model, texts=texts, dictionary=dictionary,
coherence="c_v")
    coherence_values.append(coherencemodel.get_coherence())
    perplexity_scores.append(model.log_perplexity(corpus))

return model_list, coherence_values, perplexity_scores

```

In[56]:

#Code by Selva Prabhakaran from MachineLearningPlus although the size of limit and step parameter were adjusted

```

model_list, coherence_values, perplexity_scores =
compute_coherence_values(dictionary=dictionary, corpus=doc_term_matrix,
texts=text_list, start=2, limit=50, step=1)

```

In[57]:

#Importing the Plotting Libraries

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

In[58]:

#Defining the x-axis

```
start=2
```

```
limit=50
```

```
step=1
```

```
x=range(start,limit,step)
```

#Assigning separately the corresponding coherence and perplexity score to each respective LDA topic model

```

data1=zip(x,coherence_values)
data2=zip(x,perplexity_scores)

#Assigning the corresponding coherence and perplexity score together for each respective
LDA topic model
visual_df1=pd.DataFrame(data1,columns=["Topics", "Coherence Score"])
visual_df2=pd.DataFrame(data2,columns=["Topics", "Perplexity Score"])
visual_df=pd.merge(visual_df1, visual_df2, on="Topics", how="inner")

#Preparing to illustrate multiple line graphs within one plot and setting the size of the figure
fig, ax = plt.subplots()
fig.set_size_inches(19,5)

#Plotting the Coherence Scores and customising the first y-axis to the Coherence Score
ax=sns.lineplot(x="Topics",y="Coherence Score",data=visual_df,color="steelblue")
ax.yaxis.label.set_color("steelblue")

#Enabling the option to plot a second graph
ax2 = plt.twinx()

#Plotting the Perplexity Scores and customising the second y-axis to the Perplexity Score
sns.lineplot(x="Topics",y="Perplexity Score",data=visual_df, color="r", ax=ax2)
ax2.yaxis.label.set_color("r")

# In[59]:
#Export the Scatterplot as png file
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Bar\Perplexity vs Coherence.png")

# In[60]:
#Print the model with the highest coherence score
visual_df.loc[visual_df["Coherence Score"].idxmax()]

# In[61]:

```

```
#Selecting the model with the best coherence score and its corresponding 10 words,
which occur most likely
model_with_best_topic_number= model_list[20]
topics_with_significant_words=pd.DataFrame(model_with_best_topic_number.show_topics(num_topics=22, num_words=10))
print(topics_with_significant_words)
```

```
# In[62]:
```

```
#Exporting the topics and their significant words as a csv file
topics_with_significant_words.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Bar\Infer the topics with significant words.csv")
```

```
# In[63]:
```

```
#Saving the Document Topic matrix and presenting it as a Pandas dataframe
doc_top=model_with_best_topic_number.get_document_topics(doc_term_matrix)
pd.DataFrame(model_with_best_topic_number.get_document_topics(doc_term_matrix))
```

```
# In[64]:
```

```
#Transformation into numpy due to incorrect assignment of values (row number 1) and
None = 0
correct_topics_to_docs = gensim.matutils.corpus2csc(doc_top)
final_correct_topics_to_docs = correct_topics_to_docs.T.toarray()
print(final_correct_topics_to_docs)
```

```
# In[65]:
```

```
#The creation of the document topic matrix as a correct pandas dataframe
final_correct_topics_to_docs_df=pd.DataFrame(final_correct_topics_to_docs)
```

```
# In[66]:
```

```
#Viewing if the Output changed accordingly to the last code entered
print(final_correct_topics_to_docs_df)
```

```
# In[67]:
```

```
#Exporting the document topic matrix as csv
```

```
final_correct_topics_to_docs_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Bar\Topic to Document Matrix\doc to topics.csv")
```

```
# In[68]:
```

```
#Saving the Topic to keyword matrix (Topics=rows, words=columns)
```

```
word_top=pd.DataFrame(model_with_best_topic_number.get_topics(),columns=dictionary.values())
```

```
# In[69]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```

```
print(word_top)
```

```
# In[70]:
```

```
#Saving the word topic matrix as csv file
```

```
word_top.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Bar\Word to Topic Matrix\word to topics.csv")
```

```
# In[ ]:
```

```
##### End of Section (20 000 reviews Train = 10%)#####
```

```
# In[ ]:
```

```
##Analog actions have been performed on the same cleaned Twitter data, but with the 200 000 reviews Train = 99% tested dictionary
```

```
##(Please revisit Section (20 000 reviews Train = 10%) for a commented version of these steps
```

```
# In[ ]:
```

```
#####Start Section (200 000 reviews Train = 99%)#####
```

```
# In[71]:
```

```
dict_200_000_99=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews Trainset =99%\Ultimate dict.csv")
```

```

dict_200_000_99 = dict_200_000_99.drop(columns=["Sentiment points"])
print(dict_200_000_99)

# In[72]:
new_cleaned_df=pd.merge(twitter_df_exp, dict_200_000_99, on="stemmed_words",
how="inner")
print(new_cleaned_df)

# In[73]:
new_cleaned_df.sort_values("Real_index")

# In[74]:
new_relevant_clean_matrix=new_cleaned_df.groupby("Real_index")
["stemmed_words"].apply(list)
new_relevant_clean_df=pd.DataFrame(new_relevant_clean_matrix)
print(new_relevant_clean_matrix)
print(new_relevant_clean_df)

# In[75]:
new_relevant_clean_df["Real Doc Numbers"] = new_relevant_clean_df.index
new_relevant_clean_df = new_relevant_clean_df.reset_index(drop=True)
print(new_relevant_clean_df)

# In[76]:
new_relevant_clean_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University
of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 -
Train=99%)\Bar\matched words to docs overview.csv")

# In[77]:
new_text_list=new_relevant_clean_df["stemmed_words"].tolist()
print(new_text_list[0])

# In[78]:
new_dictionary=corpora.Dictionary(new_relevant_clean_df["stemmed_words"])
print(new_dictionary)

```

```

# In[79]:
new_doc_term_matrix=[new_dictionary.doc2bow(doc) for doc in
new_relevant_clean_df["stemmed_words"] ]
print(len(new_doc_term_matrix))

# In[80]:
#Code by Selva Prabhakaran from MachineLearningPlus although the size of limit and
step parameter were adjusted
model_list, coherence_values, perplexity_scores =
compute_coherence_values(dictionary=new_dictionary, corpus=new_doc_term_matrix,
texts=new_text_list, start=2, limit=50, step=1)

# In[81]:
#Defining the x-axis
start=2
limit=50
step=1
x=range(start,limit,step)

#Assigning separately the corresponding coherence and perplexity score to each
respective LDA topic model
new_data1=zip(x,coherence_values)
new_data2=zip(x,perplexity_scores)

#Assigning the corresponding coherence and perplexity score together for each respective
LDA topic model
new_visual_df1=pd.DataFrame(new_data1,columns=["Topics", "Coherence Score"])
new_visual_df2=pd.DataFrame(new_data2,columns=["Topics", "Perplexity Score"])
new_visual_df=pd.merge(new_visual_df1, new_visual_df2, on="Topics", how="inner")

#Preparing to illustrate multiple line graphs within one plot and setting the size of the figure
fig, ax = plt.subplots()
fig.set_size_inches(19,5)

```

```
#Plotting the Coherence Scores and customising the first y-axis to the Coherence Score
ax=sns.lineplot(x="Topics",y="Coherence Score",data=new_visual_df,color="steelblue")
ax.yaxis.label.set_color("steelblue")
```

```
#Enabling the option to plot a second graph
ax2 = plt.twinx()
```

```
#Plotting the Perplexity Scores and customising the second y-axis to the Perplexity Score
sns.lineplot(x="Topics",y="Perplexity Score",data=new_visual_df, color="r", ax=ax2)
ax2.yaxis.label.set_color("r")
```

```
# In[82]:
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)
\Bar\Perplexity vs Coherence.png")
```

```
# In[83]:
#Print the model with the highest coherence score
new_visual_df.loc[new_visual_df["Coherence Score"].idxmax()]
```

```
# In[84]:
new_model_with_best_topic_number= model_list[15]

new_topics_with_significant_words=pd.DataFrame(new_model_with_best_topic_number.s
how_topics(num_topics=17, num_words=10))
print(new_topics_with_significant_words)
```

```
# In[85]:
new_topics_with_significant_words.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive
- University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000
- Train=99%)\Bar\Infer the topics with significant words.csv")
```

```
# In[86]:
new_doc_top=new_model_with_best_topic_number.get_document_topics(new_doc_term
_matrix)
```



```
pd.DataFrame(new_model_with_best_topic_number.get_document_topics(new_doc_term_matrix))
```

```
# In[87]:
```

```
new_correct_topics_to_docs = gensim.matutils.corpus2csc(new_doc_top)
new_final_correct_topics_to_docs = new_correct_topics_to_docs.T.toarray()
print(new_final_correct_topics_to_docs)
```

```
# In[88]:
```

```
new_final_correct_topics_to_docs_df=pd.DataFrame(new_final_correct_topics_to_docs)
print(new_final_correct_topics_to_docs_df)
```

```
# In[89]:
```

```
new_final_correct_topics_to_docs_df.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and
dict\Matched words (200 000 - Train=99%)\Bar\Topic to Document Matrix\doc to
topics.csv")
```

```
# In[90]:
```

```
new_word_top=pd.DataFrame(new_model_with_best_topic_number.get_topics(),columns
=new_dictionary.values())
```

```
# In[91]:
```

```
print(new_word_top)
```

```
# In[92]:
```

```
new_word_top.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)
\Bar\Word to Topic Matrix\word to topics.csv")
```

```
# In[ ]:
```

```
#####End Section (200 000 reviews Train = 99%)#####
```

End - 8.1. Bar Data Cleaning Twitter data, Matching,

..... keyword matrix creation and Topic modelling
(Both sentiment dictionaries)

Start - 8.2. Entertainment Data Cleaning Twitter data,
..... Matching, keyword matrix creation and Topic modelling
(Both sentiment dictionaries)

In[1]:

#Importing the Pandas library
import pandas as pd

In[2]:

#Importing the Twitter Data concerning the topic stationary entertainment
twitter_data=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\3 Obtaining the Twitter Data\Twitter Data\Search by topic
event.csv")

In[3]:

#Inspecting the twitter dataframe
twitter_data.head()

In[4]:

#Deleting the Screenname
twitter_data = twitter_data.drop(columns=["Screenname"])
print(twitter_data)

In[5]:

#Maximising the view window to get an impression about the original tweets
pd.set_option("display.max_rows", None)

In[6]:

#Displaying the original tweets
print(twitter_data)

In[7]:

#Importing the Regular Expression library
import re

In[8]:

```
#Removing hashtag symbols from the text data (i.e. replacing it by Nothing [""])
hashtag= re.compile(r"#")
twitter_data["Text"]= twitter_data.Text.apply(lambda x: hashtag.sub("", x))
```

```
# In[9]:
```

```
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)
```

```
# In[10]:
```

```
#Removing @ + username from the text data (i.e. replacing it by Nothing [""])
mention= re.compile(r"\s*@[\s]*")
twitter_data["Text"]= twitter_data.Text.apply(lambda x: mention.sub("", x))
```

```
# In[11]:
```

```
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)
```

```
# In[12]:
```

```
#Removing Urls from the text data (i.e. replacing it by Nothing [""])
url= re.compile(r"\s*http[\s]*")
twitter_data["Text"]= twitter_data.Text.apply(lambda x: url.sub("", x))
```

```
# In[13]:
```

```
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)
```

```
# In[14]:
```

```
#Lower casing the text data
twitter_data["Text"]=twitter_data["Text"].str.lower()
```

```
# In[15]:
```

```
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)
```

```
# In[16]:
```

```

#Importing the nltk (natural language toolkit) library
#Necessary step for removing stop_words (downloading the stop-word list) and applying
word tokenization (downloading the punkt package)
import nltk
nltk.download("stopwords")
nltk.download("punkt")

# In[17]:
#Importing the English stop-words
from nltk.corpus import stopwords
stop_words = stopwords.words("english")
print(stop_words)

# In[18]:
#Removing Stop-words = splitting each word except stop-words into a list for each tweet
respectively by maintaining the dataframe data type
twitter_data["Text"]=twitter_data["Text"].apply(lambda x: [item for item in str.split(x) if item
not in stop_words])

# In[19]:
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

# In[20]:
#Join all list items together by a space for each row of the particular text column
twitter_data["Text"] = [" ".join(map(str, l)) for l in twitter_data["Text"]]

# In[21]:
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

# In[22]:
#Replace text which is not alphanumeric or underscore or space or tabs or line brakes with
"" (i.e. Nothing)
abnormal_things = re.compile(r"[^\w\s]+")

```

```
twitter_data["Text"] = [abnormal_things.sub("", x) for x in twitter_data["Text"].tolist()]
```

```
# In[23]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

```
# In[24]:
```

```
#Removing Numbers i.e. replace Numbers with "" (\d+ means [0-9])  
twitter_data["Text"] = twitter_data["Text"].str.replace("\d+", "")
```

```
# In[25]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

```
# In[26]:
```

```
#Removing amp i.e. replace amp with ""  
amp= re.compile(r"\s*amp[^\s]*")  
twitter_data["Text"] = twitter_data.Text.apply(lambda x: amp.sub("", x))
```

```
# In[27]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

```
# In[28]:
```

```
#Importing the word tokenize function from nltk  
from nltk.tokenize import word_tokenize
```

```
# In[29]:
```

```
#Applying Word tokenization  
twitter_list=[twitter_data["Text"].apply(word_tokenize),twitter_data["ID"]]
```

```
# In[30]:
```

```
#Converting the list into a dataframe  
twitter_df=pd.DataFrame(twitter_list)
```

```
# In[31]:
#Viewing if the Output changed accordingly to the last code entered
twitter_df.head()

# In[32]:
#Transposing the dataframe for better visibility
twitter_df=twitter_df.T

# In[33]:
#Viewing if the Output changed accordingly to the last code entered
twitter_df.head()

# In[34]:
#Importing the Snowball Stemmer and save the Snowball Stemmer for English in a
Variable
from nltk.stem.snowball import SnowballStemmer

sbs = SnowballStemmer("english")

# In[35]:
#Create a new column (stemmed_words [of the dataframe train_cleaner_df]) in which
every word (i) is stemmed in list x and saved
twitter_df["Text"] = twitter_df["Text"].apply(lambda x: [sbs.stem(i) for i in x])

# In[36]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[37]:
#Inspecting the stemmed dataframe
print(twitter_df)

# In[38]:
#Exporting the data cleaned file as csv
```

```
twitter_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\5 Matching Twitter and dict\Cleaned Twitter data\Entertainment\Tweet  
lists.csv")
```

```
# In[39]:
```

```
#####Start Section (20 000 reviews Train = 10%)#####
```

```
# In[40]:
```

```
#Creating a column with the original index
```

```
twitter_df["Real_index"] = twitter_df.index
```

```
# In[41]:
```

```
#Renaming the Text column to stemmed_words
```

```
twitter_df.rename(columns={"Text":"stemmed_words"}, inplace=True)
```

```
print(twitter_df)
```

```
# In[42]:
```

```
#Removing the Tweet ID
```

```
twitter_df = twitter_df.drop(columns=["ID"])
```

```
print(twitter_df)
```

```
# In[43]:
```

```
#Having each word separately with its corresponding Real_index number
```

```
twitter_df_exp=twitter_df.explode("stemmed_words")
```

```
print(twitter_df_exp)
```

```
# In[44]:
```

```
#Importing the tested dictionary, then dropping the Sentiment points column of the tested  
dictionary
```

```
dict_20_000_10=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University  
of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews  
Trainset =10%\Ultimate dict.csv")
```

```
dict_20_000_10 = dict_20_000_10.drop(columns=["Sentiment points"])
```

```
print(dict_20_000_10)
```

```

# In[45]:
#Words which appear in the cleaned twitter data and the tested dictionary as well is stored
in a cleaned dataframe
cleaned_df=pd.merge(twitter_df_exp, dict_20_000_10, on="stemmed_words",
how="inner")
print(cleaned_df)

# In[46]:
#Sorting according to the original Tweet index
cleaned_df.sort_values("Real_index")

# In[47]:
#If the Real_index number is the same, allocate the stemmed words into their own list,
then converting the Output into a Pandas dataframe again
relevant_clean_matrix=cleaned_df.groupby("Real_index")["stemmed_words"].apply(list)
relevant_clean_df=pd.DataFrame(relevant_clean_matrix)
print(relevant_clean_matrix)
print(relevant_clean_df)

# In[48]:
#First creating a column with the words original tweet number, then a new index is
initialised to indicate how many Tweet texts have been remaining
relevant_clean_df["Real Doc Numbers"] = relevant_clean_df.index
relevant_clean_df = relevant_clean_df.reset_index(drop=True)
print(relevant_clean_df)

# In[49]:
#Exporting the dataframe of stemmed_words and real document numbers as csv
relevant_clean_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Entertainment\matched words to docs overview.csv")

# In[50]:
#Creating a separate list of lists and saving it under the variable text_list
text_list=relevant_clean_df["stemmed_words"].tolist()

```



```

# In[51]:
#Viewing if the Output changed accordingly to the last code entered
print(text_list[1])

# In[52]:
#Importing the LDA Topic Model libraries
import gensim
from gensim import corpora
from gensim.models import CoherenceModel

# In[53]:
#Creating a dictionary made out of unique words
dictionary=corpora.Dictionary(relevant_clean_df["stemmed_words"])
print(dictionary)

# In[54]:
#Creating the document term matrix where one document is one document (i.e. Tweet)
from the relevant and cleaned dataframe
doc_term_matrix=[dictionary.doc2bow(doc) for doc in
relevant_clean_df["stemmed_words"] ]
print(len(doc_term_matrix))

# In[55]:
#Code by Selva Prabhakaran from MachineLearningPlus although the passes parameter
was added and adjusted to improve the topic model output and the random_state
parameter was added to make the calculation reproducible
def compute_coherence_values(dictionary,corpus,texts,limit, start=2, step=3):
    perplexity_scores=[]
    coherence_values=[]
    model_list=[]
    for num_topics in range (start, limit, step):
        model=gensim.models.ldamodel.LdaModel(corpus=corpus, num_topics=num_topics,
id2word=dictionary, random_state=100, passes=20)
        model_list.append(model)

```

```

coherencemodel= CoherenceModel(model=model, texts=texts, dictionary=dictionary,
coherence="c_v")
coherence_values.append(coherencemodel.get_coherence())
perplexity_scores.append(model.log_perplexity(corpus))

```

```

return model_list, coherence_values, perplexity_scores

```

In[56]:

```

#Code by Selva Prabhakaran from MachineLearningPlus although the size of limit and
step parameter were adjusted

```

```

model_list, coherence_values, perplexity_scores =
compute_coherence_values(dictionary=dictionary, corpus=doc_term_matrix,
texts=text_list, start=2, limit=50, step=1)

```

In[57]:

```

#Importing the Plotting Libraries

```

```

import matplotlib.pyplot as plt

```

```

import seaborn as sns

```

In[58]:

```

#Defining the x-axis

```

```

start=2

```

```

limit=50

```

```

step=1

```

```

x=range(start,limit,step)

```

```

#Assigning separately the corresponding coherence and perplexity score to each
respective LDA topic model

```

```

data1=zip(x,coherence_values)

```

```

data2=zip(x,perplexity_scores)

```

```

#Assigning the corresponding coherence and perplexity score together for each respective
LDA topic model

```

```

visual_df1=pd.DataFrame(data1,columns=["Topics", "Coherence Score"])

```

```

visual_df2=pd.DataFrame(data2,columns=["Topics", "Perplexity Score"])

```

```

visual_df=pd.merge(visual_df1, visual_df2, on="Topics", how="inner")

#Preparing to illustrate multiple line graphs within one plot and setting the size of the figure
fig, ax = plt.subplots()
fig.set_size_inches(19,5)

#Plotting the Coherence Scores and customising the first y-axis to the Coherence Score
ax=sns.lineplot(x="Topics",y="Coherence Score",data=visual_df,color="steelblue")
ax.yaxis.label.set_color("steelblue")

#Enabling the option to plot a second graph
ax2 = plt.twinx()

#Plotting the Perplexity Scores and customising the second y-axis to the Perplexity Score
sns.lineplot(x="Topics",y="Perplexity Score",data=visual_df, color="r", ax=ax2)
ax2.yaxis.label.set_color("r")

# In[59]:
#Export the Scatterplot as png file
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Entertainment\Perplexity vs Coherence.png")

# In[60]:
#Print the model with the highest coherence score
visual_df.loc[visual_df["Coherence Score"].idxmax()]

# In[61]:
#Selecting the model with the best coherence score and its corresponding 10 words,
which occur most likely
model_with_best_topic_number= model_list[22]
topics_with_significant_words=pd.DataFrame(model_with_best_topic_number.show_topics(
num_topics=24, num_words=10))
print(topics_with_significant_words)

```

```

# In[62]:
#Exporting the topics and their significant words as a csv file
topics_with_significant_words.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Entertainment\Infer the topics with significant words.csv")

# In[63]:
#Saving the Document Topic matrix and presenting it as a Pandas dataframe
doc_top=model_with_best_topic_number.get_document_topics(doc_term_matrix)
pd.DataFrame(model_with_best_topic_number.get_document_topics(doc_term_matrix))

# In[64]:
#Transformation into numpy due to incorrect assignment of values (row number 1) and None = 0
correct_topics_to_docs = gensim.matutils.corpus2csc(doc_top)
final_correct_topics_to_docs = correct_topics_to_docs.T.toarray()
print(final_correct_topics_to_docs)

# In[65]:
#The creation of the document topic matrix as a correct pandas dataframe
final_correct_topics_to_docs_df=pd.DataFrame(final_correct_topics_to_docs)

# In[66]:
#Viewing if the Output changed accordingly to the last code entered
print(final_correct_topics_to_docs_df)

# In[67]:
#Exporting the document topic matrix as csv
final_correct_topics_to_docs_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Entertainment\Topic to Document Matrix\doc to topics.csv")

# In[68]:
#Saving the Topic to keyword matrix (Topics=rows, words=columns)

```

```
word_top=pd.DataFrame(model_with_best_topic_number.get_topics(),columns=dictionary.
values())
```

```
# In[69]:
```

```
#Viewing if the Output changed accordingly to the last code entered
print(word_top)
```

```
# In[70]:
```

```
#Saving the word topic matrix as csv file
word_top.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Entertainment\Word to Topic Matrix\word to topics.csv")
```

```
# In[ ]:
```

```
##### End of Section (20 000 reviews Train = 10%)#####
```

```
# In[ ]:
```

```
##*Analog actions have been performed on the same cleaned Twitter data, but with the 200
000 reviews Train = 99% tested dictionary
#(Please revisit Section (20 000 reviews Train = 10%) for a commented version of these
steps
```

```
# In[ ]:
```

```
#####Start Section (200 000 reviews Train = 99%)#####
```

```
# In[71]:
```

```
dict_200_000_99=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200
000 reviews Trainset =99%\Ultimate dict.csv")
dict_200_000_99 = dict_200_000_99.drop(columns=["Sentiment points"])
print(dict_200_000_99)
```

```
# In[72]:
```

```
new_cleaned_df=pd.merge(twitter_df_exp, dict_200_000_99, on="stemmed_words",
how="inner")
```

```

print(new_cleaned_df)

# In[73]:
new_cleaned_df.sort_values("Real_index")

# In[74]:
new_relevant_clean_matrix=new_cleaned_df.groupby("Real_index")
["stemmed_words"].apply(list)
new_relevant_clean_df=pd.DataFrame(new_relevant_clean_matrix)
print(new_relevant_clean_matrix)
print(new_relevant_clean_df)

# In[75]:
new_relevant_clean_df["Real Doc Numbers"] = new_relevant_clean_df.index
new_relevant_clean_df = new_relevant_clean_df.reset_index(drop=True)
print(new_relevant_clean_df)

# In[76]:
new_relevant_clean_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University
of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 -
Train=99%)\Entertainment\matched words to docs overview.csv")

# In[77]:
new_text_list=new_relevant_clean_df["stemmed_words"].tolist()
print(new_text_list[0])

# In[78]:
new_dictionary=corpora.Dictionary(new_relevant_clean_df["stemmed_words"])
print(new_dictionary)

# In[79]:
new_doc_term_matrix=[new_dictionary.doc2bow(doc) for doc in
new_relevant_clean_df["stemmed_words"] ]
print(len(new_doc_term_matrix))

```

```

# In[80]:
#Code by Selva Prabhakaran from MachineLearningPlus although the size of limit and
step parameter were adjusted
model_list, coherence_values, perplexity_scores =
compute_coherence_values(dictionary=new_dictionary, corpus=new_doc_term_matrix,
texts=new_text_list, start=2, limit=50, step=1)

# In[81]:
#Defining the x-axis
start=2
limit=50
step=1
x=range(start,limit,step)

#Assigning separately the corresponding coherence and perplexity score to each
respective LDA topic model
new_data1=zip(x,coherence_values)
new_data2=zip(x,perplexity_scores)

#Assigning the corresponding coherence and perplexity score together for each respective
LDA topic model
new_visual_df1=pd.DataFrame(new_data1,columns=["Topics", "Coherence Score"])
new_visual_df2=pd.DataFrame(new_data2,columns=["Topics", "Perplexity Score"])
new_visual_df=pd.merge(new_visual_df1, new_visual_df2, on="Topics", how="inner")

#Preparing to illustrate multiple line graphs within one plot and setting the size of the figure
fig, ax = plt.subplots()
fig.set_size_inches(19,5)

#Plotting the Coherence Scores and customising the first y-axis to the Coherence Score
ax=sns.lineplot(x="Topics",y="Coherence Score",data=new_visual_df,color="steelblue")
ax.yaxis.label.set_color("steelblue")

#Enabling the option to plot a second graph
ax2 = plt.twinx()

```

```

#Plotting the Perplexity Scores and customising the second y-axis to the Perplexity Score
sns.lineplot(x="Topics",y="Perplexity Score",data=new_visual_df, color="r", ax=ax2)
ax2.yaxis.label.set_color("r")

# In[82]:
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)
\Entertainment\Perplexity vs Coherence.png")

# In[83]:
#Print the model with the highest coherence score
new_visual_df.loc[new_visual_df["Coherence Score"].idxmax()]

# In[84]:
new_model_with_best_topic_number= model_list[22]

new_topics_with_significant_words=pd.DataFrame(new_model_with_best_topic_number.s
how_topics(num_topics=24, num_words=10))
print(new_topics_with_significant_words)

# In[85]:
new_topics_with_significant_words.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive
- University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000
- Train=99%)\Entertainment\Infer the topics with significant words.csv")

# In[86]:
new_doc_top=new_model_with_best_topic_number.get_document_topics(new_doc_term
_matrix)
pd.DataFrame(new_model_with_best_topic_number.get_document_topics(new_doc_term
_matrix))

# In[87]:
new_correct_topics_to_docs = gensim.matutils.corpus2csc(new_doc_top)
new_final_correct_topics_to_docs = new_correct_topics_to_docs.T.toarray()

```



```
print(new_final_correct_topics_to_docs)
```

```
# In[88]:
```

```
new_final_correct_topics_to_docs_df=pd.DataFrame(new_final_correct_topics_to_docs)
print(new_final_correct_topics_to_docs_df)
```

```
# In[89]:
```

```
new_final_correct_topics_to_docs_df.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and
dict\Matched words (200 000 - Train=99%)\Entertainment\Topic to Document Matrix\doc to
topics.csv")
```

```
# In[90]:
```

```
new_word_top=pd.DataFrame(new_model_with_best_topic_number.get_topics(),columns
=new_dictionary.values())
```

```
# In[91]:
```

```
print(new_word_top)
```

```
# In[92]:
```

```
new_word_top.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)
\Entertainment\Word to Topic Matrix\word to topics.csv")
```

```
# In[ ]:
```

```
#####End Section (200 000 reviews Train = 99%)#####
```

```

                End - 8.2. Entertainment Data Cleaning Twitter data,
.....          Matching, keyword matrix creation and Topic modelling          .....
                (Both sentiment dictionaries)
```

```

                Start - 8.3. Wedding Data Cleaning Twitter data, Matching,
.....          keyword matrix creation and Topic modelling          .....
                (Both sentiment dictionaries)
```

```

# In[1]:
#Importing the Pandas library
import pandas as pd

# In[2]:
#Importing the Twitter Data concerning the topic bar
twitter_data=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\3 Obtaining the Twitter Data\Twitter Data\Search by topic
wedding.csv")

# In[3]:
#Inspecting the twitter dataframe
twitter_data.head()

# In[4]:
#Deleting the Screenname
twitter_data = twitter_data.drop(columns=["Screenname"])
print(twitter_data)

# In[5]:
#Maximising the view window to get an impression about the original tweets
pd.set_option("display.max_rows", None)

# In[6]:
#Displaying the original tweets
print(twitter_data)

# In[7]:
#Importing the Regular Expression library
import re

# In[8]:
#Removing hashtag symbols from the text data (i.e. replacing it by Nothing [""])
hashtag= re.compile(r"#")
twitter_data["Text"]= twitter_data.Text.apply(lambda x: hashtag.sub("", x))

```

In[9]:

#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

In[10]:

#Removing @ + username from the text data (i.e. replacing it by Nothing [""])
mention= re.compile(r"\s*@[\s]*")
twitter_data["Text"]= twitter_data.Text.apply(lambda x: mention.sub("", x))

In[11]:

#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

In[12]:

#Removing Urls from the text data (i.e. replacing it by Nothing [""])
url= re.compile(r"\s*http[\s]*")
twitter_data["Text"]= twitter_data.Text.apply(lambda x: url.sub("", x))

In[13]:

#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

In[14]:

#Lower casing the text data
twitter_data["Text"]=twitter_data["Text"].str.lower()

In[15]:

#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

In[16]:

#Importing the nltk (natural language toolkit) library
#Necessary step for removing stop_words (downloading the stop-word list) and applying
word tokenization (downloading the punkt package)

```

import nltk
nltk.download("stopwords")
nltk.download("punkt")

# In[17]:
#Importing the English stop-words
from nltk.corpus import stopwords
stop_words = stopwords.words("english")
print(stop_words)

# In[18]:
#Removing Stop-words = splitting each word except stop-words into a list for each tweet
respectively by maintaining the dataframe data type
twitter_data["Text"]=twitter_data["Text"].apply(lambda x: [item for item in str.split(x) if item
not in stop_words])

# In[19]:
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

# In[20]:
#Join all list items together by a space for each row of the particular text column
twitter_data["Text"] = [" ".join(map(str, l)) for l in twitter_data["Text"]]

# In[21]:
#Viewing if the Output changed accordingly to the last code entered
print(twitter_data)

# In[22]:
#Replace text which is not alphanumeric or underscore or space or tabs or line brakes with
"" (i.e. Nothing)
normal_things = re.compile(r"^[^\w\s]+")
twitter_data["Text"] = [normal_things.sub("", x) for x in twitter_data["Text"].tolist()]

# In[23]:

```

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

```
# In[24]:
```

```
#Removing Numbers i.e. replace Numbers with "" (\d+ means [0-9])  
twitter_data["Text"] = twitter_data["Text"].str.replace("\d+", "")
```

```
# In[25]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

```
# In[26]:
```

```
#Removing amp i.e. replace amp with ""  
amp= re.compile(r"\s*amp[^\s]*")  
twitter_data["Text"]= twitter_data.Text.apply(lambda x: amp.sub("", x))
```

```
# In[27]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(twitter_data)
```

```
# In[28]:
```

```
#Importing the word tokenize function from nltk  
from nltk.tokenize import word_tokenize
```

```
# In[29]:
```

```
#Applying Word tokenization  
twitter_list=[twitter_data["Text"].apply(word_tokenize),twitter_data["ID"]]
```

```
# In[30]:
```

```
#Converting the list into a dataframe  
twitter_df=pd.DataFrame(twitter_list)
```

```
# In[31]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
twitter_df.head()
```

```

# In[32]:
#Transposing the dataframe for better visibility
twitter_df=twitter_df.T

# In[33]:
#Viewing if the Output changed accordingly to the last code entered
twitter_df.head()

# In[34]:
#Importing the Snowball Stemmer and save the Snowball Stemmer for English in a
Variable
from nltk.stem.snowball import SnowballStemmer

sbs = SnowballStemmer("english")

# In[35]:
#Create a new column (stemmed_words [of the dataframe train_cleaner_df]) in which
every word (i) is stemmed in list x and saved
twitter_df["Text"] = twitter_df["Text"].apply(lambda x: [sbs.stem(i) for i in x])

# In[36]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[37]:
#Inspecting the stemmed dataframe
print(twitter_df)

# In[38]:
#Exporting the data cleaned file as csv
twitter_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Cleaned Twitter data\Wedding\Tweet
lists.csv")

```

```

# In[39]:
#####Start Section (20 000 reviews Train = 10%)#####

# In[40]:
#Creating a column with the original index
twitter_df["Real_index"] = twitter_df.index

# In[41]:
#Renaming the Text column to stemmed_words
twitter_df.rename(columns={"Text":"stemmed_words"}, inplace=True)
print(twitter_df)

# In[42]:
#Removing the Tweet ID
twitter_df = twitter_df.drop(columns=["ID"])
print(twitter_df)

# In[43]:
#Having each word separately with its corresponding Real_index number
twitter_df_exp=twitter_df.explode("stemmed_words")
print(twitter_df_exp)

# In[44]:
#Importing the tested dictionary, then dropping the Sentiment points column of the tested
dictionary
dict_20_000_10=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University
of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews
Trainset =10%\Ultimate dict.csv")
dict_20_000_10 = dict_20_000_10.drop(columns=["Sentiment points"])
print(dict_20_000_10)

# In[45]:
#Words which appear in the cleaned twitter data and the tested dictionary as well is stored
in a cleaned dataframe

```

```
cleaned_df=pd.merge(twitter_df_exp, dict_20_000_10, on="stemmed_words",
how="inner")
print(cleaned_df)
```

```
# In[46]:
```

```
#Sorting according to the original Tweet index
cleaned_df.sort_values("Real_index")
```

```
# In[47]:
```

```
#If the Real_index number is the same, allocate the stemmed words into their own list,
then converting the Output into a Pandas dataframe again
relevant_clean_matrix=cleaned_df.groupby("Real_index")["stemmed_words"].apply(list)
relevant_clean_df=pd.DataFrame(relevant_clean_matrix)
print(relevant_clean_matrix)
print(relevant_clean_df)
```

```
# In[48]:
```

```
#First creating a column with the words original tweet number, then a new index is
initialised to indicate how many Tweet texts have been remaining
relevant_clean_df["Real Doc Numbers"] = relevant_clean_df.index
relevant_clean_df = relevant_clean_df.reset_index(drop=True)
print(relevant_clean_df)
```

```
# In[49]:
```

```
#Exporting the dataframe of stemmed_words and real document numbers as csv
relevant_clean_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Wedding\matched words to docs overview.csv")
```

```
# In[50]:
```

```
#Creating a separate list of lists and saving it under the variable text_list
text_list=relevant_clean_df["stemmed_words"].tolist()
```

```
# In[51]:
```

```
#Viewing if the Output changed accordingly to the last code entered
```



```
print(text_list[1])
```

```
# In[52]:
```

```
#Importing the LDA Topic Model libraries
```

```
import gensim
```

```
from gensim import corpora
```

```
from gensim.models import CoherenceModel
```

```
# In[53]:
```

```
#Creating a dictionary made out of unique words
```

```
dictionary=corpora.Dictionary(relevant_clean_df["stemmed_words"])
```

```
print(dictionary)
```

```
# In[54]:
```

```
#Creating the document term matrix where one document is one document (i.e. Tweet)  
from the relevant and cleaned dataframe
```

```
doc_term_matrix=[dictionary.doc2bow(doc) for doc in  
relevant_clean_df["stemmed_words"] ]
```

```
print(len(doc_term_matrix))
```

```
# In[55]:
```

```
#Code by Selva Prabhakaran from MachineLearningPlus although the passes parameter  
was added and adjusted to improve the topic model output and the random_state  
parameter was added to make the calculation reproducible
```

```
def compute_coherence_values(dictionary,corpus,texts,limit, start=2, step=3):
```

```
    perplexity_scores=[]
```

```
    coherence_values=[]
```

```
    model_list=[]
```

```
    for num_topics in range (start, limit, step):
```

```
        model=gensim.models.ldamodel.LdaModel(corpus=corpus, num_topics=num_topics,  
id2word=dictionary, random_state=100, passes=20)
```

```
        model_list.append(model)
```

```
        coherencemodel= CoherenceModel(model=model, texts=texts, dictionary=dictionary,  
coherence="c_v")
```

```
        coherence_values.append(coherencemodel.get_coherence())
```

```
perplexity_scores.append(model.log_perplexity(corpus))
```

```
return model_list, coherence_values, perplexity_scores
```

```
# In[56]:
```

```
#Code by Selva Prabhakaran from MachineLearningPlus although the size of limit and  
step parameter were adjusted
```

```
model_list, coherence_values, perplexity_scores =  
compute_coherence_values(dictionary=dictionary, corpus=doc_term_matrix,  
texts=text_list, start=2, limit=50, step=1)
```

```
# In[57]:
```

```
#Importing the Plotting Libraries
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# In[58]:
```

```
#Defining the x-axis
```

```
start=2
```

```
limit=50
```

```
step=1
```

```
x=range(start,limit,step)
```

```
#Assigning separately the corresponding coherence and perplexity score to each  
respective LDA topic model
```

```
data1=zip(x,coherence_values)
```

```
data2=zip(x,perplexity_scores)
```

```
#Assigning the corresponding coherence and perplexity score together for each respective  
LDA topic model
```

```
visual_df1=pd.DataFrame(data1,columns=["Topics", "Coherence Score"])
```

```
visual_df2=pd.DataFrame(data2,columns=["Topics", "Perplexity Score"])
```

```
visual_df=pd.merge(visual_df1, visual_df2, on="Topics", how="inner")
```

```
#Preparing to illustrate multiple line graphs within one plot and setting the size of the figure
```

```

fig, ax = plt.subplots()
fig.set_size_inches(19,5)

#Plotting the Coherence Scores and customising the first y-axis to the Coherence Score
ax=sns.lineplot(x="Topics",y="Coherence Score",data=visual_df,color="steelblue")
ax.yaxis.label.set_color("steelblue")

#Enabling the option to plot a second graph
ax2 = plt.twinx()

#Plotting the Perplexity Scores and customising the second y-axis to the Perplexity Score
sns.lineplot(x="Topics",y="Perplexity Score",data=visual_df, color="r", ax=ax2)
ax2.yaxis.label.set_color("r")

# In[59]:
#Export the Scatterplot as png file
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Wedding\Perplexity vs Coherence.png")

# In[60]:
#Print the model with the highest coherence score
visual_df.loc[visual_df["Coherence Score"].idxmax()]

# In[61]:
#Selecting the model with the best coherence score and its corresponding 10 words,
which occur most likely
model_with_best_topic_number= model_list[21]
topics_with_significant_words=pd.DataFrame(model_with_best_topic_number.show_topics(
num_topics=23, num_words=10))
print(topics_with_significant_words)

# In[62]:
#Exporting the topics and their significant words as a csv file

```

```
topics_with_significant_words.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Wedding\Infer the topics with significant words.csv")
```

```
# In[63]:
```

```
#Saving the Document Topic matrix and presenting it as a Pandas dataframe  
doc_top=model_with_best_topic_number.get_document_topics(doc_term_matrix)  
pd.DataFrame(model_with_best_topic_number.get_document_topics(doc_term_matrix))
```

```
# In[64]:
```

```
#Transformation into numpy due to incorrect assignment of values (row number 1) and  
None = 0  
correct_topics_to_docs = gensim.matutils.corpus2csc(doc_top)  
final_correct_topics_to_docs = correct_topics_to_docs.T.toarray()  
print(final_correct_topics_to_docs)
```

```
# In[65]:
```

```
#The creation of the document topic matrix as a correct pandas dataframe  
final_correct_topics_to_docs_df=pd.DataFrame(final_correct_topics_to_docs)
```

```
# In[66]:
```

```
#Viewing if the Output changed accordingly to the last code entered  
print(final_correct_topics_to_docs_df)
```

```
# In[67]:
```

```
#Exporting the document topic matrix as csv  
final_correct_topics_to_docs_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Wedding\Topic to Document Matrix\doc to topics.csv")
```

```
# In[68]:
```

```
#Saving the Topic to keyword matrix (Topics=rows, words=columns)  
word_top=pd.DataFrame(model_with_best_topic_number.get_topics(),columns=dictionary.  
values())
```

```

# In[69]:
#Viewing if the Output changed accordingly to the last code entered
print(word_top)

# In[70]:
#Saving the word topic matrix as csv file
word_top.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)
\Wedding\Word to Topic Matrix\word to topics.csv")

# In[ ]:
##### End of Section (20 000 reviews Train = 10%)#####

# In[ ]:
#*Analog actions have been performed on the same cleaned Twitter data, but with the 200
000 reviews Train = 99% tested dictionary
#(Please revisit Section (20 000 reviews Train = 10%) for a commented version of these
steps

# In[ ]:
#####Start Section (200 000 reviews Train = 99%)#####

# In[71]:
dict_200_000_99=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200
000 reviews Trainset =99%\Ultimate dict.csv")
dict_200_000_99 = dict_200_000_99.drop(columns=["Sentiment points"])
print(dict_200_000_99)

# In[72]:
new_cleaned_df=pd.merge(twitter_df_exp, dict_200_000_99, on="stemmed_words",
how="inner")
print(new_cleaned_df)

# In[73]:

```

```
new_cleaned_df.sort_values("Real_index")
```

```
# In[74]:
```

```
new_relevant_clean_matrix=new_cleaned_df.groupby("Real_index")  
["stemmed_words"].apply(list)  
new_relevant_clean_df=pd.DataFrame(new_relevant_clean_matrix)  
print(new_relevant_clean_matrix)  
print(new_relevant_clean_df)
```

```
# In[75]:
```

```
new_relevant_clean_df["Real Doc Numbers"] = new_relevant_clean_df.index  
new_relevant_clean_df = new_relevant_clean_df.reset_index(drop=True)  
print(new_relevant_clean_df)
```

```
# In[76]:
```

```
new_relevant_clean_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University  
of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 -  
Train=99%)\Wedding\matched words to docs overview.csv")
```

```
# In[77]:
```

```
new_text_list=new_relevant_clean_df["stemmed_words"].tolist()  
print(new_text_list[0])
```

```
# In[78]:
```

```
new_dictionary=corpora.Dictionary(new_relevant_clean_df["stemmed_words"])  
print(new_dictionary)
```

```
# In[79]:
```

```
new_doc_term_matrix=[new_dictionary.doc2bow(doc) for doc in  
new_relevant_clean_df["stemmed_words"] ]  
print(len(new_doc_term_matrix))
```

```
# In[80]:
```

```
#Code by Selva Prabhakaran from MachineLearningPlus although the size of limit and  
step parameter were adjusted
```

```
model_list, coherence_values, perplexity_scores =  
compute_coherence_values(dictionary=new_dictionary, corpus=new_doc_term_matrix,  
texts=new_text_list, start=2, limit=50, step=1)
```

```
# In[81]:
```

```
#Defining the x-axis
```

```
start=2
```

```
limit=50
```

```
step=1
```

```
x=range(start,limit,step)
```

```
#Assigning separately the corresponding coherence and perplexity score to each  
respective LDA topic model
```

```
new_data1=zip(x,coherence_values)
```

```
new_data2=zip(x,perplexity_scores)
```

```
#Assigning the corresponding coherence and perplexity score together for each respective  
LDA topic model
```

```
new_visual_df1=pd.DataFrame(new_data1,columns=["Topics", "Coherence Score"])
```

```
new_visual_df2=pd.DataFrame(new_data2,columns=["Topics", "Perplexity Score"])
```

```
new_visual_df=pd.merge(new_visual_df1, new_visual_df2, on="Topics", how="inner")
```

```
#Preparing to illustrate multiple line graphs within one plot and setting the size of the figure
```

```
fig, ax = plt.subplots()
```

```
fig.set_size_inches(19,5)
```

```
#Plotting the Coherence Scores and customising the first y-axis to the Coherence Score
```

```
ax=sns.lineplot(x="Topics",y="Coherence Score",data=new_visual_df,color="steelblue")
```

```
ax.yaxis.label.set_color("steelblue")
```

```
#Enabling the option to plot a second graph
```

```
ax2 = plt.twinx()
```

```
#Plotting the Perplexity Scores and customising the second y-axis to the Perplexity Score
```

```
sns.lineplot(x="Topics",y="Perplexity Score",data=new_visual_df, color="r", ax=ax2)
```

```
ax2.yaxis.label.set_color("r")
```

```
# In[82]:
```

```
fig.savefig(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)  
\Wedding\Perplexity vs Coherence.png")
```

```
# In[83]:
```

```
#Print the model with the highest coherence score  
new_visual_df.loc[new_visual_df["Coherence Score"].idxmax()]
```

```
# In[84]:
```

```
new_model_with_best_topic_number= model_list[21]
```

```
new_topics_with_significant_words=pd.DataFrame(new_model_with_best_topic_number.s  
how_topics(num_topics=23, num_words=10))  
print(new_topics_with_significant_words)
```

```
# In[85]:
```

```
new_topics_with_significant_words.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive  
- University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000  
- Train=99%)\Wedding\Infer the topics with significant words.csv")
```

```
# In[86]:
```

```
new_doc_top=new_model_with_best_topic_number.get_document_topics(new_doc_term  
_matrix)  
pd.DataFrame(new_model_with_best_topic_number.get_document_topics(new_doc_term  
_matrix))
```

```
# In[87]:
```

```
new_correct_topics_to_docs = gensim.matutils.corpus2csc(new_doc_top)  
new_final_correct_topics_to_docs = new_correct_topics_to_docs.T.toarray()  
print(new_final_correct_topics_to_docs)
```

```
# In[88]:
```



```
new_final_correct_topics_to_docs_df=pd.DataFrame(new_final_correct_topics_to_docs)
print(new_final_correct_topics_to_docs_df)
```

```
# In[89]:
```

```
new_final_correct_topics_to_docs_df.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and
dict\Matched words (200 000 - Train=99%)\Wedding\Topic to Document Matrix\doc to
topics.csv")
```

```
# In[90]:
```

```
new_word_top=pd.DataFrame(new_model_with_best_topic_number.get_topics(),columns
=new_dictionary.values())
```

```
# In[91]:
```

```
print(new_word_top)
```

```
# In[92]:
```

```
new_word_top.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West
London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)
\Wedding\Word to Topic Matrix\word to topics.csv")
```

```
# In[ ]:
```

```
#####End Section (200 000 reviews Train = 99%)#####
```

End - 8.3. Wedding Data Cleaning Twitter data, Matching,

```
..... keyword matrix creation and Topic modelling .....
              (Both sentiment dictionaries)
```

Start - 9 Finding Opportunities

```
..... (Optimal document-topic and topic-word matrices of both opportunity systems) .....
```

```
# In[1]:
```

```
import pandas as pd
```

```
# In[2]:
```

```
##### Start 20 000 reviews Train = 10 %#####
```

```
# In[ ]:
```

```
#*Start Topic Stock Section*
```

```
# In[ ]:
```

```
#Bar Start
```

```
# In[3]:
```

```
#Importing of the document to topic matrix of the overall topic bar
```

```
doc_top_20_10_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Bar\Topic to Document Matrix\doc to topics.csv")
```

```
print(doc_top_20_10_bar)
```

```
# In[4]:
```

```
#Removing the column Unnamed: 0
```

```
doc_top_20_10_bar= doc_top_20_10_bar.drop(columns=["Unnamed: 0"])
```

```
print(doc_top_20_10_bar)
```

```
# In[5]:
```

```
#Transposing the document topic matrix
```

```
doc_top_20_10_bar=doc_top_20_10_bar.T
```

```
print(doc_top_20_10_bar)
```

```
# In[6]:
```

```
#Summing up the absolute total score of importance for each topic in the topic category bar respectively
```

```
doc_top_20_10_bar["Total"] = doc_top_20_10_bar.sum(axis=1)
```

```
doc_top_20_10_bar.head()
```

```
# In[7]:
```

```
#Finding and saving the minimum value of the absolute total score of importance in the topic category bar in a variable
```

```
minimum_doc_top_20_10_bar=doc_top_20_10_bar["Total"].min()
```

```
print(minimum_doc_top_20_10_bar)
```

```
# In[8]:
```

```
#Finding and saving the maximal value of the absolute total score of importance in the  
topic category bar in a variable
```

```
maximum_doc_top_20_10_bar=doc_top_20_10_bar["Total"].max()
```

```
print(maximum_doc_top_20_10_bar)
```

```
# In[ ]:
```

```
#Bar END
```

```
# In[ ]:
```

```
#Entertainment Start
```

```
# In[9]:
```

```
#Importing the document to topic matrix for stationery entertainment
```

```
doc_top_20_10_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-  
VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and  
dict\Matched words (20 000 - Train=10%)\Entertainment\Topic to Document Matrix\doc to  
topics.csv")
```

```
print(doc_top_20_10_entertainment)
```

```
# In[10]:
```

```
#Removing the column Unnamed: 0
```

```
d o c _ t o p _ 2 0 _ 1 0 _ e n t e r t a i n m e n t =  
doc_top_20_10_entertainment.drop(columns=["Unnamed: 0"])
```

```
print(doc_top_20_10_entertainment)
```

```
# In[11]:
```

```
#Transposing the document to topic matrix of the topic stationary entertainment
```

```
doc_top_20_10_entertainment=doc_top_20_10_entertainment.T
```

```
print(doc_top_20_10_entertainment)
```

```
# In[12]:
```

```
#Summing up the absolute total score of importance for each topic in the topic category  
entertainment respectively
```

```
doc_top_20_10_entertainment["Total"] = doc_top_20_10_entertainment.sum(axis=1)
doc_top_20_10_entertainment.head()
```

```
# In[13]:
```

```
#Finding and saving the minimum value of the absolute total score of importance in the
topic category entertainment in a variable
```

```
minimum_doc_top_20_10_entertainment=doc_top_20_10_entertainment["Total"].min()
print(minimum_doc_top_20_10_entertainment)
```

```
# In[14]:
```

```
#Finding and saving the maximum value of the absolute total score of importance in the
topic category entertainment in a variable
```

```
maximum_doc_top_20_10_entertainment=doc_top_20_10_entertainment["Total"].max()
print(maximum_doc_top_20_10_entertainment)
```

```
# In[ ]:
```

```
#Entertainment END
```

```
# In[ ]:
```

```
#Wedding Start
```

```
# In[15]:
```

```
#Importing the doc to topic matrix for the overall topic wedding
```

```
doc_top_20_10_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 -
Train=10%)\Wedding\Topic to Document Matrix\doc to topics.csv")
print(doc_top_20_10_wedding)
```

```
# In[16]:
```

```
#Dropping column Unnamed: 0
```

```
doc_top_20_10_wedding= doc_top_20_10_wedding.drop(columns=["Unnamed: 0"])
print(doc_top_20_10_wedding)
```

```
# In[17]:
```

```
#Transposing the document to topic dataframe
```

```
doc_top_20_10_wedding=doc_top_20_10_wedding.T
print(doc_top_20_10_wedding)
```

```
# In[18]:
```

```
#Summing up the absolute total score of importance for each topic in the topic category
wedding respectively
```

```
doc_top_20_10_wedding["Total"] = doc_top_20_10_wedding.sum(axis=1)
doc_top_20_10_wedding.head()
```

```
# In[19]:
```

```
#Finding and saving the minimal value of the absolute total score of importance in the
topic category wedding in a variable
```

```
minimum_doc_top_20_10_wedding=doc_top_20_10_wedding["Total"].min()
print(minimum_doc_top_20_10_wedding)
```

```
# In[20]:
```

```
#Finding and saving the maximal value of the absolute total score of importance in the
topic category wedding in a variable
```

```
maximum_doc_top_20_10_wedding=doc_top_20_10_wedding["Total"].max()
print(maximum_doc_top_20_10_wedding)
```

```
# In[ ]:
```

```
# Wedding End
```

```
# In[21]:
```

```
#Opening the viewing window to a maximum
```

```
pd.set_option("display.max_rows", None)
```

```
# In[22]:
```

```
#Calculating the relative score of importance in the topic category bar and saving it in a
new column in the bar document topic dataframe
```

```
doc_top_20_10_bar["Topic Stock"]=(doc_top_20_10_bar["Total"]-
minimum_doc_top_20_10_bar)/(maximum_doc_top_20_10_bar-
minimum_doc_top_20_10_bar)
print(doc_top_20_10_bar)
```

In[23]:

#Calculating the relative score of importance in the topic category entertainment and saving it in a new column in the entertainment document topic dataframe

```
doc_top_20_10_entertainment["Topic Stock"]=(doc_top_20_10_entertainment["Total"]-
minimum_doc_top_20_10_entertainment)/(maximum_doc_top_20_10_entertainment-
minimum_doc_top_20_10_entertainment)
print(doc_top_20_10_entertainment)
```

In[24]:

#Calculating the relative score of importance in the topic category wedding and saving it in a new column in the wedding document topic dataframe

```
doc_top_20_10_wedding["Topic Stock"]=(doc_top_20_10_wedding["Total"]-
minimum_doc_top_20_10_wedding)/(maximum_doc_top_20_10_wedding-
minimum_doc_top_20_10_wedding)
print(doc_top_20_10_wedding)
```

In[25]:

#Scaling up the relative score of importance in the topic category bar by 10, before converting it into a dataframe.

#In addition the index number is set as topic ID and a new column topic category is created which is equal to Bar for all of the bar topics.

```
Topic_stock_vector_20_10_bar=10*doc_top_20_10_bar["Topic Stock"]
Topic_stock_vector_20_10_bar=pd.DataFrame(Topic_stock_vector_20_10_bar,columns=["
Topic Stock"])
Topic_stock_vector_20_10_bar["Topics"] = Topic_stock_vector_20_10_bar.index
Topic_stock_vector_20_10_bar["Topic Category"]="Bar"

print(Topic_stock_vector_20_10_bar)
```

In[26]:

#Scaling up the relative score of importance in the topic category entertainment by 10, before converting it into a dataframe.

#In addition the index number is set as topic ID and a new column topic category is created which is equal to entertainment for all of the entertainment topics.

```

Topic_stock_vector_20_10_entertainment=10*doc_top_20_10_entertainment["Topic
Stock"]
Topic_stock_vector_20_10_entertainment=pd.DataFrame(Topic_stock_vector_20_10_ent
ertainment,columns=["Topic Stock"])
Topic_stock_vector_20_10_entertainment["Topics"] =
Topic_stock_vector_20_10_entertainment.index
Topic_stock_vector_20_10_entertainment["Topic Category"]="Entertainment"

print(Topic_stock_vector_20_10_entertainment)

```

In[27]:

#Scaling up the relative score of importance in the topic category wedding by 10, before converting it into a dataframe.

#In addition the index number is set as topic ID and a new column topic category is created which is equal to wedding for all of the wedding topics.

```

Topic_stock_vector_20_10_wedding=10*doc_top_20_10_wedding["Topic Stock"]
Topic_stock_vector_20_10_wedding=pd.DataFrame(Topic_stock_vector_20_10_wedding,
columns=["Topic Stock"])
Topic_stock_vector_20_10_wedding["Topics"] = Topic_stock_vector_20_10_wedding.index
Topic_stock_vector_20_10_wedding["Topic Category"]="Wedding"

print(Topic_stock_vector_20_10_wedding)

```

In[28]:

#Reordering the columns of the bar topic importance dataframe

```

reordered_topic_df_bar= Topic_stock_vector_20_10_bar[["Topics","Topic Category" ,"Topic
Stock"]]
print(reordered_topic_df_bar)

```

In[29]:

#Reordering the columns of the entertainment topic importance dataframe

```

reordered_topic_df_entertainment =
Topic_stock_vector_20_10_entertainment[["Topics","Topic Category" ,"Topic Stock"]]
print(reordered_topic_df_entertainment)

```

```

# In[30]:
#Reordering the columns of the wedding topic importance dataframe
reordered_topic_df_wedding= Topic_stock_vector_20_10_wedding[["Topics","Topic
Category" ,"Topic Stock"]]
print(reordered_topic_df_wedding)

# In[31]:
#Saving the bar topic importance dataframe as csv file
reordered_topic_df_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Bar\Topic Importance\Topic Importance.csv",index=False)

# In[32]:
#Saving the entertainment topic importance dataframe as csv file
reordered_topic_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive
- University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Entertainment\Topic Importance\Topic Importance.csv",index=False)

# In[33]:
#Saving the wedding topic importance dataframe as csv file
reordered_topic_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Wedding\Topic Importance\Topic Importance.csv",index=False)

# In[ ]:
##*END Topic Stock Section*

# In[34]:
#Resetting the max view option to the default value
pd.reset_option("display.max_rows")

# In[ ]:
##*Start Sentiment Stock Section*

# In[35]:

```



```
#Importing the word to topic matrix as pandas dataframe of the overall topic Bar
word_top_20_10_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Bar\Word to Topic Matrix\word to topics.csv")
print(word_top_20_10_bar)
```

```
# In[36]:
#Dropping column Unnamed: 0
word_top_20_10_bar = word_top_20_10_bar.drop(columns=["Unnamed: 0"])
print(word_top_20_10_bar)
```

```
# In[37]:
#Transposing the word to topic dataframe about bar topics
word_top_20_10_bar=word_top_20_10_bar.T
print(word_top_20_10_bar)
```

```
# In[38]:
#Setting the index as a column at the first column to the left, then renaming the new column index to stemmed_words
word_top_20_10_bar.reset_index(level=0, inplace=True)
word_top_20_10_bar.rename(columns={"index":"stemmed_words"}, inplace=True)
print(word_top_20_10_bar)
```

```
# In[39]:
#Importing the tested sentiment dictionary
sent_dict_20_10=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\20 000 reviews Trainset =10%\Ultimate dict.csv")
print(sent_dict_20_10)
```

```
# In[40]:
#Assigning the sentiment of the dictionary (-1 = negative, +1 = positive) to the words of the word topic matrix of the general topic bar
concrete_sentiment_df_bar=pd.merge(word_top_20_10_bar, sent_dict_20_10, on="stemmed_words", how="inner")
```

```

print(concrete_sentiment_df_bar)

# In[41]:
#Saving the sentiment values and sentiment order for the word topic matrix of the overall
topic bar
Sentiment_20_10_series_bar=concrete_sentiment_df_bar["Sentiment points"]
print(Sentiment_20_10_series_bar)

# In[42]:
#Saving the unique words in the word topic matrix of the overall topic bar into a variable
words_bar=concrete_sentiment_df_bar["stemmed_words"]

#Dropping both columns which have been saved prior
concrete_sentiment_df_bar= concrete_sentiment_df_bar.drop(columns=["Sentiment
points"])
c o n c r e t e _ s e n t i m e n t _ d f _ b a r =
concrete_sentiment_df_bar.drop(columns=["stemmed_words"])
print(concrete_sentiment_df_bar)

# In[43]:
#Multiplying remaining word topic matrix the assigned sentiment value (-1 or +1) for each
word
concrete_sentiment_df_bar=concrete_sentiment_df_bar.mul(Sentiment_20_10_series_bar
, axis = 0)
print(concrete_sentiment_df_bar)
print(words_bar)

# In[44]:
#Adding the stemmed words first as column and second as index to the sentiment word
topic dataframe of the overall topic bar
final_concrete_sentiment_df_bar=pd.concat([words_bar, concrete_sentiment_df_bar],
axis=1)
final_concrete_sentiment_df_bar.set_index("stemmed_words", inplace=True)
print(final_concrete_sentiment_df_bar)

```

In[45]:

#Transposing the sentiment word topic matrix of the overall topic bar

final_concrete_sentiment_df_bar=final_concrete_sentiment_df_bar.T

print(final_concrete_sentiment_df_bar)

In[46]:

#Renaming index as Topics and then saving the sentiment word topics dataframe of the overall topic bar as csv file

final_concrete_sentiment_df_bar=final_concrete_sentiment_df_bar.rename_axis(None, axis=1).rename_axis("Topics", axis=0)

final_concrete_sentiment_df_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%) \Bar\Topic words table with sentiment\Topic words table with sentiment.csv")

In[47]:

#Generating the absolute total sentiment for each subtopic of the overall topic bar

final_concrete_sentiment_df_bar["Total"] = final_concrete_sentiment_df_bar.sum(axis=1)

final_concrete_sentiment_df_bar.head()

In[48]:

#Finding and saving the minimum of the absolute total sentiment of each bar subtopic

minimum_final_sen_20_10_bar=final_concrete_sentiment_df_bar["Total"].min()

print(minimum_final_sen_20_10_bar)

In[49]:

#Finding and saving the maximum of the absolute total sentiment of each bar subtopic

maximum_final_sen_20_10_bar=final_concrete_sentiment_df_bar["Total"].max()

print(maximum_final_sen_20_10_bar)

In[50]:

#Importing the word topic matrix of the general topic entertainment

word_top_20_10_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Entertainment\Word to Topic Matrix\word to topics.csv")

```
print(word_top_20_10_entertainment)
```

```
# In[51]:
```

```
#Removing the column Unnamed: 0
```

```
word_top_20_10_entertainment
```

```
=
```

```
word_top_20_10_entertainment.drop(columns=["Unnamed: 0"])
```

```
print(word_top_20_10_entertainment)
```

```
# In[52]:
```

```
#Transposing the word to topic matrix of the general topic of entertainment
```

```
word_top_20_10_entertainment=word_top_20_10_entertainment.T
```

```
print(word_top_20_10_entertainment)
```

```
# In[53]:
```

```
#Setting the index as column and rename the column with the name index with the name  
stemmed_words
```

```
word_top_20_10_entertainment.reset_index(level=0, inplace=True)
```

```
word_top_20_10_entertainment.rename(columns={"index":"stemmed_words"},  
inplace=True)
```

```
print(word_top_20_10_entertainment)
```

```
# In[54]:
```

```
#Assigning a sentiment of the tested dictionary to each word of the word topic dataframe  
of the general topic entertainment
```

```
concrete_sentiment_df_entertainment=pd.merge(word_top_20_10_entertainment,  
sent_dict_20_10, on="stemmed_words", how="inner")
```

```
print(concrete_sentiment_df_entertainment)
```

```
# In[55]:
```

```
#Saving the assigned sentiment for the entertainment words in a new variable
```

```
Sentiment_20_10_series_entertainment=concrete_sentiment_df_entertainment["Sentimen  
t points"]
```

```
print(Sentiment_20_10_series_entertainment)
```

```
# In[56]:
```

#Saving the entertainment words in a new variable

```
words_entertainment=concrete_sentiment_df_entertainment["stemmed_words"]
```

#Removing the columns sentiment points and stemmed_words which have been saved prior to this action

```
concrete_sentiment_df_entertainment =  
concrete_sentiment_df_entertainment.drop(columns=["Sentiment points"])
```

```
concrete_sentiment_df_entertainment =  
concrete_sentiment_df_entertainment.drop(columns=["stemmed_words"])
```

```
print(concrete_sentiment_df_entertainment)
```

In[57]:

#Multiplying the word topic matrix with the assigned sentiment from the dictionary (-1 = Negative or 1 = Positive)

```
concrete_sentiment_df_entertainment=concrete_sentiment_df_entertainment.mul(Sentiment_20_10_series_entertainment, axis = 0)
```

```
print(concrete_sentiment_df_entertainment)
```

```
print(words_entertainment)
```

In[58]:

#Adding the column stemmed_words, then setting these stemmed entertainment words as index

```
final_concrete_sentiment_df_entertainment=pd.concat([words_entertainment,  
concrete_sentiment_df_entertainment], axis=1)
```

```
final_concrete_sentiment_df_entertainment.set_index("stemmed_words", inplace=True)
```

```
print(final_concrete_sentiment_df_entertainment)
```

In[59]:

#Transposing the sentiment word topic matrix belonging to the overall topic entertainment

```
final_concrete_sentiment_df_entertainment=final_concrete_sentiment_df_entertainment.T  
print(final_concrete_sentiment_df_entertainment)
```

In[60]:

#Renaming the index to Topics, then exporting the sentiment entertainment word topic matrix as csv file

```

final_concrete_sentiment_df_entertainment=final_concrete_sentiment_df_entertainment.re
name_axis(None, axis=1).rename_axis("Topics", axis=0)
final_concrete_sentiment_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews
(Train = 10%)\Entertainment\Topic words table with sentiment\Topic words table with
sentiment.csv")

```

```
# In[61]:
```

```
#Generating the absolute total sentiment for each subtopic of the overall topic
entertainment
```

```

final_concrete_sentiment_df_entertainment["Total"]      =
final_concrete_sentiment_df_entertainment.sum(axis=1)
final_concrete_sentiment_df_entertainment.head()

```

```
# In[62]:
```

```
#Finding and saving the minimum of the absolute total sentiment of each entertainment
subtopic
```

```

minimum_final_sen_20_10_entertainment=final_concrete_sentiment_df_entertainment["To
tal"].min()
print(minimum_final_sen_20_10_entertainment)

```

```
# In[63]:
```

```
#Finding and saving the maximum of the absolute total sentiment of each entertainment
subtopic
```

```

maximum_final_sen_20_10_entertainment=final_concrete_sentiment_df_entertainment["T
otal"].max()
print(maximum_final_sen_20_10_entertainment)

```

```
# In[64]:
```

```
#Importing the word to topic matrix for the overall topic wedding
```

```

word_top_20_10_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 -
Train=10%)\Wedding\Word to Topic Matrix\word to topics.csv")
print(word_top_20_10_wedding)

```

```

# In[65]:
#Removing the column Unnamed: 0
word_top_20_10_wedding= word_top_20_10_wedding.drop(columns=["Unnamed: 0"])
print(word_top_20_10_wedding)

# In[66]:
#Transposing the word to topic dataframe of the general topic wedding
word_top_20_10_wedding=word_top_20_10_wedding.T
print(word_top_20_10_wedding)

# In[67]:
#Setting the index as column at the first place, then renaming it to stemmed_words and
the new index will consist out of numbers
word_top_20_10_wedding.reset_index(level=0, inplace=True)
word_top_20_10_wedding.rename(columns={"index":"stemmed_words"}, inplace=True)
print(word_top_20_10_wedding)

# In[68]:
#Assigning sentiment of the tested dictionary to the stemmed wedding words (-1 =
Negative, +1 = Positive)
concrete_sentiment_df_wedding=pd.merge(word_top_20_10_wedding, sent_dict_20_10,
on="stemmed_words", how="inner")
print(concrete_sentiment_df_wedding)

# In[69]:
#Saving the assigned sentiment and the order in a new variable as a series
Sentiment_20_10_series_wedding=concrete_sentiment_df_wedding["Sentiment points"]
print(Sentiment_20_10_series_wedding)

# In[70]:
#Saving the stemmed wedding words in a new variable
words_wedding=concrete_sentiment_df_wedding["stemmed_words"]

#Removing the columns stemmed_words and sentiment points

```

```

c o n c r e t e _ s e n t i m e n t _ d f _ w e d d i n g =
concrete_sentiment_df_wedding.drop(columns=["Sentiment points"])
c o n c r e t e _ s e n t i m e n t _ d f _ w e d d i n g =
concrete_sentiment_df_wedding.drop(columns=["stemmed_words"])
print(concrete_sentiment_df_wedding)

```

In[71]:

```

#Multiplying the sentiment to the word to topic dataframe of the general topic wedding
concrete_sentiment_df_wedding=concrete_sentiment_df_wedding.mul(Sentiment_20_10_
series_wedding, axis = 0)
print(concrete_sentiment_df_wedding)
print(words_wedding)

```

In[72]:

```

#Adding the stemmed_words as column and then setting the stemmed_words column as
index
final_concrete_sentiment_df_wedding=pd.concat([words_wedding,
concrete_sentiment_df_wedding], axis=1)
final_concrete_sentiment_df_wedding.set_index("stemmed_words", inplace=True)
print(final_concrete_sentiment_df_wedding)

```

In[73]:

```

#Transposing the sentiment word topic dataframe for the general topic wedding
final_concrete_sentiment_df_wedding=final_concrete_sentiment_df_wedding.T
print(final_concrete_sentiment_df_wedding)

```

In[74]:

```

#Renaming the index to Topic and then saving the sentiment word to topic dataframe for
the overall topic wedding
final_concrete_sentiment_df_wedding=final_concrete_sentiment_df_wedding.rename_axi
s(None, axis=1).rename_axis("Topics", axis=0)
final_concrete_sentiment_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews
(Train = 10%)\Wedding\Topic words table with sentiment\Topic words table with
sentiment.csv")

```



```
# In[75]:
```

```
#Generating the absolute total sentiment for each subtopic of the overall topic wedding
```

```
final_concrete_sentiment_df_wedding["Total"] =  
final_concrete_sentiment_df_wedding.sum(axis=1)  
final_concrete_sentiment_df_wedding.head()
```

```
# In[76]:
```

```
#Finding and saving the minimum of the absolute total sentiment of each wedding subtopic
```

```
minimum_final_sen_20_10_wedding=final_concrete_sentiment_df_wedding["Total"].min()  
print(minimum_final_sen_20_10_wedding)
```

```
# In[77]:
```

```
#Finding and saving the maximum of the absolute total sentiment of each wedding  
subtopic
```

```
maximum_final_sen_20_10_wedding=final_concrete_sentiment_df_wedding["Total"].max()  
print(maximum_final_sen_20_10_wedding)
```

```
# In[ ]:
```

```
#Last Word to Topic End
```

```
# In[78]:
```

```
#Opening up the viewing window to the maximum
```

```
pd.set_option("display.max_rows", None)
```

```
# In[79]:
```

```
#Calculating the relative sentiment score for the topic category bar and saving it in a new  
column in the bar sentiment word topic dataframe
```

```
final_concrete_sentiment_df_bar["Sentiment  
Stock"]=(final_concrete_sentiment_df_bar["Total"]-minimum_final_sen_20_10_bar)/  
(maximum_final_sen_20_10_bar-minimum_final_sen_20_10_bar)  
print(final_concrete_sentiment_df_bar)
```

```
# In[80]:
```

#Calculating the relative sentiment score for the topic category entertainment and saving it in a new column in the entertainment sentiment word topic dataframe

```
final_concrete_sentiment_df_entertainment["Sentiment Stock"]=(final_concrete_sentiment_df_entertainment["Total"]-minimum_final_sen_20_10_entertainment)/(maximum_final_sen_20_10_entertainment-minimum_final_sen_20_10_entertainment)
print(final_concrete_sentiment_df_entertainment)
```

In[81]:

#Calculating the relative sentiment score for the topic category wedding and saving it in a new column in the wedding sentiment word topic dataframe

```
final_concrete_sentiment_df_wedding["Sentiment Stock"]=(final_concrete_sentiment_df_wedding["Total"]-minimum_final_sen_20_10_wedding)/(maximum_final_sen_20_10_wedding-minimum_final_sen_20_10_wedding)
print(final_concrete_sentiment_df_wedding)
```

In[82]:

#Scaling up the relative sentiment score in the topic category bar by 10, before converting it into a dataframe.

#In addition the index number is copied as the new column topic ID.

```
normalized_sentiment_for_topics_bar=10*final_concrete_sentiment_df_bar["Sentiment Stock"]
print(normalized_sentiment_for_topics_bar)
Sentiments_bar=pd.DataFrame(normalized_sentiment_for_topics_bar)
print(Sentiments_bar)
Sentiments_bar.reset_index(level=0, inplace=True)
print(Sentiments_bar)
```

In[83]:

#Scaling up the relative sentiment score in the topic category entertainment by 10, before converting it into a dataframe.

#In addition the index number is copied as the new column topic ID.

```
normalized_sentiment_for_topics_entertainment=10*final_concrete_sentiment_df_entertainment["Sentiment Stock"]
```

```

print(normalized_sentiment_for_topics_entertainment)
Sentiments_entertainment=pd.DataFrame(normalized_sentiment_for_topics_entertainmen
t)
print(Sentiments_entertainment)
Sentiments_entertainment.reset_index(level=0, inplace=True)
print(Sentiments_entertainment)

```

In[84]:

```

#Scaling up the relative sentiment score in the topic category wedding by 10, before
converting it into a dataframe.

```

```

#In addition the index number is copied as the new column topic ID.

```

```

normalized_sentiment_for_topics_wedding=10*final_concrete_sentiment_df_wedding["Se
ntiment Stock"]

```

```

print(normalized_sentiment_for_topics_wedding)

```

```

Sentiments_wedding=pd.DataFrame(normalized_sentiment_for_topics_wedding)

```

```

print(Sentiments_wedding)

```

```

Sentiments_wedding.reset_index(level=0, inplace=True)

```

```

print(Sentiments_wedding)

```

In[85]:

```

#Exporting the sentiment to topic dataframe for bar

```

```

Sentiments_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)\Bar\Topic
Sentiment\Topic Sentiment.csv")

```

In[86]:

```

#Exporting the sentiment to topic dataframe for entertainment

```

```

Sentiments_entertainment.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Entertainment\Topic Sentiment\Topic Sentiment.csv")

```

In[87]:

```

#Exporting the sentiment to topic dataframe for wedding

```

```
Sentiments_wedding.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University  
of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)\Wedding\Topic  
Sentiment\Topic Sentiment.csv")
```

```
# In[ ]:
```

```
##End Sentiment Stock Section*
```

```
# In[ ]:
```

```
##Start Calculating the Opportunity scores*
```

```
# In[88]:
```

```
##Viewing the datatype of the topic importance dataframe of the overall topic bar  
reordered_topic_df_bar.dtypes
```

```
# In[89]:
```

```
##Changing the datatype of the topic importance dataframes of the column "Topics" to  
numeric
```

```
reordered_topic_df_bar["Topics"]=pd.to_numeric(reordered_topic_df_bar["Topics"])
```

```
reordered_topic_df_entertainment["Topics"]=pd.to_numeric(reordered_topic_df_entertainm  
ent["Topics"])
```

```
reordered_topic_df_wedding["Topics"]=pd.to_numeric(reordered_topic_df_wedding["Topic  
s"])
```

```
# In[90]:
```

```
##Viewing the datatype of the topic importance dataframe of the overall topic bar again  
reordered_topic_df_bar.dtypes
```

```
# In[91]:
```

```
##Viewing the datatype of the topic satisfaction dataframe of the overall topic bar  
Sentiments_bar.dtypes
```

```
# In[92]:
```

```
##Uniting the topic importance and the topic satisfaction dataframe on the "Topics" (i.e.  
Topic ID) for the overall topic bar
```

```
opportunity_df_bar=pd.merge(reordered_topic_df_bar, Sentiments_bar, on="Topics",
how="inner")
print(opportunity_df_bar)
```

```
# In[93]:
```

```
#Setting up a new column, which displays the difference between Topic Stock and
Sentiment Stock for each topic of the general topic bar
opportunity_df_bar["Topic Stock-Sentiment Stock"]=opportunity_df_bar["Topic Stock"]-
opportunity_df_bar["Sentiment Stock"]
print(opportunity_df_bar)
```

```
# In[94]:
```

```
#If the stock difference falls below zero the difference for the specific topic is set to zero in
the bar opportunity dataframe
opportunity_df_bar.loc[opportunity_df_bar["Topic Stock-Sentiment Stock"] < 0, "Topic
Stock-Sentiment Stock"] = 0
print(opportunity_df_bar)
```

```
# In[95]:
```

```
#Generating the Opportunity Score in the bar opportunity dataframe by adding to the Topic
Stock the difference between Topic Stock and Sentiment Stock
opportunity_df_bar["Opportunity Score"]=opportunity_df_bar["Topic Stock"]
+opportunity_df_bar["Topic Stock-Sentiment Stock"]
print(opportunity_df_bar)
```

```
# In[96]:
```

```
#Uniting the topic importance and the topic satisfaction dataframe on the "Topics" (i.e.
Topic ID) for the overall topic entertainment
opportunity_df_entertainment=pd.merge(reordered_topic_df_entertainment,
Sentiments_entertainment, on="Topics", how="inner")
print(opportunity_df_entertainment)
```

```
# In[97]:
```

```
#Setting up a new column, which displays the difference between Topic Stock and
Sentiment Stock for each topic of the general topic entertainment
```

```

opportunity_df_entertainment["Topic    Stock-Sentiment
Stock"]=opportunity_df_entertainment["Topic    Stock"]-
opportunity_df_entertainment["Sentiment Stock"]
print(opportunity_df_entertainment)

```

In[98]:

#If the stock difference falls below zero the difference for the specific topic is set to zero in the entertainment opportunity dataframe

```

opportunity_df_entertainment.loc[opportunity_df_entertainment["Topic    Stock-Sentiment
Stock"] < 0, "Topic Stock-Sentiment Stock"] = 0
print(opportunity_df_entertainment)

```

In[99]:

#Generating the Opportunity Score in the entertainment opportunity dataframe by adding to the Topic Stock the difference between Topic Stock and Sentiment Stock

```

opportunity_df_entertainment["Opportunity    Score"]=opportunity_df_entertainment["Topic
Stock"]+opportunity_df_entertainment["Topic Stock-Sentiment Stock"]
print(opportunity_df_entertainment)

```

In[100]:

#Uniting the topic importance and the topic satisfaction dataframe on the "Topics" (i.e. Topic ID) for the overall topic wedding

```

opportunity_df_wedding=pd.merge(reordered_topic_df_wedding,    Sentiments_wedding,
on="Topics", how="inner")
print(opportunity_df_wedding)

```

In[101]:

#Setting up a new column, which displays the difference between Topic Stock and Sentiment Stock for each topic of the general topic wedding

```

opportunity_df_wedding["Topic    Stock-Sentiment    Stock"]=opportunity_df_wedding["Topic
Stock"]-opportunity_df_wedding["Sentiment Stock"]
print(opportunity_df_wedding)

```

In[102]:

```
#If the stock difference falls below zero the difference for the specific topic is set to zero in
the wedding opportunity dataframe
opportunity_df_wedding.loc[opportunity_df_wedding["Topic Stock-Sentiment Stock"] < 0,
"Topic Stock-Sentiment Stock"] = 0
print(opportunity_df_wedding)
```

```
# In[103]:
```

```
#Generating the Opportunity Score in the wedding opportunity dataframe by adding to the
Topic Stock the difference between Topic Stock and Sentiment Stock
opportunity_df_wedding["Opportunity Score"]=opportunity_df_wedding["Topic Stock"]
+opportunity_df_wedding["Topic Stock-Sentiment Stock"]
print(opportunity_df_wedding)
```

```
# In[104]:
```

```
#Viewing the entirety of a column
pd.set_option("max_colwidth", None)
```

```
# In[105]:
```

```
#Reading in the respective topic IDs and their 10 most prominent words for the overall
topic bar
Topics_20_10_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 -
Train=10%)\Bar\Infer the topics with significant words.csv")
Topics_20_10_bar=Topics_20_10_bar.drop(columns="Unnamed: 0")
Topics_20_10_bar.head(22)
```

```
# In[106]:
```

```
#Assigning each topic a name and saving it in a new column of the bar opportunity
dataframe
opportunity_df_bar["Topic Name"]=["Strip club", "Eating shrimps under the moonlight",
"Lounge nudity", "Dancing in the kitchen with friends", "Price performance ratio of eating",
"Magic in the library", "Drinking shots with friends", "To waitress", "Fashion vendors",
"Grilling by a restaurant", "Singing at the dinner table", "Golf & brewery tour", "Roasting
pineapples live", "Retail seller's verbal communication", "Afternoon dessert", "Bar
```

```
ambience", "Singing with the family", "Bistro garden", "Chocolate", "Design of a chandelier", "Legal phone examination", "Traditional eating style"]
print(opportunity_df_bar)
```

```
# In[107]:
```

```
#Reading in the respective topic IDs and their 10 most prominent words for the overall topic entertainment
```

```
Topics_20_10_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Entertainment\Infer the topics with significant words.csv")
```

```
Topics_20_10_entertainment=Topics_20_10_entertainment.drop(columns="Unnamed: 0")
```

```
Topics_20_10_entertainment.head(24)
```

```
# In[108]:
```

```
#Assigning each topic a name and saving it in a new column of the entertainment opportunity dataframe
```

```
opportunity_df_entertainment["Topic Name"]=["Misleading show title", "Live cast of actors", "Place of the theater", "Youtube series actor in the library", "Sing & fashion style competition", "Health of the viewers", "Visiting a movie apartment", "Vibrance in the vision of colours", "Magic with a car", "Plot scenery", "Sales event", "Art of a woman", "Emergency call", "Dancing show", "Drought", "Talking talent", "Smoking marijuana", "Wrestling", "Charity", "Event cancellation", "Sports team in the finals", "Excuse messages", "Awaiting a club scene", "Summer fun with friends"]
```

```
print(opportunity_df_entertainment)
```

```
# In[109]:
```

```
#Reading in the respective topic IDs and their 10 most prominent words for the overall topic wedding
```

```
Topics_20_10_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (20 000 - Train=10%)\Wedding\Infer the topics with significant words.csv")
```

```
Topics_20_10_wedding=Topics_20_10_wedding.drop(columns="Unnamed: 0")
```

```
Topics_20_10_wedding.head(23)
```



```
# In[110]:
```

```
#Assigning each topic a name and saving it in a new column of the wedding opportunity dataframe
```

```
opportunity_df_wedding["Topic Name"]=["Chapel", "Fashion style of wedding clothes",  
"Decoration", "Fraud", "Desert atmosphere", "News about an eloped aunt", "Traditional gun  
of the grandparents", "Wedding entertainment", "Joking about marriage", "Personality of  
the wedding partner", "Wedding photography", "Dressing up like a bride for prom", "Elvis  
performance in a chapel", "Exclusivity of the ceremony", "Health of the customers", "Shop  
for wedding utensils", "Renewing the marriage", "Engagement memories", "Organising a  
wedding", "Reflections on wedding dream(s)", "Unprofessional wedding photographer",  
"Dealing with attention during a wedding", "Valentine's day photography"]  
print(opportunity_df_wedding)
```

```
# In[111]:
```

```
#Resetting the column width to the default value
```

```
pd.reset_option("max_colwidth")
```

```
# In[112]:
```

```
#Reordering the columns of the bar opportunity dataframe
```

```
reordered_opportunity_df_bar = opportunity_df_bar[["Topics", "Topic Name", "Topic  
Category","Topic Stock", "Sentiment Stock", "Topic Stock-Sentiment Stock", "Opportunity  
Score"]]  
print(reordered_opportunity_df_bar)
```

```
# In[113]:
```

```
#Reordering the columns of the entertainment opportunity dataframe
```

```
reordered_opportunity_df_entertainment = opportunity_df_entertainment[["Topics", "Topic  
Name","Topic Category", "Topic Stock", "Sentiment Stock", "Topic Stock-Sentiment Stock",  
"Opportunity Score"]]  
print(reordered_opportunity_df_entertainment)
```

```
# In[114]:
```

```
#Reordering the columns of the wedding opportunity dataframe
```

```
reordered_opportunity_df_wedding = opportunity_df_wedding[["Topics", "Topic Name",
"Topic Category","Topic Stock", "Sentiment Stock", "Topic Stock-Sentiment Stock",
"Opportunity Score"]]
print(reordered_opportunity_df_wedding)
```

In[115]:

```
#Exporting the maximal expanded version of the bar opportunity dataframe as csv
reordered_opportunity_df_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Bar\Opportunity Scores\Maximum Expansion\Opportunities expanded.csv")
```

In[116]:

```
#Exporting the maximal expanded version of entertainment opportunity dataframe as csv
reordered_opportunity_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews
(Train = 10%)\Entertainment\Opportunity Scores\Maximum Expansion\Opportunities
expanded.csv")
```

In[117]:

```
#Exporting the maximal expanded version of wedding opportunity dataframe as csv
reordered_opportunity_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive
- University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Wedding\Opportunity Scores\Maximum Expansion\Opportunities expanded.csv")
```

In[118]:

```
#Deleting the Stock difference columns for all 3 distinct dataframes
reordered_opportunity_df_bar=reordered_opportunity_df_bar.drop(columns=["Topic Stock-
Sentiment Stock"])
reordered_opportunity_df_entertainment=reordered_opportunity_df_entertainment.drop(co
lumns=["Topic Stock-Sentiment Stock"])
reordered_opportunity_df_wedding=reordered_opportunity_df_wedding.drop(columns=["T
opic Stock-Sentiment Stock"])
```

In[119]:

```
#Exporting the user-friendly version of the bar opportunity dataframe as csv
```

```
reordered_opportunity_df_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%) \Bar\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
# In[120]:
```

```
#Exporting the user-friendly version of the entertainment opportunity dataframe as csv
reordered_opportunity_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)\Entertainment\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
# In[121]:
```

```
#Exporting the user-friendly version of the wedding opportunity dataframe as csv
reordered_opportunity_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%) \Wedding\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
# In[ ]:
```

```
##### End 20 000 reviews Train = 10 %#####
```

```
# In[ ]:
```

```
##*Analog actions have been performed on the doc to topic matrices and on the word to topic matrices which resulted
```

```
#from LDA topic modelling the merged dataset of Twitter data and the 200 000 reviews Train = 99% tested dictionary
```

```
#Please revisit Section (20 000 reviews Train = 10%) for a commented version of these steps*
```

```
# In[ ]:
```

```
##### Start 200 000 reviews Train = 99 %#####
```

```
# In[ ]:
```

```
##*Start Topic Stock Selection*
```

```
# In[ ]:
```

```
#Bar Start
```

```
# In[122]:
```

```
doc_top_200_99_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -  
University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 -  
Train=99%)\Bar\Topic to Document Matrix\doc to topics.csv")  
print(doc_top_200_99_bar)
```

```
# In[123]:
```

```
doc_top_200_99_bar= doc_top_200_99_bar.drop(columns=["Unnamed: 0"])  
print(doc_top_200_99_bar)
```

```
# In[124]:
```

```
doc_top_200_99_bar=doc_top_200_99_bar.T  
print(doc_top_200_99_bar)
```

```
# In[125]:
```

```
doc_top_200_99_bar["Total"] = doc_top_200_99_bar.sum(axis=1)  
doc_top_200_99_bar.head()
```

```
# In[126]:
```

```
minimum_doc_200_99_bar=doc_top_200_99_bar["Total"].min()  
print(minimum_doc_200_99_bar)
```

```
# In[127]:
```

```
maximum_doc_200_99_bar=doc_top_200_99_bar["Total"].max()  
print(maximum_doc_200_99_bar)
```

```
# In[ ]:
```

```
#Bar End
```

```
# In[ ]:
```

```
#Entertainment Start
```

```
# In[128]:
```

```
doc_top_200_99_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and
dict\Matched words (200 000 - Train=99%)\Entertainment\Topic to Document Matrix\doc to
topics.csv")
```

```
print(doc_top_200_99_entertainment)
```

```
# In[129]:
```

```
d o c _ t o p _ 2 0 0 _ 9 9 _ e n t e r t a i n m e n t =
doc_top_200_99_entertainment.drop(columns=["Unnamed: 0"])
print(doc_top_200_99_entertainment)
```

```
# In[130]:
```

```
doc_top_200_99_entertainment=doc_top_200_99_entertainment.T
print(doc_top_200_99_entertainment)
```

```
# In[131]:
```

```
doc_top_200_99_entertainment["Total"] = doc_top_200_99_entertainment.sum(axis=1)
doc_top_200_99_entertainment.head()
```

```
# In[132]:
```

```
minimum_doc_200_99_entertainment=doc_top_200_99_entertainment["Total"].min()
print(minimum_doc_200_99_entertainment)
```

```
# In[133]:
```

```
maximum_doc_200_99_entertainment=doc_top_200_99_entertainment["Total"].max()
print(maximum_doc_200_99_entertainment)
```

```
# In[ ]:
```

```
#Entertainment End
```

```
# In[ ]:
```

```
#Wedding Start
```

```
# In[134]:
```

```

doc_top_200_99_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 -
Train=99%)\Wedding\Topic to Document Matrix\doc to topics.csv")
print(doc_top_200_99_wedding)

# In[135]:
doc_top_200_99_wedding=doc_top_200_99_wedding.drop(columns=["Unnamed: 0"])
print(doc_top_200_99_wedding)

# In[136]:
doc_top_200_99_wedding=doc_top_200_99_wedding.T
print(doc_top_200_99_wedding)

# In[137]:
doc_top_200_99_wedding["Total"] = doc_top_200_99_wedding.sum(axis=1)
doc_top_200_99_wedding.head()

# In[138]:
minimum_doc_200_99_wedding=doc_top_200_99_wedding["Total"].min()
print(minimum_doc_200_99_wedding)

# In[139]:
maximum_doc_200_99_wedding=doc_top_200_99_wedding["Total"].max()
print(maximum_doc_200_99_wedding)

# In[ ]:
#Wedding End

# In[140]:
pd.set_option("display.max_rows", None)

# In[141]:
doc_top_200_99_bar["Topic Stock"]=(doc_top_200_99_bar["Total"]-
minimum_doc_200_99_bar)/(maximum_doc_200_99_bar-minimum_doc_200_99_bar)
print(doc_top_200_99_bar)

```

In[142]:

```
doc_top_200_99_entertainment["Topic Stock"]=(doc_top_200_99_entertainment["Total"]-
minimum_doc_200_99_entertainment)/(maximum_doc_200_99_entertainment-
minimum_doc_200_99_entertainment)
print(doc_top_200_99_entertainment)
```

In[143]:

```
doc_top_200_99_wedding["Topic Stock"]=(doc_top_200_99_wedding["Total"]-
minimum_doc_200_99_wedding)/(maximum_doc_200_99_wedding-
minimum_doc_200_99_wedding)
print(doc_top_200_99_wedding)
```

In[144]:

```
Topic_stock_vector_200_99_bar=10*doc_top_200_99_bar["Topic Stock"]
Topic_stock_vector_200_99_bar=pd.DataFrame(Topic_stock_vector_200_99_bar,columns
=["Topic Stock"])
Topic_stock_vector_200_99_bar["Topics"] = Topic_stock_vector_200_99_bar.index
Topic_stock_vector_200_99_bar["Topic Category"]="Bar"

print(Topic_stock_vector_200_99_bar)
```

In[145]:

```
Topic_stock_vector_200_99_entertainment=10*doc_top_200_99_entertainment["Topic
Stock"]
Topic_stock_vector_200_99_entertainment=pd.DataFrame(Topic_stock_vector_200_99_e
ntertainment,columns=["Topic Stock"])
Topic_stock_vector_200_99_entertainment["Topics"] =
Topic_stock_vector_200_99_entertainment.index
Topic_stock_vector_200_99_entertainment["Topic Category"]="Entertainment"

print(Topic_stock_vector_200_99_entertainment)
```

In[146]:

```
Topic_stock_vector_200_99_wedding=10*doc_top_200_99_wedding["Topic Stock"]
```

```

Topic_stock_vector_200_99_wedding=pd.DataFrame(Topic_stock_vector_200_99_wedding,columns=["Topic Stock"])
Topic_stock_vector_200_99_wedding["Topics"] =
Topic_stock_vector_200_99_wedding.index
Topic_stock_vector_200_99_wedding["Topic Category"]="Wedding"

```

```

print(Topic_stock_vector_200_99_wedding)

```

```

# In[147]:

```

```

reordered_topic_df_bar_200_99=Topic_stock_vector_200_99_bar[["Topics", "Topic Category", "Topic Stock"]]
print(reordered_topic_df_bar_200_99)

```

```

# In[148]:

```

```

reordered_topic_df_entertainment_200_99=Topic_stock_vector_200_99_entertainment[["Topics", "Topic Category", "Topic Stock"]]
print(reordered_topic_df_entertainment_200_99)

```

```

# In[149]:

```

```

reordered_topic_df_wedding_200_99=Topic_stock_vector_200_99_wedding[["Topics", "Topic Category", "Topic Stock"]]
print(reordered_topic_df_wedding_200_99)

```

```

# In[150]:

```

```

reordered_topic_df_bar_200_99.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%) \Bar\Topic Importance\Topic Importance.csv")

```

```

# In[151]:

```

```

reordered_topic_df_entertainment_200_99.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%)\Entertainment\Topic Importance\Topic Importance.csv")

```

```

# In[152]:

```



```
reordered_topic_df_wedding_200_99.to_csv(r"C:\Users\Max.DESKTOP-  
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews  
(Train = 99%)\Wedding\Topic Importance\Topic Importance.csv")
```

```
# In[ ]:
```

```
##*END Topic Stock Section
```

```
# In[153]:
```

```
pd.reset_option("display.max_rows")
```

```
# In[ ]:
```

```
##*Start Sentiment Stock Section*
```

```
# In[154]:
```

```
word_top_200_99_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -  
University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 -  
Train=99%)\Bar\Word to Topic Matrix\word to topics.csv")  
print(word_top_200_99_bar)
```

```
# In[155]:
```

```
word_top_200_99_bar= word_top_200_99_bar.drop(columns=["Unnamed: 0"])  
print(word_top_200_99_bar)
```

```
# In[156]:
```

```
word_top_200_99_bar=word_top_200_99_bar.T  
print(word_top_200_99_bar)
```

```
# In[157]:
```

```
word_top_200_99_bar.reset_index(level=0, inplace=True)  
word_top_200_99_bar.rename(columns={"index":"stemmed_words"},inplace=True)  
print(word_top_200_99_bar)
```

```
# In[158]:
```

```
sent_dict_200_99=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\4 Creating & evaluating the sentiment dictionary\200 000 reviews Trainset =99%\Ultimate dict.csv")
print(sent_dict_200_99)
```

```
# In[159]:
```

```
new_concrete_sentiment_df_bar=pd.merge(word_top_200_99_bar, sent_dict_200_99,
on="stemmed_words", how="inner")
print(new_concrete_sentiment_df_bar)
```

```
# In[160]:
```

```
Sentiment_200_99_series_bar=new_concrete_sentiment_df_bar["Sentiment points"]
print(Sentiment_200_99_series_bar)
```

```
# In[161]:
```

```
words_bar=new_concrete_sentiment_df_bar["stemmed_words"]
new_concrete_sentiment_df_bar=new_concrete_sentiment_df_bar.drop(columns=["Sentiment points"])
new_concrete_sentiment_df_bar=new_concrete_sentiment_df_bar.drop(columns=["stemmed_words"])
print(new_concrete_sentiment_df_bar)
```

```
# In[162]:
```

```
new_concrete_sentiment_df_bar=new_concrete_sentiment_df_bar.mul(Sentiment_200_99_series_bar,axis=0)
print(new_concrete_sentiment_df_bar)
print(words_bar)
```

```
# In[163]:
```

```
new_final_concrete_sentiment_df_bar=pd.concat([words_bar,new_concrete_sentiment_df_bar],axis=1)
new_final_concrete_sentiment_df_bar.set_index("stemmed_words",inplace=True)
print(new_final_concrete_sentiment_df_bar)
```

```
# In[164]:
```

```

new_final_concrete_sentiment_df_bar=new_final_concrete_sentiment_df_bar.T
print(new_final_concrete_sentiment_df_bar)

# In[165]:
new_final_concrete_sentiment_df_bar=new_final_concrete_sentiment_df_bar.rename_axis(
    None, axis=1).rename_axis("Topics", axis=0)
new_final_concrete_sentiment_df_bar.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Bar\Topic words table with sentiment\Topic words table with sentiment.csv")

# In[166]:
new_final_concrete_sentiment_df_bar["Total"]=new_final_concrete_sentiment_df_bar.sum
    (axis=1)
new_final_concrete_sentiment_df_bar.head()

# In[167]:
minimum_final_sen_200_99_bar=new_final_concrete_sentiment_df_bar["Total"].min()
print(minimum_final_sen_200_99_bar)

# In[168]:
maximum_final_sen_200_99_bar=new_final_concrete_sentiment_df_bar["Total"].max()
print(maximum_final_sen_200_99_bar)

# In[ ]:
#New Word to Topic Start

# In[169]:
word_top_200_99_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and
dict\Matched words (200 000 - Train=99%)\Entertainment\Word to Topic Matrix\word to
topics.csv")
print(word_top_200_99_entertainment)

# In[170]:

```

```
w o r d _ t o p _ 2 0 0 _ 9 9 _ e n t e r t a i n m e n t =  
word_top_200_99_entertainment.drop(columns=["Unnamed: 0"])  
print(word_top_200_99_entertainment)
```

```
# In[171]:  
word_top_200_99_entertainment=word_top_200_99_entertainment.T  
print(word_top_200_99_entertainment)
```

```
# In[172]:  
word_top_200_99_entertainment.reset_index(level=0, inplace=True)  
word_top_200_99_entertainment.rename(columns={"index": "stemmed_words"}, inplace=True)  
print(word_top_200_99_entertainment)
```

```
# In[173]:  
new_concrete_sentiment_df_entertainment=pd.merge(word_top_200_99_entertainment,  
sent_dict_200_99, on="stemmed_words", how="inner")  
print(new_concrete_sentiment_df_entertainment)
```

```
# In[174]:  
Sentiment_200_99_series_entertainment=new_concrete_sentiment_df_entertainment["Sentiment points"]  
print(Sentiment_200_99_series_entertainment)
```

```
# In[175]:  
words_entertainment=new_concrete_sentiment_df_entertainment["stemmed_words"]  
new_concrete_sentiment_df_entertainment=new_concrete_sentiment_df_entertainment.drop(columns=["Sentiment points"])  
new_concrete_sentiment_df_entertainment=new_concrete_sentiment_df_entertainment.drop(columns=["stemmed_words"])  
print(new_concrete_sentiment_df_entertainment)
```

```
# In[176]:  
new_concrete_sentiment_df_entertainment=new_concrete_sentiment_df_entertainment.mul(Sentiment_200_99_series_entertainment,axis=0)
```

```
print(new_concrete_sentiment_df_entertainment)
print(words_entertainment)
```

```
# In[177]:
```

```
new_final_concrete_sentiment_df_entertainment=pd.concat([words_entertainment,new_c
oncrete_sentiment_df_entertainment],axis=1)
new_final_concrete_sentiment_df_entertainment.set_index("stemmed_words",inplace=True)
print(new_final_concrete_sentiment_df_entertainment)
```

```
# In[178]:
```

```
new_final_concrete_sentiment_df_entertainment=new_final_concrete_sentiment_df_entertainment.T
print(new_final_concrete_sentiment_df_entertainment)
```

```
# In[179]:
```

```
new_final_concrete_sentiment_df_entertainment=new_final_concrete_sentiment_df_entertainment.rename_axis(None, axis=1).rename_axis("Topics", axis=0)
new_final_concrete_sentiment_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%)\Entertainment\Topic words table with sentiment\Topic words table with sentiment.csv")
```

```
# In[180]:
```

```
new_final_concrete_sentiment_df_entertainment["Total"]=new_final_concrete_sentiment_df_entertainment.sum(axis=1)
new_final_concrete_sentiment_df_entertainment.head()
```

```
# In[181]:
```

```
minimum_final_sen_200_99_entertainment=new_final_concrete_sentiment_df_entertainment["Total"].min()
print(minimum_final_sen_200_99_entertainment)
```

```
# In[182]:
```

```
maximum_final_sen_200_99_entertainment=new_final_concrete_sentiment_df_entertainment["Total"].max()
print(maximum_final_sen_200_99_entertainment)
```

```
# In[ ]:
```

```
#New Word to Topic End
```

```
# In[ ]:
```

```
#Last Word to Topic Start
```

```
# In[183]:
```

```
word_top_200_99_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)\Wedding\Word to Topic Matrix\word to topics.csv")
print(word_top_200_99_wedding)
```

```
# In[184]:
```

```
word_top_200_99_wedding= word_top_200_99_wedding.drop(columns=["Unnamed: 0"])
print(word_top_200_99_wedding)
```

```
# In[185]:
```

```
word_top_200_99_wedding=word_top_200_99_wedding.T
print(word_top_200_99_wedding)
```

```
# In[186]:
```

```
word_top_200_99_wedding.reset_index(level=0, inplace=True)
word_top_200_99_wedding.rename(columns={"index":"stemmed_words"},inplace=True)
print(word_top_200_99_wedding)
```

```
# In[187]:
```

```
new_concrete_sentiment_df_wedding=pd.merge(word_top_200_99_wedding,
sent_dict_200_99, on="stemmed_words", how="inner")
print(new_concrete_sentiment_df_wedding)
```

```
# In[188]:
```

```

Sentiment_200_99_series_wedding=new_concrete_sentiment_df_wedding["Sentiment
points"]
print(Sentiment_200_99_series_wedding)

# In[189]:
words_wedding=new_concrete_sentiment_df_wedding["stemmed_words"]
new_concrete_sentiment_df_wedding=new_concrete_sentiment_df_wedding.drop(column
s=["Sentiment points"])
new_concrete_sentiment_df_wedding=new_concrete_sentiment_df_wedding.drop(column
s=["stemmed_words"])
print(new_concrete_sentiment_df_wedding)

# In[190]:
new_concrete_sentiment_df_wedding=new_concrete_sentiment_df_wedding.mul(Sentime
nt_200_99_series_wedding,axis=0)
print(new_concrete_sentiment_df_wedding)
print(words_wedding)

# In[191]:
new_final_concrete_sentiment_df_wedding=pd.concat([words_wedding,new_concrete_se
ntiment_df_wedding],axis=1)
new_final_concrete_sentiment_df_wedding.set_index("stemmed_words",inplace=True)
print(new_final_concrete_sentiment_df_wedding)

# In[192]:
new_final_concrete_sentiment_df_wedding=new_final_concrete_sentiment_df_wedding.T
print(new_final_concrete_sentiment_df_wedding)

# In[193]:
new_final_concrete_sentiment_df_wedding=new_final_concrete_sentiment_df_wedding.re
name_axis(None, axis=1).rename_axis("Topics", axis=0)
new_final_concrete_sentiment_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Wedding\Topic words table with sentiment\Topic words table with
sentiment.csv")

```

```
# In[194]:
new_final_concrete_sentiment_df_wedding["Total"]=new_final_concrete_sentiment_df_wedding.sum(axis=1)
new_final_concrete_sentiment_df_wedding.head()
```

```
# In[195]:
minimum_final_sen_200_99_wedding=new_final_concrete_sentiment_df_wedding["Total"].min()
print(minimum_final_sen_200_99_wedding)
```

```
# In[196]:
maximum_final_sen_200_99_wedding=new_final_concrete_sentiment_df_wedding["Total"].max()
print(maximum_final_sen_200_99_wedding)
```

```
# In[ ]:
#Last Word to Topic End
```

```
# In[ ]:
#Continue Calc Sentiment Stock
```

```
# In[197]:
pd.set_option("display.max_rows",None)
```

```
# In[198]:
new_final_concrete_sentiment_df_bar["Sentiment Stock"]=(new_final_concrete_sentiment_df_bar["Total"]-minimum_final_sen_200_99_bar)/(maximum_final_sen_200_99_bar-minimum_final_sen_200_99_bar)
print(new_final_concrete_sentiment_df_bar)
```

```
# In[199]:
new_final_concrete_sentiment_df_entertainment["Sentiment Stock"]=(new_final_concrete_sentiment_df_entertainment["Total"]-
```



```
minimum_final_sen_200_99_entertainment)/(maximum_final_sen_200_99_entertainment-
minimum_final_sen_200_99_entertainment)
print(new_final_concrete_sentiment_df_entertainment)
```

```
# In[200]:
```

```
new_final_concrete_sentiment_df_wedding["Sentiment
Stock"]=(new_final_concrete_sentiment_df_wedding["Total"]-
minimum_final_sen_200_99_wedding)/(maximum_final_sen_200_99_wedding-
minimum_final_sen_200_99_wedding)
print(new_final_concrete_sentiment_df_wedding)
```

```
# In[201]:
```

```
new_normalized_sentiment_for_topics_bar=10*new_final_concrete_sentiment_df_bar["Se
ntiment Stock"]
print(new_normalized_sentiment_for_topics_bar)
new_Sentiments_bar=pd.DataFrame(new_normalized_sentiment_for_topics_bar)
print(new_Sentiments_bar)
new_Sentiments_bar.reset_index(level=0, inplace=True)
print(new_Sentiments_bar)
```

```
# In[202]:
```

```
new_normalized_sentiment_for_topics_entertainment=10*new_final_concrete_sentiment_
df_entertainment["Sentiment Stock"]
print(new_normalized_sentiment_for_topics_entertainment)
new_Sentiments_entertainment=pd.DataFrame(new_normalized_sentiment_for_topics_en
tertainment)
print(new_Sentiments_entertainment)
new_Sentiments_entertainment.reset_index(level=0, inplace=True)
print(new_Sentiments_entertainment)
```

```
# In[203]:
```

```
new_normalized_sentiment_for_topics_wedding=10*new_final_concrete_sentiment_df_w
edding["Sentiment Stock"]
print(new_normalized_sentiment_for_topics_wedding)
```

```

new_Sentiments_wedding=pd.DataFrame(new_normalized_sentiment_for_topics_wedding)
print(new_Sentiments_wedding)
new_Sentiments_wedding.reset_index(level=0, inplace=True)
print(new_Sentiments_wedding)

```

In[204]:

```

new_Sentiments_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%) \Bar\Topic Sentiment\Topic Sentiment.csv")

```

In[205]:

```

new_Sentiments_entertainment.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%) \Entertainment\Topic Sentiment\Topic Sentiment.csv")

```

In[206]:

```

new_Sentiments_wedding.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%) \Wedding\Topic Sentiment\Topic Sentiment.csv")

```

In[]:

```

##*End Sentiment Stock Section

```

In[]:

```

##*Start Calculating the opportunity scores

```

In[207]:

```

reordered_topic_df_bar_200_99["Topics"]=pd.to_numeric(reordered_topic_df_bar_200_99["Topics"])
reordered_topic_df_entertainment_200_99["Topics"]=pd.to_numeric(reordered_topic_df_entertainment_200_99["Topics"])
reordered_topic_df_wedding_200_99["Topics"]=pd.to_numeric(reordered_topic_df_wedding_200_99["Topics"])

```

```

# In[208]:
reordered_topic_df_bar_200_99.dtypes

# In[209]:
new_Sentiments_bar.dtypes

# In[210]:
new_opportunity_df_bar=pd.merge(reordered_topic_df_bar_200_99,new_Sentiments_bar,
on="Topics", how="inner")
print(new_opportunity_df_bar)

# In[211]:
new_opportunity_df_bar["Topic Stock-Sentiment Stock"]=new_opportunity_df_bar["Topic
Stock"]-new_opportunity_df_bar["Sentiment Stock"]
print(new_opportunity_df_bar)

# In[212]:
new_opportunity_df_bar.loc[new_opportunity_df_bar["Topic Stock-Sentiment Stock"] < 0,
"Topic Stock-Sentiment Stock"]=0
print(new_opportunity_df_bar)

# In[213]:
new_opportunity_df_bar["Opportunity Score"]=new_opportunity_df_bar["Topic Stock"]
+new_opportunity_df_bar["Topic Stock-Sentiment Stock"]
print(new_opportunity_df_bar)

# In[214]:
new_opportunity_df_entertainment=pd.merge(reordered_topic_df_entertainment_200_99,
new_Sentiments_entertainment, on="Topics", how="inner")
print(new_opportunity_df_entertainment)

# In[215]:
new_opportunity_df_entertainment["Topic Stock-Sentiment
Stock"]=new_opportunity_df_entertainment["Topic Stock"]-
new_opportunity_df_entertainment["Sentiment Stock"]

```

```
print(new_opportunity_df_entertainment)
```

```
# In[216]:
```

```
new_opportunity_df_entertainment.loc[new_opportunity_df_entertainment["Topic Stock-Sentiment Stock"] < 0, "Topic Stock-Sentiment Stock"]=0  
print(new_opportunity_df_entertainment)
```

```
# In[217]:
```

```
new_opportunity_df_entertainment["Opportunity Score"]=new_opportunity_df_entertainment["Topic Stock"]  
+new_opportunity_df_entertainment["Topic Stock-Sentiment Stock"]  
print(new_opportunity_df_entertainment)
```

```
# In[218]:
```

```
new_opportunity_df_wedding=pd.merge(reordered_topic_df_wedding_200_99,new_Sentiments_wedding, on="Topics", how="inner")  
print(new_opportunity_df_wedding)
```

```
# In[219]:
```

```
new_opportunity_df_wedding["Topic Stock-Sentiment Stock"]=new_opportunity_df_wedding["Topic Stock"]-  
new_opportunity_df_wedding["Sentiment Stock"]  
print(new_opportunity_df_wedding)
```

```
# In[220]:
```

```
new_opportunity_df_wedding.loc[new_opportunity_df_wedding["Topic Stock-Sentiment Stock"] < 0, "Topic Stock-Sentiment Stock"]=0  
print(new_opportunity_df_wedding)
```

```
# In[221]:
```

```
new_opportunity_df_wedding["Opportunity Score"]=new_opportunity_df_wedding["Topic Stock"]+new_opportunity_df_wedding["Topic Stock-Sentiment Stock"]  
print(new_opportunity_df_wedding)
```

```
# In[222]:
```

```
pd.set_option("max_colwidth", None)
```

```
# In[223]:
```

```
Topics_200_99_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)\Bar\Infer the topics with significant words.csv")  
Topics_200_99_bar=Topics_200_99_bar.drop(columns="Unnamed: 0")  
Topics_200_99_bar.head(17)
```

```
# In[224]:
```

```
new_opportunity_df_bar["Topic Name"]=["Art of cooking", "Evening preferences for groceries", "Dancing", "Nightlife", "Oyster bar", "Legal disputes", "Lounge nudity", "Drink & drive", "Singing & snacks", "Fire screen", "Taste of wine", "Traditional wine", "Karaoke", "Bar atmosphere", "Affordability of wine", "Brew shop", "Interior design of a pub"]  
print(new_opportunity_df_bar)
```

```
# In[225]:
```

```
Topics_200_99_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)\Entertainment\Infer the topics with significant words.csv")  
Topics_200_99_entertainment=Topics_200_99_entertainment.drop(columns="Unnamed: 0")  
Topics_200_99_entertainment.head(24)
```

```
# In[226]:
```

```
new_opportunity_df_entertainment["Topic Name"]=["Singing performance", "Acting performance", "Corporate celebration", "Costume design", "Studio emotions", "Advertisement for buying theater tickets", "Cinematic effects", "Decor design", "Producing a score", "Plot comedy", "Entertainment craft", "Thoughts about (a) plot(s)", "The fox as figure in history and culture", "Asking in an unordinary fashion for a nightclub dance", "Live performance", "Security staff dealing with inappropriate behaviour", "Comedy performance supported by laser lights", "Comedy rock album", "Seminar for singing", "An interview about the dream of dancing", "Amusement tour", "Advertising tickets with friends for a charity event", "A detective play", "Feedback on a tour payment process"]
```

```
print(new_opportunity_df_entertainment)
```

```
# In[227]:
```

```
Topics_200_99_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\5 Matching Twitter and dict\Matched words (200 000 - Train=99%)\Wedding\Infer the topics with significant words.csv")
Topics_200_99_wedding=Topics_200_99_wedding.drop(columns="Unnamed: 0")
Topics_200_99_wedding.head(23)
```

```
# In[228]:
```

```
new_opportunity_df_wedding["Topic Name"]=["Renewing the marriage vow","Disputes about real estate" ,"Broken marriage", "Misbehaving actor at a wedding", "Beauty of a chapel", "Wedding expenditures", "Waiting for a marriage license", "Ceremony with Elvis photography", "Eloped woman", "Marrying at the museum", "Sexual violence within a marriage", "Features of a singer", "Wedding ring", "Appearance of women at the marriage", "Wedding expectations", "Omitting the goal of marrying", "Dancing in a chapel", "Traditional values in a marriage", "Happiness among the best friends due to the wedding", "Wedding celebration style", "Becoming engaged", "Mother's feelings about a marriage", "Beautiful hair when being photographed"]
print(new_opportunity_df_wedding)
```

```
# In[229]:
```

```
pd.reset_option("max_colwidth")
```

```
# In[230]:
```

```
new_reordered_opportunity_df_bar = new_opportunity_df_bar[["Topics", "Topic Name","Topic Category", "Topic Stock", "Sentiment Stock", "Topic Stock-Sentiment Stock", "Opportunity Score"]]
print(new_reordered_opportunity_df_bar)
```

```
# In[231]:
```

```
new_reordered_opportunity_df_entertainment = new_opportunity_df_entertainment[["Topics", "Topic Name","Topic Category", "Topic Stock", "Sentiment Stock", "Topic Stock-Sentiment Stock", "Opportunity Score"]]
print(new_reordered_opportunity_df_entertainment)
```

In[232]:

```
new_reordered_opportunity_df_wedding = new_opportunity_df_wedding[["Topics", "Topic  
Name", "Topic Category", "Topic Stock", "Sentiment Stock", "Topic Stock-Sentiment Stock",  
"Opportunity Score"]]  
print(new_reordered_opportunity_df_wedding)
```

In[233]:

```
new_reordered_opportunity_df_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive  
- University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%)  
\Bar\Opportunity Scores\Maximum Expansion\Opportunities expanded.csv")
```

In[234]:

```
new_reordered_opportunity_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-  
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews  
(Train = 99%)\Entertainment\Opportunity Scores\Maximum Expansion\Opportunities  
expanded.csv")
```

In[235]:

```
new_reordered_opportunity_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-  
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews  
(Train = 99%)\Wedding\Opportunity Scores\Maximum Expansion\Opportunities  
expanded.csv")
```

In[236]:

```
new_reordered_opportunity_df_bar=new_reordered_opportunity_df_bar.drop(columns=["T  
opic Stock-Sentiment Stock"])  
new_reordered_opportunity_df_entertainment=new_reordered_opportunity_df_entertainm  
ent.drop(columns=["Topic Stock-Sentiment Stock"])  
new_reordered_opportunity_df_wedding=new_reordered_opportunity_df_wedding.drop(co  
lumns=["Topic Stock-Sentiment Stock"])
```

In[237]:

```
new_reordered_opportunity_df_bar.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%) \Bar\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
# In[238]:
```

```
new_reordered_opportunity_df_entertainment.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%)\Entertainment\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
# In[239]:
```

```
new_reordered_opportunity_df_wedding.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%)\Wedding\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
# In[ ]:
```

```
##### End 200 000 reviews Train = 99 %#####
```

End - 9 Finding Opportunities

```
..... (Optimal document-topic and topic-word matrices of both opportunity systems) .....
```

Start - 10 Dash Visualisation

```
..... (Opportunity elements for both selected systems) .....
```

```
# In[1]:
```

```
import pandas as pd
```

```
# In[ ]:
```

```
#####For 20 000 Reviews and Train=10%
```

```
# In[2]:
```

```
#Importing the topic words table for the general topic bar and then secondly transposing it
```



```
topics_words_table_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%) \Bar\Topic words table with sentiment\Topic words table with sentiment.csv")
topics_words_table_bar=topics_words_table_bar.T
print(topics_words_table_bar)
```

```
# In[3]:
```

```
#Importing the topic words table for the general topic entertainment and then secondly transposing it
```

```
topics_words_table_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)\Entertainment\Topic words table with sentiment\Topic words table with sentiment.csv")
```

```
topics_words_table_entertainment=topics_words_table_entertainment.T
```

```
print(topics_words_table_entertainment)
```

```
# In[4]:
```

```
#Importing the topic words table for the general topic wedding and then secondly transposing it
```

```
topics_words_table_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)\Wedding\Topic words table with sentiment\Topic words table with sentiment.csv")
```

```
topics_words_table_wedding=topics_words_table_wedding.T
```

```
print(topics_words_table_wedding)
```

```
# In[5]:
```

```
#Importing the bar opportunity dataframe and then secondly removing column Unnamed: 0
```

```
opportunity_df_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%) \Bar\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
```

```
opportunity_df_bar=opportunity_df_bar.drop(columns="Unnamed: 0")
```

```
opportunity_df_bar.head()
```

```
# In[6]:
```

```
#Importing the entertainment opportunity dataframe and then secondly removing column
Unnamed: 0
opportunity_df_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\20 000 reviews
(Train = 10%)\Entertainment\Opportunity Scores\Minimum Expansion\Opportunities for
Users.csv")
opportunity_df_entertainment=opportunity_df_entertainment.drop(columns="Unnamed: 0")
opportunity_df_entertainment.head()
```

In[7]:

```
#Importing the wedding opportunity dataframe and then secondly removing column
Unnamed: 0
opportunity_df_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\6 Opportunities\20 000 reviews (Train = 10%)
\Wedding\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
opportunity_df_wedding=opportunity_df_wedding.drop(columns="Unnamed: 0")
opportunity_df_wedding.head()
```

In[8]:

```
#Creating an empty bar negative words list and an empty bar topic number list
negative_words_bar=[]
topic_number_bar=[]
```

In[9]:

```
#For each column of the topic words table for the general topic bar, select the 10 words
with the most negative connotation,
#append them to the bar negative words list as well as append the topic number (i.e.
column number) to the bar topic number list
for i in range(0,22,1):
    a=topics_words_table_bar.iloc[1:964,i]
    b=a.nsmallest(10).index.to_list()
    negative_words_bar.append(b)
    topic_number_bar.append(i)
```

In[10]:

```
#Creating a dictionary made out of the bar topic number list and the bar negative words list
and print it
```

```
Topics2negwords_dict_bar=dict(zip(topic_number_bar, negative_words_bar))
print(Topics2negwords_dict_bar)
```

```
#Converting the dictionary into a pandas dataframe
```

```
Topics2negwords_bar=pd.DataFrame(Topics2negwords_dict_bar)
```

```
#Transposing the pandas dataframe
```

```
Topics2negwords_bar=Topics2negwords_bar.T
```

```
#Place the index as a column under the name Topics (=Topic ID) and finally print the
dataframe
```

```
Topics2negwords_bar.reset_index(level=0, inplace=True)
```

```
Topics2negwords_bar.rename(columns={"index":"Topics"}, inplace=True)
```

```
print(Topics2negwords_bar)
```

```
# In[11]:
```

```
#Combining the bar opportunity dataframe and the bar topic to negative words dataframe
on Topics (=Topic ID)
```

```
dash_df_bar=pd.merge(opportunity_df_bar, Topics2negwords_bar, on="Topics",
how="inner")
```

```
print(dash_df_bar)
```

```
# In[12]:
```

```
#Creating an empty entertainment negative words list and an empty entertainment topic
number list
```

```
negative_words_entertainment=[]
```

```
topic_number_entertainment=[]
```

```
# In[13]:
```

```
#For each column of the topic words table for the general topic entertainment, select the
10 words with the most negative connotation,
```

```
#append them to the entertainment negative words list as well as append the topic number
(i.e. column number) to the entertainment topic number list
```

```

for i in range(0,24,1):
    a=topics_words_table_entertainment.iloc[1:1052,i]
    b=a.nsmallest(10).index.to_list()
    negative_words_entertainment.append(b)
    topic_number_entertainment.append(i)

# In[14]:
#Creating a dictionary made out of the entertainment topic number list and the
entertainment negative words list and print it
Topics2negwords_dict_entertainment=dict(zip(topic_number_entertainment,
negative_words_entertainment))
print(Topics2negwords_dict_entertainment)

#Converting the dictionary into a pandas dataframe
Topics2negwords_entertainment=pd.DataFrame(Topics2negwords_dict_entertainment)

#Transposing the pandas dataframe
Topics2negwords_entertainment=Topics2negwords_entertainment.T

#Place the index as a column under the name Topics (=Topic ID) and finally print the
dataframe
Topics2negwords_entertainment.reset_index(level=0, inplace=True)
Topics2negwords_entertainment.rename(columns={"index":"Topics"}, inplace=True)
print(Topics2negwords_entertainment)

# In[15]:
#Combining the entertainment opportunity dataframe and the entertainment topic to
negative words dataframe on Topics (=Topic ID)
dash_df_entertainment=pd.merge(opportunity_df_entertainment,
Topics2negwords_entertainment, on="Topics", how="inner")
print(dash_df_entertainment)

# In[16]:
#Creating an empty wedding negative words list and an empty wedding topic number list

```

```

negative_words_wedding=[]
topic_number_wedding=[]

# In[17]:
#For each column of the topic words table for the general topic wedding, select the 10
words with the most negative connotation,
#append them to the wedding negative words list as well as append the topic number (i.e.
column number) to the wedding topic number list
for i in range(0,23,1):
    a=topics_words_table_wedding.iloc[1:899,i]
    b=a.nsmallest(10).index.to_list()
    negative_words_wedding.append(b)
    topic_number_wedding.append(i)

# In[18]:
#Creating a dictionary made out of the wedding topic number list and the wedding
negative words list and print it
Topics2negwords_dict_wedding=dict(zip(topic_number_wedding,
negative_words_wedding))
print(Topics2negwords_dict_wedding)

#Converting the dictionary into a pandas dataframe
Topics2negwords_wedding=pd.DataFrame(Topics2negwords_dict_wedding)

#Transposing the pandas dataframe
Topics2negwords_wedding=Topics2negwords_wedding.T

#Place the index as a column under the name Topics (=Topic ID) and finally print the
dataframe
Topics2negwords_wedding.reset_index(level=0, inplace=True)
Topics2negwords_wedding.rename(columns={"index":"Topics"}, inplace=True)
print(Topics2negwords_wedding)

# In[19]:

```

```
#Combining the wedding opportunity dataframe and the wedding topic to negative words
dataframe on Topics (=Topic ID)
dash_df_wedding=pd.merge(opportunity_df_wedding, Topics2negwords_wedding,
on="Topics", how="inner")
print(dash_df_wedding)
```

```
# In[20]:
#Setting the viewing window to the maximum option
pd.set_option("display.max_rows", None)
```

```
# In[21]:
#Merging the 3 dataframes of different general topics into one, by connecting them as new
row entries
#due to them having the same column names and quantity of columns
entire_dash=pd.concat([dash_df_bar,dash_df_entertainment,dash_df_wedding],ignore_ind
ex=True)
```

```
#Setting the index to count starting with one, then the index is set as column and renamed
from index to Topic ID, whereby
#a new index is initialised starting to count at 0. Next the local topic ID (=Topics) is
removed and the entire dataframe is shown
entire_dash.index += 1
entire_dash.reset_index(level=0, inplace=True)
entire_dash=entire_dash.drop(columns="Topics")
entire_dash.rename(columns={"index":"Topic ID"}, inplace=True)
entire_dash.head(69)
```

```
# In[22]:
#Selecting manually what a Service Topic is and what no Service Topic is (for each Topic
sep)
entire_dash["Topic Class"]=["Service Topic","No Service Topic","Service Topic", "No
Service Topic",
                        "Service Topic","Service Topic","No Service Topic","Service Topic","No
Service Topic",
```

```

        "Service Topic","Service Topic", "Service Topic","Service Topic", "Service
Topic","No Service Topic",
        "No Service Topic","No Service Topic","No Service Topic","No Service
Topic","No Service Topic",
        "No Service Topic","No Service Topic","No Service Topic","Service Topic",
"No Service Topic",
        "No Service Topic","Service Topic","Service Topic","Service Topic","No
Service Topic",
        "Service Topic","No Service Topic","Service Topic","No Service
Topic","Service Topic",
        "Service Topic","No Service Topic", "Service Topic","No Service
Topic","Service Topic",
        "Service Topic","Service Topic","No Service Topic","Service Topic",
"Service Topic",
        "No Service Topic","No Service Topic","No Service Topic","No Service
Topic","Service Topic",
        "No Service Topic","No Service Topic","No Service Topic","Service
Topic","No Service Topic",
        "No Service Topic","Service Topic","Service Topic","Service
Topic","Service Topic", "Service Topic",
        "No Service Topic","Service Topic","No Service Topic","Service
Topic","No Service Topic",
        "Service Topic","No Service Topic","Service Topic" ]

```

#Reordering the columns of the entire dash dataframe

```

entire_dash = entire_dash[["Topic ID", "Topic Name", "Topic Class", "Topic Category",
"Topic Stock",

```

```

        "Sentiment Stock","Opportunity Score",0,1,2,3,4,5,6,7,8,9]]

```

#

```

entire_dash=entire_dash.round({"Opportunity Score":2})
entire_dash=entire_dash.round({"Topic Stock":2})
entire_dash=entire_dash.round({"Sentiment Stock":2})

```

#Selecting Topics which are classified as "Service Topic" due to their relevance for this project

```
#Secondly starting a newly initialised index count, while printing the entire dash dataframe  
as the last action
```

```
entire_dash= entire_dash[entire_dash["Topic Class"]=="Service Topic"]  
entire_dash = entire_dash.reset_index(drop=True)  
print(entire_dash)
```

```
# In[23]:
```

```
#Resetting the max view option to the default value  
pd.reset_option("display.max_rows")
```

```
# In[24]:
```

```
#Exporting the table which will be later on used for the Scatterplot  
entire_dash.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West  
London\Desktop\7 Dash on Final Table\20 000 reviews (Train = 10%)\Extended Table for  
Dash Scatterplot\ Dash Scatterplot Table.csv",index=False)
```

```
# In[25]:
```

```
#Creating a dash table dataframe  
dash_table_df=entire_dash.copy()
```

```
#Dropping these columns due to simplicity for the end user
```

```
dash_table_df=dash_table_df.drop(columns=["Topic Stock","Sentiment Stock","Topic ID",  
"Topic Class"])
```

```
#Renaming the negative word columns due to a better visibility for the end user
```

```
dash_table_df.rename(columns={0:"1st neg. word"}, inplace=True)  
dash_table_df.rename(columns={1:"2nd neg. word"}, inplace=True)  
dash_table_df.rename(columns={2:"3rd neg. word"}, inplace=True)  
dash_table_df.rename(columns={3:"4th neg. word"}, inplace=True)  
dash_table_df.rename(columns={4:"5th neg. word"}, inplace=True)  
dash_table_df.rename(columns={5:"6th neg. word"}, inplace=True)  
dash_table_df.rename(columns={6:"7th neg. word"}, inplace=True)  
dash_table_df.rename(columns={7:"8th neg. word"}, inplace=True)  
dash_table_df.rename(columns={8:"9th neg. word"}, inplace=True)  
dash_table_df.rename(columns={9:"10th neg. word"}, inplace=True)
```



```
#Sorting the dash table per default to start with the service topic with the highest opportunity score
```

```
dash_table_df=dash_table_df.sort_values(by=["Opportunity Score"],ascending=False)
```

```
#Taking a look of the newly created dash table
```

```
print(dash_table_df)
```

```
# In[26]:
```

```
#Exporting the dash table as csv file
```

```
dash_table_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of West London\Desktop\7 Dash on Final Table\20 000 reviews (Train = 10%)\Minimal Table for Dash Table\ Dash Table.csv",index=False)
```

```
# In[27]:
```

```
#Generating a subset of the entire dash table, where just Topics of Topic Category Bar are occurring
```

```
bar_entire_dash=entire_dash[entire_dash["Topic Category"]=="Bar"]
```

```
# In[28]:
```

```
#Generating a subset of the entire dash table, where just Topics of Topic Category Entertainment are occurring
```

```
entertainment_entire_dash=entire_dash[entire_dash["Topic Category"]=="Entertainment"]
```

```
# In[29]:
```

```
#Generating a subset of the entire dash table, where just Topics of Topic Category wedding are occurring
```

```
wedding_entire_dash=entire_dash[entire_dash["Topic Category"]=="Wedding"]
```

```
# In[30]:
```

```
#Saving the bar topic names in string format
```

```
text_bar=bar_entire_dash["Topic Name"].map(str)
```

```
print(text_bar)
```

```
# In[31]:
```

```
#Saving the entertainment topic names in string format
text_entertainment=entertainment_entire_dash["Topic Name"].map(str)
print(text_entertainment)
```

```
# In[32]:
```

```
#Saving the wedding topic names in string format
text_wedding=wedding_entire_dash["Topic Name"].map(str)
print(text_wedding)
```

```
# In[33]:
```

```
#Importing the plotting libraries
import dash
import plotly.graph_objs as go
import dash_core_components as dcc
import dash_html_components as html
import dash_table
```

```
# In[34]:
```

```
#Initialising the dash application
app=dash.Dash()
```

```
# In[35]:
```

```
app.layout=html.Div([
```

```
    #Setting the Main Header
```

```
        html.H1("Welcome to the Discovery of Hotel Service Opportunities in Las Vegas",style={"text-align":"center","font-family":"Arial","size":70, "text-decoration":"underline"}),
```

```
    #Calling the Graphing component
```

```
    dcc.Graph(
```

```
        id="scatter",
```

```
        #Setting what belongs into the one graph
```

```
        figure={
```

```

"data": [
    #Exercising a 3d scatterplot
    go.Scatter3d(
        #Name of the trace
        name="Bar",
        #Setting x values for the general topic bar
        x=bar_entire_dash["Topic Stock"],
        #Setting y values for the general topic bar
        y=bar_entire_dash["Sentiment Stock"],
        #Setting z values for the general topic bar
        z=bar_entire_dash["Opportunity Score"],
        #Plotting as dots (not as e.g. lines+markers, which combine the distinct dots)
        mode="markers",
        #Setting the color of the marker for this trace
        marker=dict(color="midnightblue"),
        #Providing the individual topic names of the general topic bar
        hovertext=text_bar,
        #Choosing what text is shown when hovering over the bar data and how it is
display (e.g. bold)
        hovertemplate="<span style='font-weight:bold'>Topic: %{hovertext}<br>
Opportunity Score: %{z:.2f}</span><br>Importance: %{x:.2f}<br>Satisfaction: %{y:.2f}<br>
<extra></extra>"
    ),

    #Exercising a 3d scatterplot
    go.Scatter3d(
        #Name of the trace
        name="Entertainment",
        #Setting x values for the general topic entertainment
        x=entertainment_entire_dash["Topic Stock"],
        #Setting y values for the general topic entertainment
        y=entertainment_entire_dash["Sentiment Stock"],
        #Setting z values for the general topic entertainment
        z=entertainment_entire_dash["Opportunity Score"],

```

```

#Plotting as dots (not as e.g. lines+markers, which combine the distinct dots)
mode="markers",
#Setting the color of the marker for this trace
marker=dict(color="darkgreen"),
#Providing the individual topic names of the general topic entertainment
hovertext=text_entertainment,
#Choosing what text is shown when hovering over the entertainment data and how
it is display (e.g. bold)
        hovertemplate="""<span style="font-weight:bold">Topic: %{hovertext}
<br>Opportunity Score: %{z:.2f}</span><br>Importance: %{x:.2f}<br>Satisfaction: %{y:.2f}
<extra></extra>"""
    ),

```

```

#Exercising a 3d scatterplot
go.Scatter3d(
#Name of the trace
name="Wedding",
#Setting x values for the general topic wedding
x=wedding_entire_dash["Topic Stock"],
#Setting y values for the general topic wedding
y=wedding_entire_dash["Sentiment Stock"],
#Setting z values for the general topic wedding
z=wedding_entire_dash["Opportunity Score"],
#Plotting as dots (not as e.g. lines+markers, which combine the distinct dots)
mode="markers",
#Setting the color of the marker for this trace
marker=dict(color="mediumvioletred"),
#Providing the individual topic names of the general topic wedding
hovertext=text_wedding,
#Choosing what text is shown when hovering over the wedding data and how it is
display (e.g. bold)
        hovertemplate="""<span style="font-weight:bold">Topic: %{hovertext}
<br>Opportunity Score: %{z:.2f}</span><br>Importance: %{x:.2f}<br>Satisfaction: %{y:.2f}
<extra></extra>"""
    )

```

```

],

#Setting the layout of the scatterplot
"layout":go.Layout(
    #Setting the title
    title= {"text":"<b>Scatterplot of all monitored Hotel Service Opportunities<b>","font":
{"family":"Arial","size":30},"x":0.5,"xanchor":"center","y":0.9,"yanchor":"bottom"},
    #Setting under what conditions the hover template should appear
    hovermode="closest",
    #Selecting the background color
    template="simple_white",
    #Setting the width of the plot
    width=1250,
    #Setting the height of the plot
    height=600,
    #Setting the top and bottom margins of the plot
    margin={"t":20,"b":10},
    #Setting the legend position of the plot
    legend={"x":0.8,"y":0.5},
    #Setting the axes labels and the view angle at the beginning, which is changeable
    interactively if desired by the user
    scene={
        "xaxis":{"title":"Importance"},
        "yaxis":{"title":"Satisfaction"},
        "zaxis":{"title":"Opportunity Score"},
        "camera":{"eye":{"x":1.25,"y":1.25,"z":1.65}},
    }

)
}

```

```
),
```

```
#2 line breaks
```

```
html.Br(),
```

```
html.Br(),
```

```
#Header of the Dash Table
```

```
html.H2("Service Opportunity Table",style={"text-align":"center","font-  
family":"Arial","size":30}),
```

```
#Calling the dash table component
```

```
dash_table.DataTable(  
    id="table",
```

```
#Defining the quantity of columns and their names
```

```
columns=[{"name":i,"id":i} for i in dash_table_df.columns],
```

```
#Defining the input data (i.e. records)
```

```
data=dash_table_df.to_dict("records"),
```

```
#Setting interactive options, which may be executed by the user
```

```
sort_action="native",
```

```
sort_mode="single",
```

```
row_deletable=True,
```

```
#Setting styling options for the cell, the header of the dash table and the design of the  
dash_table in general
```

```
style_cell={"fontSize":11, "font-family":"Arial","textAlign":"center"},
```

```
style_as_list_view=True,
```

```
style_header={"backgroundColor": "white"},
```

```
style_cell_conditional=[
```

```
{
```

```
"if":{"column_id":i},
```

```
"textAlign":"left"
```

```

        }for i in ["Topic Name","Topic Category","1st neg. word","2nd neg. word","3rd neg.
word","4th neg. word",
                "5th neg. word","6th neg. word","7th neg. word","8th neg. word","9th neg.
word","10th neg. word"]],

```

```

)

```

```

])

```

```

# In[36]:

```

```

#Running the Dash application

```

```

if __name__ == "__main__":

```

```

    app.run_server(port=8050)

```

```

# In[ ]:

```

```

#####End Section 20 000 reviews Train = 10%

```

```

# In[ ]:

```

```

#*Analog actions have been performed for the topics words tables and the opportunity
dataframe created through the use of

```

```

#the also tested dictionary 200 000 review Train = 99%

```

```

#(Please revisit Section (20 000 reviews Train = 10%) for a commented version of these
steps*

```

```

# In[ ]:

```

```

#####Start 200 000 review Train = 99%

```

```

# In[37]:

```

```
new_topics_words_table_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Bar\Topic words table with sentiment\Topic words table with sentiment.csv")
new_topics_words_table_bar=new_topics_words_table_bar.T
print(new_topics_words_table_bar)
```

```
# In[38]:
```

```
new_topics_words_table_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Entertainment\Topic words table with sentiment\Topic words table with
sentiment.csv")
new_topics_words_table_entertainment=new_topics_words_table_entertainment.T
print(new_topics_words_table_entertainment)
```

```
# In[39]:
```

```
new_topics_words_table_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Wedding\Topic words table with sentiment\Topic words table with
sentiment.csv")
new_topics_words_table_wedding=new_topics_words_table_wedding.T
print(new_topics_words_table_wedding)
```

```
# In[40]:
```

```
new_opportunity_df_bar=pd.read_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive -
University of West London\Desktop\6 Opportunities\200 000 reviews (Train = 99%)
\Bar\Opportunity Scores\Minimum Expansion\Opportunities for Users.csv")
new_opportunity_df_bar=new_opportunity_df_bar.drop(columns="Unnamed: 0")
new_opportunity_df_bar.head()
```

```
# In[41]:
```

```
new_opportunity_df_entertainment=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Entertainment\Opportunity Scores\Minimum Expansion\Opportunities for
Users.csv")
```



```
new_opportunity_df_entertainment=new_opportunity_df_entertainment.drop(columns="Unnamed: 0")
new_opportunity_df_entertainment.head()
```

```
# In[42]:
```

```
new_opportunity_df_wedding=pd.read_csv(r"C:\Users\Max.DESKTOP-
VPIVGKN\OneDrive - University of West London\Desktop\6 Opportunities\200 000 reviews
(Train = 99%)\Wedding\Opportunity Scores\Minimum Expansion\Opportunities for
Users.csv")
new_opportunity_df_wedding=new_opportunity_df_wedding.drop(columns="Unnamed: 0")
new_opportunity_df_wedding.head()
```

```
# In[43]:
```

```
new_negative_words_bar=[]
new_topic_number_bar=[]
```

```
# In[44]:
```

```
for i in range(0,17,1):
    a=new_topics_words_table_bar.iloc[1:904,i]
    b=a.nsmallest(10).index.to_list()
    new_negative_words_bar.append(b)
    new_topic_number_bar.append(i)
```

```
# In[45]:
```

```
new_Topics2negwords_dict_bar=dict(zip(new_topic_number_bar,new_negative_words_bar))
print(new_Topics2negwords_dict_bar)
new_Topics2negwords_bar=pd.DataFrame(new_Topics2negwords_dict_bar)
new_Topics2negwords_bar=new_Topics2negwords_bar.T
new_Topics2negwords_bar.reset_index(level=0,inplace=True)
new_Topics2negwords_bar.rename(columns={"index":"Topics"},inplace=True)
print(new_Topics2negwords_bar)
```

```
# In[46]:
```

```
new_dash_df_bar=pd.merge(new_opportunity_df_bar,new_Topics2negwords_bar,on="Topics",how="inner")
print(new_dash_df_bar)
```

```
# In[47]:
```

```
new_negative_words_entertainment=[]
new_topic_number_entertainment=[]
```

```
# In[48]:
```

```
for i in range(0,24,1):
    a=new_topics_words_table_entertainment.iloc[1:960,i]
    b=a.nsmallest(10).index.to_list()
    new_negative_words_entertainment.append(b)
    new_topic_number_entertainment.append(i)
```

```
# In[49]:
```

```
new_Topics2negwords_dict_entertainment=dict(zip(new_topic_number_entertainment,new_negative_words_entertainment))
print(new_Topics2negwords_dict_entertainment)
new_Topics2negwords_entertainment=pd.DataFrame(new_Topics2negwords_dict_entertainment)
new_Topics2negwords_entertainment=new_Topics2negwords_entertainment.T
new_Topics2negwords_entertainment.reset_index(level=0,inplace=True)
new_Topics2negwords_entertainment.rename(columns={"index":"Topics"},inplace=True)
print(new_Topics2negwords_entertainment)
```

```
# In[50]:
```

```
new_dash_df_entertainment=pd.merge(new_opportunity_df_entertainment,new_Topics2negwords_entertainment,on="Topics",how="inner")
print(new_dash_df_entertainment)
```

```
# In[51]:
```

```
new_negative_words_wedding=[]
new_topic_number_wedding=[]
```

```

# In[52]:
for i in range(0,23,1):
    a=new_topics_words_table_wedding.iloc[1:822,i]
    b=a.nsmallest(10).index.to_list()
    new_negative_words_wedding.append(b)
    new_topic_number_wedding.append(i)

# In[53]:
new_Topics2negwords_dict_wedding=dict(zip(new_topic_number_wedding,new_negative
_words_wedding))
print(new_Topics2negwords_dict_wedding)
new_Topics2negwords_wedding=pd.DataFrame(new_Topics2negwords_dict_wedding)
new_Topics2negwords_wedding=new_Topics2negwords_wedding.T
new_Topics2negwords_wedding.reset_index(level=0,inplace=True)
new_Topics2negwords_wedding.rename(columns={"index":"Topics"},inplace=True)
print(new_Topics2negwords_wedding)

# In[54]:
new_dash_df_wedding=pd.merge(new_opportunity_df_wedding,new_Topics2negwords_w
edding,on="Topics",how="inner")
print(new_dash_df_wedding)

# In[55]:
pd.set_option("display.max_rows",None)

# In[56]:
new_entire_dash=pd.concat([new_dash_df_bar,new_dash_df_entertainment,new_dash_d
f_wedding],ignore_index=True)
new_entire_dash.index += 1
new_entire_dash.reset_index(level=0, inplace=True)
new_entire_dash=new_entire_dash.drop(columns="Topics")
new_entire_dash.rename(columns={"index":"Topic ID"}, inplace=True)
new_entire_dash.head(64)

# In[57]:

```

```

new_entire_dash["Topic Class"]=["Service Topic","No Service Topic","Service Topic",
"Service Topic","Service Topic","No Service Topic","Service Topic", "No Service Topic","No
Service Topic","No Service Topic","No Service Topic","No Service Topic","Service
Topic","No Service Topic","No Service Topic","No Service Topic","No Service
Topic","Service Topic","Service Topic","Service Topic","No Service Topic","No Service
Topic","Service Topic","Service Topic","No Service Topic","Service Topic","Service
Topic","No Service Topic","No Service Topic","No Service Topic","No Service
Topic","Service Topic","Service Topic","Service Topic","No Service Topic","Service
Topic","No Service Topic","Service Topic","Service Topic","Service Topic","Service
Topic","Service Topic","No Service Topic","No Service Topic","No Service Topic","No
Service Topic","Service Topic","Service Topic","Service Topic","No Service Topic","No
Service Topic","No Service Topic","Service Topic","No Service Topic","No Service
Topic","No Service Topic","No Service Topic","Service Topic","No Service Topic","No
Service Topic","Service Topic","Service Topic","No Service Topic","Service Topic"]
new_entire_dash = new_entire_dash[["Topic ID", "Topic Name", "Topic Class", "Topic
Category", "Topic Stock","Sentiment Stock","Opportunity Score",0,1,2,3,4,5,6,7,8,9]]
new_entire_dash=new_entire_dash.round({"Opportunity Score":2})
new_entire_dash=new_entire_dash.round({"Topic Stock":2})
new_entire_dash=new_entire_dash.round({"Sentiment Stock":2})
new_entire_dash= new_entire_dash[new_entire_dash["Topic Class"]=="Service Topic"]
new_entire_dash = new_entire_dash.reset_index(drop=True)
print(new_entire_dash)

# In[58]:
pd.reset_option("display.max_rows")

# In[59]:
new_entire_dash.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\7 Dash on Final Table\200 000 reviews (Train=99%)\Extended
Table for Dash Scatterplot\Dash Scatterplot Table.csv",index=False)

# In[60]:
new_dash_table_df=new_entire_dash.copy()
new_dash_table_df=new_dash_table_df.drop(columns=["Topic Stock","Sentiment
Stock","Topic ID","Topic Class"])

```

```

new_dash_table_df.rename(columns={0:"1st neg. word"}, inplace=True)
new_dash_table_df.rename(columns={1:"2nd neg. word"}, inplace=True)
new_dash_table_df.rename(columns={2:"3rd neg. word"}, inplace=True)
new_dash_table_df.rename(columns={3:"4th neg. word"}, inplace=True)
new_dash_table_df.rename(columns={4:"5th neg. word"}, inplace=True)
new_dash_table_df.rename(columns={5:"6th neg. word"}, inplace=True)
new_dash_table_df.rename(columns={6:"7th neg. word"}, inplace=True)
new_dash_table_df.rename(columns={7:"8th neg. word"}, inplace=True)
new_dash_table_df.rename(columns={8:"9th neg. word"}, inplace=True)
new_dash_table_df.rename(columns={9:"10th neg. word"}, inplace=True)

new_dash_table_df=new_dash_table_df.sort_values(by=["Opportunity
Score"],ascending=False)

print(new_dash_table_df)

# In[61]:
new_dash_table_df.to_csv(r"C:\Users\Max.DESKTOP-VPIVGKN\OneDrive - University of
West London\Desktop\7 Dash on Final Table\200 000 reviews (Train=99%)\Minimal Table
for Dash Table\Dash Table.csv",index=False)

# In[62]:
bar_new_entire_dash= new_entire_dash[new_entire_dash["Topic Category"]=="Bar"]

# In[63]:
entertainment_new_entire_dash= new_entire_dash[new_entire_dash["Topic
Category"]=="Entertainment"]

# In[64]:
wedding_new_entire_dash= new_entire_dash[new_entire_dash["Topic
Category"]=="Wedding"]

# In[65]:
new_text_bar=bar_new_entire_dash["Topic Name"].map(str)

```

```
print(new_text_bar)
```

```
# In[66]:
```

```
new_text_entertainment=entertainment_new_entire_dash["Topic Name"].map(str)
print(new_text_entertainment)
```

```
# In[67]:
```

```
new_text_wedding=wedding_new_entire_dash["Topic Name"].map(str)
print(new_text_wedding)
```

```
# In[68]:
```

```
app=dash.Dash()
```

```
# In[69]:
```

```
app.layout=html.Div([
```

```
    html.H1("Welcome to the Discovery of Hotel Service Opportunities in Las Vegas",style={"text-align":"center","font-family":"Arial","size":70, "text-decoration":"underline"}),
```

```
    dcc.Graph(
        id="scatter",
        figure={
```

```
            "data": [
```

```
                go.Scatter3d(
                    name="Bar",
                    x=bar_new_entire_dash["Topic Stock"],
                    y=bar_new_entire_dash["Sentiment Stock"],
                    z=bar_new_entire_dash["Opportunity Score"],
                    mode="markers",
                    marker=dict(color="midnightblue"),
```

```

        hovertext=new_text_bar,
            hovertemplate=""<span style="font-weight:bold">Topic: %{hovertext}
<br>Opportunity Score: %{z:.2f}</span><br>Importance: %{x:.2f}<br>Satisfaction: %{y:.2f}
<extra></extra>""
    ),

    go.Scatter3d(
        name="Entertainment",
        x=entertainment_new_entire_dash["Topic Stock"],
        y=entertainment_new_entire_dash["Sentiment Stock"],
        z=entertainment_new_entire_dash["Opportunity Score"],
        mode="markers",
        marker=dict(color="darkgreen"),
        hovertext=new_text_entertainment,
            hovertemplate=""<span style="font-weight:bold">Topic: %{hovertext}
<br>Opportunity Score: %{z:.2f}</span><br>Importance: %{x:.2f}<br>Satisfaction: %{y:.2f}
<extra></extra>""
    ),

    go.Scatter3d(
        name="Wedding",
        x=wedding_new_entire_dash["Topic Stock"],
        y=wedding_new_entire_dash["Sentiment Stock"],
        z=wedding_new_entire_dash["Opportunity Score"],
        mode="markers",
        marker=dict(color="mediumvioletred"),
        hovertext=new_text_wedding,
            hovertemplate=""<span style="font-weight:bold">Topic: %{hovertext}
<br>Opportunity Score: %{z:.2f}</span><br>Importance: %{x:.2f}<br>Satisfaction: %{y:.2f}
<extra></extra>""
    )
],

"layout":go.Layout(

```

```

        title= {"text": "<b>Scatterplot of all monitored Hotel Service Opportunities<b>", "font":
{"family": "Arial", "size": 30}, "x": 0.5, "xanchor": "center", "y": 0.9, "yanchor": "bottom"},
        hovermode="closest",
        template="simple_white",
        width=1250,
        height=600,
        margin={"t": 20, "b": 10},
        legend={"x": 0.8, "y": 0.5},

        scene={
            "xaxis":{"title": "Importance"},
            "yaxis":{"title": "Satisfaction"},
            "zaxis":{"title": "Opportunity Score"},
            "camera":{"eye":{"x": 1.25, "y": 1.25, "z": 1.65}},
        }

    )
}

),
html.Br(),
html.Br(),
        html.H2("Service Opportunity Table", style={"text-align": "center", "font-
family": "Arial", "size": 30}),
        dash_table.DataTable(
            id="table",
            columns=[{"name": i, "id": i} for i in new_dash_table_df.columns],
            data=new_dash_table_df.to_dict("records"),
            sort_action="native",
            sort_mode="single",
            row_deletable=True,
            style_cell={"fontSize": 11, "font-family": "Arial", "text-align": "center"},

```



```

style_as_list_view=True,
style_header={"backgroundColor": "white"},
style_cell_conditional=[
    {
        "if":{"column_id":i},
        "textAlign":"left"
    }for i in ["Topic Name","Topic Category","1st neg. word","2nd neg. word","3rd neg.
word","4th neg. word",
                "5th neg. word","6th neg. word","7th neg. word","8th neg. word","9th neg.
word","10th neg. word"]]

)

])

# In[70]:
if __name__ == "__main__":
    app.run_server(port=8051)

```

End - 10 Dash Visualisation

..... (Opportunity elements for both selected systems)